EECE 435L: Software Tools Lab

Final Project: Sprint I Technical Report

Security Incident Detection and Response Management System

Backend Services

Team members

Adam Yahya	Services 1, 2 and 7
Leen Abdallah	Services 4, 5 and 6
Youssef Al Hajj Youness	Services 3, 8 and 9

Youssef Al Hajj Youness: 202407879

Adam Yahya: 202402786

Leen Abdallah: 202400276

Instructor: Iman Ghalayini

GitHub repository:
`https://github.com/yy2jisj/EECE435L-Security-Incident-Response-System`

Table of Contents

#	Section	Page	Team member responsible
1	Title Page	i	
2	Table of Contents	1	
3	Introduction	2	
4	System Architecture	4	
5	Implementation Details	6	
5.1	Service 1: Users and Identity	6	(Adam Yahya)
5.2	Service 2: Asset Management	12	(Adam Yahya)
5.3	Service 3: Incident Management	24	(Youssef Al Hajj Youness)
5.4	Service 4: Threat Intelligence	34	(Leen Abdallah)
5.5	Service 5: Security Audit Logging	55	(Leen Abdallah)
5.6	Service 6: Security Reports	70	(Leen Abdallah)
5.7	Service 7: Vulnerability Management	78	(Adam Yahya)
5.8	Service 8: Notifications and Alerts	86	(Youssef Al Hajj Youness)
5.9	Service 9: Security Playbooks	96	(Youssef Al Hajj Youness)
6	Database Design	105	
7	Error Handling and Validation	108	
8	API Documentation	139	
9	Authentication and Authorization	144	
10	GitHub and Version Control	184	
11	References	189	

3 Introduction

3.1 Project Overview

The Security Incident Detection and Response Management System records incidents and coordinates investigation. It links reports to assets, assigned analysts and notes, supported by threat indicators, vulnerabilities, notifications, audit records and playbooks. Administrators, analysts, employees and auditors access these records through role-specific APIs.

The project connects asset details, investigation updates and responsibility in one relational database. APIs enforce role and record-level access. Incident status rules preserve the investigation sequence, and audit records support review of changes and access denials.

Sprint I covers nine Python/Flask REST services, SQLite persistence, authentication, validation and API documentation. Postman captures demonstrate selected operations. Separate processes share a database and Python modules, retaining deployment and data dependencies.

3.2 Objectives

- Implement and document nine services with role and record-level access rules.
- Separate routing, entity validation and persistence where that structure fits the service; use source adapters and statistics functions for reporting.
- Enforce relational links, required values and allowed categories through application validation and database constraints.
- Authenticate accounts with password hashes and signed tokens, and check current account permissions on protected requests.
- Return consistent JSON responses and document validation failures, lifecycle conflicts and unavailable dependencies.
- Supply the Postman collection, execution captures and version-control evidence, while identifying remaining coverage gaps.

3.3 Team Work Distribution

Student	Services implemented	API routes carry
Adam Yahya	1 Users and Identity, 2 Asset Management, 7 Vulnerability Management	`AdamYahya`
Leen Abdallah	4 Threat Intelligence, 5 Security Audit Logging, 6 Security Reports	`leen-abdallah`
Youssef Al Hajj Youness	3 Incident Management, 8 Notifications, 9 Security Playbooks	`Youssef-Al-Hajj-Youness`

4 System Architecture

4.1 Overall System Design

Nine Flask applications expose HTTP/JSON endpoints on ports 5001–5009 (Figure 1). They share SQLite, initialized from `database/schema.sql`, enabling foreign keys and cross-entity transactions while retaining schema and database-file coupling.

The `common/` package supplies basic helper functions that are used throughout the project. They are a combination of codes from older projects we have worked on as well as some newer functions specific to this project.

Most entity services separate routes, entity validation and persistence into `app.py`, `models.py` and `repository.py`. Service 6 separates source access from statistics. Shared authentication and audit modules also run SQL.

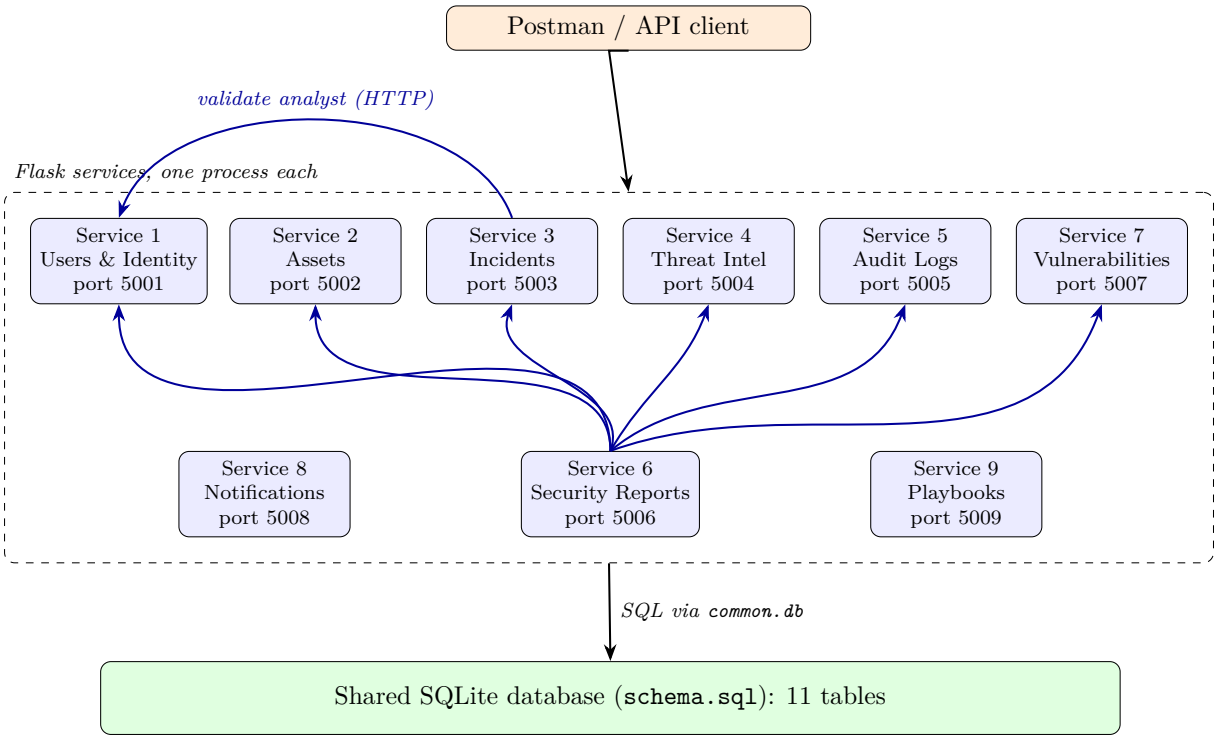


Figure 1: Service processes and shared dependencies. Blue arrows represent HTTP calls; black arrows represent client access and SQL access. Incident assignment also uses the notification repository within one SQLite transaction.

4.2 Description of Each Service

#	Service	Port	Responsibility
1	Users and Identity (Adam Yahya)	5001	Registration, login (password hashing, JWT issue), user retrieval, profile updates and deletion; login checks stored account status.
2	Asset Management (Adam Yahya)	5002	Inventory of servers, workstations, databases, applications, network devices and cloud resources with owner, department, criticality and status.
3	Incident Management (Youssef Al Hajj Youness)	5003	Incident reporting, assignment to analysts, enforced lifecycle Open → Under Investigation → Mitigated → Resolved → Closed, investigation notes.
4	Threat Intelligence (Leen Abdallah)	5004	Threat indicators (IP, domain, file hash, malware signature, other) with type-specific validation, uniqueness, search.
5	Security Audit Logging (Leen Abdallah)	5005	Create, read, search and delete audit records; a trigger protects stored log content, subject to the foreign-key nulling exception.
6	Security Reports (Leen Abdallah)	5006	Read-only summary, incident, asset, threat, vulnerability and activity reports built from the other services.
7	Vulnerability Management (Adam Yahya)	5007	CVE records tied to assets, CVSS and severity, enforced remediation lifecycle.
8	Notifications and Alerts (Youssef Al Hajj Youness)	5008	Notifications per user, unread/read tracking, search, automatic notification on incident assignment.
9	Security Playbooks (Youssef Al Hajj Youness)	5009	Response playbooks with ordered response actions, assignment, completion tracking and retained action-status history; response commands are not executed.

4.3 Communication Between Services

Clients call services over HTTP/JSON; protected calls use bearer authentication. Service 6 forwards the caller's token to Services 1, 2, 3, 4, 5 and 7 and aggregates their records sequentially. Unavailable or invalid source responses return `503`; reports are not transactionally consistent snapshots.

Service 3 verifies an Active Security Analyst through Service 1, with a five-second timeout. Dependency failure returns `503` without changing assignment. Assignment, notification insertion and audit entries then commit together. Notification creation calls a shared repository, not Service 8 over HTTP.

Audit helpers write SQL directly; Service 5 exposes those records. Standalone logging is best effort, while audit writes on an existing connection participate in that transaction.

5 Implementation Details

Each service description connects its API contract and persistence behavior to the execution evidence. Protected operations combine authentication with role, ownership and input checks. Responses use the shared JSON envelope documented in Section 8; public authentication, reads and reporting have distinct workflows.

Development captures establish the displayed request, result or database state. Added repository images complete selected responses and notification ownership examples; the figures do not cover every role or input.

5.1 Service 1: Users and Identity Management (Adam Yahya)

Responsibility. Service 1 registers accounts, verifies credentials and issues the JWT accepted by all services. Login requires an Active account. Public registration creates a Regular User; creating a privileged role requires administrator authorization.

API contract. Registration requires `full_name`, `username`, `email`, `password`, `department` and accepts an optional `role`. It returns the created account with `201`. Login accepts a username or email and a password, then returns a bearer token and public account fields. Profile updates accept only `full_name`, `email`, `department`; role and account-status changes have no exposed update API.

Persistence. The `users` table stores a Werkzeug password hash (was chosen because it ships with Flask and provides deliberately slow password hashing with a constant-time check, which a plain fast hash such as SHA-256 does not) and unique username and email values. The application checks username duplicates without case distinctions; the schema's uniqueness constraint does not itself implement that comparison. Login updates `last_login`. Public account responses exclude `password_hash`. The database query below displays account fields, so password-hash storage is established by the source implementation rather than that image.

Implemented APIs.

Method	Endpoint	Access
POST	`/users/register/AdamYahya`	public (Regular User role); other roles need an administrator
POST	`/users/login/AdamYahya`	public
GET	`/users/AdamYahya`	Administrator, Auditor
GET	`/users/<username>/AdamYahya`	Administrator, Auditor; Regular User own record only
PUT	`/users/<id>/AdamYahya`	Administrator; Regular User own record only
DELETE	`/users/<id>/AdamYahya`	Administrator

Figures 2–5: Startup, registration and login establish the account workflow. The query confirms stored account fields; the login response shows token issuance.

```
Lab6 — Python -m user_service.app — 97×26

aazelmac@Adams-MacBook-Air ~ % cd Desktop
aazelmac@Adams-MacBook-Air Desktop % cd Lab6
aazelmac@Adams-MacBook-Air Lab6 % ls
asset_service           notification_service    tests
claude_screenshots      postman                 user_service
common                  README.md               vulnerability_service
database                requirements.txt
docs                    screenshots
aazelmac@Adams-MacBook-Air Lab6 % python3 -m user_service.app
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [17/Oct/2026 15:56:33] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [17/Oct/2026 15:56:33] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [17/Oct/2026 15:58:39] "POST /users/login/AdamYahya HTTP/1.1" 200 -
127.0.0.1 - - [17/Oct/2026 16:00:01] "POST /users/login/AdamYahya HTTP/1.1" 200 -
127.0.0.1 - - [17/Oct/2026 16:00:27] "POST /users/login/AdamYahya HTTP/1.1" 400 -
127.0.0.1 - - [17/Oct/2026 16:00:47] "POST /users/login/AdamYahya HTTP/1.1" 401 -
efore127.0.0.1 - - [17/Oct/2026 16:11:21] "POST /users/login/AdamYahya HTTP/1.1" 200 -
127.0.0.1 - - [17/Oct/2026 16:12:05] "POST /users/login/AdamYahya HTTP/1.1" 401 -
```

Figure 2: Adam Yahya: Service 1 started on port 5001

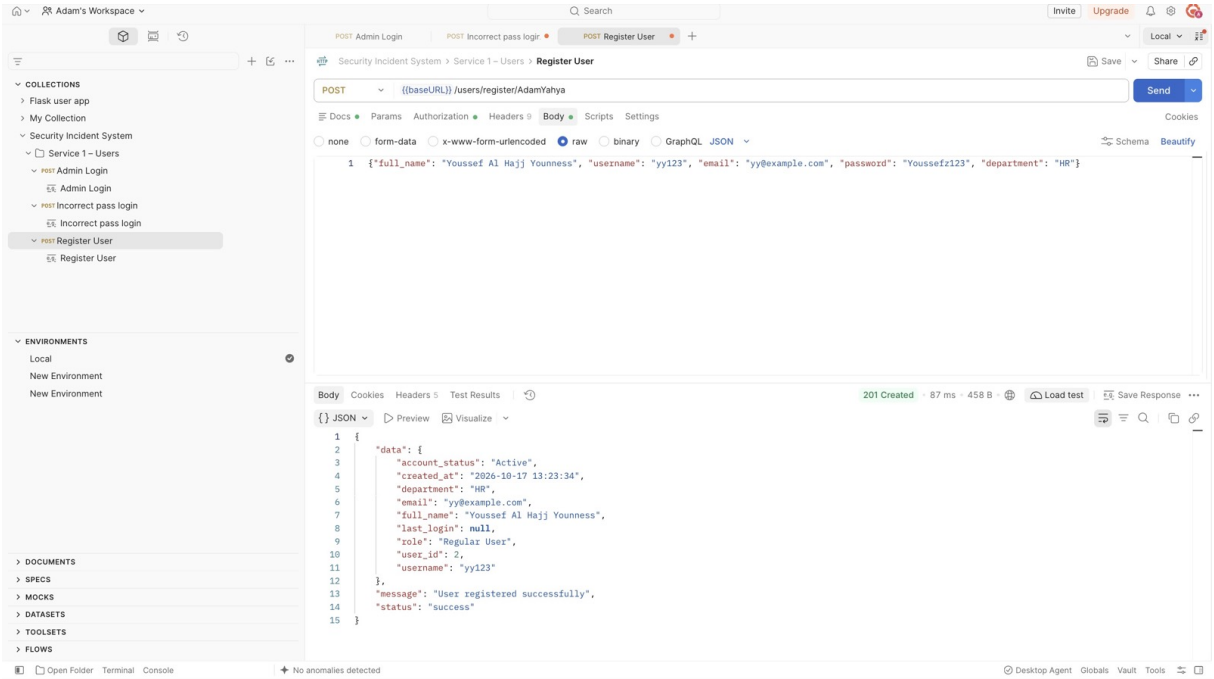


Figure 3: Adam Yahya: Register User (POST /users/register/AdamYahya): creates a new Regular User account, 201

```
aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT user_id, username, email,
 department, role, account_status FROM users;"
user_id     username                email                      department            role                  account_status
-------     --------------  -------------------------  ----------  --------------------  --------------
1           admin           admin@example.com          IT Security  Security Administrator  Active
3           youssef.younnes youssef.younnes@example.com Security     Regular User          Active
4           leen.abdallah   leen.abdallah@example.com   Audit        Regular User          Active
5           adam.yahya      adam.yahya@example.com      Finance      Regular User          Active
7           yy123           yy@example.com              HR           Regular User          Active
8           jad             jad@example.com             Security     Security Analyst      Active
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 4: Adam Yahya: Database query: account fields after registration; this query does not display password hashes

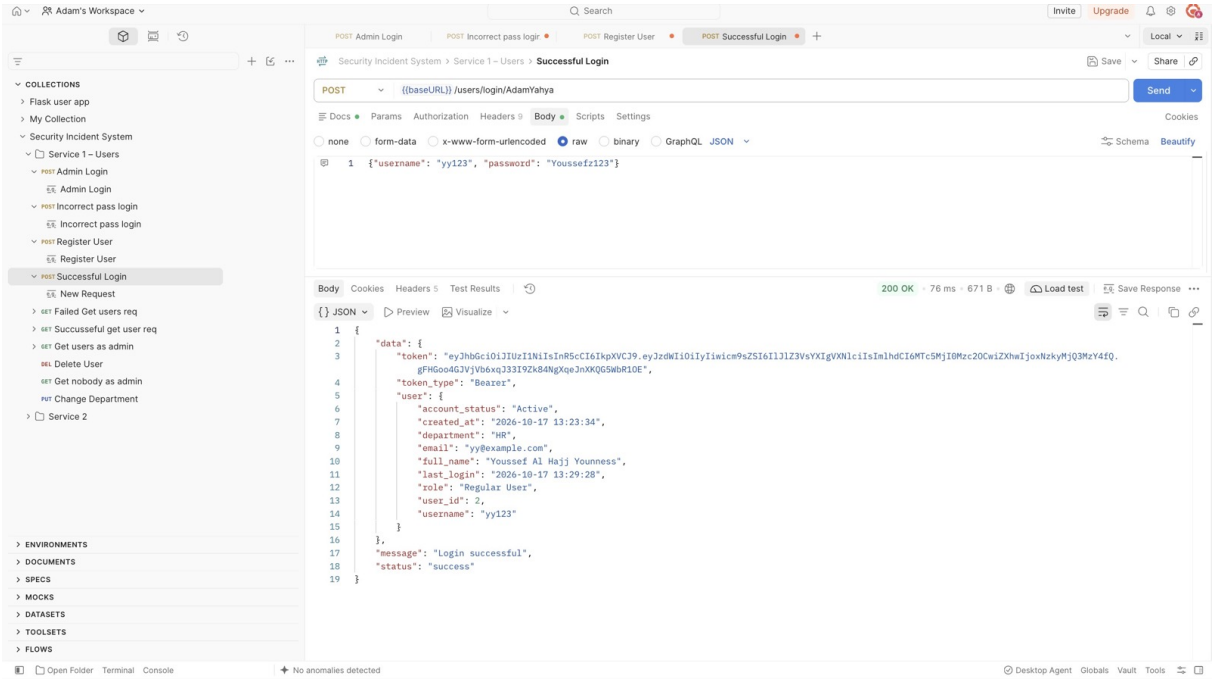


Figure 5: Adam Yahya: Login (POST /users/login/AdamYahya): valid credentials return a JWT, 200

Figures 6–8: The retrieval captures distinguish a successful administrator list, an owner reading their profile and an unknown username returning 404.

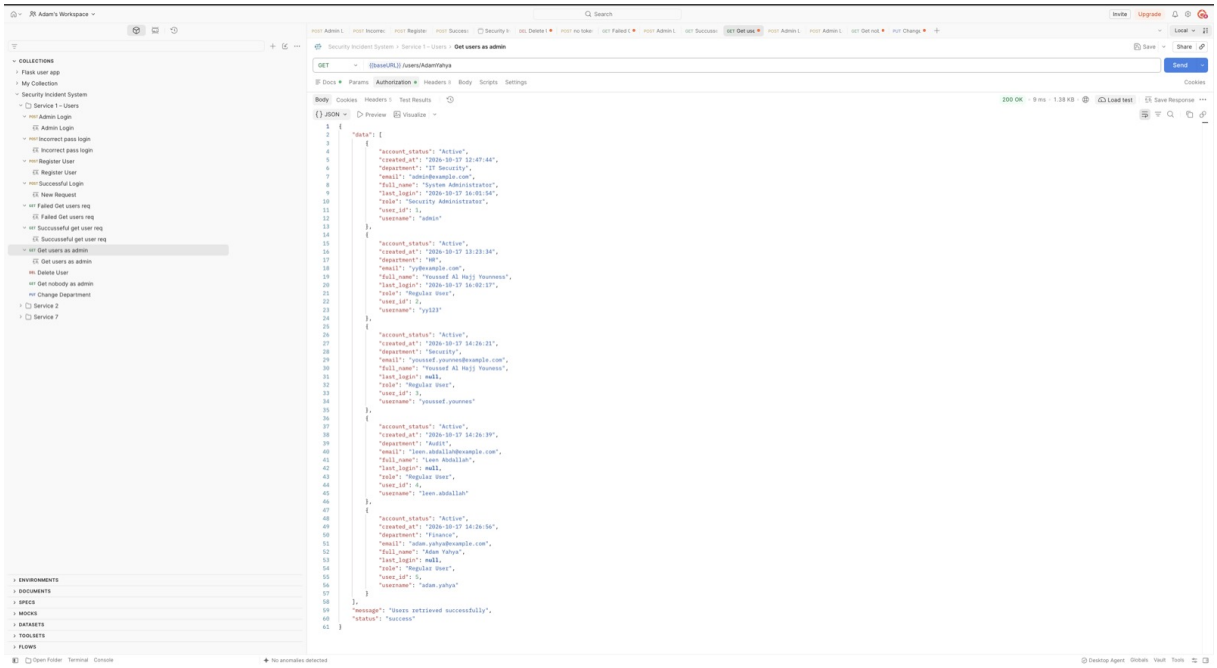


Figure 6: Adam Yahya: Get All Users (GET /users/AdamYahya): administrator receives every user, 200

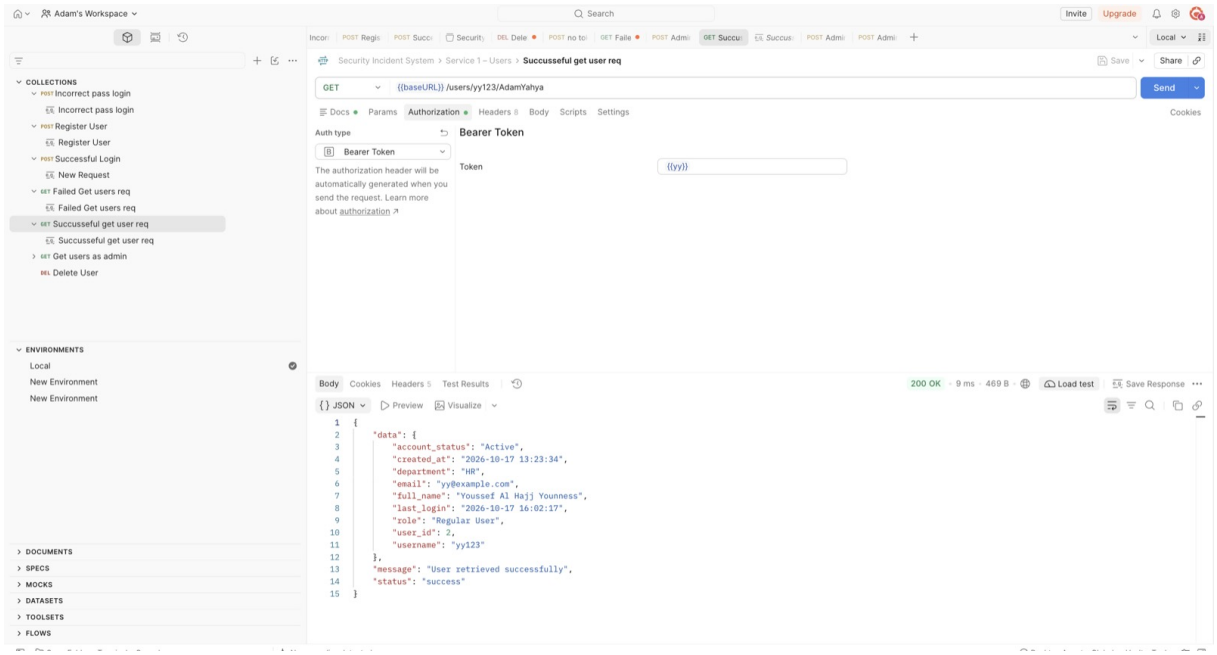


Figure 7: Adam Yahya: Get User by Username: a Regular User reads their own record, 200

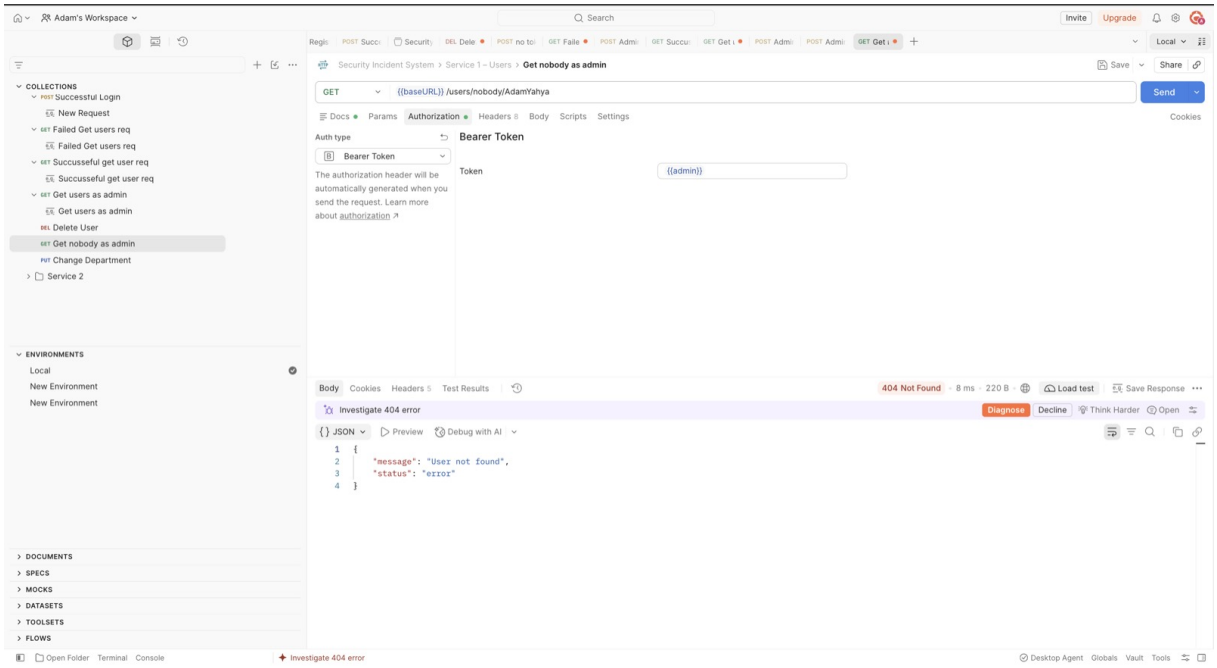


Figure 8: Adam Yahya: Get User by Username: unknown username, 404

Figures 9–10: The profile update changes the caller's department. The later database query shows the other account remained unchanged after a refused update.

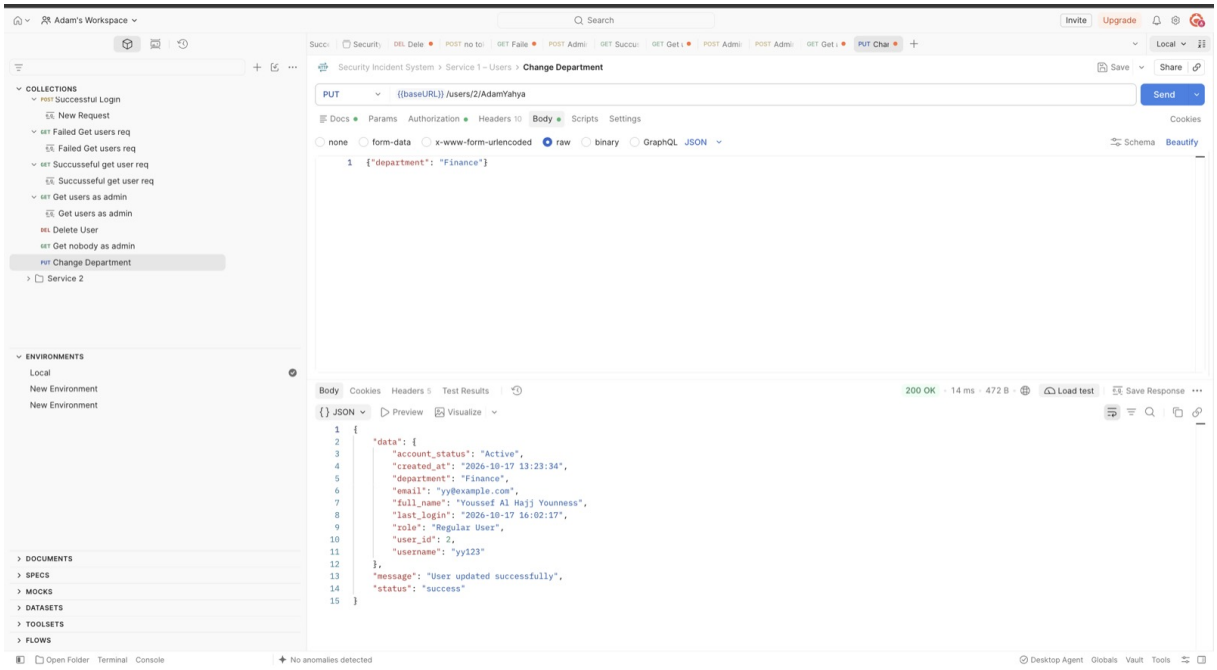


Figure 9: Adam Yahya: Update User (PUT /users/:id/AdamYahya): user updates own department, 200

```
Last login: Sat Oct 17 15:48:00 on ttys017
[aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT u]
ser_id, username, email, department, role, account_status FROM users;"
user_id    username              email                 department           role                 account_status
-------  --------------  --------------------  -----------  --------------------  --------------
      1  admin           admin@example.com      IT Security  Security Administrator  Active

      2  yy123           yy@example.com         Finance      Regular User            Active

      3  youssef.younnes youssef.younnes@example Security     Regular User            Active
                         .com

      4  leen.abdallah   leen.abdallah@example.  Audit        Regular User            Active
                         com

      5  adam.yahya      adam.yahya@example.com  Finance      Regular User            Active
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 10: Adam Yahya: Database: users table after a refused update of another user, record unchanged

Figures 11–13: Deletion is followed by a database view, while the visible audit excerpt records user changes and permission denials. The excerpt does not show every event type.

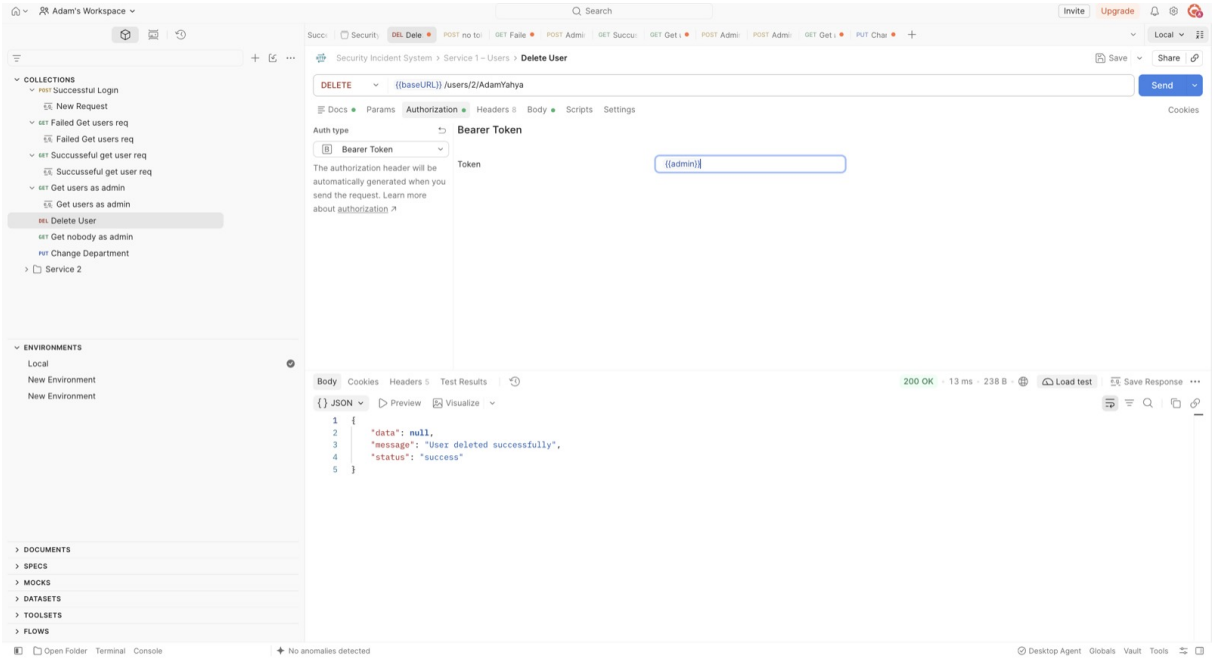


Figure 11: Adam Yahya: Delete User (DELETE /users/:id/AdamYahya): administrator deletes an account, 200

```
aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT u
ser_id, username, email, department, role, account_status FROM users;"
user_id    username             email          department           role          account_status
-------  ---------------  ---------------------  -----------  --------------------  --------------
      1  admin            admin@example.com      IT Security  Security Administrator  Active

      3  youssef.younnes  youssef.younnes@example  Security     Regular User          Active
                          .com

      4  leen.abdallah    leen.abdallah@example.   Audit        Regular User          Active
                          com

      5  adam.yahya       adam.yahya@example.com   Finance      Regular User          Active
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 12: Adam Yahya: Database: users table after the deletion

26	Unauthorized access attempt	PUT /users/1/ AdamYahya: You can only update your own user information	Warning	users	2026-10-17 16:
27	User updated	User 2 updated fields: department	Information	users	2026-10-17 16:
28	Unauthorized access attempt	DELETE /users/2/ AdamYahya: User does not have permission to perform this ...	Warning	users	2026-10-17 16:
29	Unauthorized access attempt	DELETE /users/2/ AdamYahya: User does not have permission to perform this ...	Warning	users	2026-10-17 16:
30	1 User deleted	Account 'yy123' (id 2) deleted	Warning	users	2026-10-17 16:

Figure 13: Adam Yahya: Database: visible user-update, user-deletion and permission-denial audit records

5.2 Service 2: Asset Management (Adam Yahya)

Responsibility. Service 2 maintains the asset inventory used by incident and vulnerability records. Administrators create, update and delete assets; analysts retrieve individual assets and search, while auditors can also list the inventory.

API contract. Creation requires `asset_name`, `asset_type`, `ip_address`, `department`, `owner_id`, `criticality_level`; `operating_system`, `location`, `status` are optional. Search accepts name, type, IP address, department and criticality filters. Create returns one asset with `201`; retrieval, update and search return `200`, with search results represented as a list.

Persistence and history. Each `assets.owner_id` references an existing user. CHECK constraints restrict criticality and status, while application validation checks asset types and IP addresses. Specific-asset retrieval includes current related incidents and vulnerabilities. Audit

entries identify modifications, but the service does not expose a complete revision history or retain a before-and-after snapshot of every asset change.

Implemented APIs.

Method	Endpoint	Access
POST	`/assets/AdamYahya`	Administrator
GET	`/assets/AdamYahya`	Administrator, Auditor
GET	`/assets/<id>/AdamYahya`	Administrator, Auditor, Analyst
GET	`/assets/search/AdamYahya`	Administrator, Auditor, Analyst
PUT	`/assets/<id>/AdamYahya`	Administrator
DELETE	`/assets/<id>/AdamYahya`	Administrator

Figures 14–21: The creation captures cover five submitted asset types and a database schema view.

```
aazelmac@Adams-MacBook-Air Lab6 % python3 -m asset_service.app
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

Figure 14: Adam Yahya: Service 2 started on port 5002

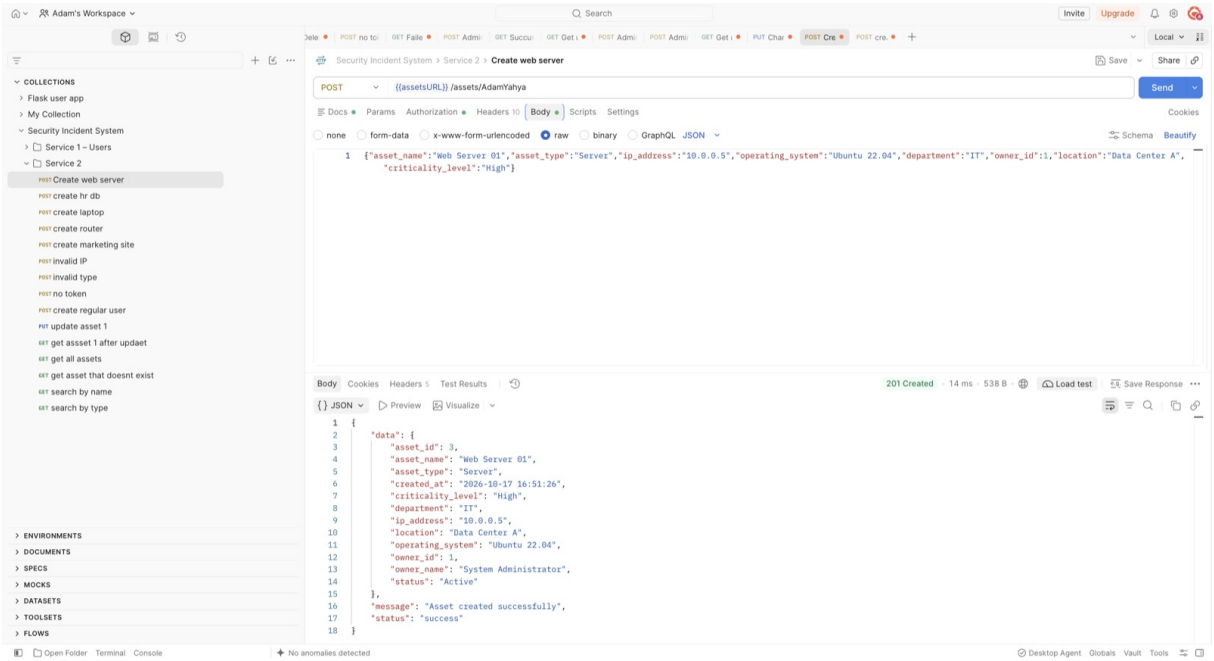


Figure 15: Adam Yahya: Add Asset (POST /assets/AdamYahya): creates WebServer01 as asset 3 with High criticality, 201

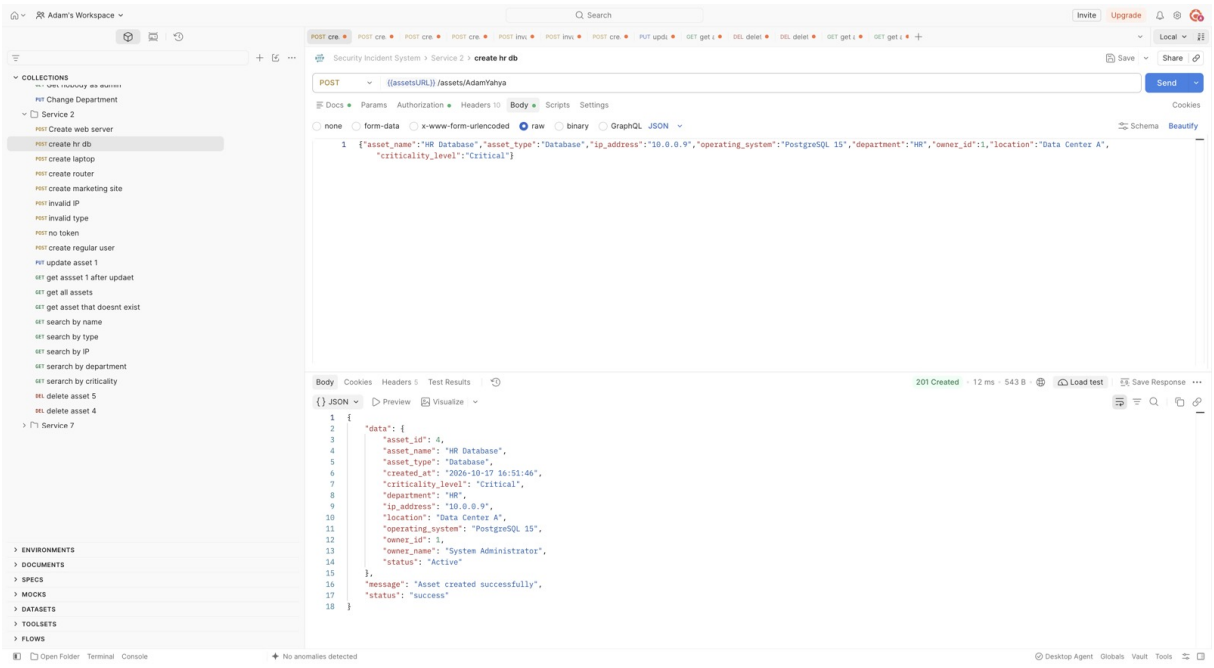


Figure 16: Adam Yahya: Add Asset: creates the HR database asset, 201

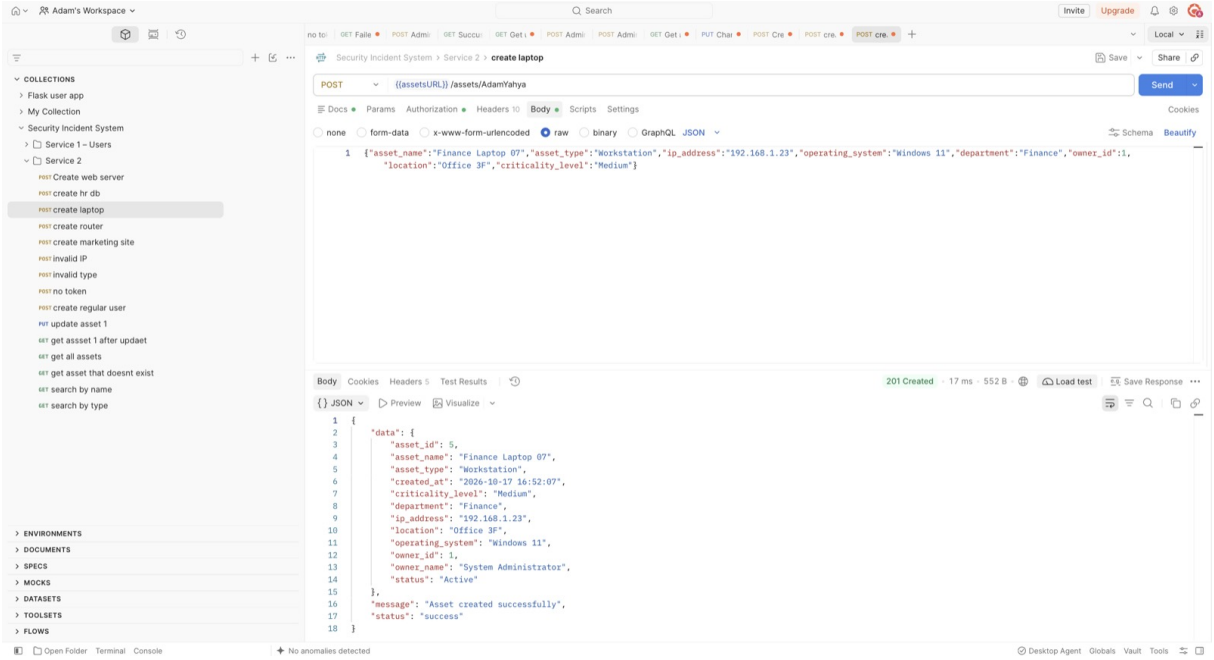


Figure 17: Adam Yahya: Add Asset: creates a finance workstation, 201

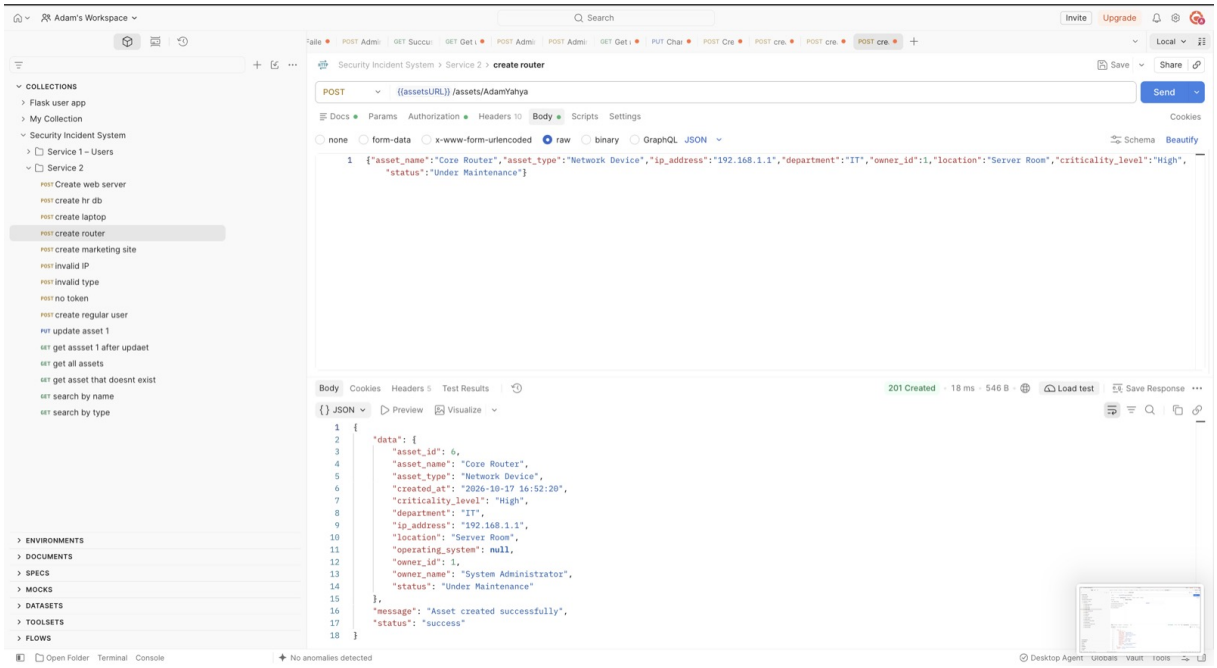


Figure 18: Adam Yahya: Add Asset: creates the core router, 201

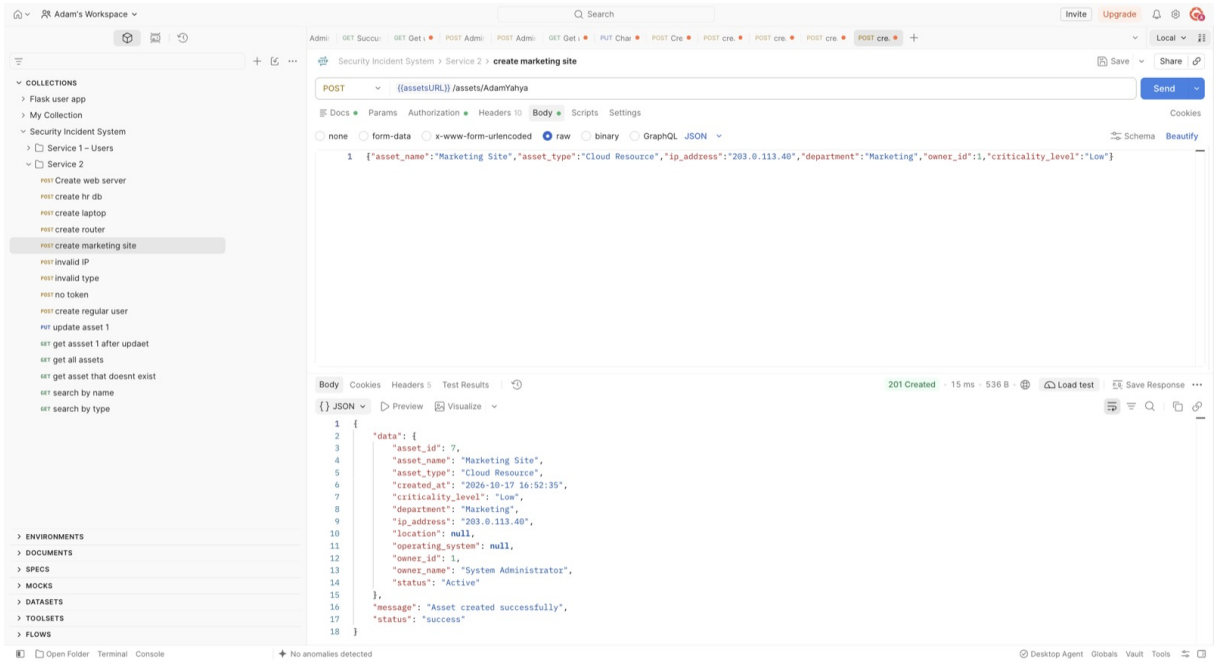


Figure 19: Adam Yahya: Add Asset: creates the marketing site, 201

```
Last login: Sat Oct 17 19:05:01 on ttys018
[aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT asset_id, asset_name, ass]
et_type, ip_address, department, owner_id, criticality_level, status FROM assets;"
asset_id asset_nam   asset_type    ip_address  department owner_id criticality_      status
            e                                                        level
-------- --------- -------------- ------------ ---------- -------- ------------ ------------------
       1 Web       Server         10.0.0.5     IT                1 High         Active
         Server 01

       2 Web       Server         10.0.0.5     IT                1 High         Active
         Server 01

       3 Web       Server         10.0.0.5     IT                1 High         Active
         Server 01

       4 HR Databa Database       10.0.0.9     HR                1 Critical     Active
         se

       5 Finance   Workstation    192.168.1.23 Finance           1 Medium       Active
         Laptop 07

       6 Core      Network Device 192.168.1.1  IT                1 High         Under Maintenance
         Router

       7 Marketing Cloud Resource 203.0.113.40 Marketing         1 Low          Active
         Site
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 20: Adam Yahya: Database: assets table with the inserted records

```
[aazelmac@Adams-MacBook-Air Lab6 % sqlite3 database/security_incident.db ".schema assets"
CREATE TABLE assets (
    asset_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    asset_name        TEXT NOT NULL,
    asset_type        TEXT NOT NULL
                      CHECK (asset_type IN ('Server', 'Workstation', 'Database',
                                            'Application', 'Network Device',
                                            'Cloud Resource')),
    ip_address        TEXT NOT NULL,
    operating_system  TEXT,                      -- optional ("if applicable")
    department        TEXT NOT NULL,
    owner_id          INTEGER NOT NULL REFERENCES users(user_id),
    location          TEXT,
    criticality_level TEXT NOT NULL
                      CHECK (criticality_level IN ('Low', 'Medium', 'High', 'Critical')),
    status            TEXT NOT NULL DEFAULT 'Active'
                      CHECK (status IN ('Active', 'Inactive', 'Under Maintenance')),
    created_at        TEXT NOT NULL DEFAULT (datetime('now'))
);
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 21: Adam Yahya: Database: assets schema showing how criticality is stored (TEXT with a CHECK constraint)

Figures 22–24: For the actual before-and-after comparison, asset 1 is High and Active in the earlier database table. The PUT response, subsequent GET and stored row show Critical, Under Maintenance, owner 3 and department Infrastructure.

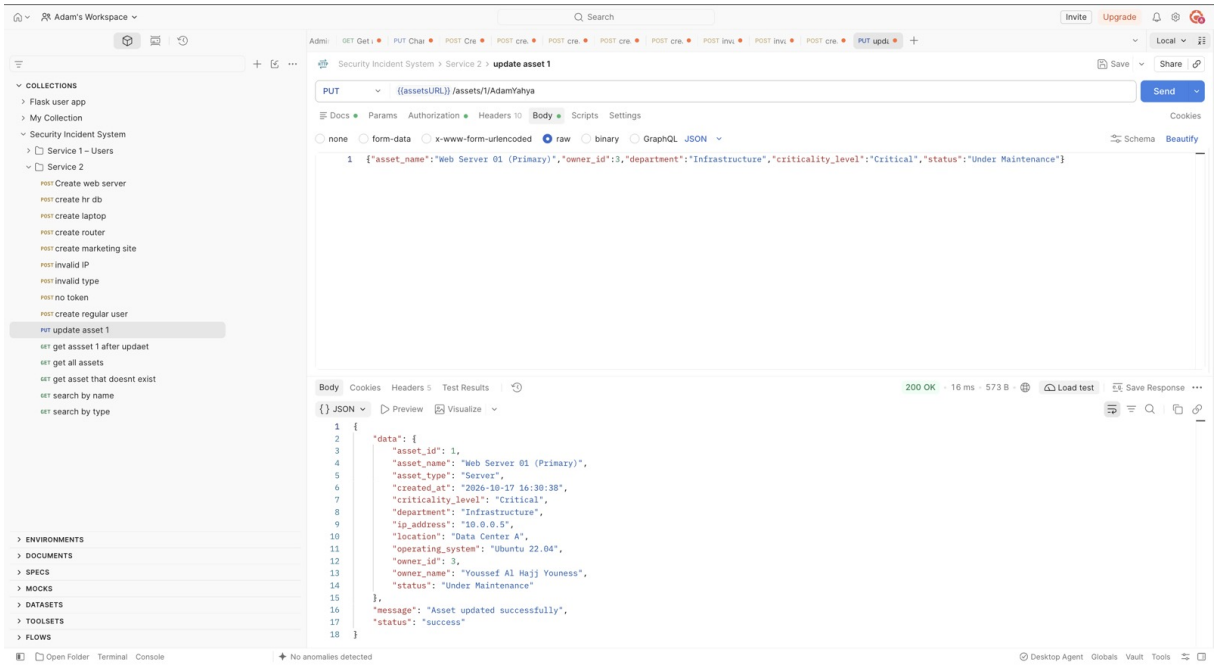


Figure 22: Adam Yahya: Update Asset (PUT /assets/:id/AdamYahya): administrator updates asset 1, 200

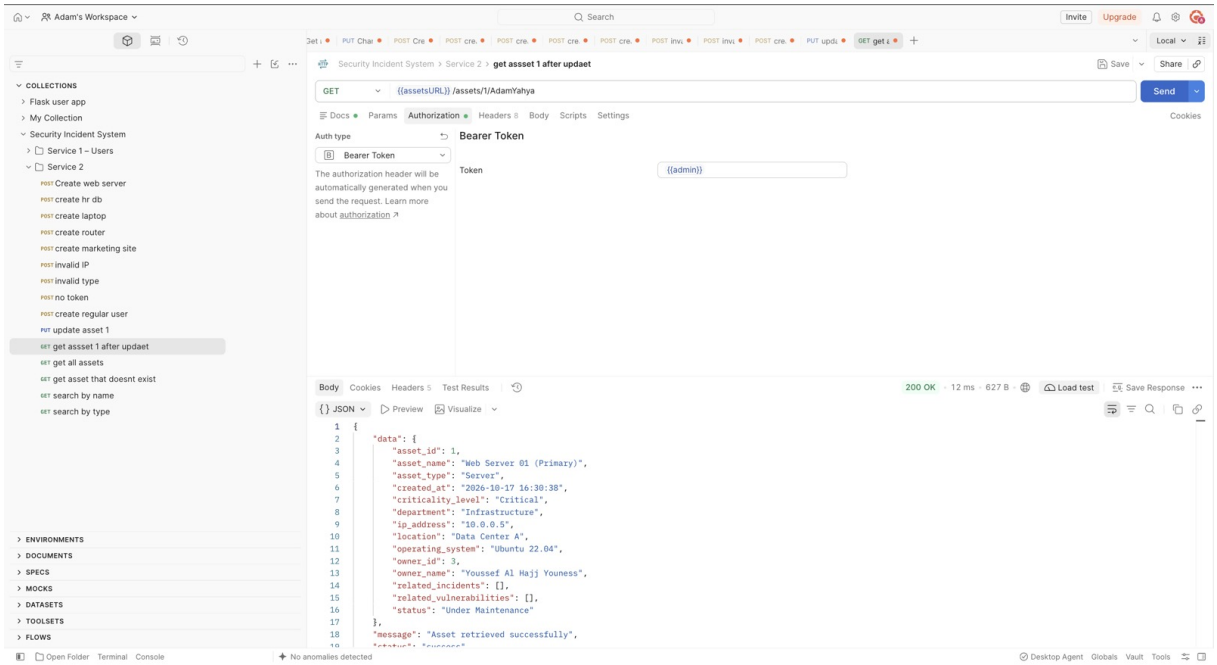


Figure 23: Adam Yahya: Get Asset: asset 1 after the update (the after state), 200

```
aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT * FROM assets WHERE asset
_id = 1;"
asset_id  asset_name  asset_type ip_address operatin   department   owner_id   location    criticali  status   created_at
                                 g_system                                     ty_level
--------  ----------  ---------- ---------- --------  -----------  --------  ----------  ---------  -------  ----------
       1  Web Server  Server     10.0.0.5   Ubuntu    Infrastructure        3  Data Center A  Critical   Under    2026-10-17
          01 (Primary)                      22.04                                                    Maintenan  16:30:38
                                                                                                     ce

aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 24: Adam Yahya: Database: asset 1 row after the update

Figures 25–29: The administrator deletes asset 5, lists the remaining inventory and receives 404 for a missing asset. Continuation images preserve the rest of the list response.

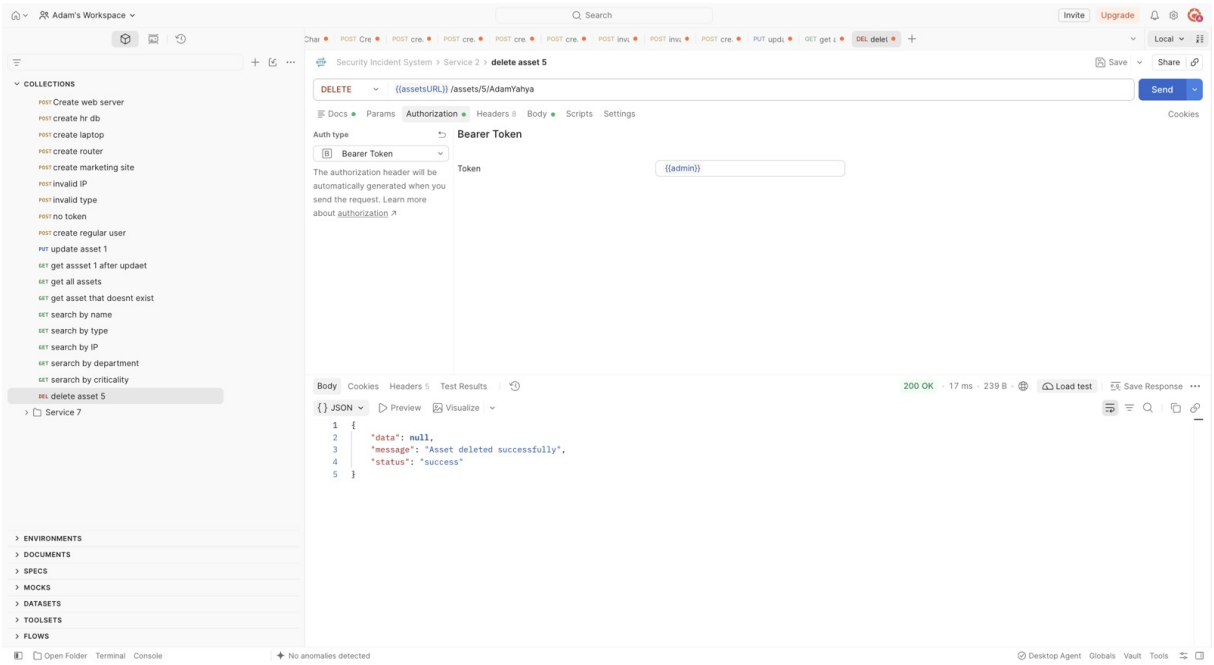


Figure 25: Adam Yahya: Delete Asset (DELETE /assets/:id/AdamYahya): administrator deletes asset 5, 200

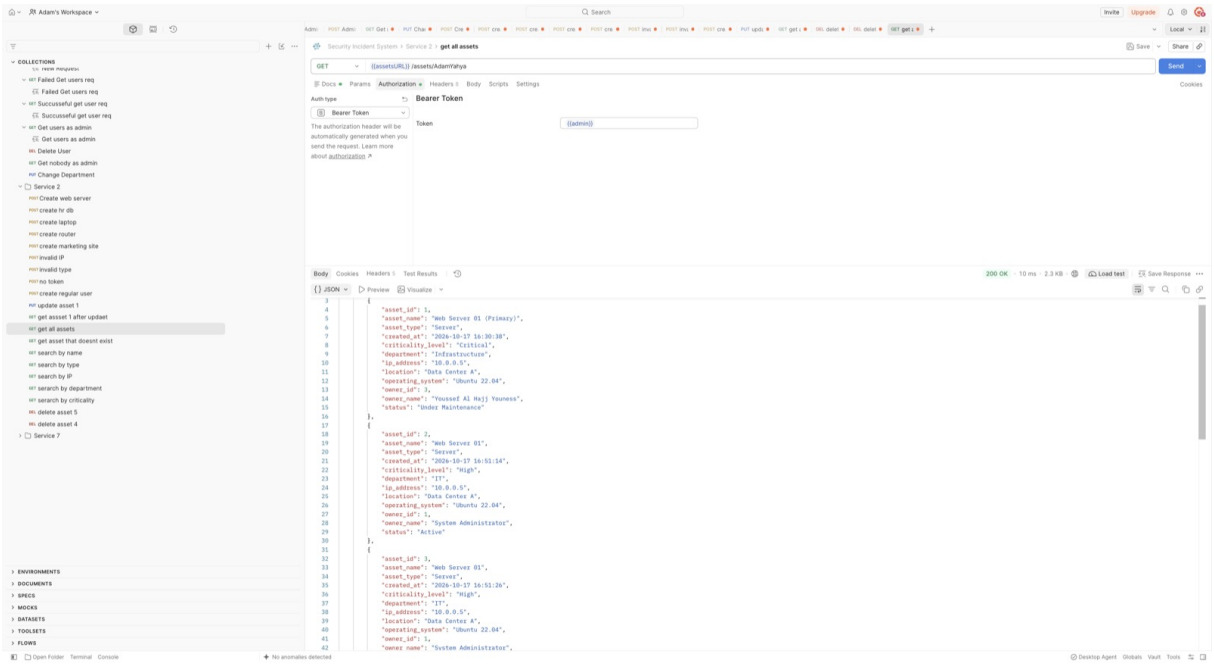


Figure 26: Adam Yahya: Get All Assets (GET /assets/AdamYahya): administrator lists assets, 200

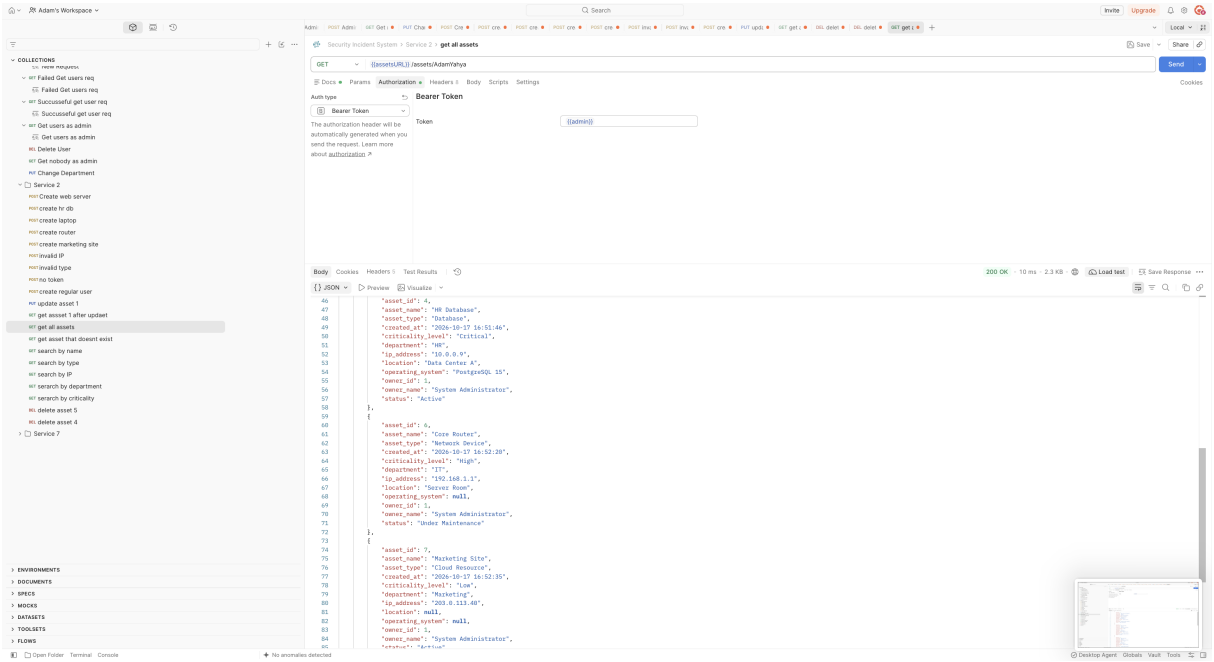


Figure 27: Adam Yahya: Get All Assets: continuation of the same 200 response

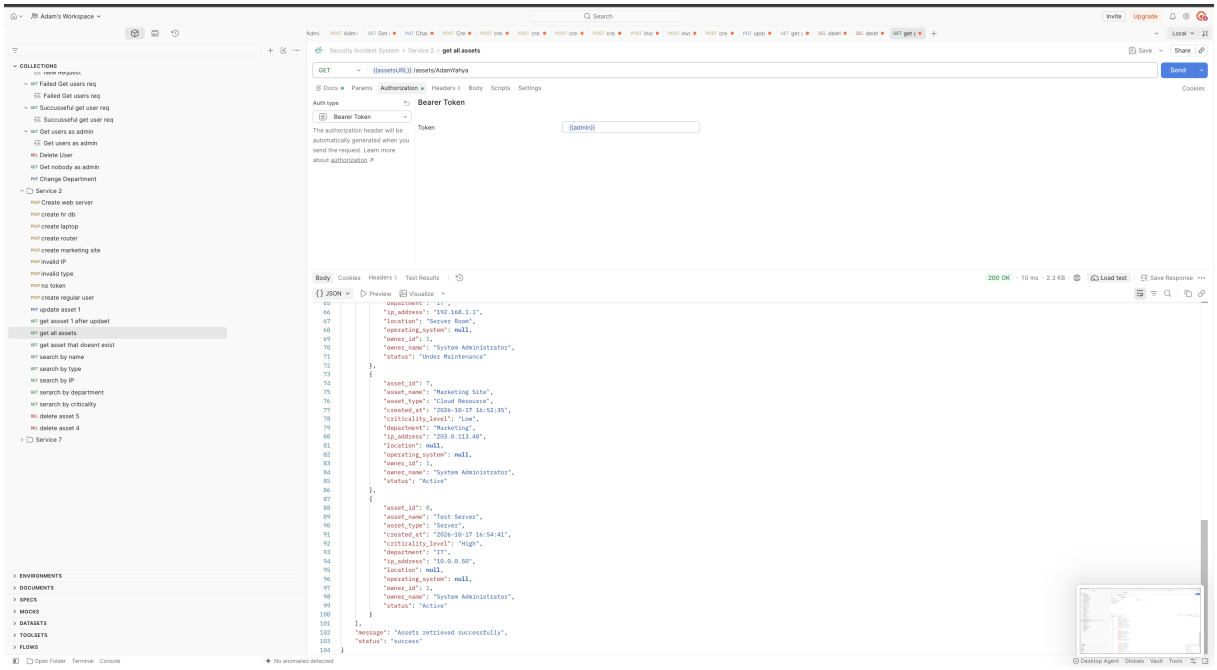


Figure 28: Adam Yahya: Get All Assets: continuation of the same 200 response

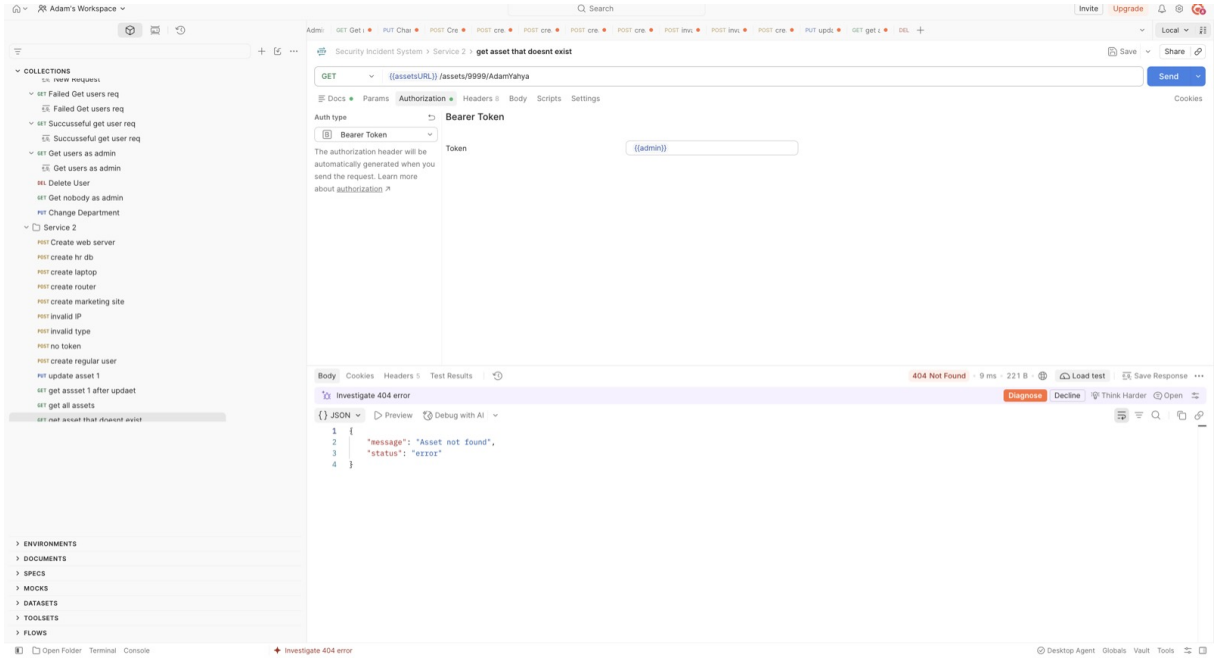


Figure 29: Adam Yahya: Get Asset: invalid asset ID, 404

Figures 30–36: Search captures demonstrate name, type, IP, department and criticality filters. The added continuations show the remaining name and department results.

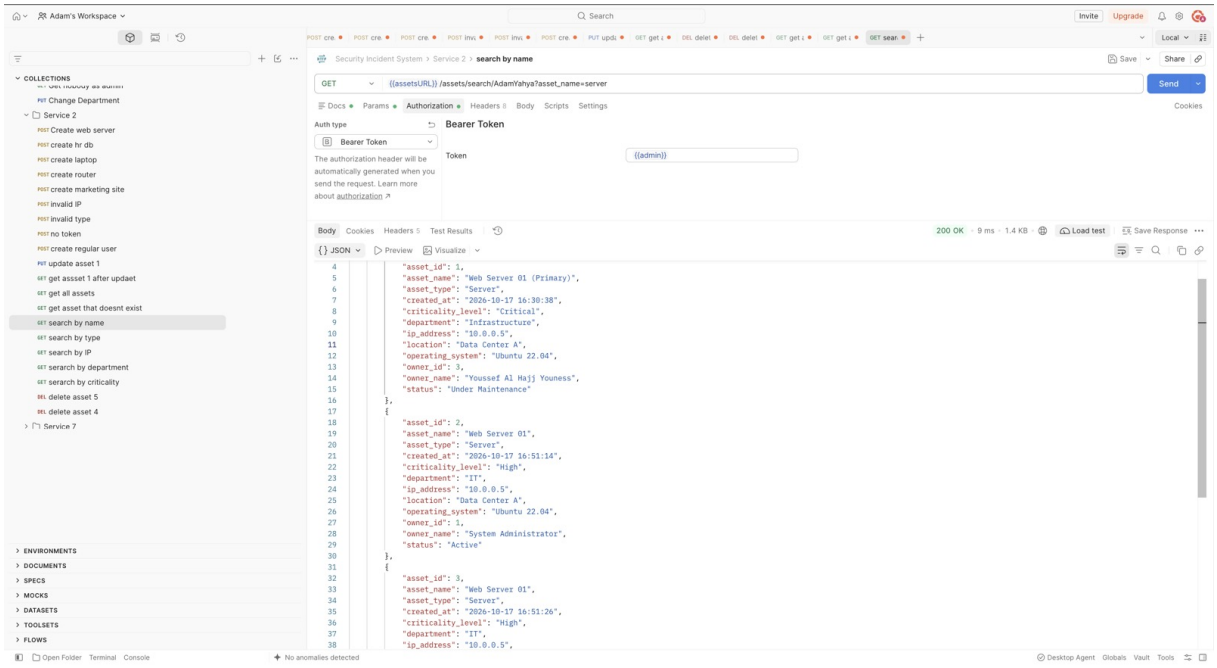


Figure 30: Adam Yahya: Search Assets (GET /assets/search/AdamYahya): search by asset name, 200

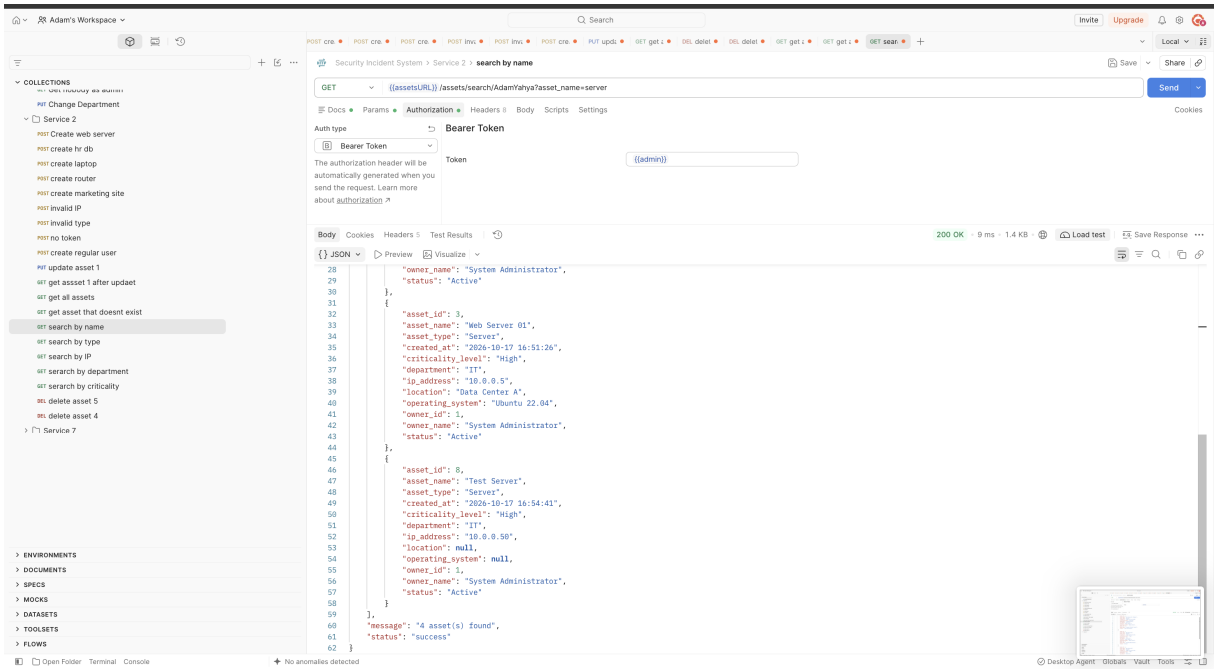


Figure 31: Adam Yahya: Asset-name search: continuation of the 200 response

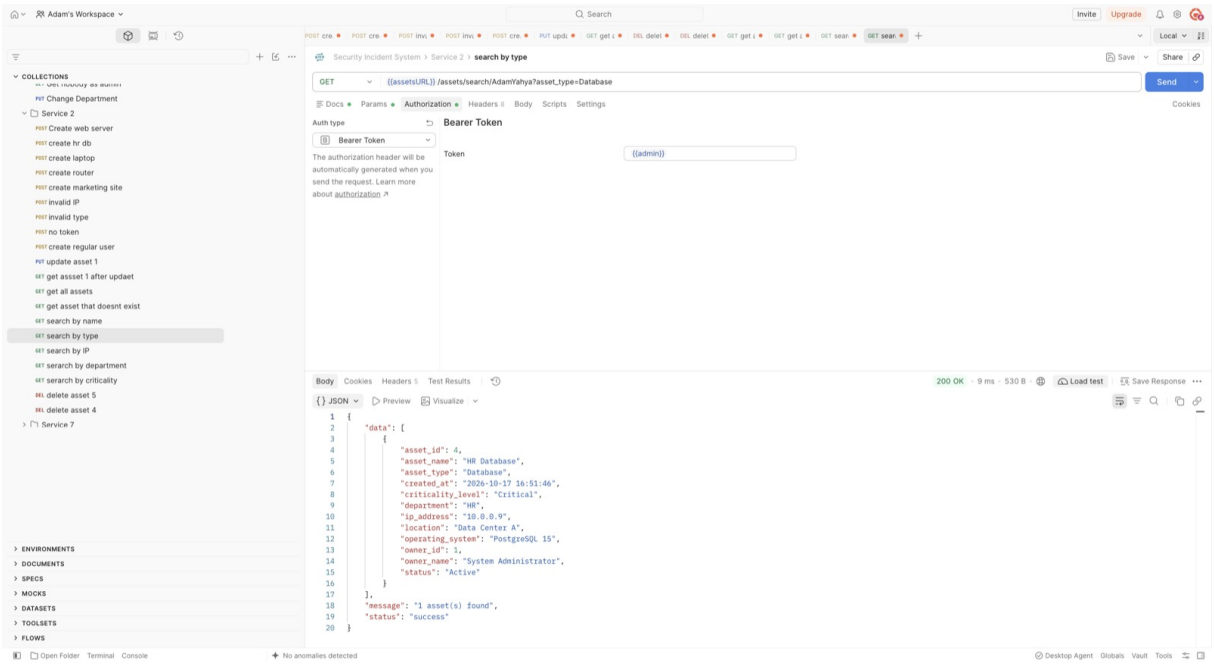


Figure 32: Adam Yahya: Search Assets: search by asset type, 200

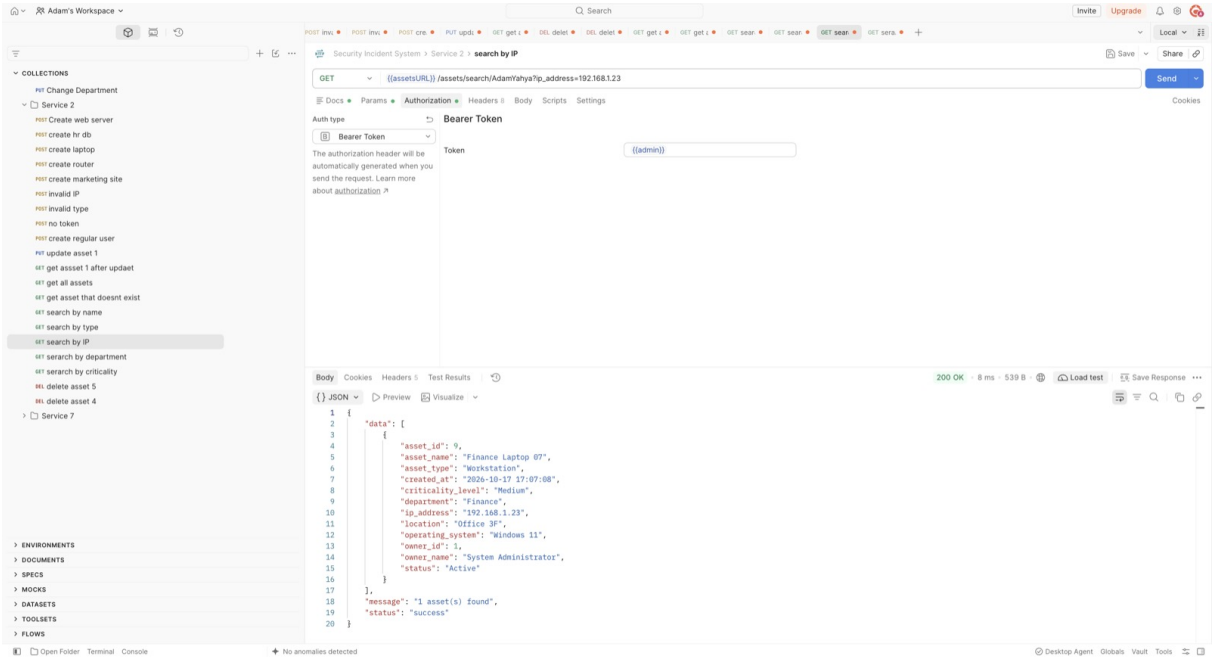


Figure 33: Adam Yahya: Search Assets: search by IP address, 200

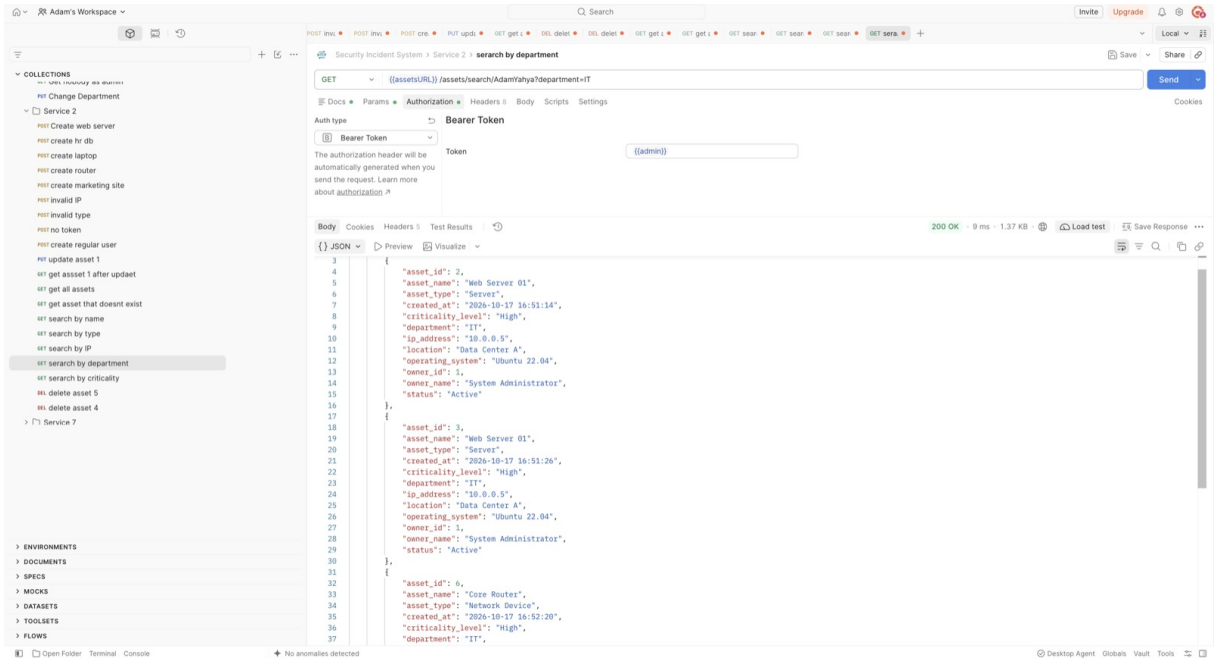


Figure 34: Adam Yahya: Search Assets: search by department, 200

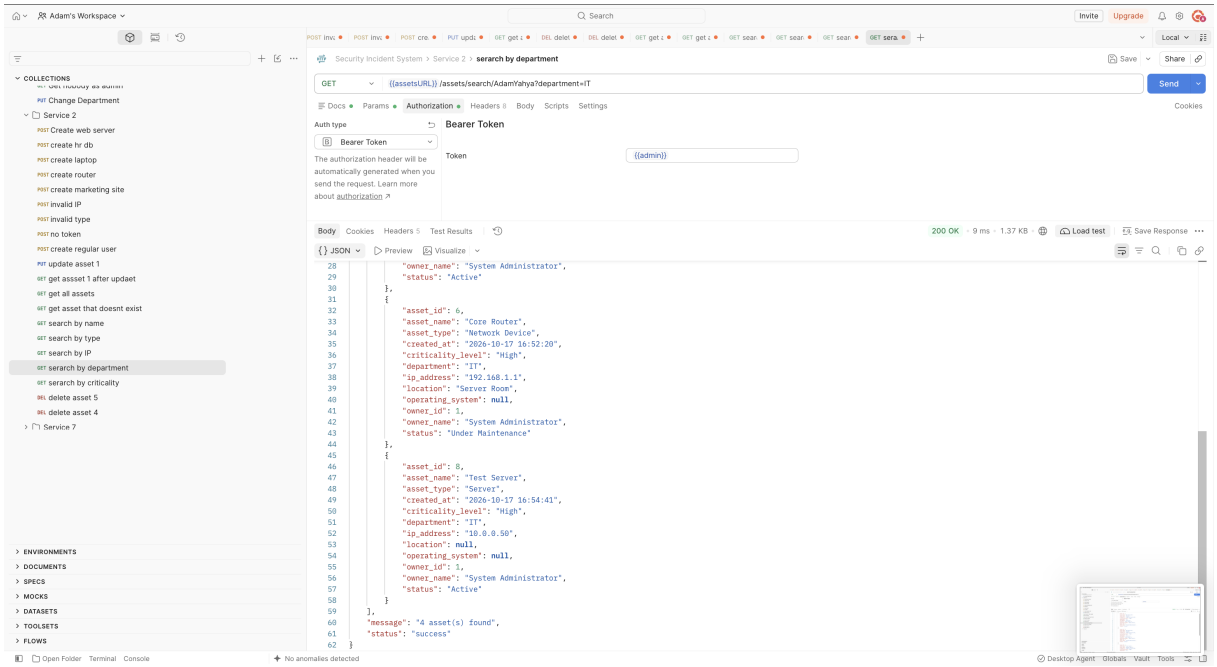


Figure 35: Adam Yahya: Department search: continuation of the 200 response

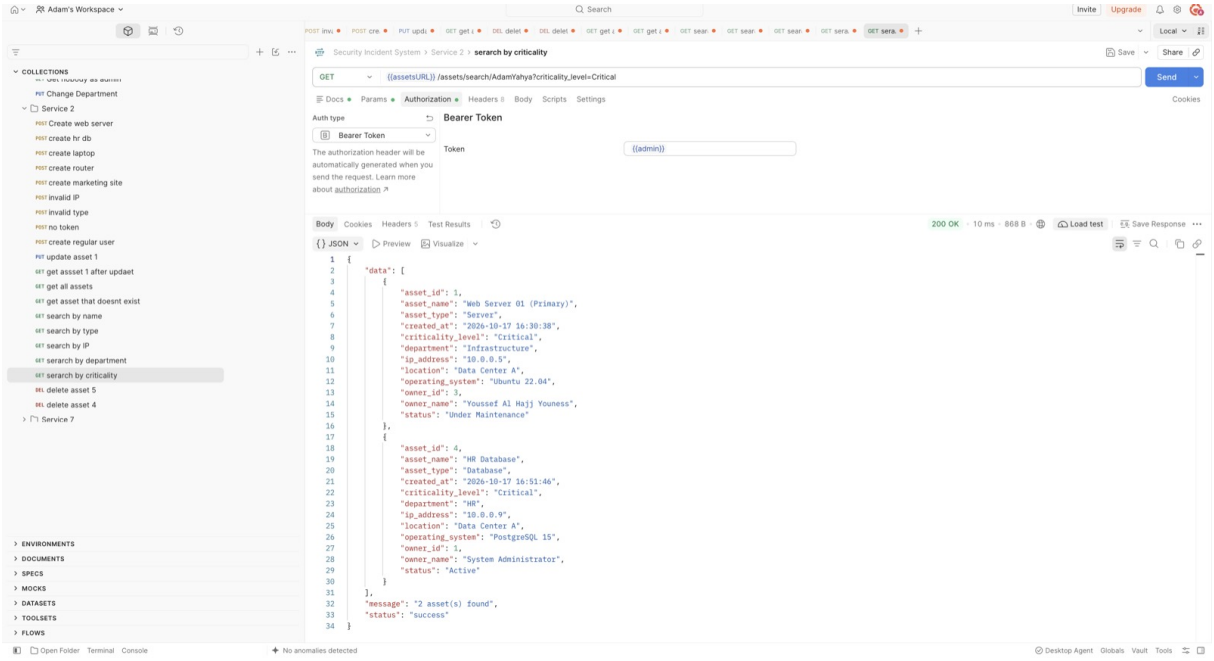


Figure 36: Adam Yahya: Search Assets: search by criticality level, 200

5.3 Service 3: Incident Management (Youssef Al Hajj Youness)

Responsibility. Service 3 records incidents, assigns analysts and stores investigation notes. Regular Users report incidents as themselves and read their own reports. Analysts read and update only assigned incidents; administrators manage assignment and the lifecycle, while auditors have read access.

API contract. Creation requires `title`, `description`, `category`, `severity_level`, `reporter_id`, `asset_id` and returns an Open incident with `201`. Assignment accepts `analyst_id`; a status update accepts `status`; a note accepts `note_content`. The lifecycle is Open → Under Investigation → Mitigated → Resolved → Closed. A skipped step returns `409`.

Persistence. `incidents` links the reporter and optional assignee to `users`, and the affected asset to `assets`. `investigation_notes` stores notes separately. Assignment, its notification and audit entries commit together in a `BEGIN IMMEDIATE` transaction. Reaching Resolved sets the resolution date; closure preserves it.

Severity rationale. The caller supplies severity. All four examples concern Web Server 01 (Primary): Low describes one blocked login attempt without access or compromise; Medium describes repeated blocked logins without an established privileged session; High describes multiple suspicious administrator logins; Critical describes an active privileged compromise requiring immediate containment. The increasing stated access and impact explain the submitted ratings. These descriptions are test scenarios, not independently confirmed security events or a formal risk assessment.

Implemented APIs.

Method	Endpoint (prefix `/Youssef-Al-Hajj-Youness`)	Access
POST	`/incidents`	Administrator, Regular User (as themselves)
GET	`/incidents`	Administrator, Auditor
GET	`/incidents/<id>`	all roles, scoped: users see their own reports, analysts their assigned incidents
PUT	`/incidents/<id>/assign`	Administrator
PUT	`/incidents/<id>/status`	Administrator, Analyst
POST	`/incidents/<id>/notes`	Administrator, Analyst

Figures 37–41: Creation and list responses show the submitted severity examples. Their ratings describe the sample scenarios, rather than a measured detection result.

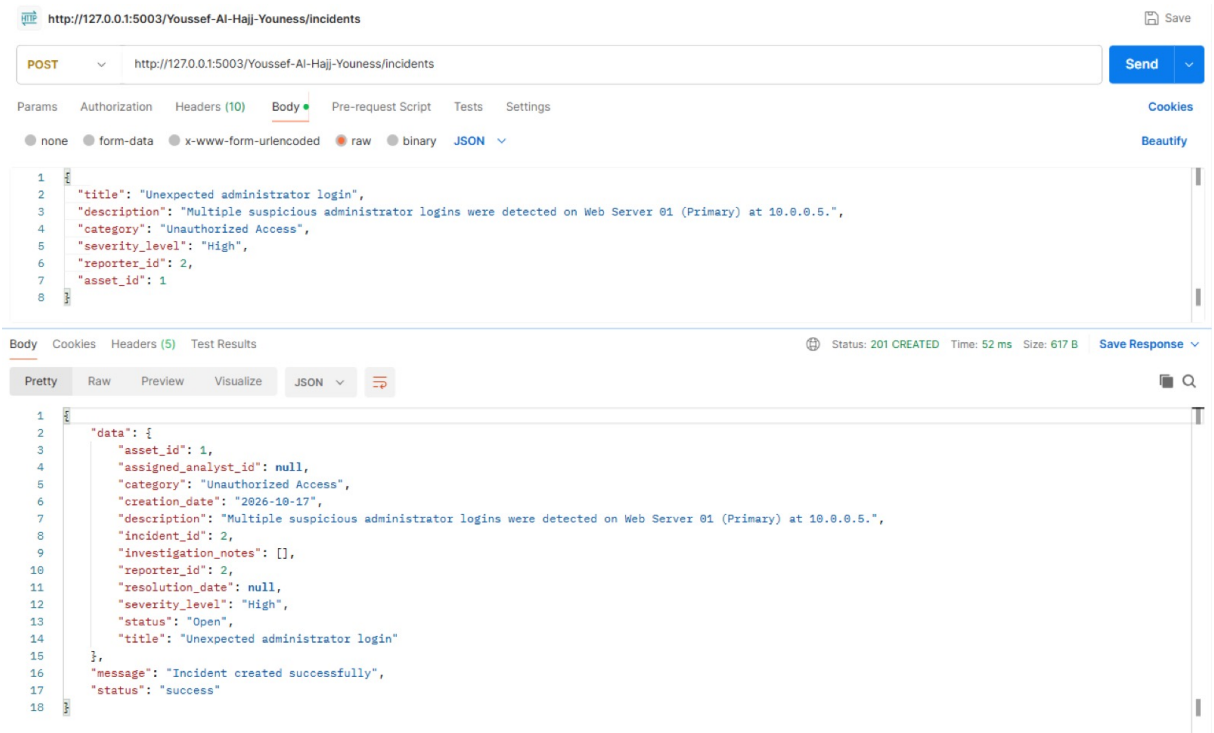


Figure 37: Youssef Al Hajj Youness: Create Incident (POST /Youssef-Al-Hajj-Youness/incidents): High severity incident, 201

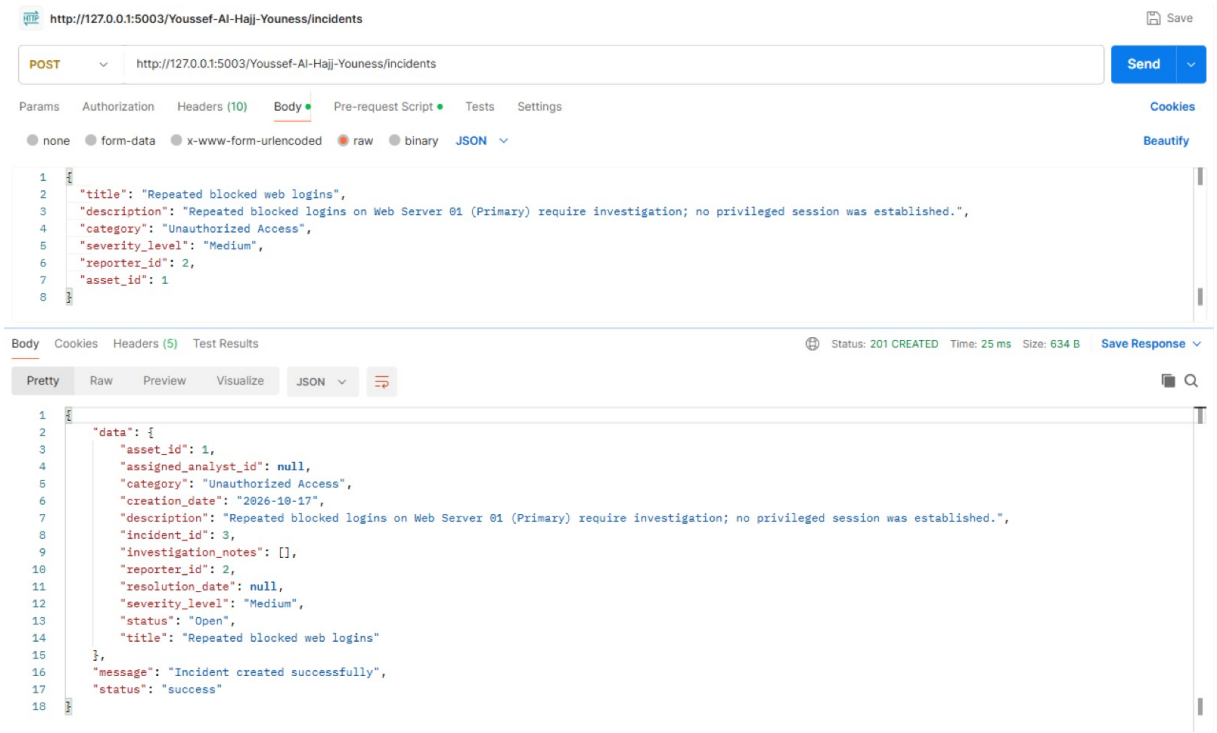


Figure 38: Youssef Al Hajj Youness: Create Incident: Medium severity incident, 201

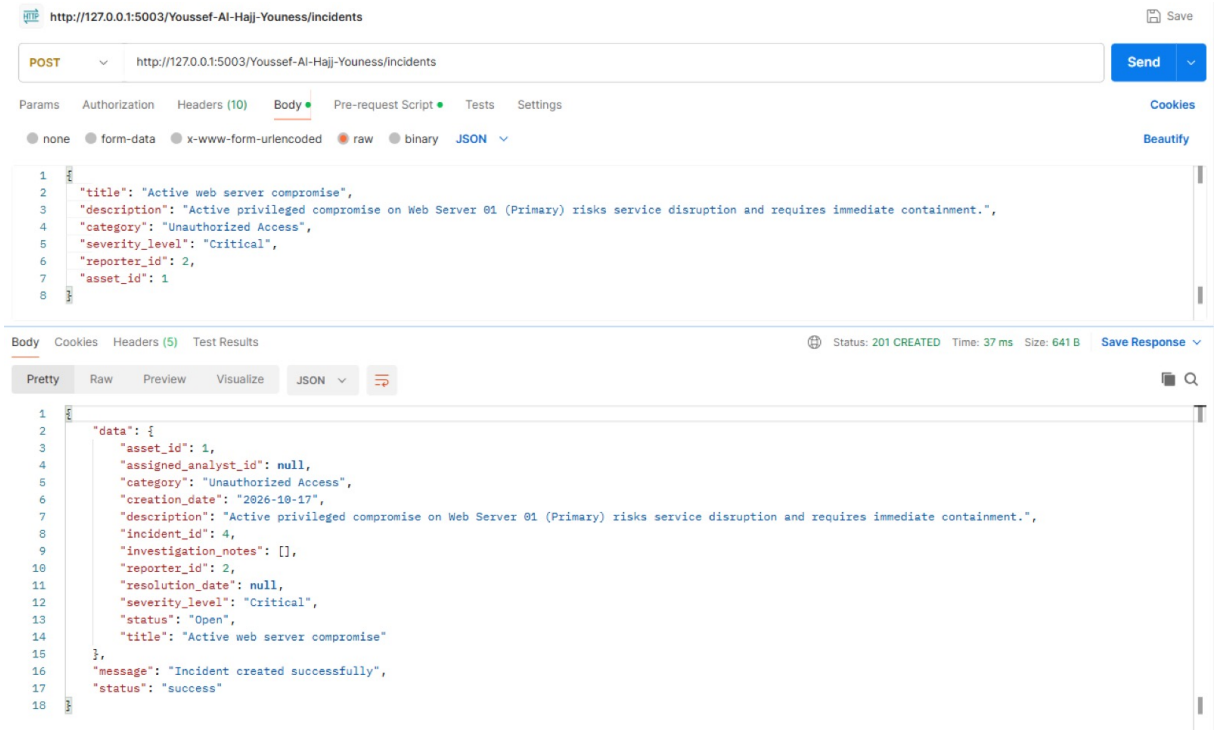


Figure 39: Youssef Al Hajj Youness: Create Incident: Critical severity incident, 201

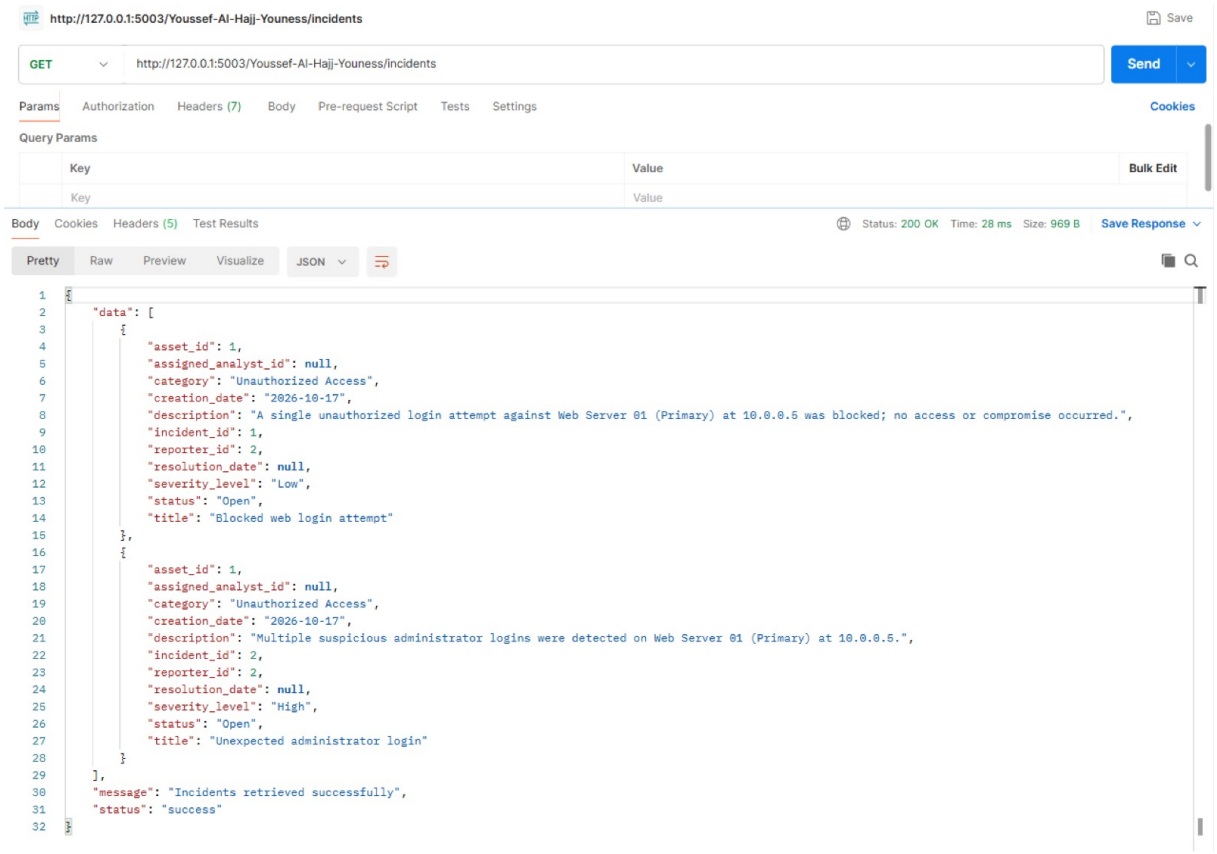


Figure 40: Youssef Al Hajj Youness: Retrieve All Incidents: several incidents, 200

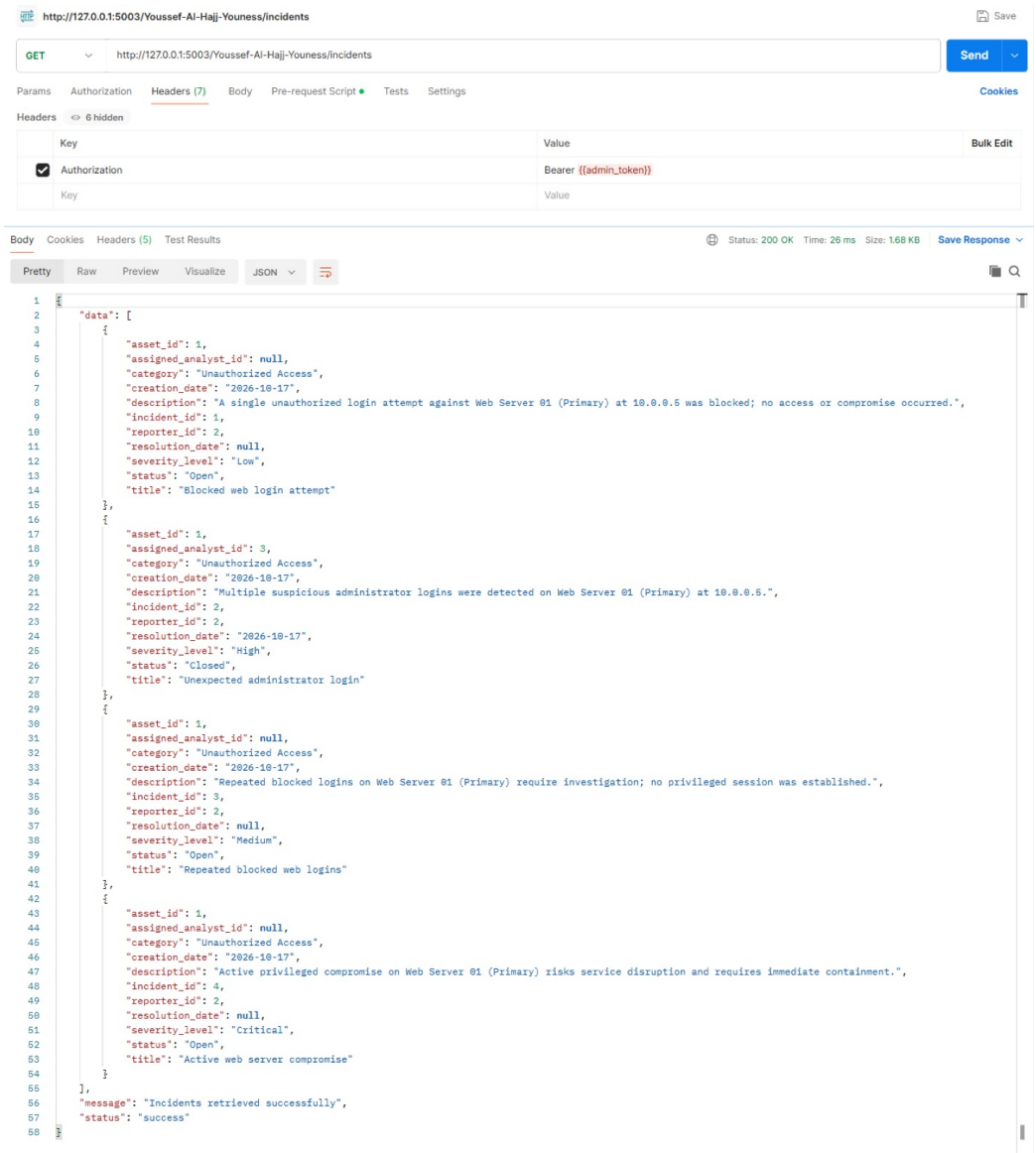


Figure 41: Youssef Al Hajj Youness: Retrieve All Incidents: incidents of four severities, 200

Figures 42–44: The assigned analyst can retrieve an incident, an absent record returns 404 for the permitted caller, and the administrator assigns the analyst.

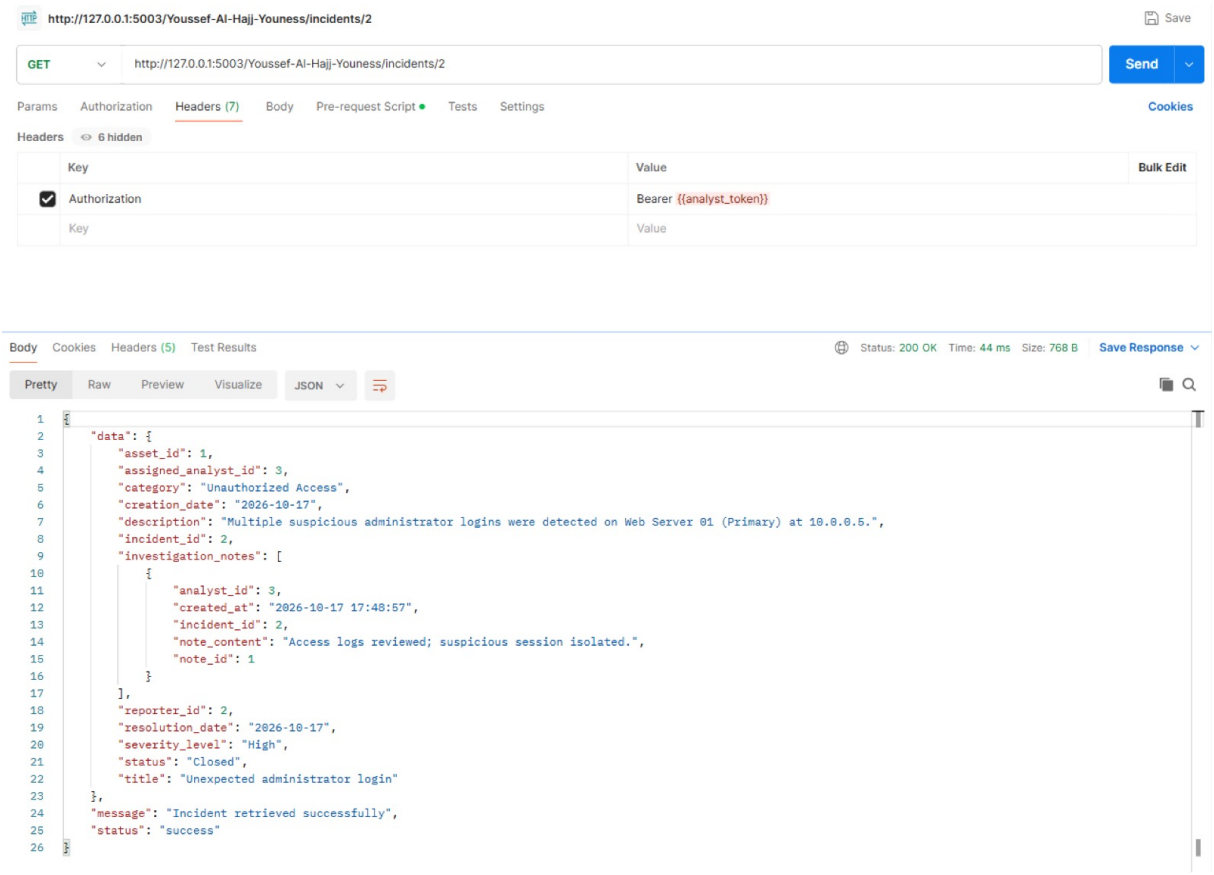


Figure 42: Youssef Al Hajj Youness: Retrieve Specific Incident: assigned analyst, 200

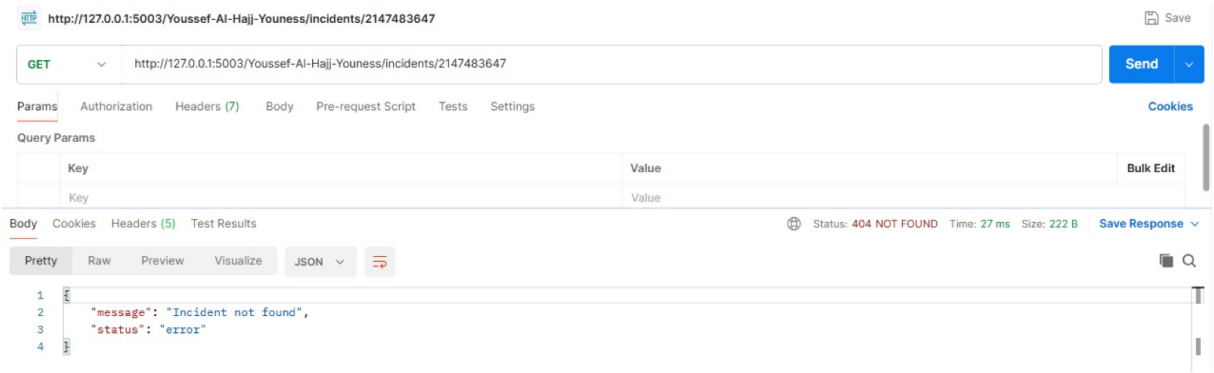


Figure 43: Youssef Al Hajj Youness: Retrieve Specific Incident: invalid incident, 404

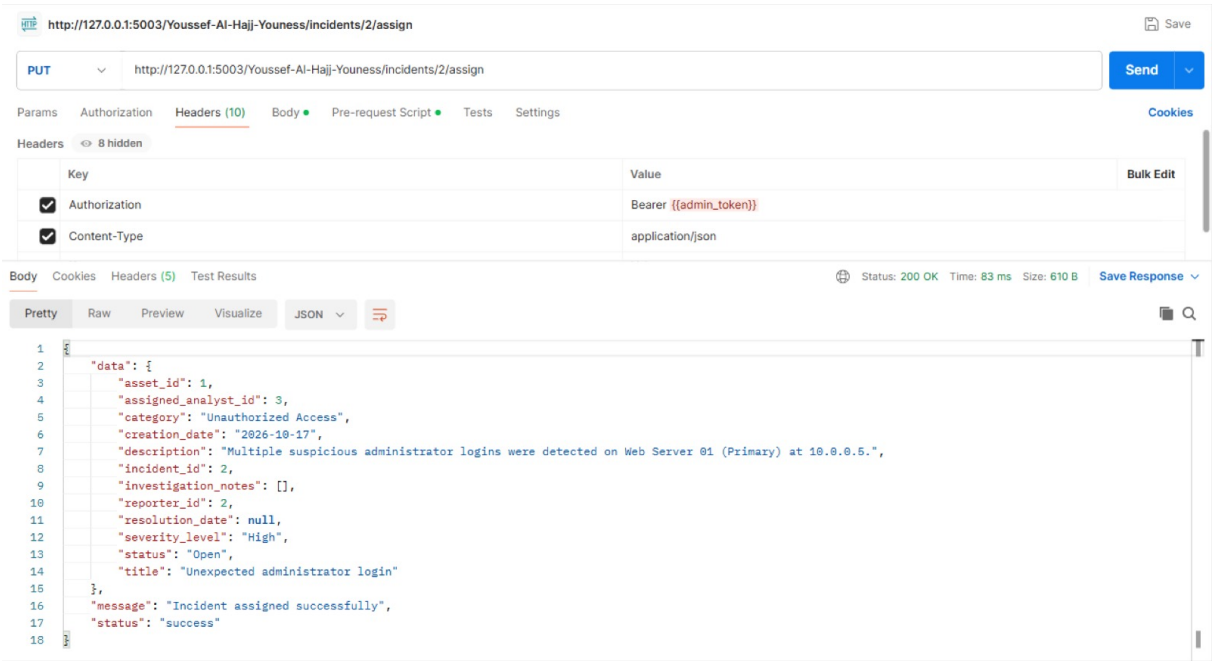


Figure 44: Youssef Al Hajj Youness: Assign Incident (PUT .../incidents/:id/assign): administrator assigns an analyst, 200

Figures 45–50: The sequence follows every permitted lifecycle step, inserts an investigation note and retrieves the Closed record with its note. The resolution date is preserved on closure.

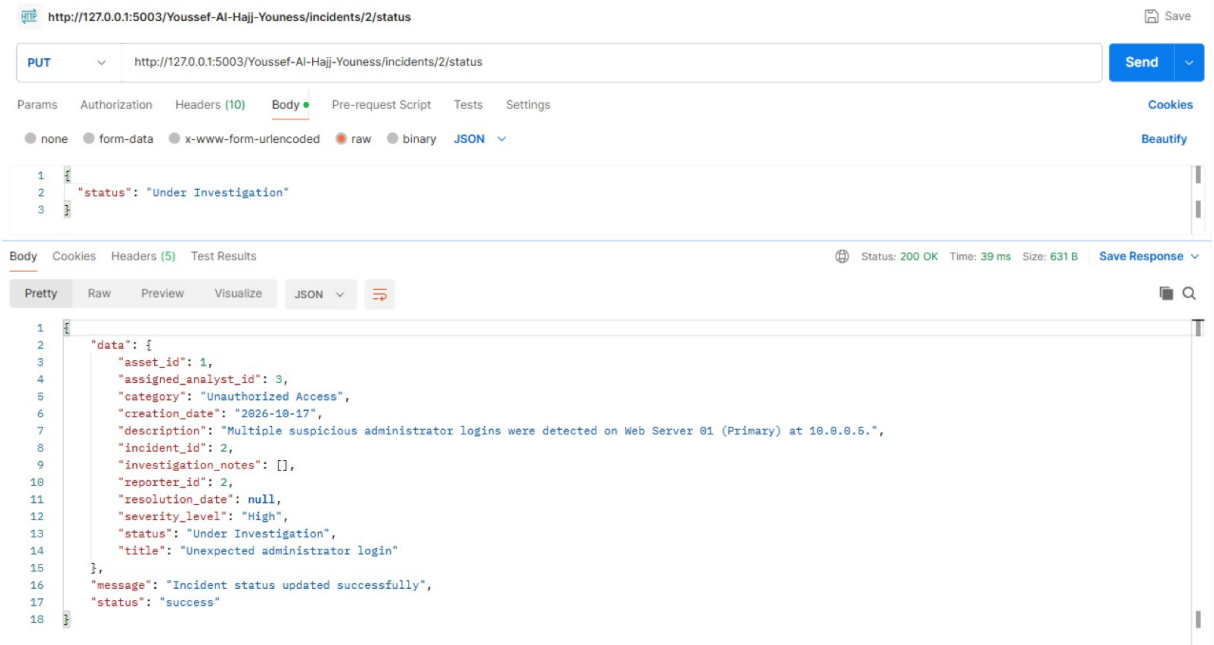


Figure 45: Youssef Al Hajj Youness: Update Incident Status: Open to Under Investigation, 200

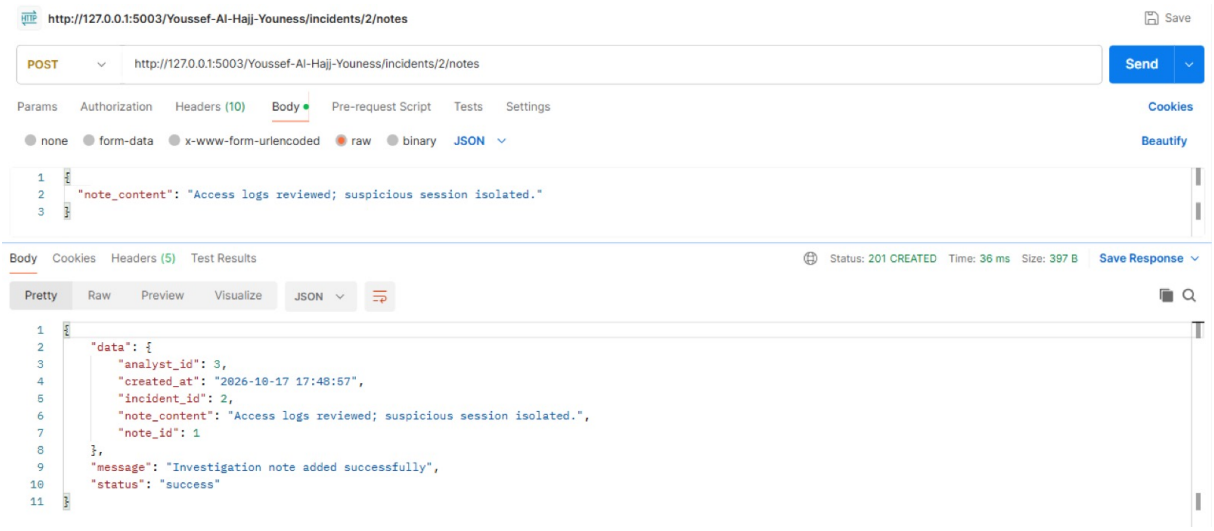


Figure 46: Youssef Al Hajj Youness: Add Investigation Notes (POST .../incidents/:id/notes): 201

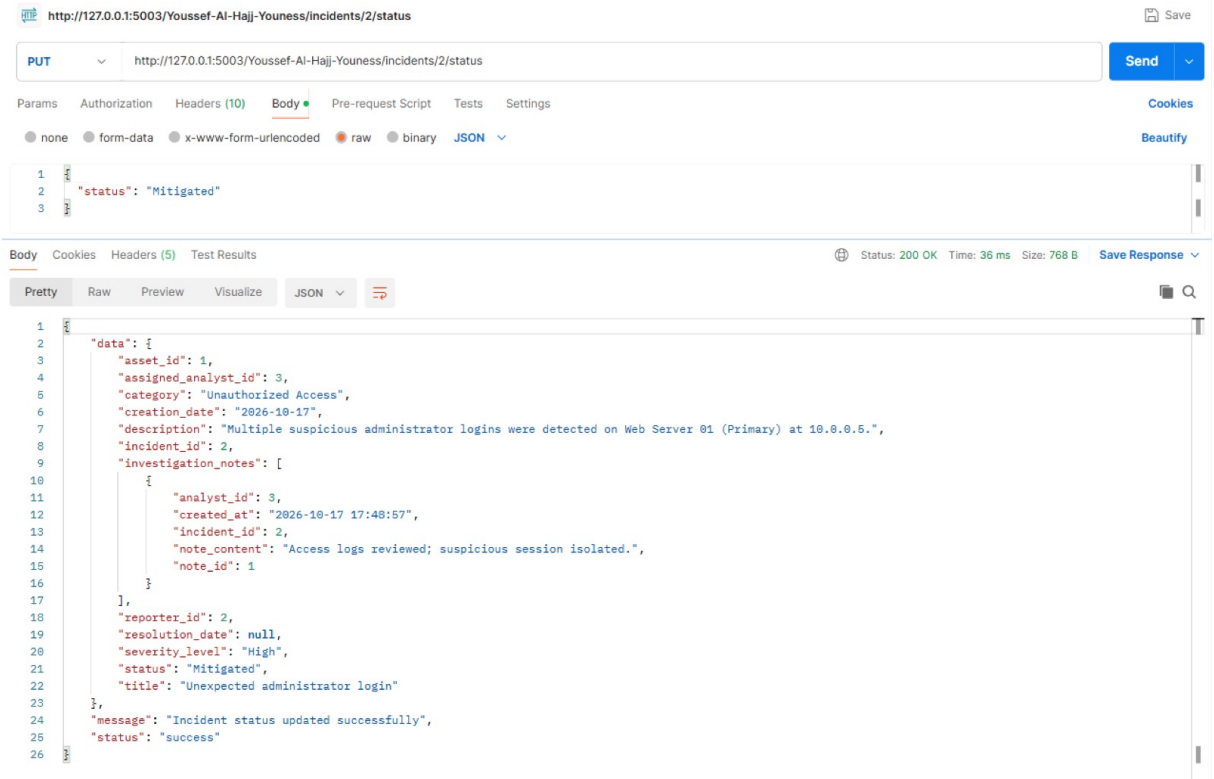


Figure 47: Youssef Al Hajj Youness: Update Incident Status: Under Investigation to Mitigated, 200

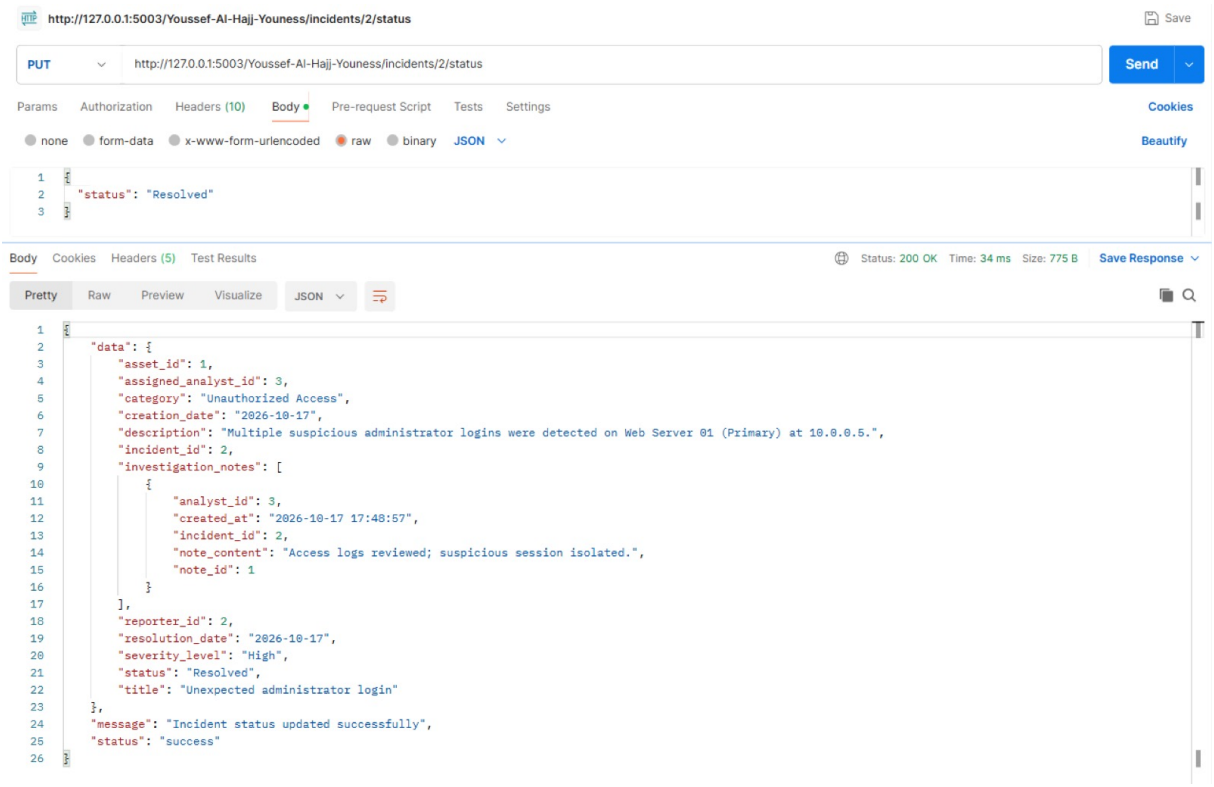


Figure 48: Youssef Al Hajj Youness: Update Incident Status: Mitigated to Resolved, 200

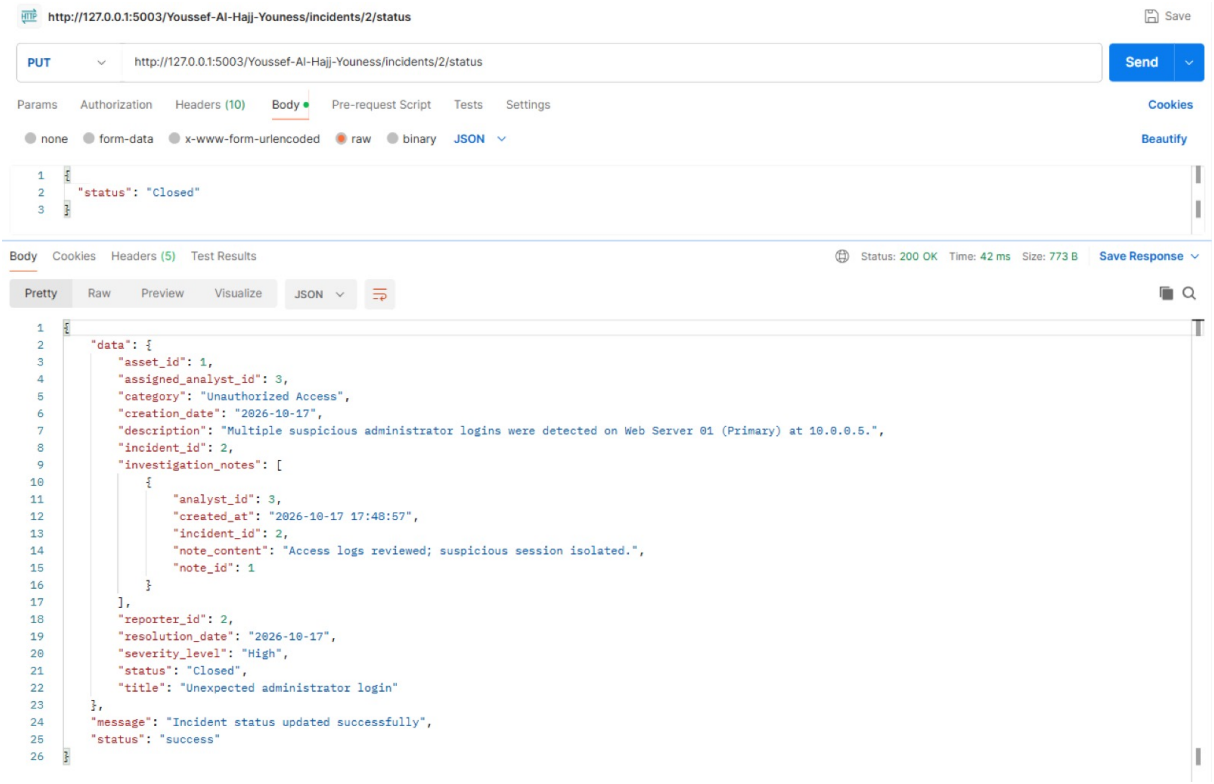


Figure 49: Youssef Al Hajj Youness: Update Incident Status: Resolved to Closed, 200

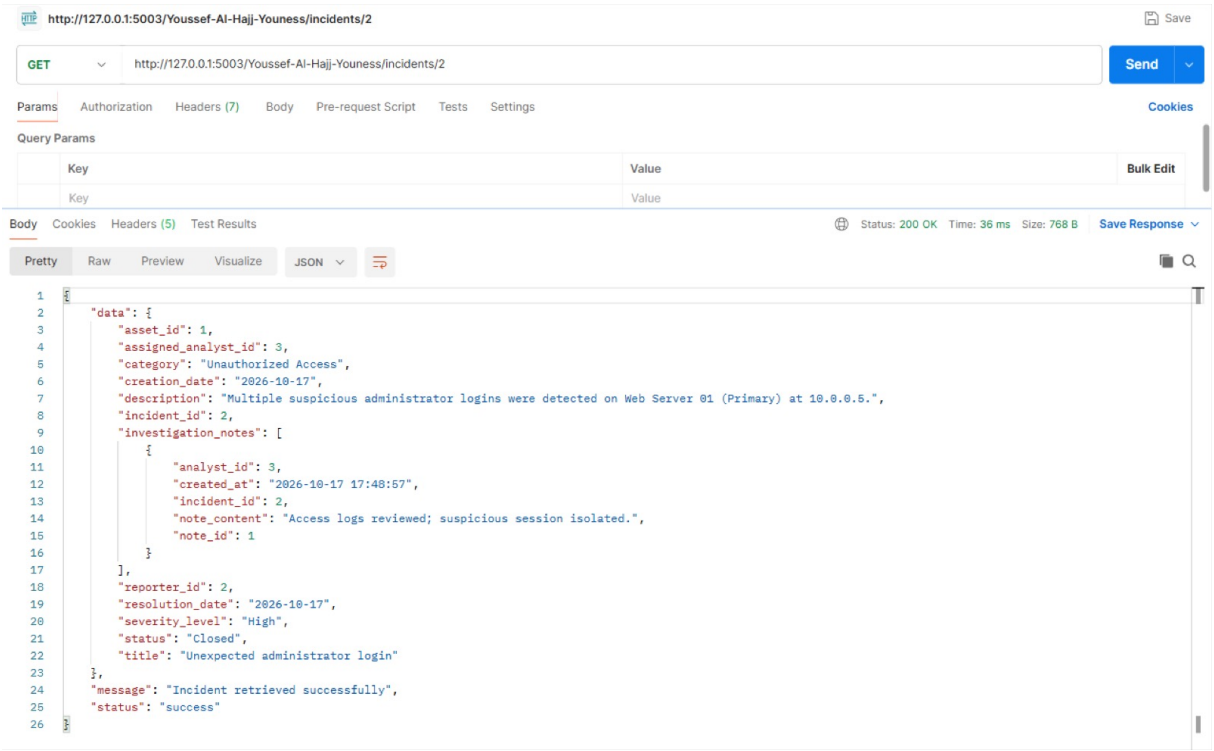


Figure 50: Youssef Al Hajj Youness: Retrieve Specific Incident: final state with its investigation notes, 200

Figures 51–52: Database captures connect incident rows, investigation notes, table structure and audit events to the API workflow.

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
QUERY FILE: 27_incidents_notes_audit.sql

SELECT incident_id AS id, severity_level AS severity, status,
       reporter_id, asset_id, assigned_analyst_id AS analyst_id,
       creation_date, resolution_date
FROM incidents ORDER BY incident_id;
id  severity  status  reporter_id  asset_id  analyst_id  creation_date  resolution_date
--  --------  ------  -----------  --------  ----------  -------------  ---------------
1   Low       Open    2            1         NULL        2026-10-17     NULL
2   High      Closed  2            1         3           2026-10-17     2026-10-17
(2 rows)

SELECT note_id, incident_id, analyst_id, note_content, created_at
FROM investigation_notes ORDER BY note_id;
note_id  incident_id  analyst_id  note_content                                    created_at
-------  -----------  ----------  ----------------------------------------------  -------------------
1        2            3           Access logs reviewed; suspicious session isolated.  2026-10-17 17:48:57
(1 rows)

SELECT log_id, user_id, action, description, timestamp
FROM logs
WHERE related_service = 'incidents' AND user_id IS NOT NULL
  AND action IN ('Incident assigned', 'Unauthorized access attempt',
                 'Investigation note added', 'Incident status updated')
ORDER BY log_id;
log_id  user_id  action                      description                                                                                          timestamp
------  -------  --------------------------  ---------------------------------------------------------------------------------------------------  -------------------
19      2        Unauthorized access attempt  PUT /Youssef-Al-Hajj-Youness/incidents/2/assign: User does not have permission to perform this operation  2026-10-17 17:47:42
20      1        Incident assigned           Incident 2: analyst None -> 3                                                                        2026-10-17 17:47:53
22      3        Incident status updated     Incident 2: Open -> Under Investigation                                                              2026-10-17 17:48:46
23      3        Investigation note added    Note 1 added to incident 2                                                                           2026-10-17 17:48:57
24      3        Incident status updated     Incident 2: Under Investigation -> Mitigated                                                         2026-10-17 17:49:08
25      3        Incident status updated     Incident 2: Mitigated -> Resolved                                                                    2026-10-17 17:49:20
26      3        Incident status updated     Incident 2: Resolved -> Closed                                                                       2026-10-17 17:49:31
(7 rows)

SELECT 'incidents' AS child, "from" AS column_name, "table" AS parent, "to" AS parent_key
FROM pragma_foreign_key_list('incidents')
UNION ALL
SELECT 'investigation_notes', "from", "table", "to"
FROM pragma_foreign_key_list('investigation_notes');
child               column_name         parent     parent_key
------------------  ------------------  ---------  ----------
incidents           assigned_analyst_id  users      user_id
incidents           asset_id            assets     asset_id
incidents           reporter_id         users      user_id
investigation_notes  analyst_id          users      user_id
investigation_notes  incident_id         incidents  incident_id
(5 rows)
```

Figure 51: Youssef Al Hajj Youness: Database: incidents, investigation notes and audit log rows

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db

SELECT sql FROM sqlite_master WHERE type='table' AND name IN ('incidents','investigation_notes');
CREATE TABLE incidents (
    incident_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    title               TEXT NOT NULL,
    description         TEXT NOT NULL,
    category            TEXT NOT NULL,
    severity_level      TEXT NOT NULL
                        CHECK (severity_level IN ('Low', 'Medium', 'High', 'Critical')),
    reporter_id         INTEGER NOT NULL REFERENCES users(user_id),
    asset_id            INTEGER NOT NULL REFERENCES assets(asset_id),
    assigned_analyst_id INTEGER REFERENCES users(user_id),
    status              TEXT NOT NULL DEFAULT 'Open'
                        CHECK (status IN ('Open', 'Under Investigation', 'Mitigated',
                                          'Resolved', 'Closed')),
    creation_date       TEXT NOT NULL DEFAULT (date('now')),
    resolution_date     TEXT
);
CREATE TABLE investigation_notes (
    note_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    incident_id   INTEGER NOT NULL REFERENCES incidents(incident_id) ON DELETE CASCADE,
    analyst_id    INTEGER NOT NULL REFERENCES users(user_id),
    note_content  TEXT NOT NULL,
    created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

SELECT l.log_id, u.role, l.description, l.timestamp
FROM logs l LEFT JOIN users u USING(user_id)
WHERE l.related_service='incidents' AND l.action='Unauthorized access attempt'
ORDER BY l.log_id DESC LIMIT 4;
log_id  role             description                                                                                          timestamp
------  ---------------  ---------------------------------------------------------------------------------------------------  -------------------
53      Auditor          POST /Youssef-Al-Hajj-Youness/incidents/2/notes: User does not have permission to perform this operation  2026-10-17 18:28:12
52      Regular User     PUT /Youssef-Al-Hajj-Youness/incidents/2/status: User does not have permission to perform this operation  2026-10-17 18:27:50
51      Regular User     GET /Youssef-Al-Hajj-Youness/incidents/2: You can only view your own submitted incidents             2026-10-17 18:26:08
50      Security Analyst GET /Youssef-Al-Hajj-Youness/incidents/3: You can only access incidents assigned to you             2026-10-17 18:25:48
(4 rows)
```

Figure 52: Youssef Al Hajj Youness: Database: incidents table structure and audit entries

5.4 Service 4: Threat Intelligence (Leen Abdallah)

Responsibility. Service 4 stores threat indicators for use during investigations. Administrators create, update and delete records. Analysts and auditors can retrieve indicators; search is implemented for administrators and analysts. The assignment's general Auditor description allows threat search, but its Service 4 permission list omits it. The implemented restriction and this specification ambiguity remain explicit.

Figures 67, 68, 176, 195, 225 and 227 come from an earlier test dataset, so their record IDs differ from indicators 1–5.

API contract. Required: `threat_type`, `threat_value`, `description`, `severity_level`, `source`. POST returns the new record with `201`; reads, search and update return `200`. A record includes these fields, `threat_id` and `date_added`; list/search responses put records in the shared envelope's `data` array. Search requires at least one filter. Type and severity match their canonical categories exactly, value and source use substring matching, and combined filters use AND. Update accepts any nonempty subset of description, severity and source; type and indicator value remain fixed. DELETE returns `200` with `data: null`.

Persistence and interpretation. The shared SQLite `threats` table stores the seven record fields, generates the ID/date and has a unique type/value pair. Repository INSERT, UPDATE and DELETE statements commit through the shared database helper. The application performs case-insensitive duplicate checks and type-specific IP, domain and hash validation; the SQL UNIQUE constraint alone uses exact stored values. Hash checks accept 32-, 40- or 64-character hexadecimal values, not evidence of maliciousness. Indicator severity and source are submitted metadata: naming VirusTotal or another provider does not demonstrate a lookup against that provider.

Sample threat rationale. The new High IP example describes a port scanner, while the Critical domain example describes a phishing site. Those descriptions explain the submitted network-investigation and account-risk priorities without measuring actual impact. The file

hash is submitted as Low with the description “Known malware sample”; the capture does not explain why it was rated Low, so no stronger risk conclusion is supported. The Medium malware-signature example names a Trojan, and the Medium Other example records a suspicious user-agent string. These are test records and supplied ratings, not externally verified threat assessments.

Implemented APIs.

Method	Endpoint (prefix `/leen-abdallah`)	Access
POST	`/threats`	Administrator
GET	`/threats`	Administrator, Analyst, Auditor
GET	`/threats/<id>`	Administrator, Analyst, Auditor
GET	`/threats/search`	Administrator, Analyst
PUT	`/threats/<id>`	Administrator
DELETE	`/threats/<id>`	Administrator

Figures 53 and 54 show the table definition and initially empty query. The schema records required fields, allowed types/severities and the unique type/value pair.

```
[leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 database/securi]
ty_incident.db ".schema threats"
CREATE TABLE threats (
    threat_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    threat_type    TEXT NOT NULL
                   CHECK (threat_type IN ('IP Address', 'Domain', 'File Hash',
                                          'Malware Signature', 'Other')),
    threat_value   TEXT NOT NULL,
    description    TEXT NOT NULL,
    severity_level TEXT NOT NULL
                   CHECK (severity_level IN ('Low', 'Medium', 'High', 'Critical')),
    source         TEXT NOT NULL,
    date_added     TEXT NOT NULL DEFAULT (date('now')),
    UNIQUE (threat_type, threat_value)
);
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 53: Leen Abdallah: Database schema for Service 4 threats: primary key, required fields, type/severity checks and indicator uniqueness

```
[leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column]
 database/security_incident.db "select * from threats;"
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 54: Leen Abdallah: Database query of threats before insertion: no stored rows are returned

Figures 55–59: The five creation responses cover IP Address (High), Domain (Critical), File Hash (Low), Malware Signature (Medium) and Other (Medium). They demonstrate accepted records and supplied severities; they do not verify the indicators externally.

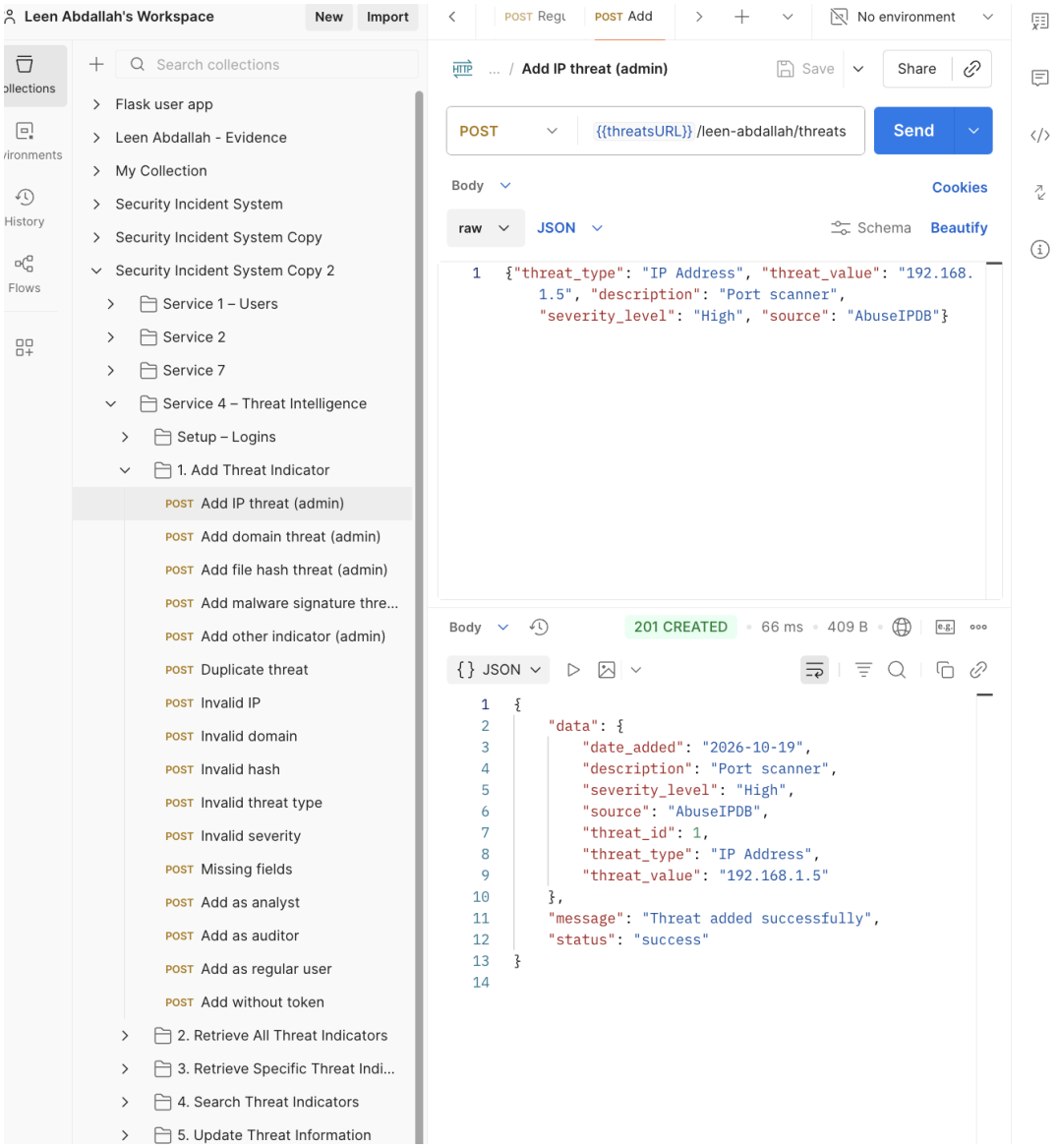


Figure 55: Leen Abdallah: Add Threat Indicator (POST /leen-abdallah/threats): IP Address 192.168.1.5, indicator 1, High, 201

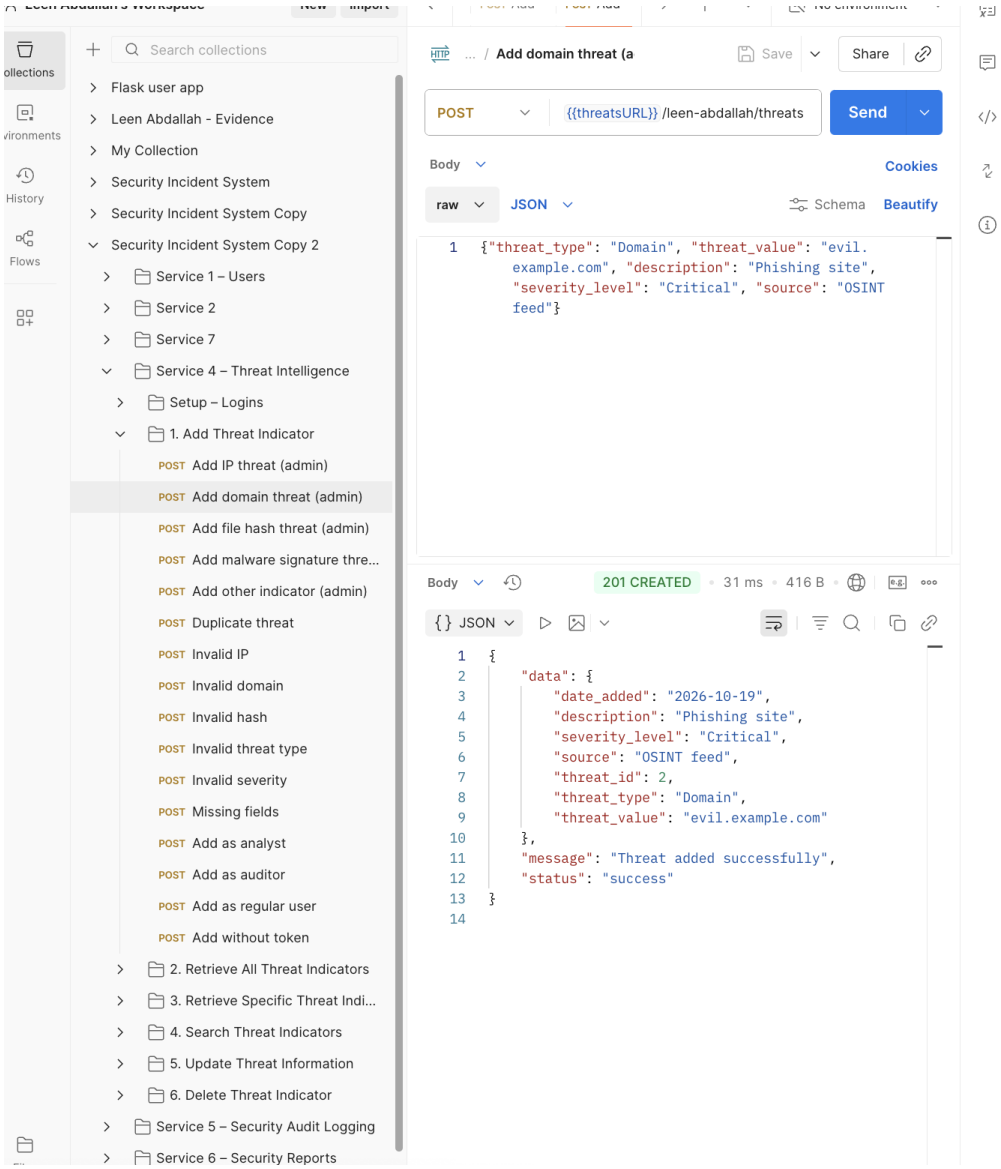


Figure 56: Leen Abdallah: Add Threat Indicator: evil.example.com, indicator 2, Domain, Critical, 201

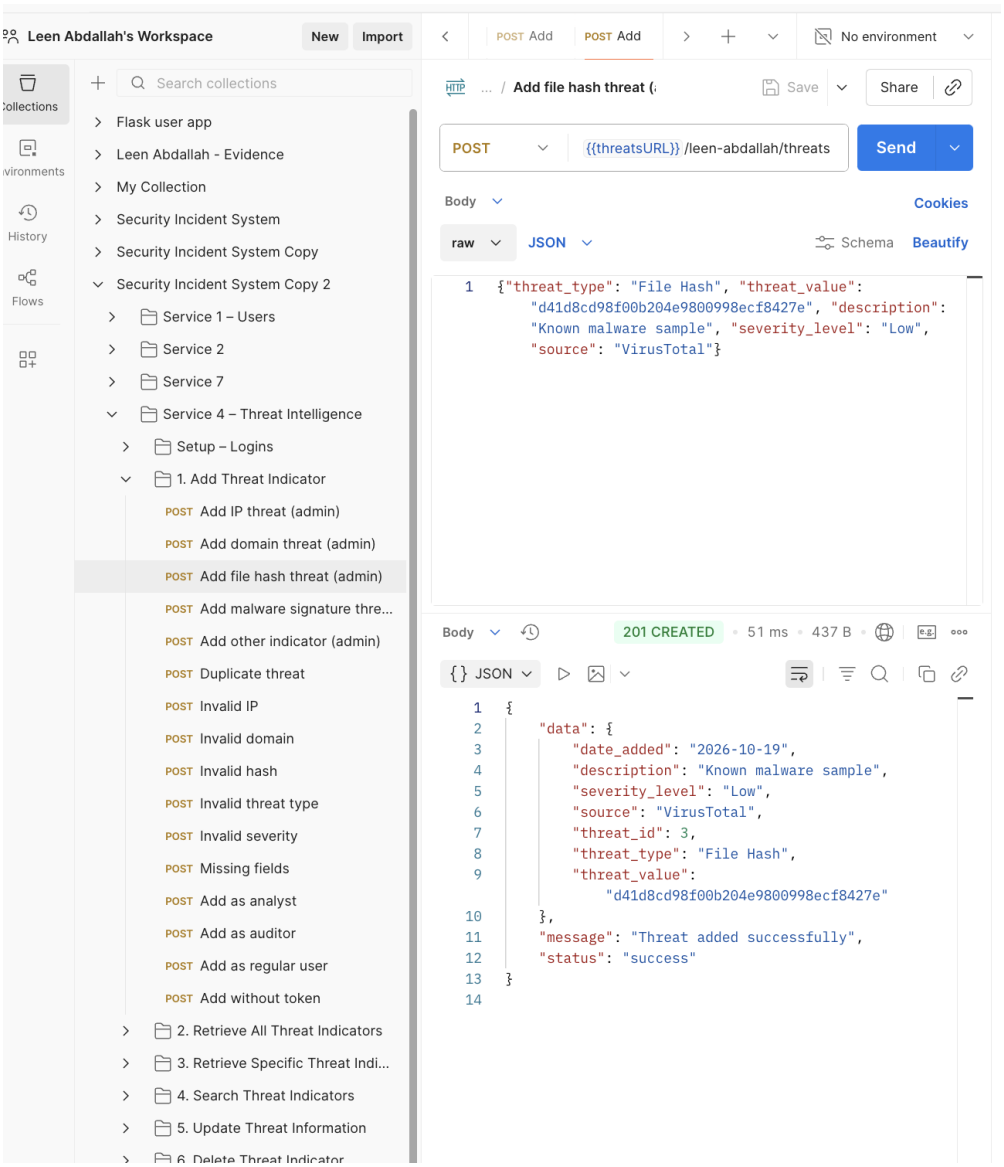


Figure 57: Leen Abdallah: Add Threat Indicator: 32-character File Hash, indicator 3, Low, 201

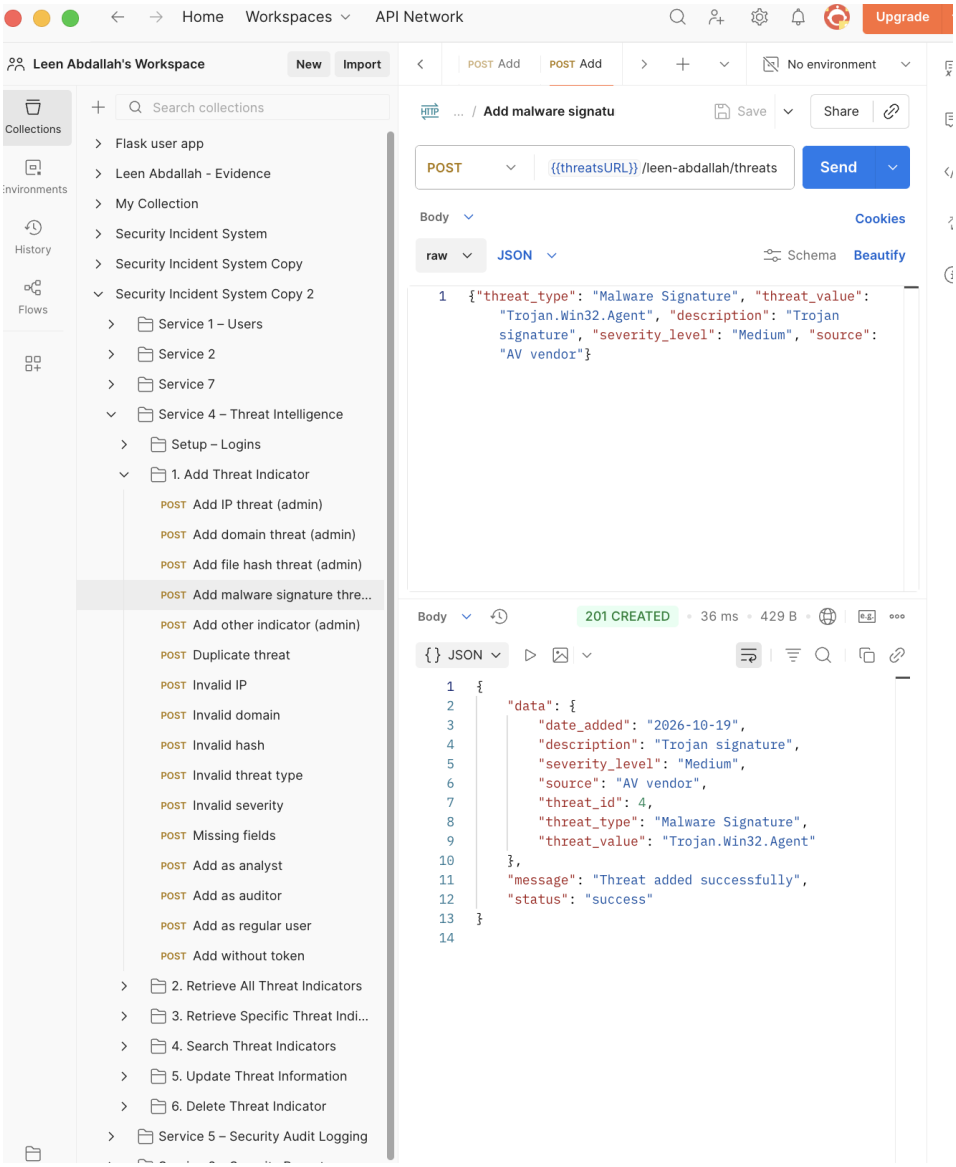


Figure 58: Leen Abdallah: Add Threat Indicator: Trojan.Win32.Agent, indicator 4, Malware Signature, Medium, 201

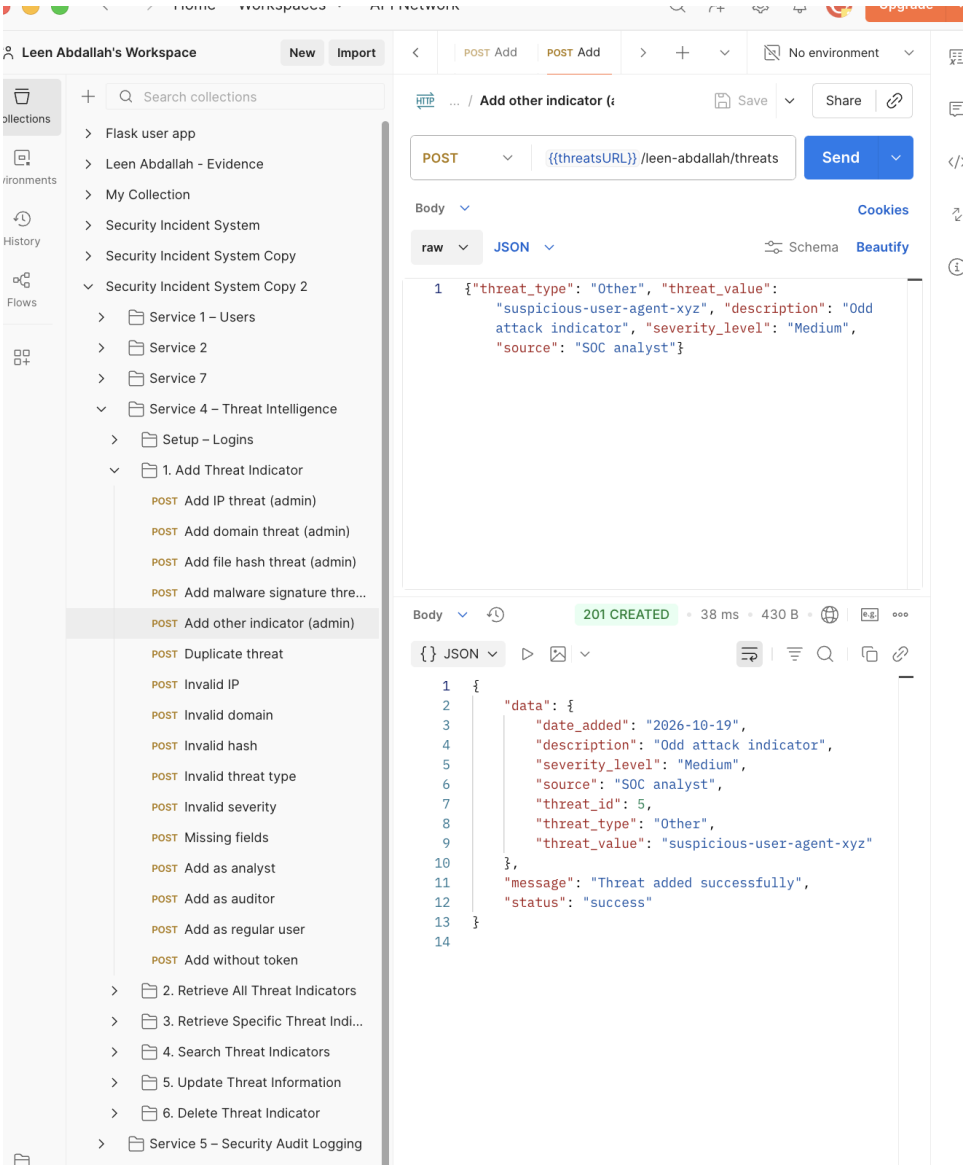


Figure 59: Leen Abdallah: Add Threat Indicator: suspicious-user-agent-xyz, indicator 5, Other, Medium, 201

Figure 60: the SQL query shows all five created indicators with matching IDs, types, values, descriptions, severities, sources and dates. Figure 61 shows five recent "Threat added" entries and authorization-denial entries for Service 4. The projected log columns omit request details, so these rows alone do not identify the denied methods.

```
[leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column
 database/security_incident.db "select * from threats;"
threat_id  threat_type        threat_value                      description           severity_level
 source       date_added
---------  -----------------  --------------------------------  --------------------  --------------
 -----------  ----------
1          IP Address         192.168.1.5                       Port scanner          High
 AbuseIPDB    2026-10-19
2          Domain             evil.example.com                  Phishing site         Critical
 OSINT feed   2026-10-19
3          File Hash          d41d8cd98f00b204e9800998ecf8427e  Known malware sample  Low
 VirusTotal   2026-10-19
4          Malware Signature  Trojan.Win32.Agent                Trojan signature      Medium
 AV vendor    2026-10-19
5          Other              suspicious-user-agent-xyz         Odd attack indicator  Medium
 SOC analyst  2026-10-19
```

Figure 60: Leen Abdallah: Database query after threat insertion: indicators 1–5 and all seven stored fields, dated 19 October 2026

```
 SOC analyst  2026-10-19
[leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column]
 database/security_incident.db "select log_id,user_id,action,severity_level,related_service from logs
 where related_service='threats' order by log_id desc limit 20;"
log_id  user_id  action                        severity_level  related_service
------  -------  ----------------------------  --------------  ---------------
78               Unauthorized access attempt   Warning         threats
77      4        Unauthorized access attempt   Warning         threats
76      3        Unauthorized access attempt   Warning         threats
75      2        Unauthorized access attempt   Warning         threats
74      1        Threat added                  Information     threats
73      1        Threat added                  Information     threats
72      1        Threat added                  Information     threats
71      1        Threat added                  Information     threats
70      1        Threat added                  Information     threats
33      1        Threat deleted                Critical        threats
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 61: Leen Abdallah: Database audit query filtered to threats: five recent insertion entries and authorization-denial entries

Figures 62–64: The administrator list capture shows the beginning of the `data` array, not the complete response. Specific-record retrieval returns indicator 1 with `200`; an absent record returns `404` and “Threat not found”. The complete five-row stored dataset is visible in Figure 60.

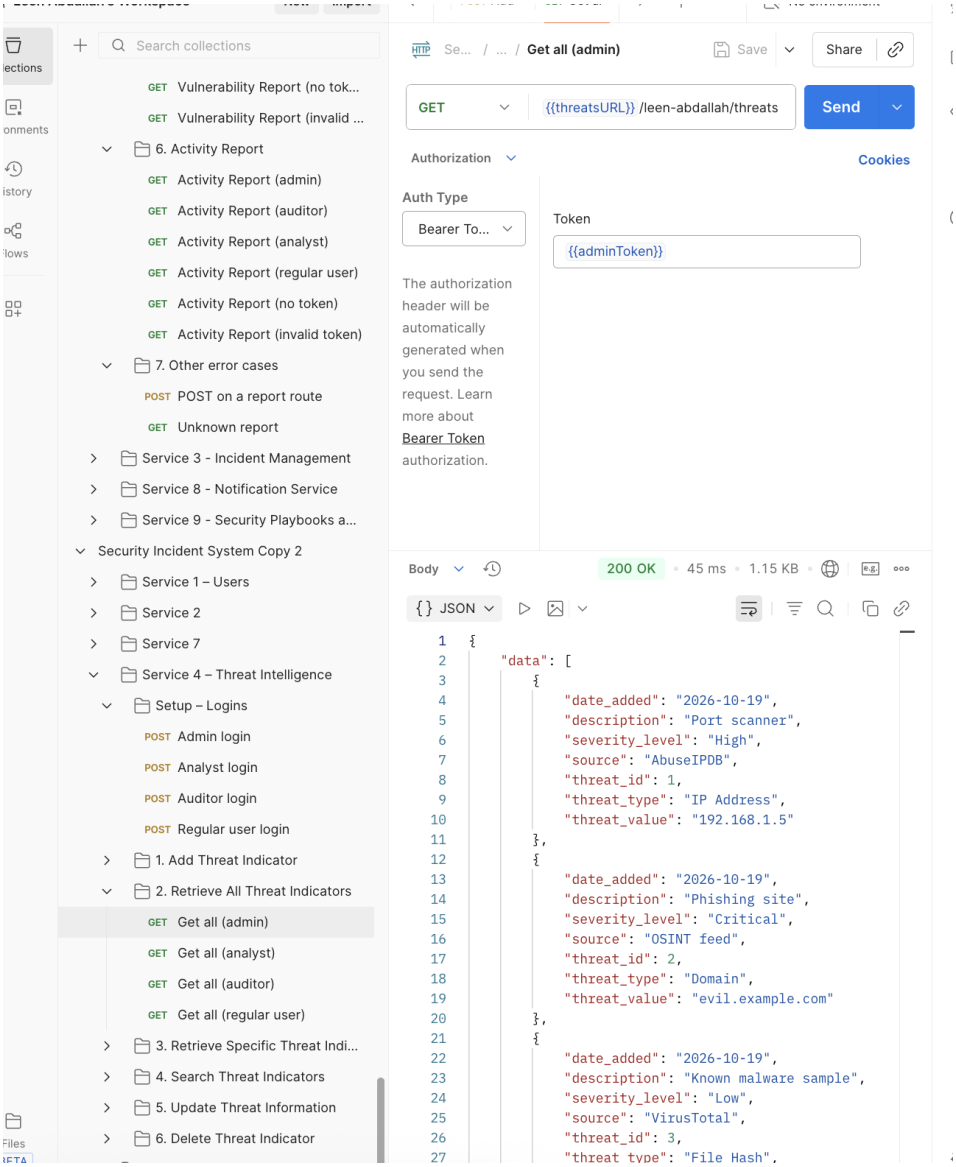


Figure 62: Leen Abdallah: Retrieve All Threat Indicators (GET /leen-abdallah/threats): administrator, 200; visible beginning of the list response

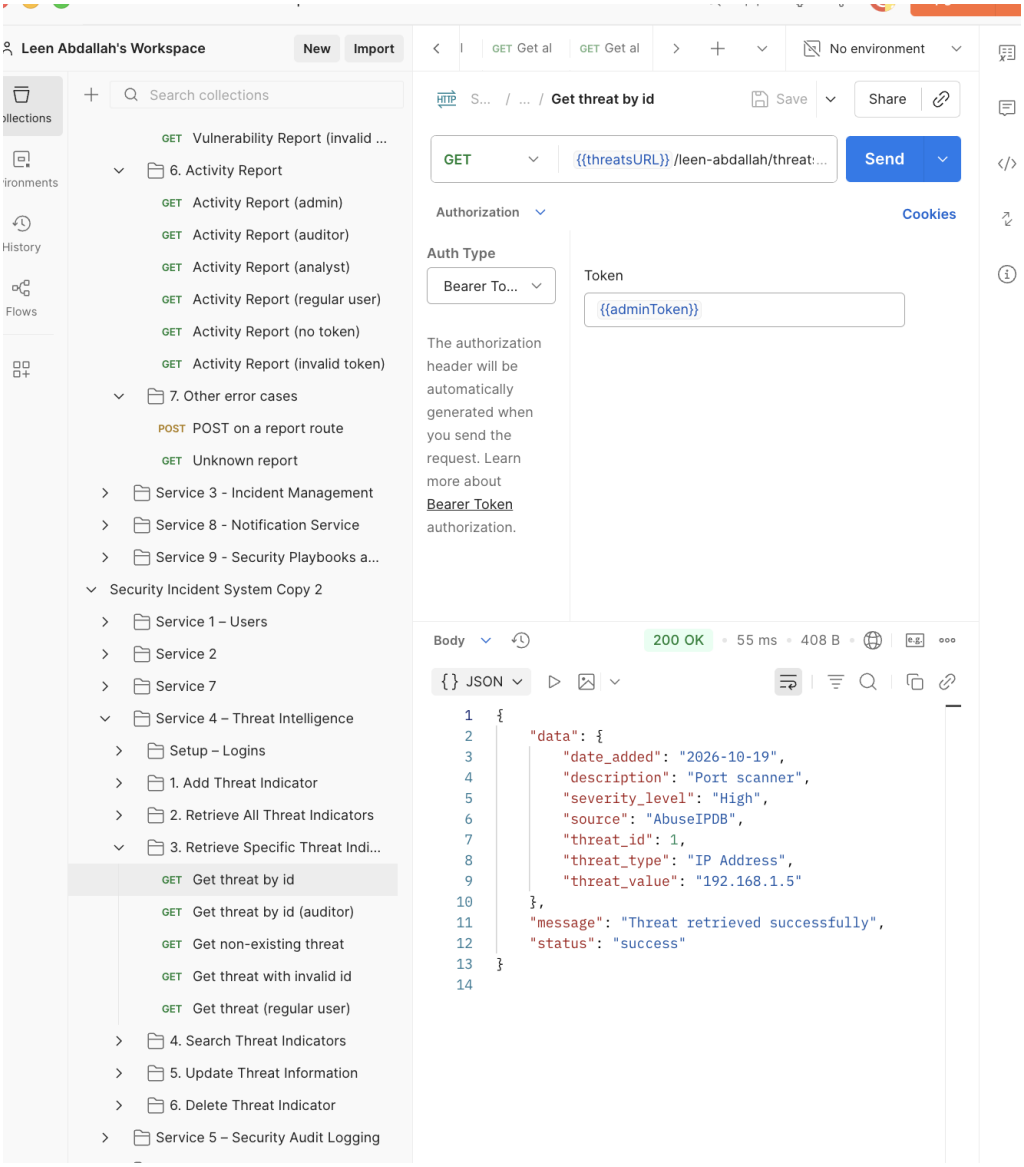


Figure 63: Leen Abdallah: Retrieve Specific Threat Indicator (GET /leen-abdallah/threats/:id): administrator retrieves indicator 1, 200

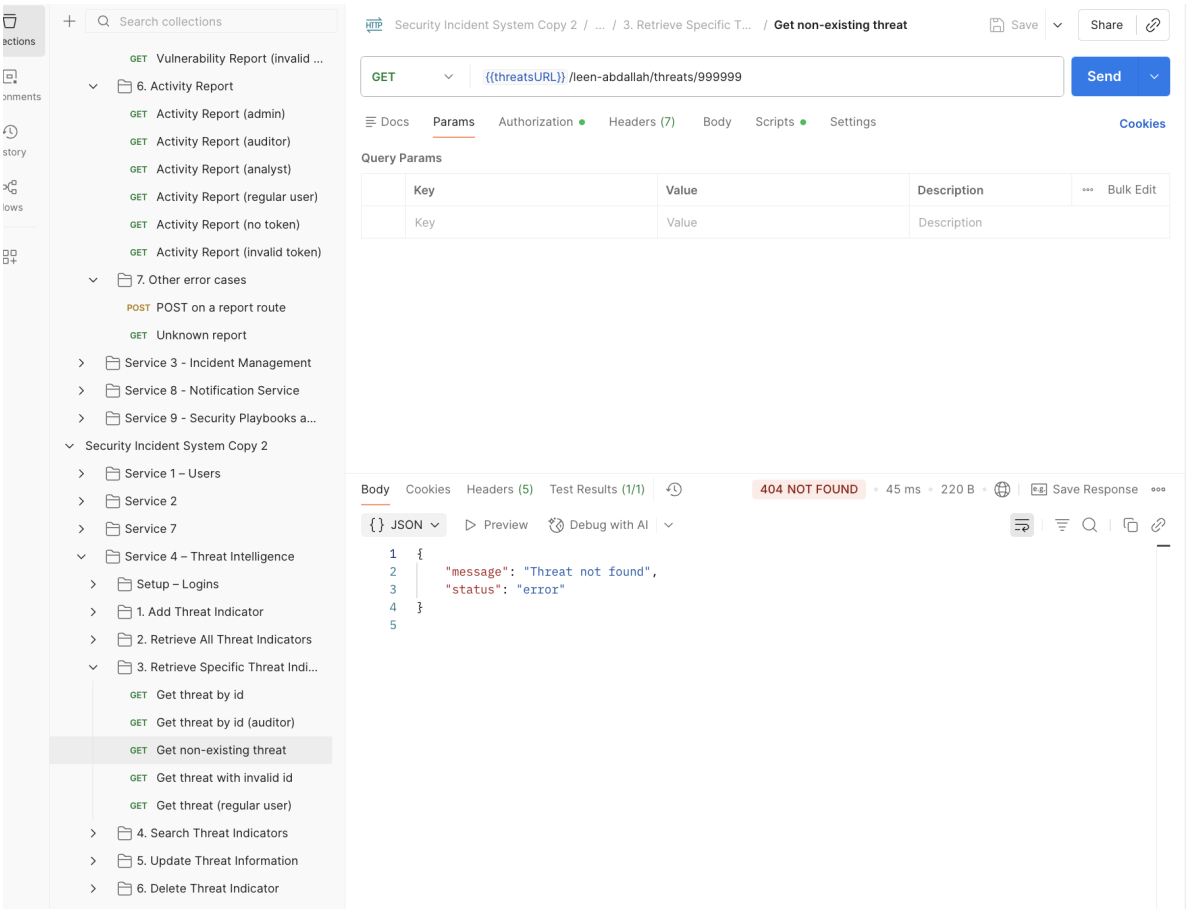


Figure 64: Leen Abdallah: Retrieve Specific Threat Indicator: non-existing indicator 999999, 404

Figures 65–69: New type, value and combined-filter captures use the 19 October dataset. The retained severity-as-analyst and source captures demonstrate those filters on the earlier dataset. Each result is interpreted within its own capture; combined Domain and Critical filters return the phishing-domain record.

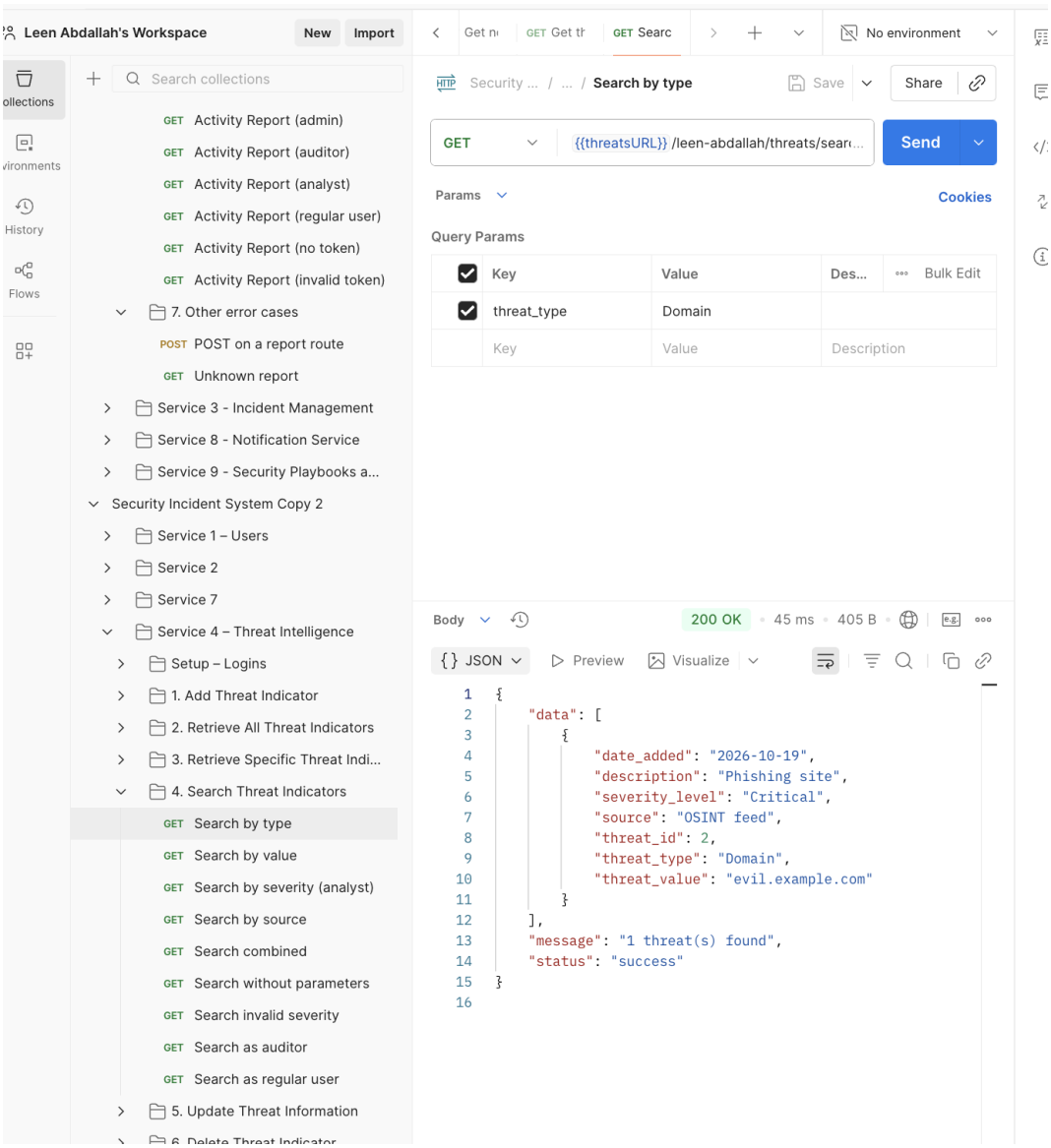


Figure 65: Leen Abdallah: Search Threat Indicators (GET /leen-abdallah/threats/search): Domain type, 200

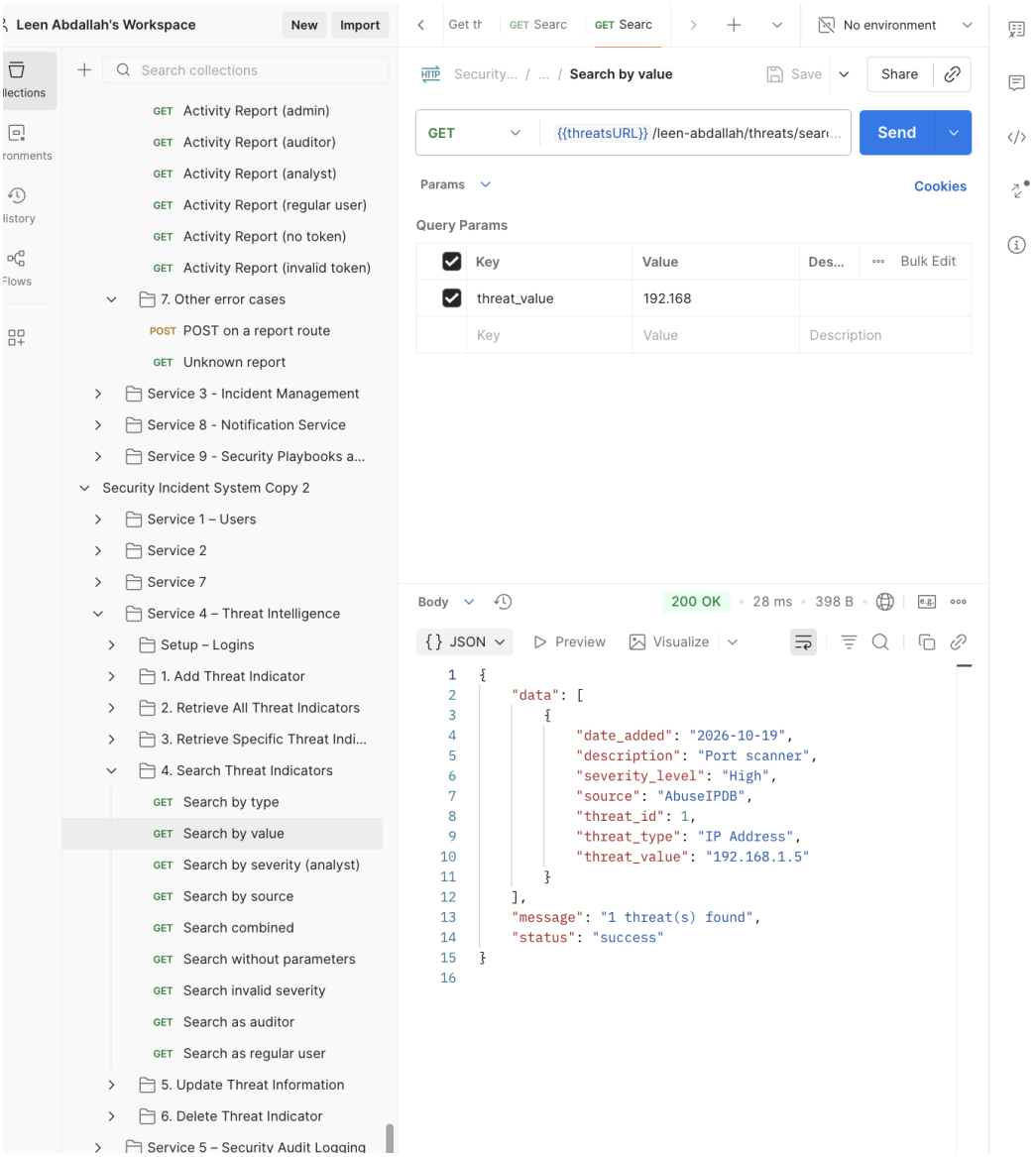


Figure 66: Leen Abdallah: Search Threat Indicators: value substring 192.168 matches indicator 1, 200

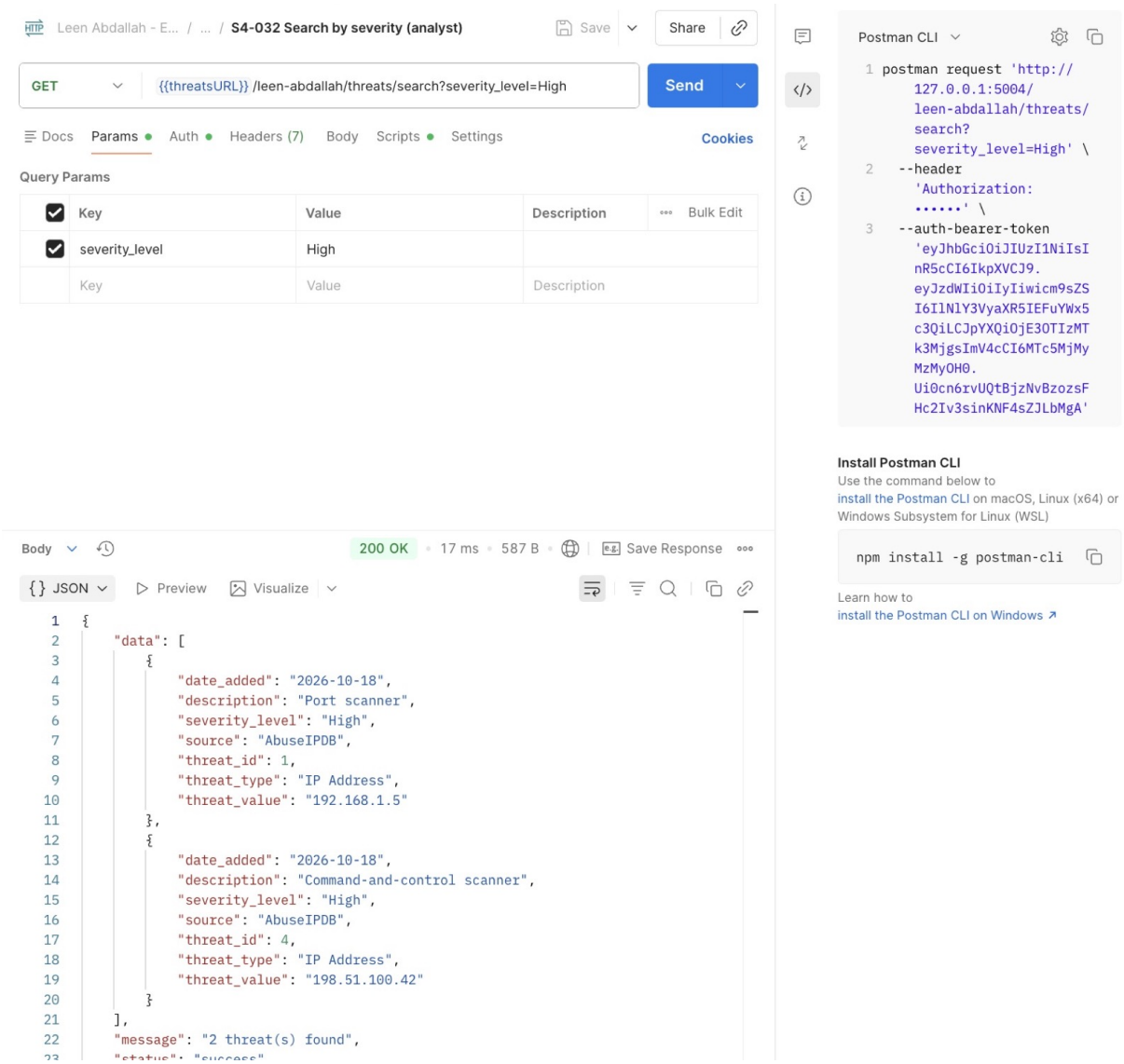


Figure 67: Leen Abdallah: Search Threat Indicators: by severity as analyst on the earlier dataset, 200

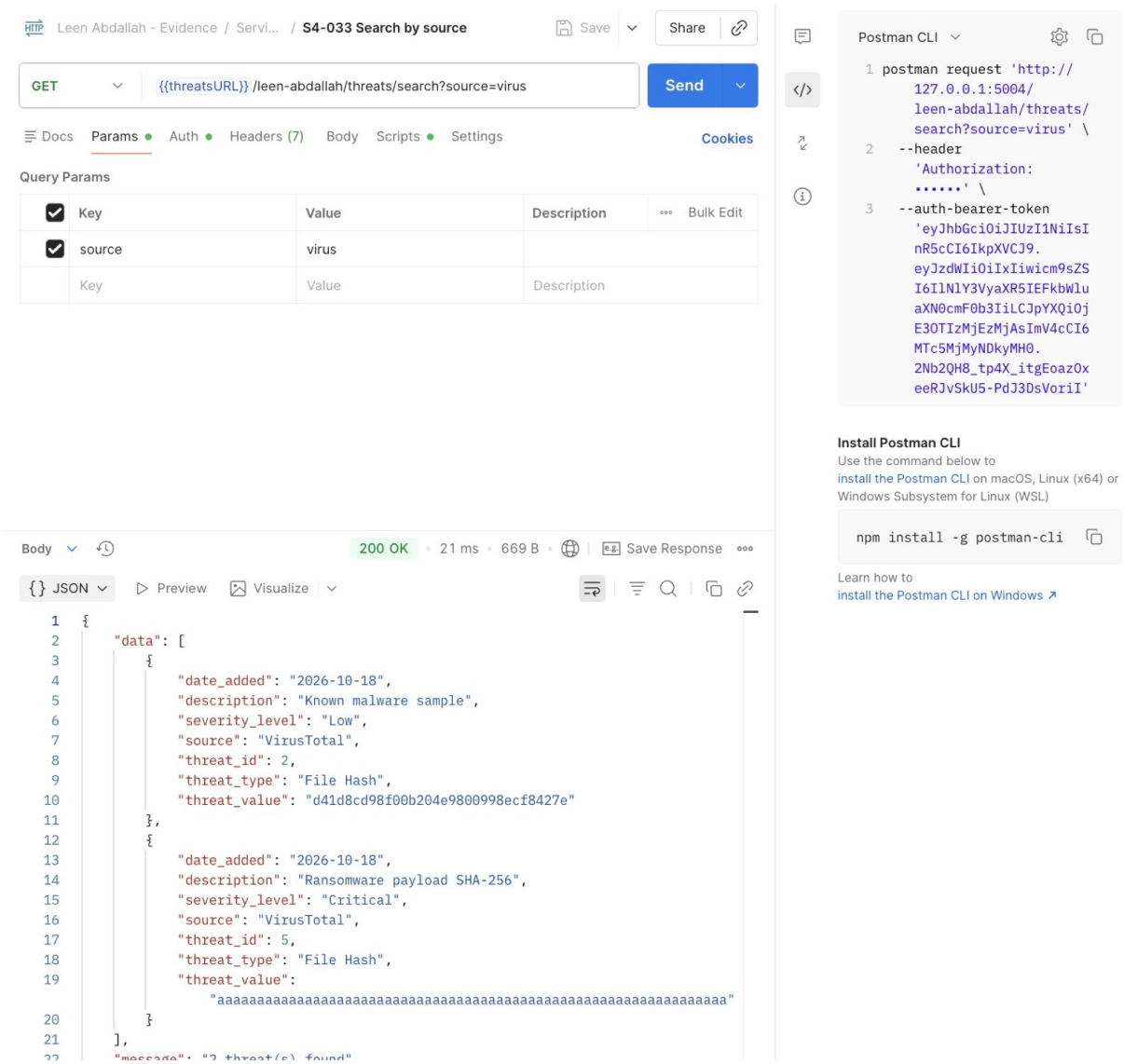


Figure 68: Leen Abdallah: Search Threat Indicators: by source on the earlier dataset, 200

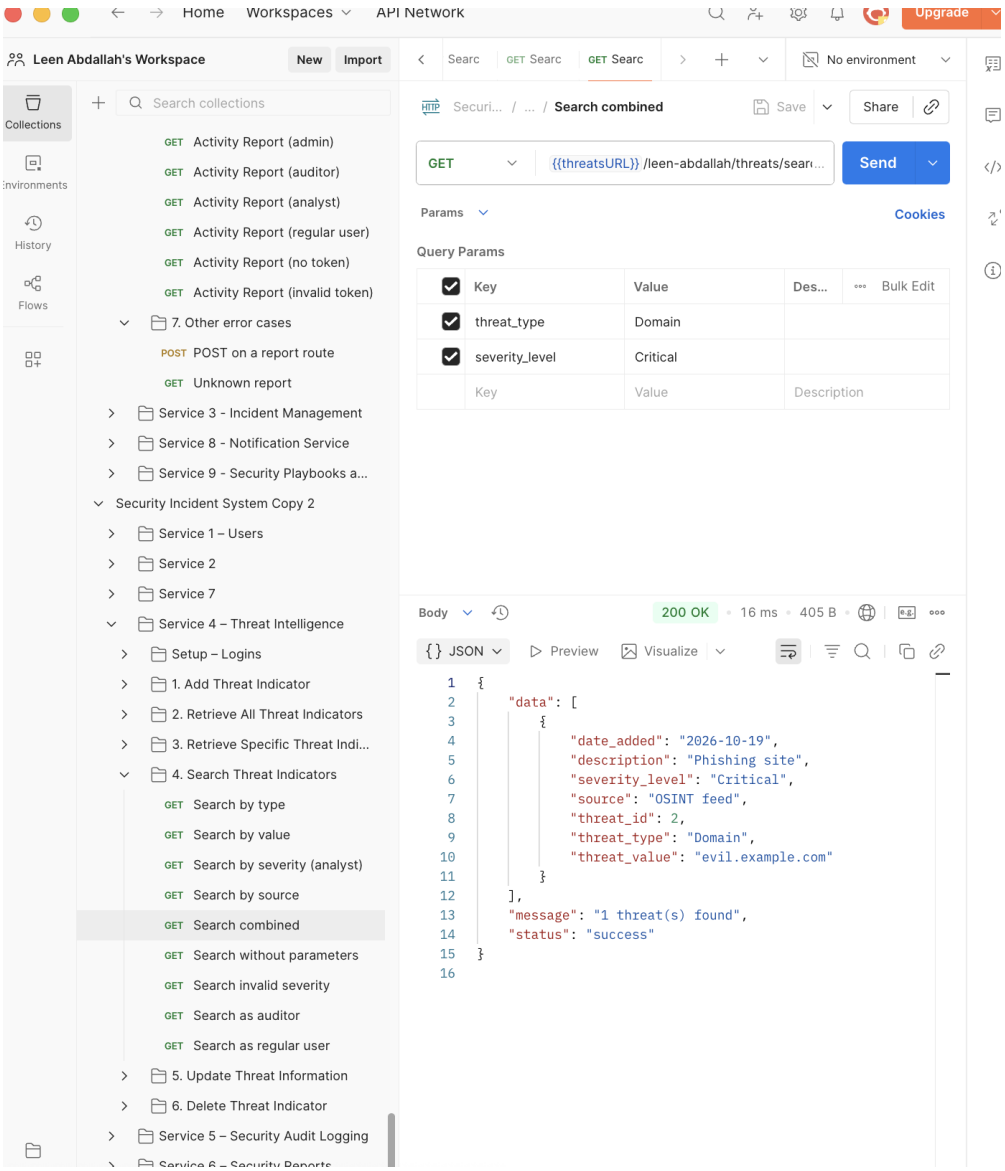


Figure 69: Leen Abdallah: Search Threat Indicators: combined Domain type and Critical severity filters, 200

Figures 70–72: The matching GET/PUT/GET sequence changes indicator 1 from “Port scanner”, High, AbuseIPDB to “Updated description”, Critical, “New source”. The final GET confirms the saved update. ID 1, IP Address, 192.168.1.5 and the creation date remain unchanged. Update input validation and analyst denial are shown in Sections 7 and 9.

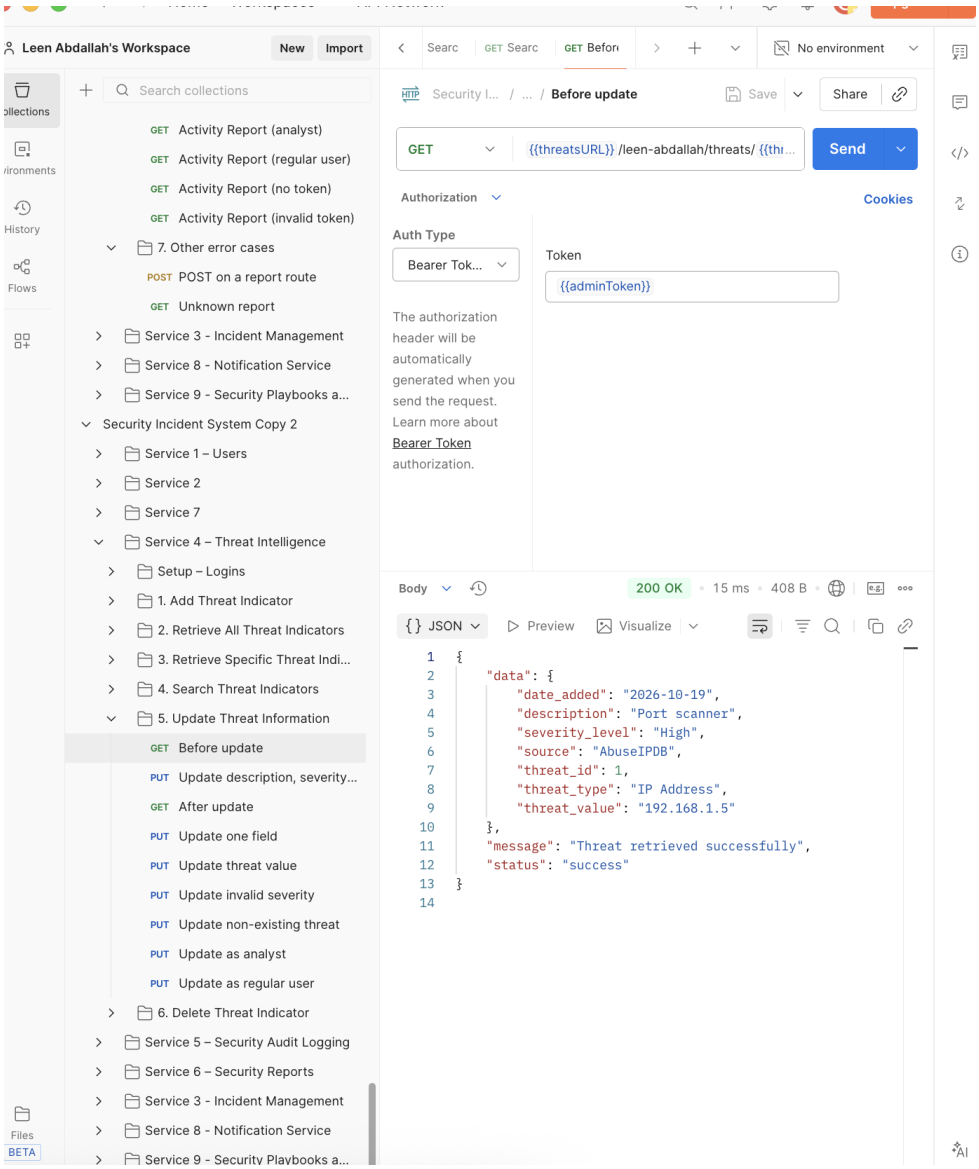


Figure 70: Leen Abdallah: Retrieve Threat before update: indicator 1, Port scanner, High, AbuseIPDB, 200

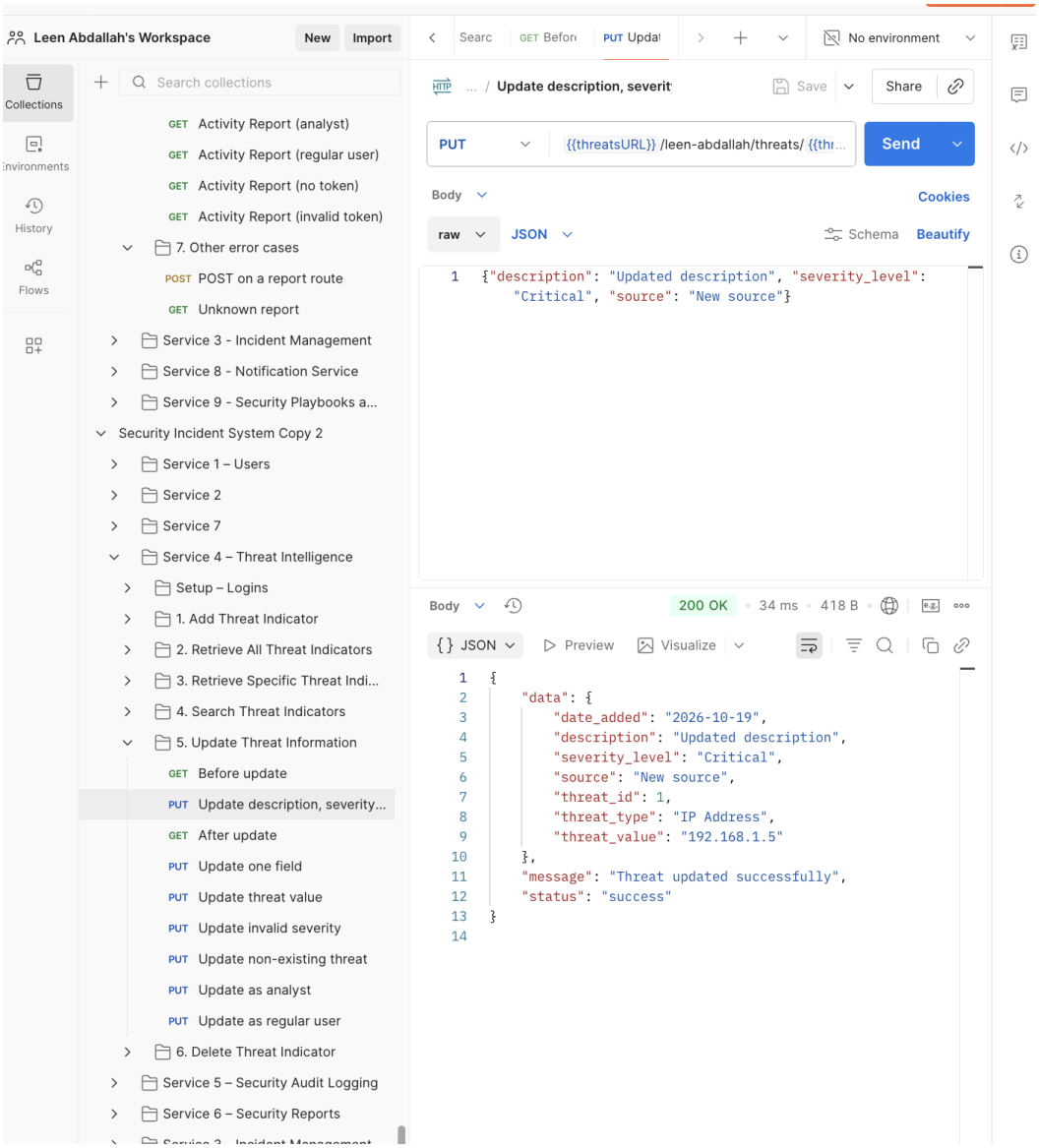


Figure 71: Leen Abdallah: Update Threat Information (PUT /leen-abdallah/threats/:id): description, severity and source changed, 200

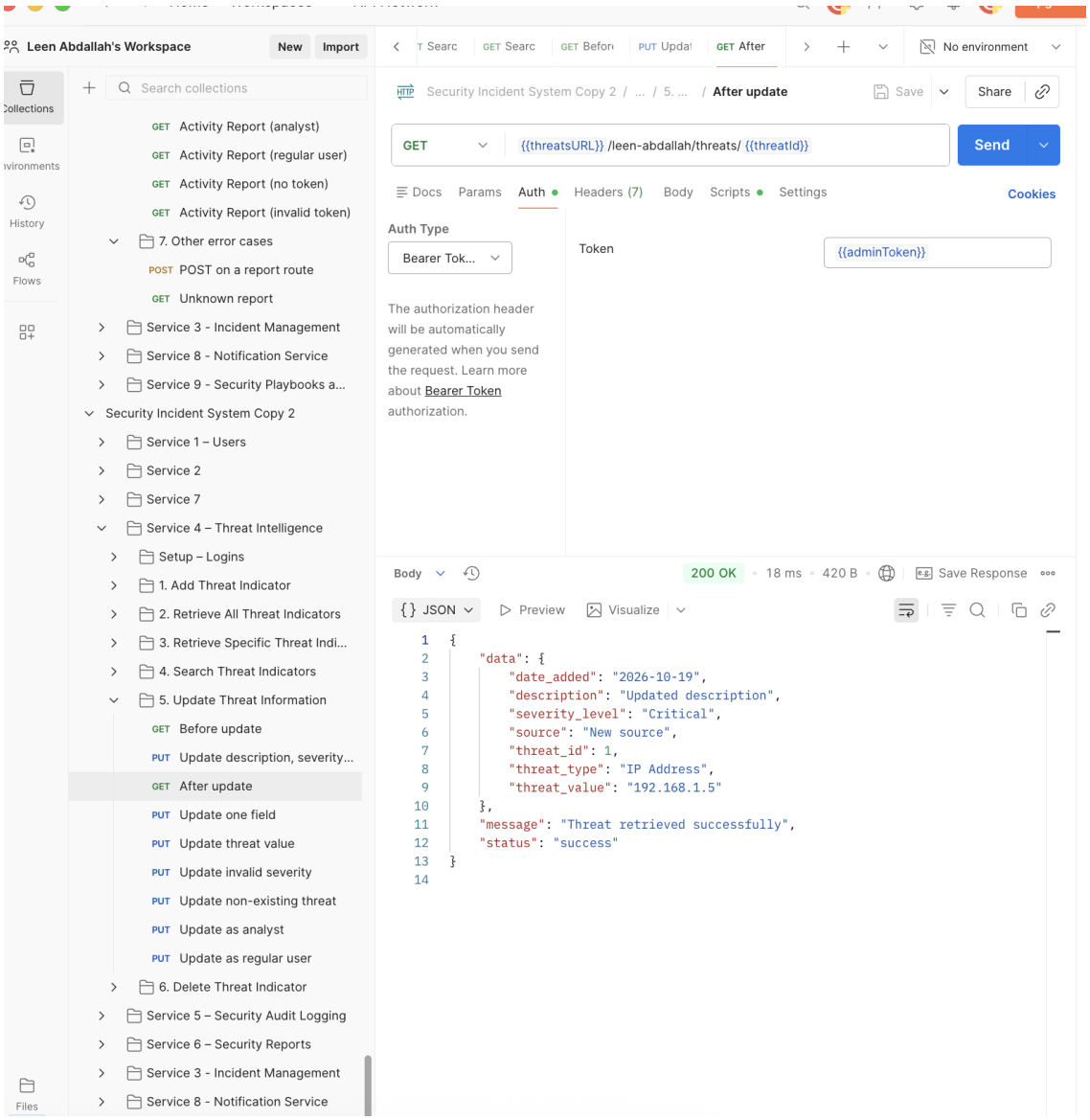


Figure 72: Leen Abdallah: Retrieve Threat after update: indicator 1, Updated description, Critical, New source, 200

Figures 73–74: Administrator deletion returns `200`, followed by `404` when retrieving the deleted indicator. The request uses a separate `deleteThreatId` variable; its numeric value is not visible, so it is not identified as indicator 1. Section 9 shows a completed analyst DELETE denial with 403. A Regular User DELETE returns 403 and a DELETE without a token returns 401, distinguishing role rejection from authentication failure.

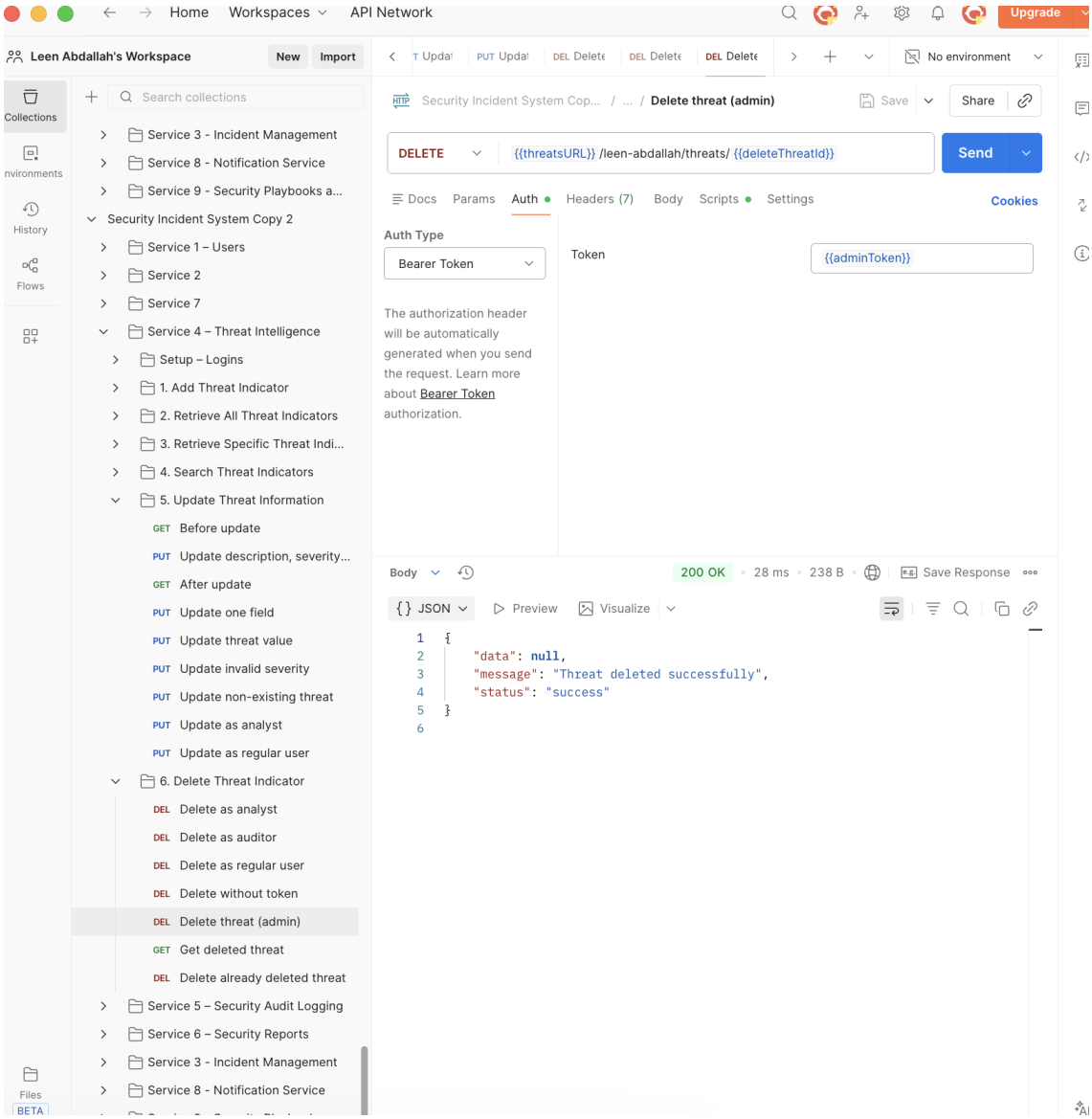


Figure 73: Leen Abdallah: Delete Threat Indicator (DELETE /leen-abdallah/threats/:id): administrator, 200 with null data

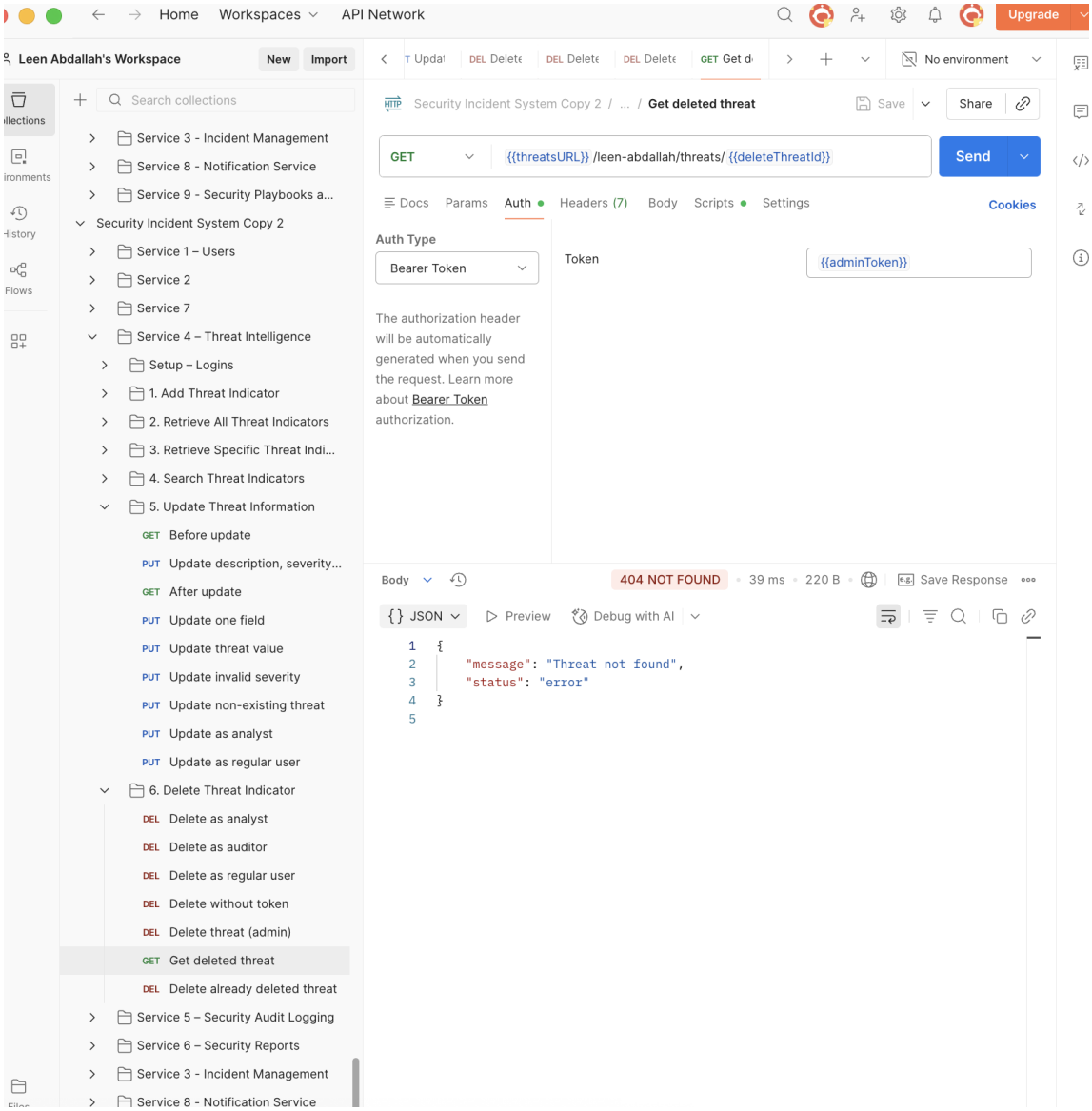


Figure 74: Leen Abdallah: Retrieve Specific Threat Indicator after deletion: same deleteThreatId variable, 404

Figure 75 shows stored threat-update entries and a recent “Threat deleted” entry with Warning severity. Its authorization-denial entries omit request descriptions and are interpreted alongside the completed DELETE/403 in Section 9, rather than identifying the denied method independently. The older Critical deletion entry is part of the pre-existing log dataset, not proof that the current route logs deletion at Critical severity.

```
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column database/security_incident.db "select log_id,user_id,actic
n,severity_level,related_service from logs where related_service='threats' order by log_id desc limit 25;"
log_id  user_id  action                       severity_level  related_service
------  -------  ---------------------------  --------------  ---------------
88      1        Threat deleted               Warning         threats
87               Unauthorized access attempt  Warning         threats
86      4        Unauthorized access attempt  Warning         threats
85      4        Unauthorized access attempt  Warning         threats
84      2        Unauthorized access attempt  Warning         threats
83      1        Threat updated               Information     threats
82      1        Threat updated               Information     threats
81      4        Unauthorized access attempt  Warning         threats
80      3        Unauthorized access attempt  Warning         threats
79      4        Unauthorized access attempt  Warning         threats
78               Unauthorized access attempt  Warning         threats
77      4        Unauthorized access attempt  Warning         threats
76      3        Unauthorized access attempt  Warning         threats
75      2        Unauthorized access attempt  Warning         threats
74      1        Threat added                 Information     threats
73      1        Threat added                 Information     threats
72      1        Threat added                 Information     threats
71      1        Threat added                 Information     threats
70      1        Threat added                 Information     threats
33      1        Threat deleted               Critical        threats
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 75: Leen Abdallah: Final Service 4 database audit query: threat updates, recent deletion at Warning severity and authorization-denial entries

5.5 Service 5: Security Audit Logging (Leen Abdallah)

Responsibility. Service 5 exposes security audit records for review. Shared helpers record implemented events such as login outcomes, entity changes and permission denials. Administrators can also create manual records and delete logs. Because no role-change API exists, automatic logging of an actual role-change workflow is not demonstrated.

API contract. Manual creation requires `user_id`, `action`, `description`, `severity_level`, `related_service`; the server generates the ID and timestamp. Search accepts user, date, action, severity and related-service filters. A known user with no matching logs produces `200` and an empty list. An unknown user supplied to creation produces `400`, not `404`.

Persistence and integrity. The `logs_no_update` trigger blocks changes to stored log content, and there is no PUT route. Its narrow exception permits `user_id` to become NULL during user deletion while preserving the other fields. Logs remain administratively deletable, and standalone audit writes can fail without failing the main request. The design therefore supports review and content protection, rather than an immutable or independently tamper-proof audit store.

Implemented APIs.

Method	Endpoint (prefix `/leen-abdallah`)	Access
POST	`/logs`	Administrator
GET	`/logs`	Administrator, Auditor
GET	`/logs/<id>`	Administrator, Auditor, Analyst
GET	`/logs/search`	Administrator, Auditor, Analyst
GET	`/logs/user/<user_id>`	Administrator, Auditor
DELETE	`/logs/<id>`	Administrator

Figures 76–80: The schema and initial rows establish the log structure. An administrator login is followed by manual incident-event and asset-modification records; these two requests do not prove automatic integration from those services.

```
Last login: Sat Oct 17 18:39:17 on ttys001
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 database/securi
ty_incident.db ".schema logs"
CREATE TABLE logs (
    log_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id         INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
    action          TEXT NOT NULL,
    description     TEXT NOT NULL,
    timestamp       TEXT NOT NULL DEFAULT (datetime('now')),
    severity_level  TEXT NOT NULL DEFAULT 'Information'
                    CHECK (severity_level IN ('Information', 'Warning', 'Error',
                                              'Critical')),
    related_service TEXT NOT NULL
);
CREATE TRIGGER logs_no_update
BEFORE UPDATE ON logs
WHEN NEW.log_id          IS NOT OLD.log_id
  OR NEW.action          IS NOT OLD.action
  OR NEW.description     IS NOT OLD.description
  OR NEW.timestamp       IS NOT OLD.timestamp
  OR NEW.severity_level  IS NOT OLD.severity_level
  OR NEW.related_service IS NOT OLD.related_service
  OR NEW.user_id         IS NOT NULL
BEGIN
    SELECT RAISE(ABORT, 'Audit logs cannot be modified after creation');
END;
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 76: Leen Abdallah: Database: logs table schema

```
END;
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column
 database/security_incident.db "select log_id,user_id,action,severity_level,related_service from logs
;"
log_id  user_id  action            severity_level  related_service
------  -------  ----------------  --------------  ---------------
1       1        Successful login  Information     users
2       1        User created      Information     users
3       1        User created      Information     users
4       4        User created      Information     users
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 77: Leen Abdallah: Database: logs table before the tests

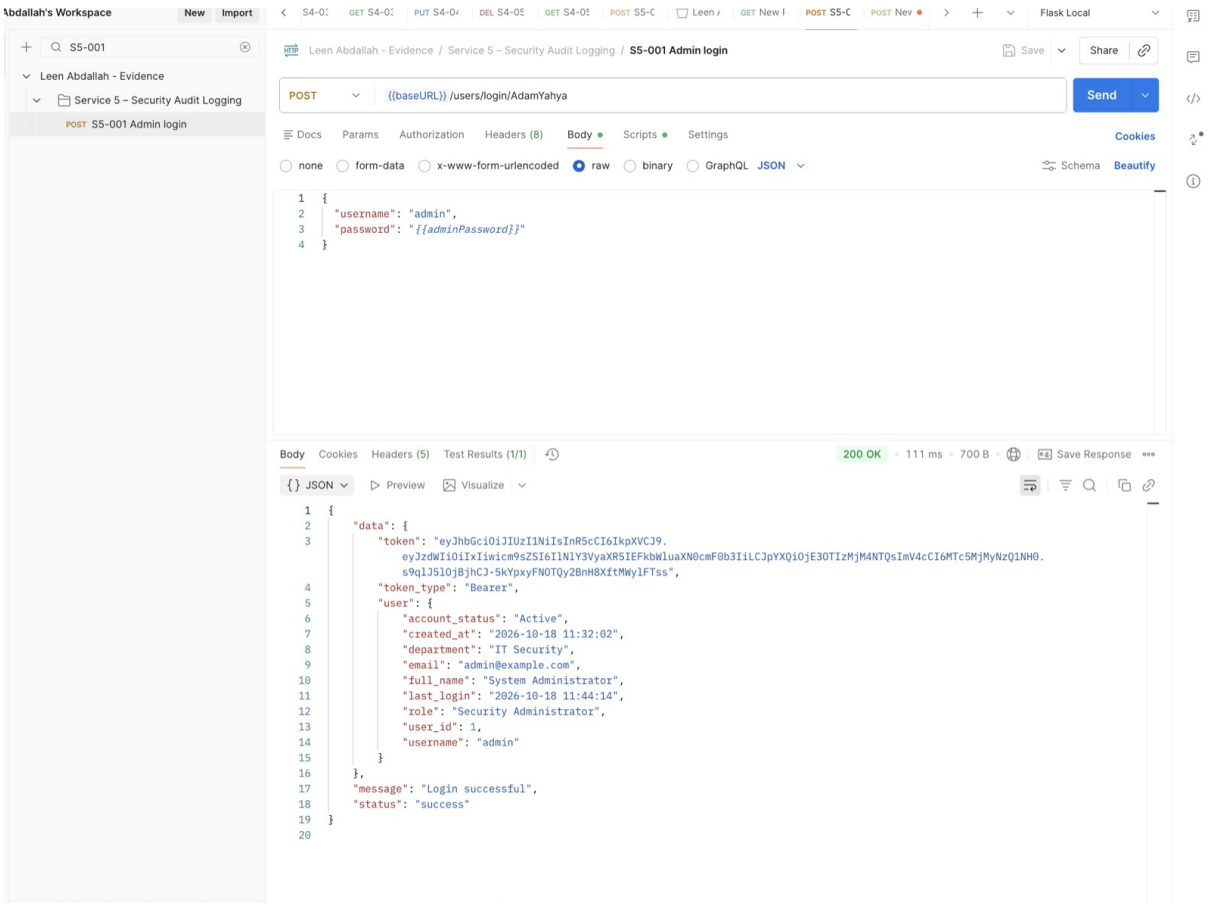


Figure 78: Leen Abdallah: Login: administrator token used for Service 5

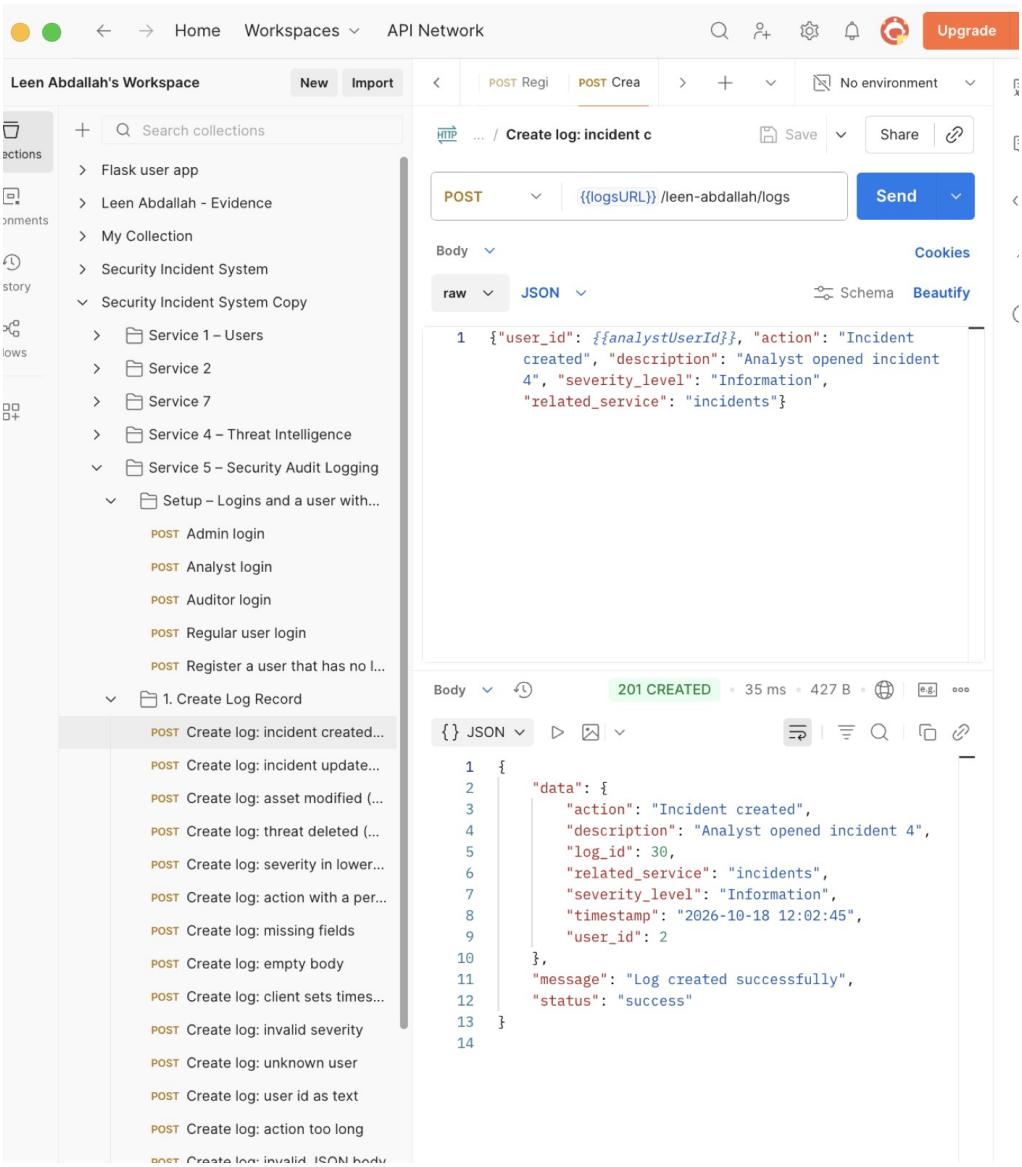


Figure 79: Leen Abdallah: Create Log Record (POST /leen-abdallah/logs): incident event, 201

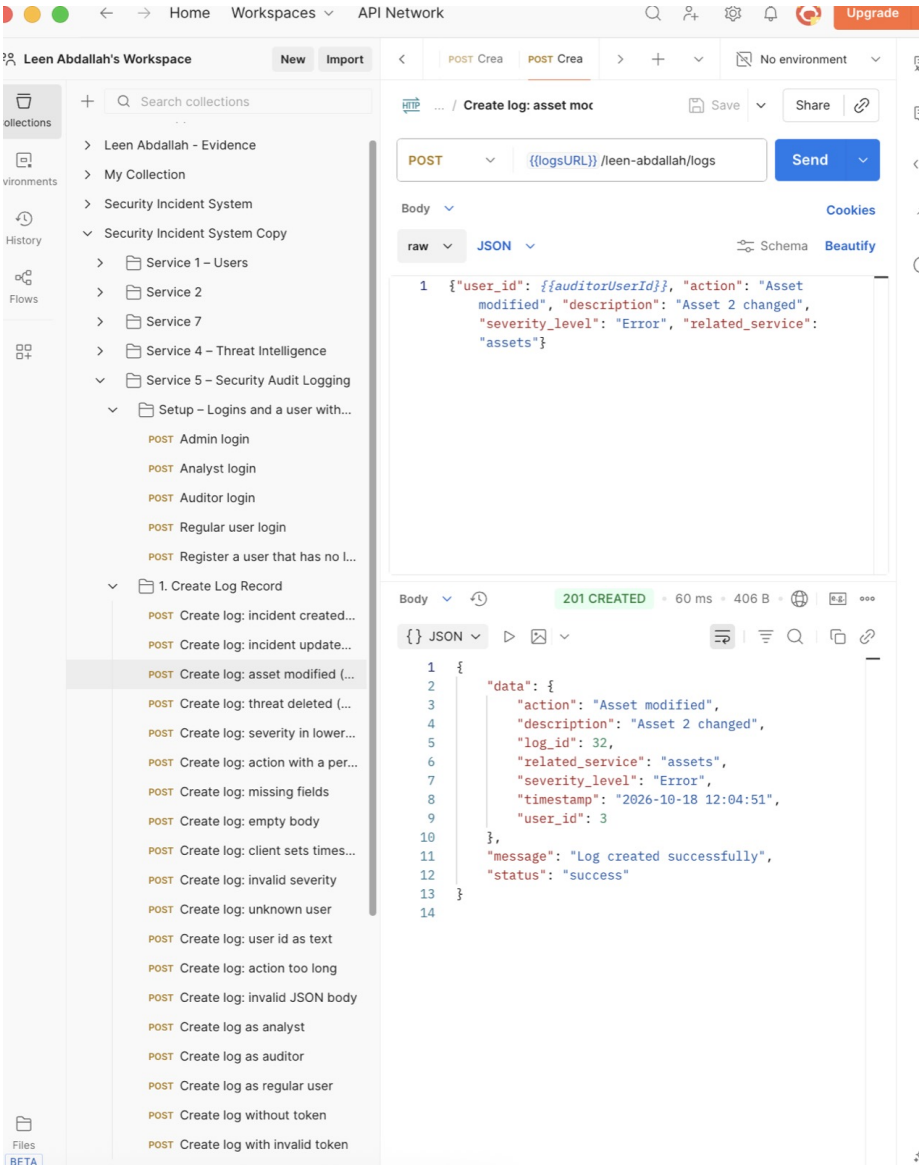


Figure 80: Leen Abdallah: Create Log Record: asset modification event, 201

Figures 81–87: List, single-record and filtered retrieval illustrate review access. The user-history example with no records returns an empty list rather than a missing-resource error.

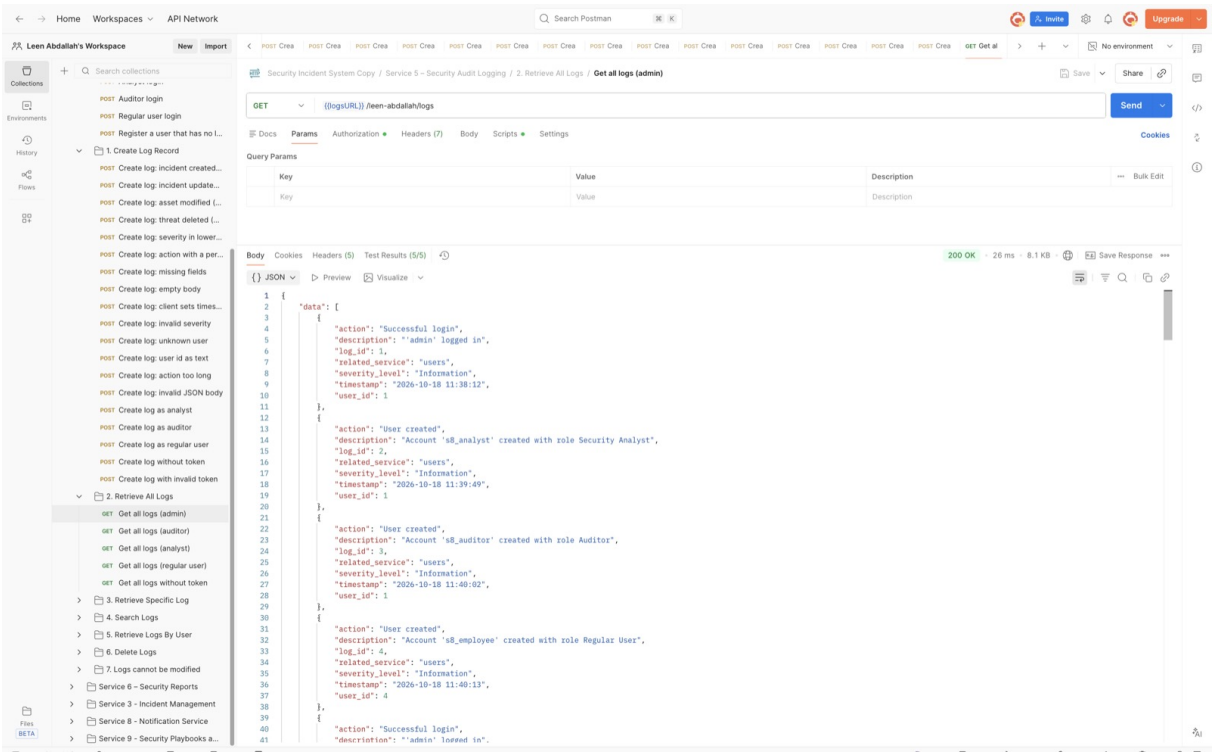


Figure 81: Leen Abdallah: Retrieve All Logs (GET /leen-abdallah/logs): administrator, 200

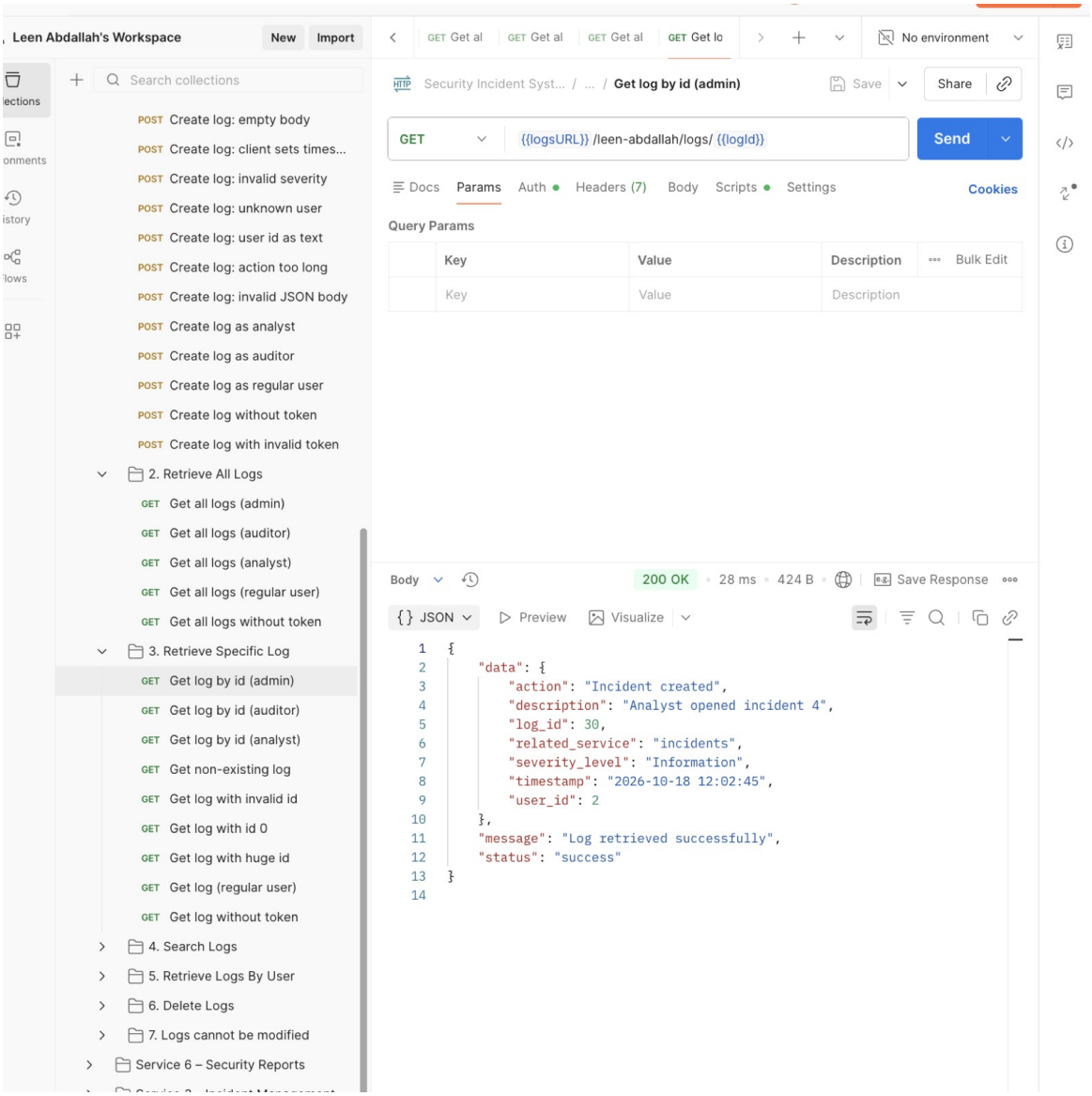


Figure 82: Leen Abdallah: Retrieve Specific Log (GET /leen-abdallah/logs/:id), 200

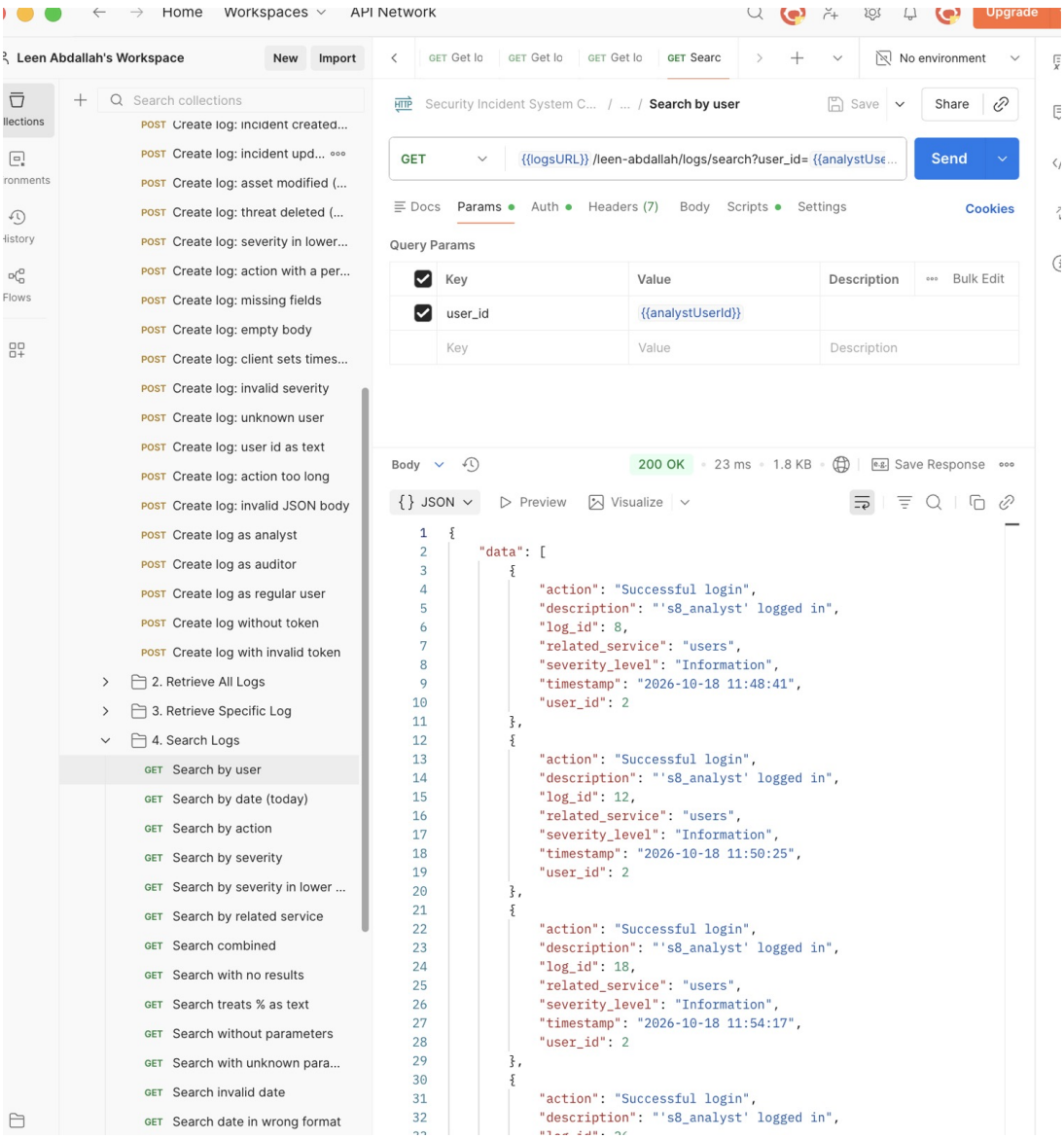


Figure 83: Leen Abdallah: Search Logs (GET /leen-abdallah/logs/search): by user, 200

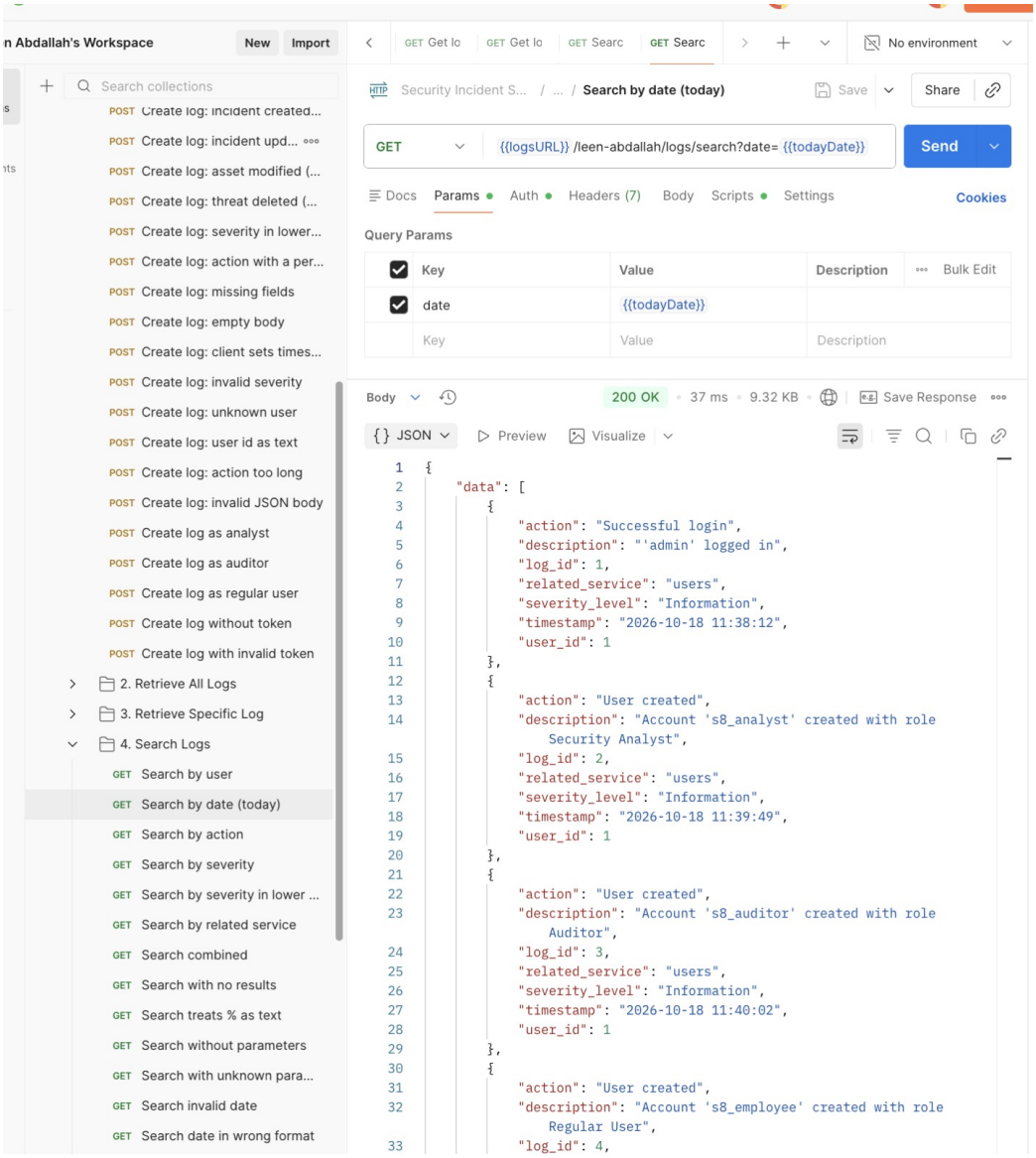


Figure 84: Leen Abdallah: Search Logs: by date, 200

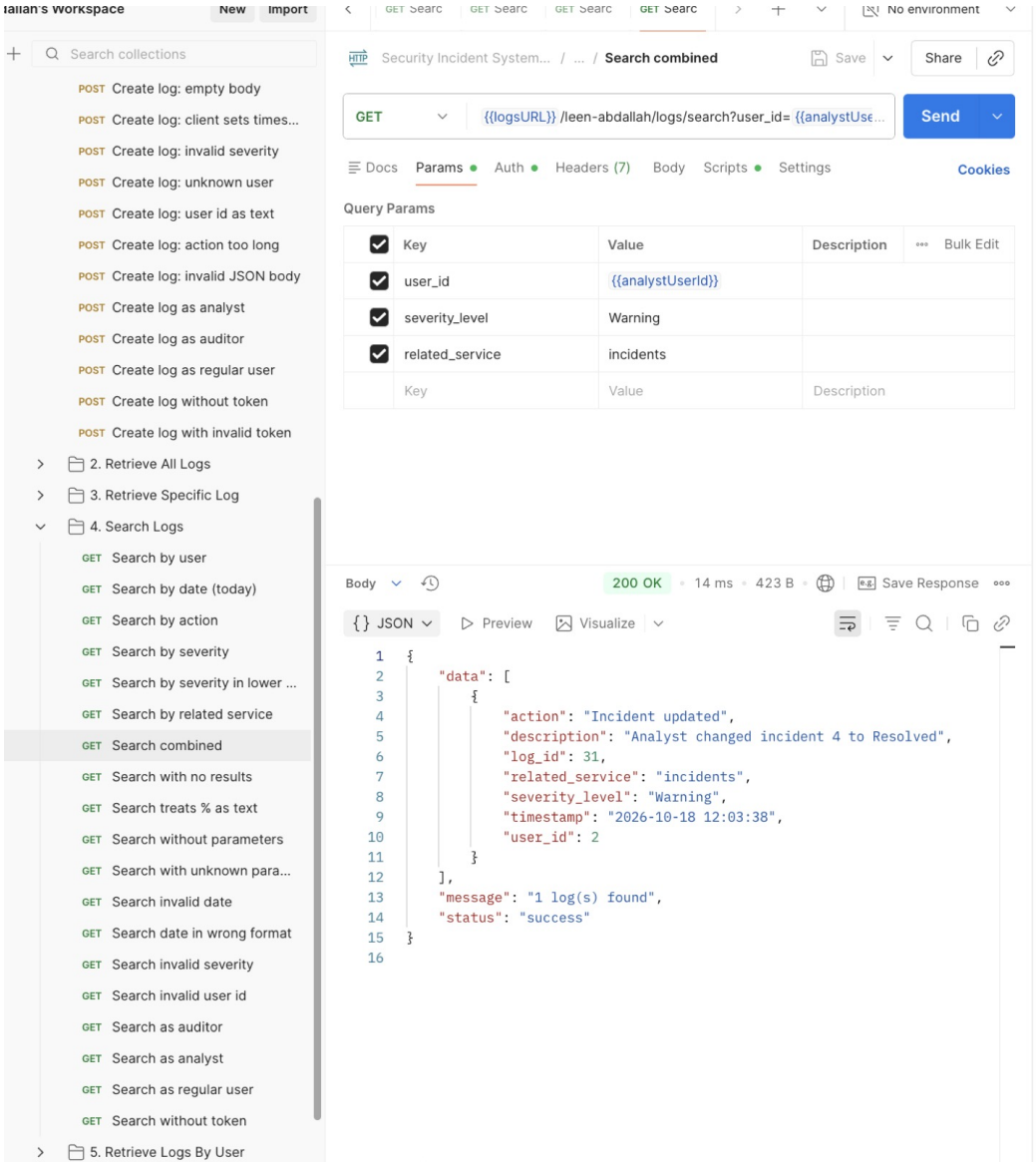


Figure 85: Leen Abdallah: Search Logs: combined filters, 200

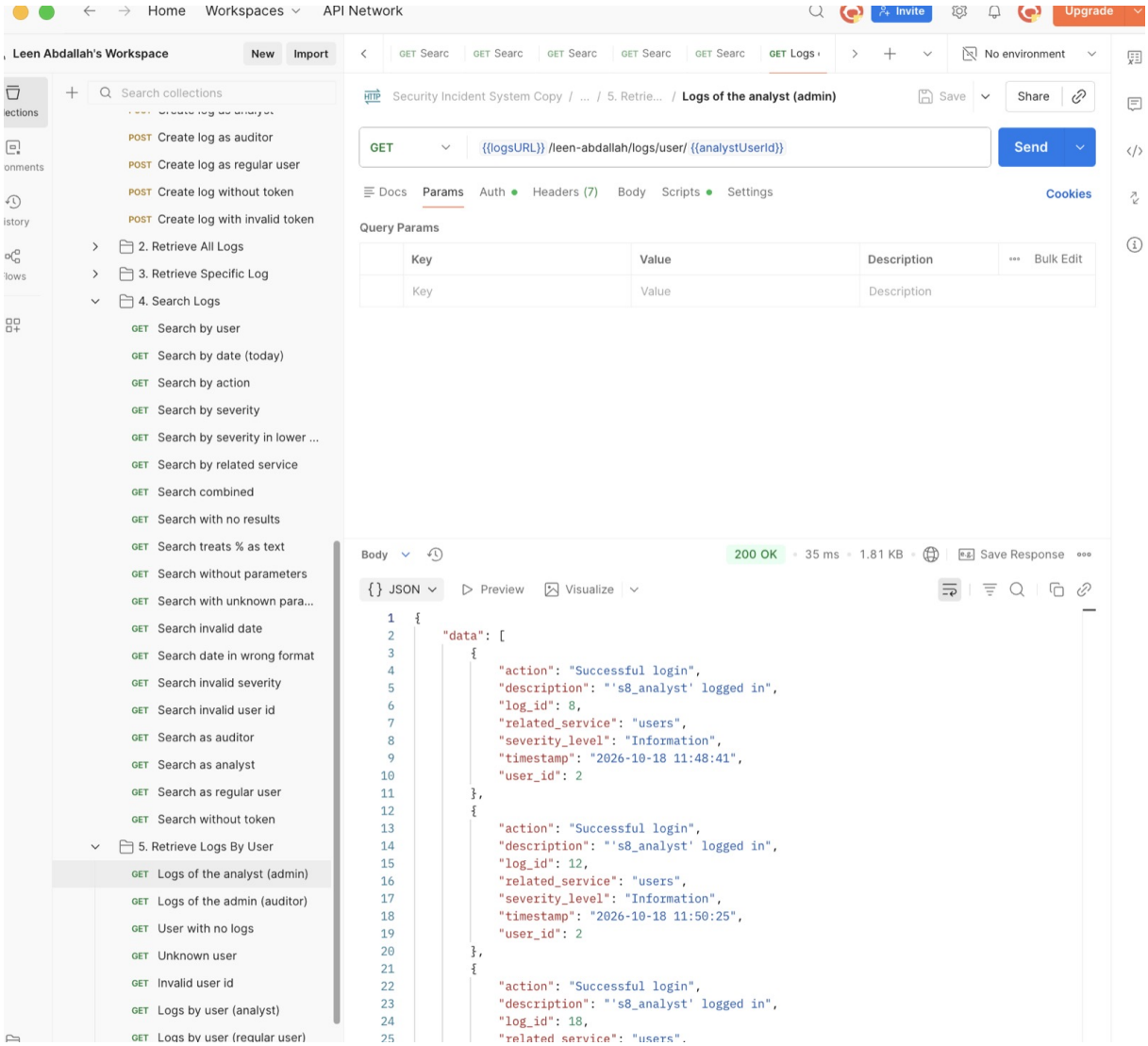


Figure 86: Leen Abdallah: Retrieve Logs By User (GET /leen-abdallah/logs/user/:user_id): 200

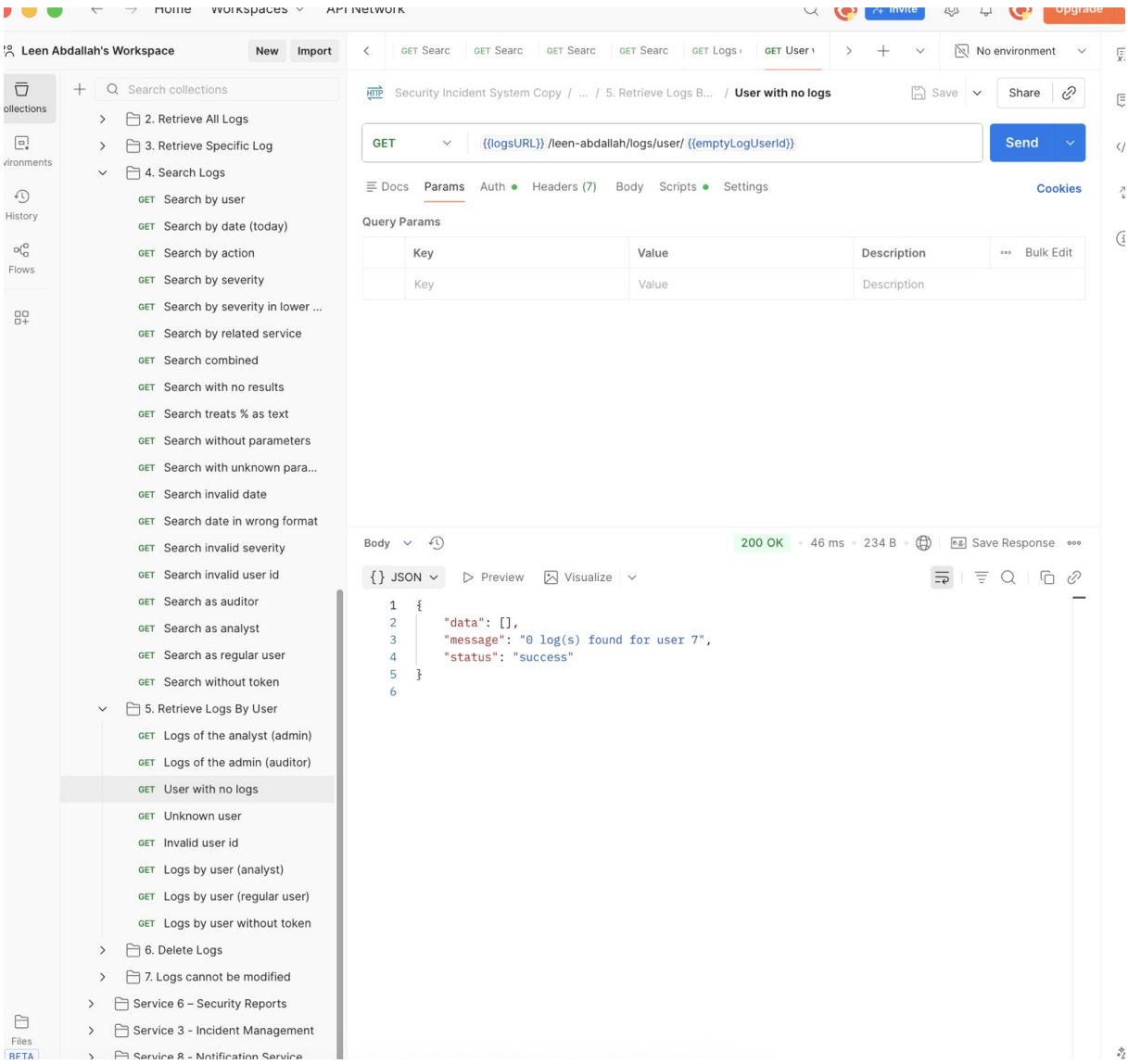


Figure 87: Leen Abdallah: Retrieve Logs By User: user without logs, 200 with empty list

Figures 88–92: Deletion is confirmed by a subsequent 404. The search excerpt shows denied POST and GET requests, not visible DELETE-denial records. The rejected PUT establishes the absent update API; the separate SQLite UPDATE failure demonstrates the content-protection trigger.

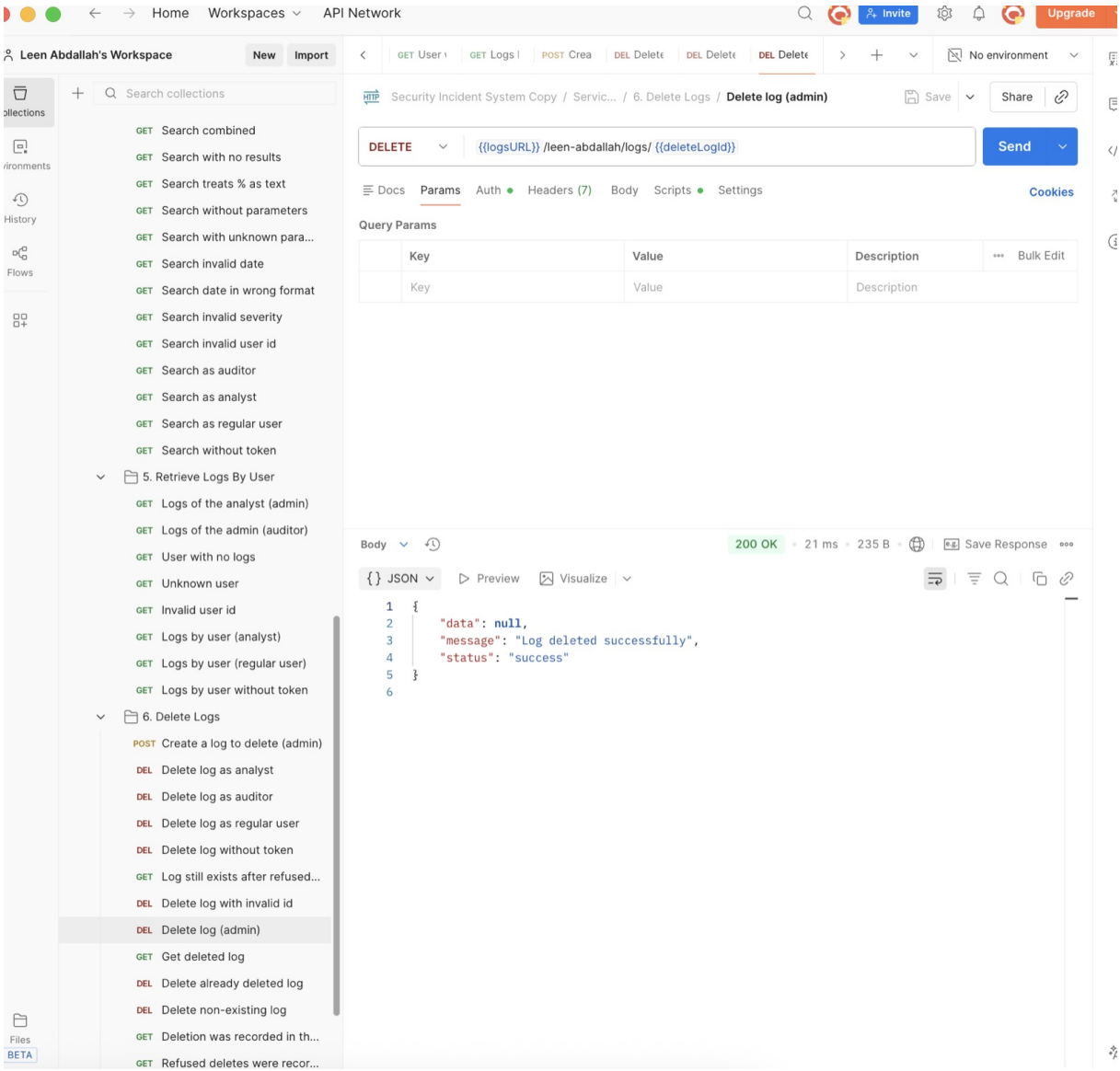


Figure 88: Leen Abdallah: Delete Logs (DELETE /leen-abdallah/logs/:id): administrator, 200

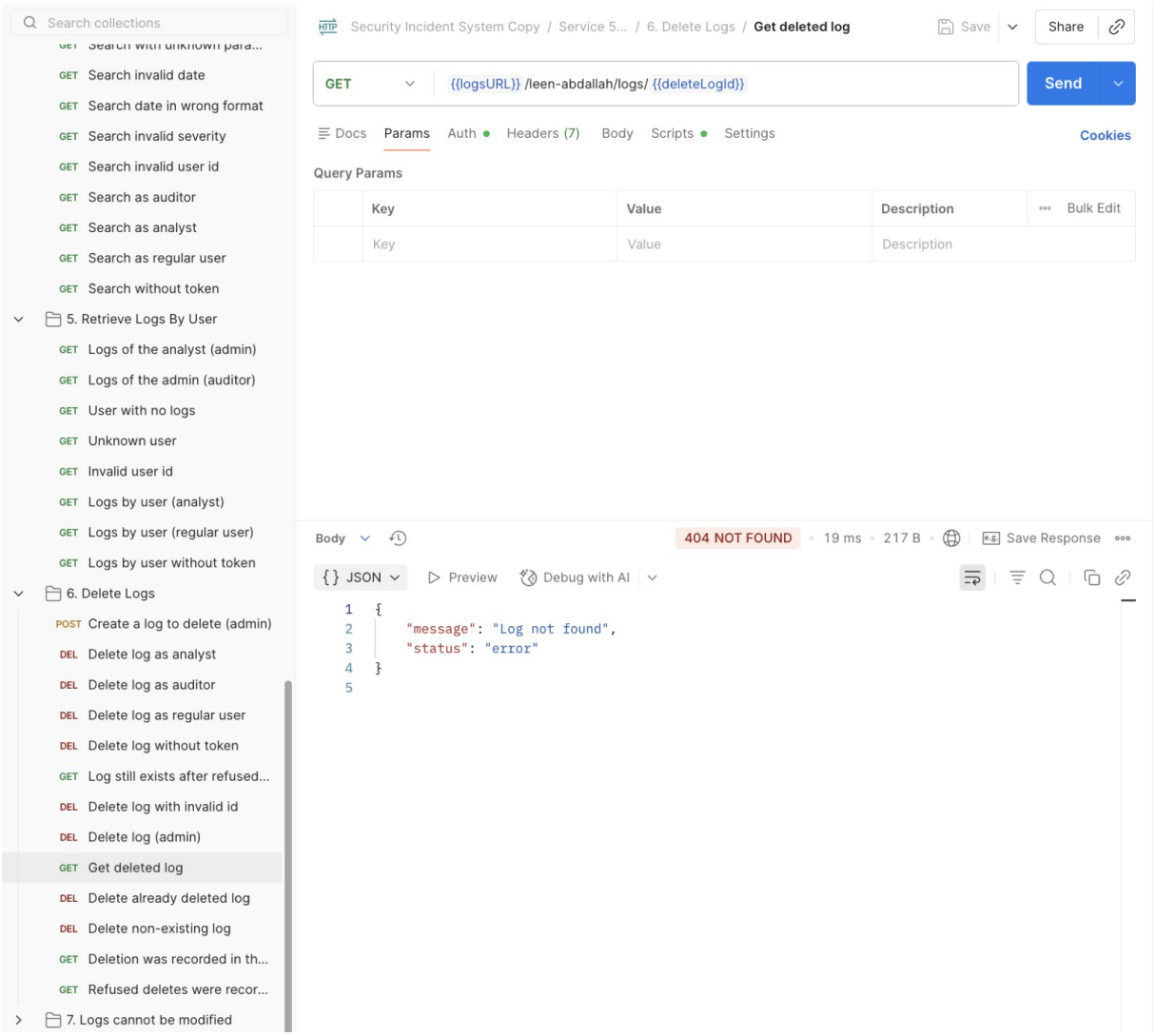


Figure 89: Leen Abdallah: Retrieve Specific Log after deletion, 404

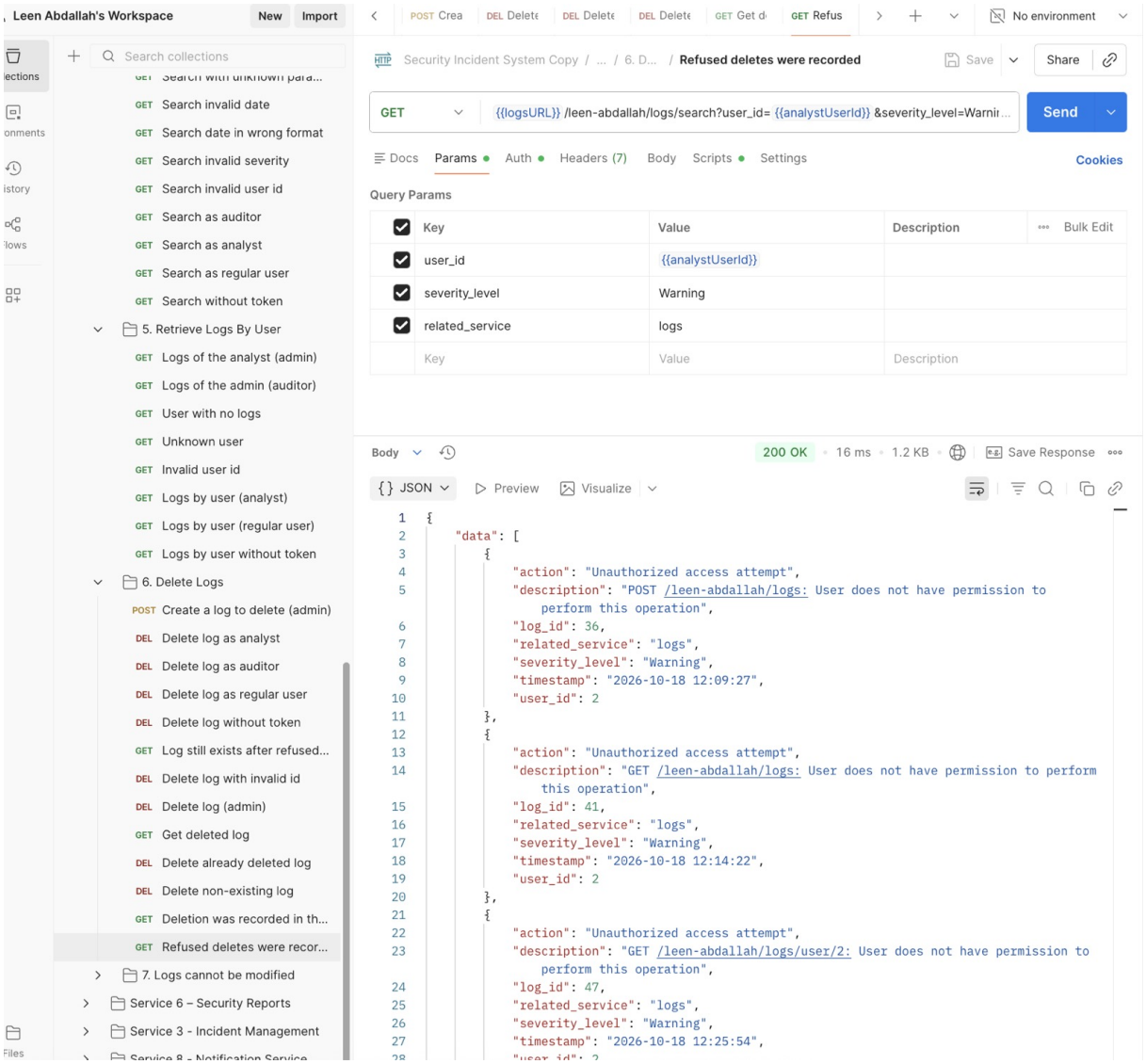


Figure 90: Leen Abdallah: Search Logs (GET /leen-abdallah/logs/search): visible permission-denial records for POST and GET requests, 200

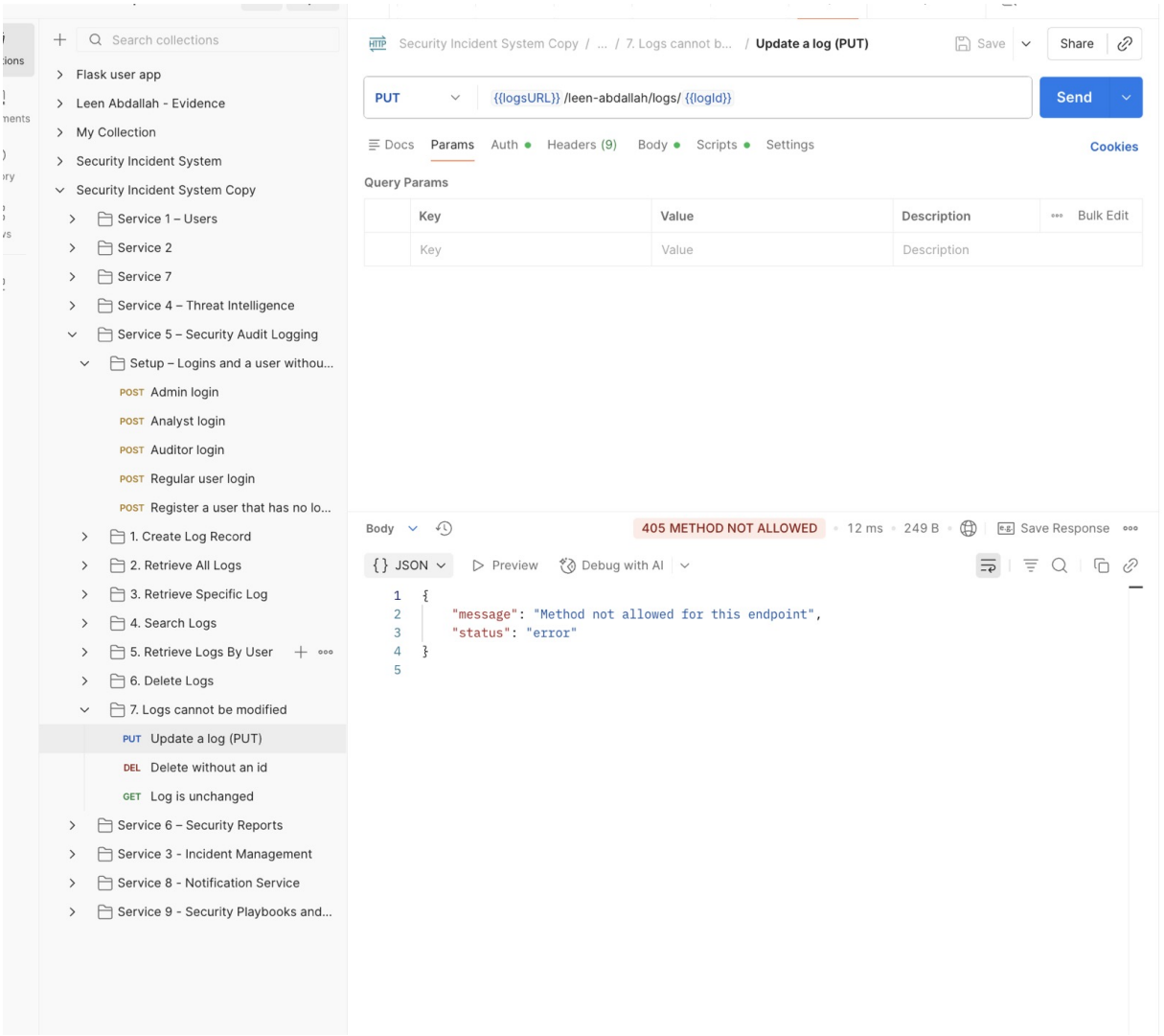


Figure 91: Leen Abdallah: PUT on the logs resource returns 405; there is no log-update API

```
26      2       Successful login            Information     users
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 database/securi
ty_incident.db "update logs set action='hacked' where log_id=1;"
Error: stepping, Audit logs cannot be modified after creation (19)
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 92: Leen Abdallah: Database: trigger blocks modification of stored logs

5.6 Service 6: Security Reports (Leen Abdallah)

Responsibility. Service 6 provides six reports to administrators and auditors. It has no report tables and does not modify source business records. Authentication and denied-access handling can still produce audit records.

API contract. The summary, incident, asset, threat, vulnerability and activity routes accept GET requests without a body or query filters. Each returns a statistics object with `generated_at`; categorical breakdowns retain zero-count categories. Unsupported methods return 405.

Data sources and numerical checks. The service retrieves source records over HTTP and computes statistics in `report_service/statistics.py`. The displayed database counts are 7

users, 4 assets, 6 incidents, 5 threats, 5 vulnerabilities and 54 logs. The matching summary reports 7 users, 4 assets, 6 total incidents, 4 open incidents, 5 threats and 2 critical threats. Here, "open" includes Open, Under Investigation and Mitigated incidents: 2 + 1 + 1 = 4. It does not mean only the two records whose literal status is Open.

The incident severity counts sum to 6 (2 Critical, 2 High, 1 Medium, 1 Low), and its status counts also sum to 6. Asset criticality counts sum to 4 (1 Critical, 2 High, 1 Medium, 0 Low). Vulnerability severities sum to 5 (2 Critical, 2 High, 1 Medium). The activity report shows 24 login attempts: 23 successful and 1 failed. Its later log records must not be reconciled as though they were captured at the same instant as the 54-log database count. Manually entered action labels in that report are log data, not evidence that an unrelated business feature exists.

Implemented APIs. Administrator and Auditor: GET `/leen-abdallah/reports/summary`, `/leen-abdallah/reports/incidents`, `/leen-abdallah/reports/assets`, `/leen-abdallah/reports/threats`, `/leen-abdallah/reports/vulnerabilities`, `/leen-abdallah/reports/activity`.

Figures 93–95: The running-process capture and matching database counts support the summary comparison described above.

```
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % pkill -f "_service.app"
; sleep 1
for s in user asset incident threat log vulnerability report; do nohup .venv/bin/python -m ${s}_servi
ce.app > /tmp/$s.log 2>&1 & done; sleep 3
lsof -iTCP:5001-5007 -sTCP:LISTEN -P
[2] 42945
[3] 42946
[4] 42947
[5] 42948
[6] 42949
[7] 42950
[8] 42951
COMMAND   PID         USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
Python  42945 leenabdallah    3u  IPv4 0xe53940e836ac2c09      0t0  TCP localhost:5001 (LISTEN)
Python  42946 leenabdallah    3u  IPv4 0x53d519fd21c66f51      0t0  TCP localhost:5002 (LISTEN)
Python  42947 leenabdallah    3u  IPv4 0x8568b746e868a982      0t0  TCP localhost:5003 (LISTEN)
Python  42948 leenabdallah    3u  IPv4 0x16e4b0f2e989dc4f      0t0  TCP localhost:5004 (LISTEN)
Python  42949 leenabdallah    3u  IPv4 0xfd00040c6890345d      0t0  TCP localhost:5005 (LISTEN)
Python  42950 leenabdallah    3u  IPv4 0x3a72487745bb0cd0      0t0  TCP localhost:5007 (LISTEN)
Python  42951 leenabdallah    3u  IPv4 0x78fb41b0d4708888      0t0  TCP localhost:5006 (LISTEN)
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 93: Leen Abdallah: Terminal: processes for ports 5001–5007, including the six report data sources

```
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column
 database/security_incident.db "select (select count(*) from users) users,(select count(*) from asset
s) assets,(select count(*) from incidents) incidents,(select count(*) from threats) threats,(select c
ount(*) from vulnerabilities) vulnerabilities,(select count(*) from logs) logs;"
users  assets  incidents  threats  vulnerabilities  logs
-----  ------  ---------  -------  ---------------  ----
7      4       6          5        5                54
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 94: Leen Abdallah: Database: record counts used to verify the reports

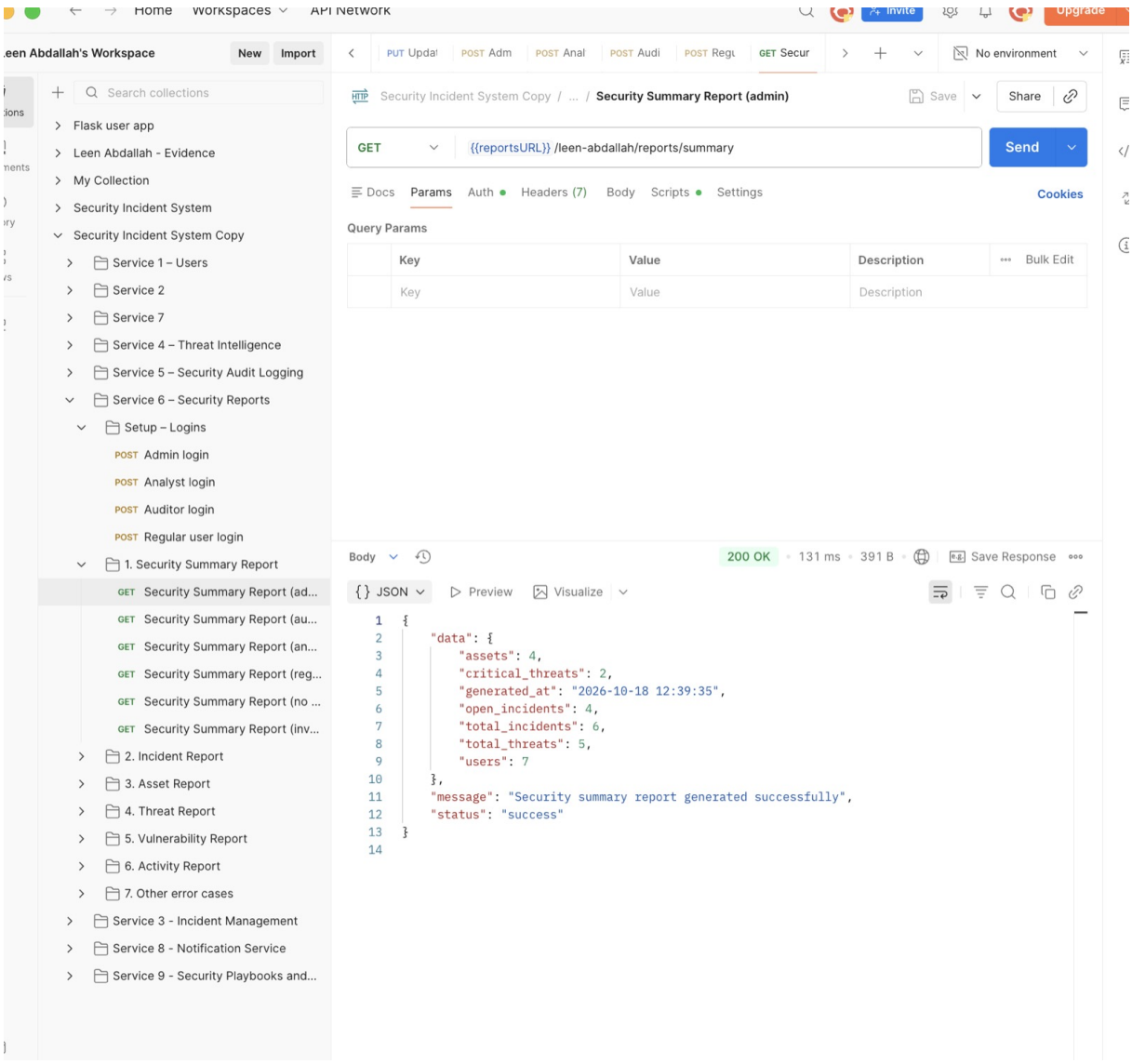


Figure 95: Leen Abdallah: Generate Security Summary Report (GET /leen-abdallah/reports/summary), 200

Figures 96–99: Incident, asset, threat and vulnerability reports summarize their respective source datasets. The visible threat panel is partial, so only its displayed values are treated as screenshot evidence.

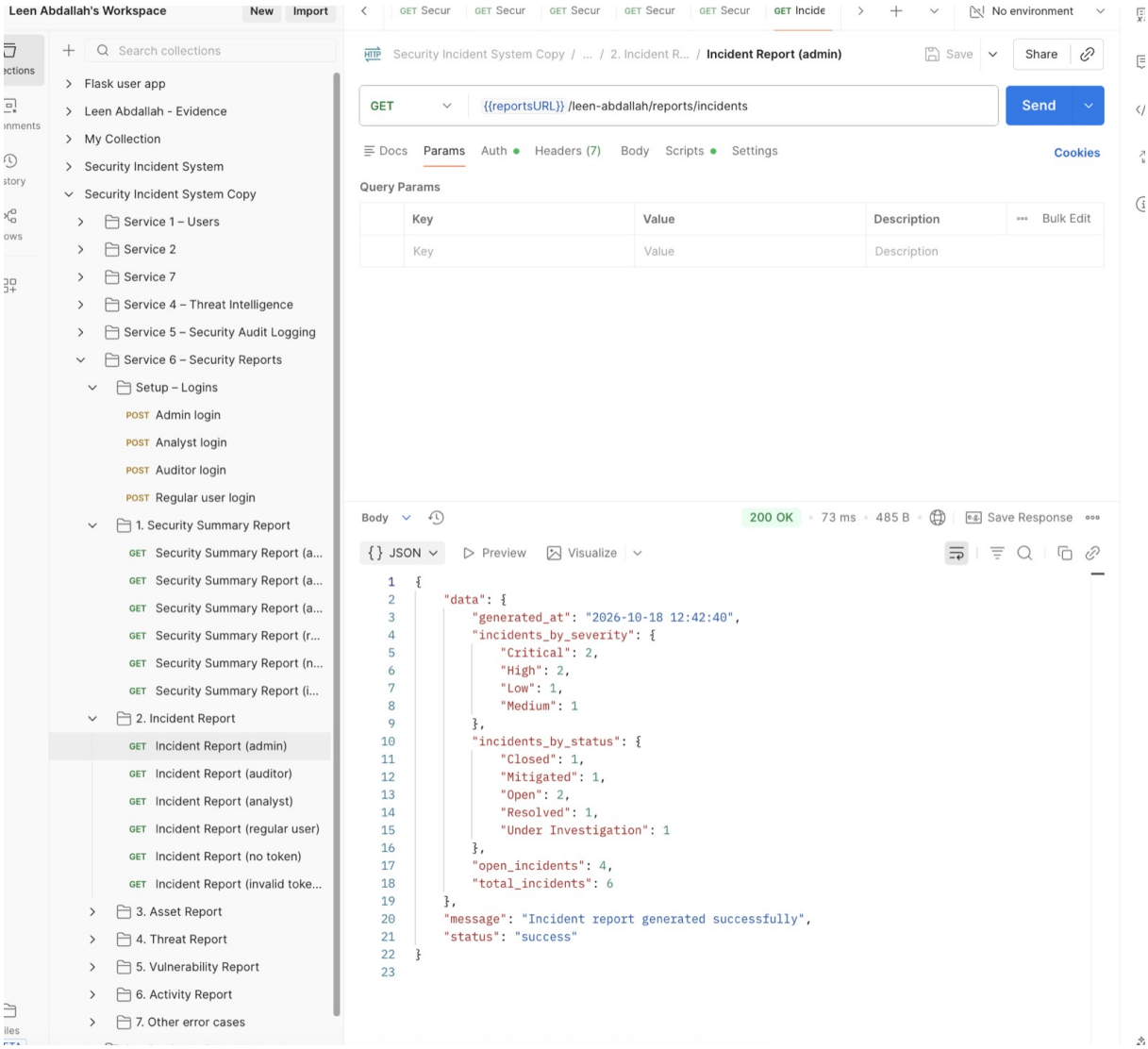


Figure 96: Leen Abdallah: Generate Incident Report (GET /leen-abdallah/reports/incidents), 200

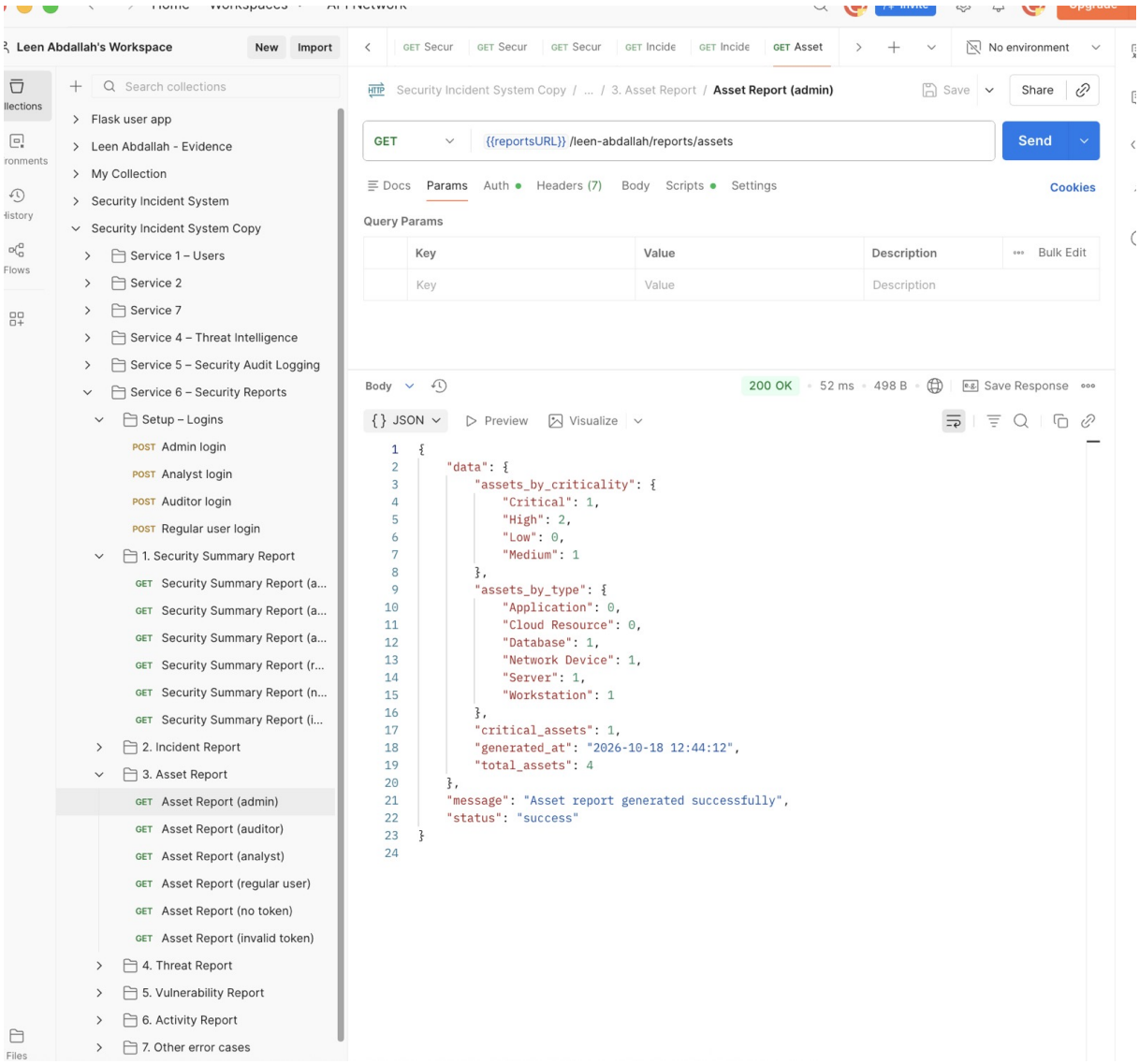


Figure 97: Leen Abdallah: Generate Asset Report (GET /leen-abdallah/reports/assets), 200

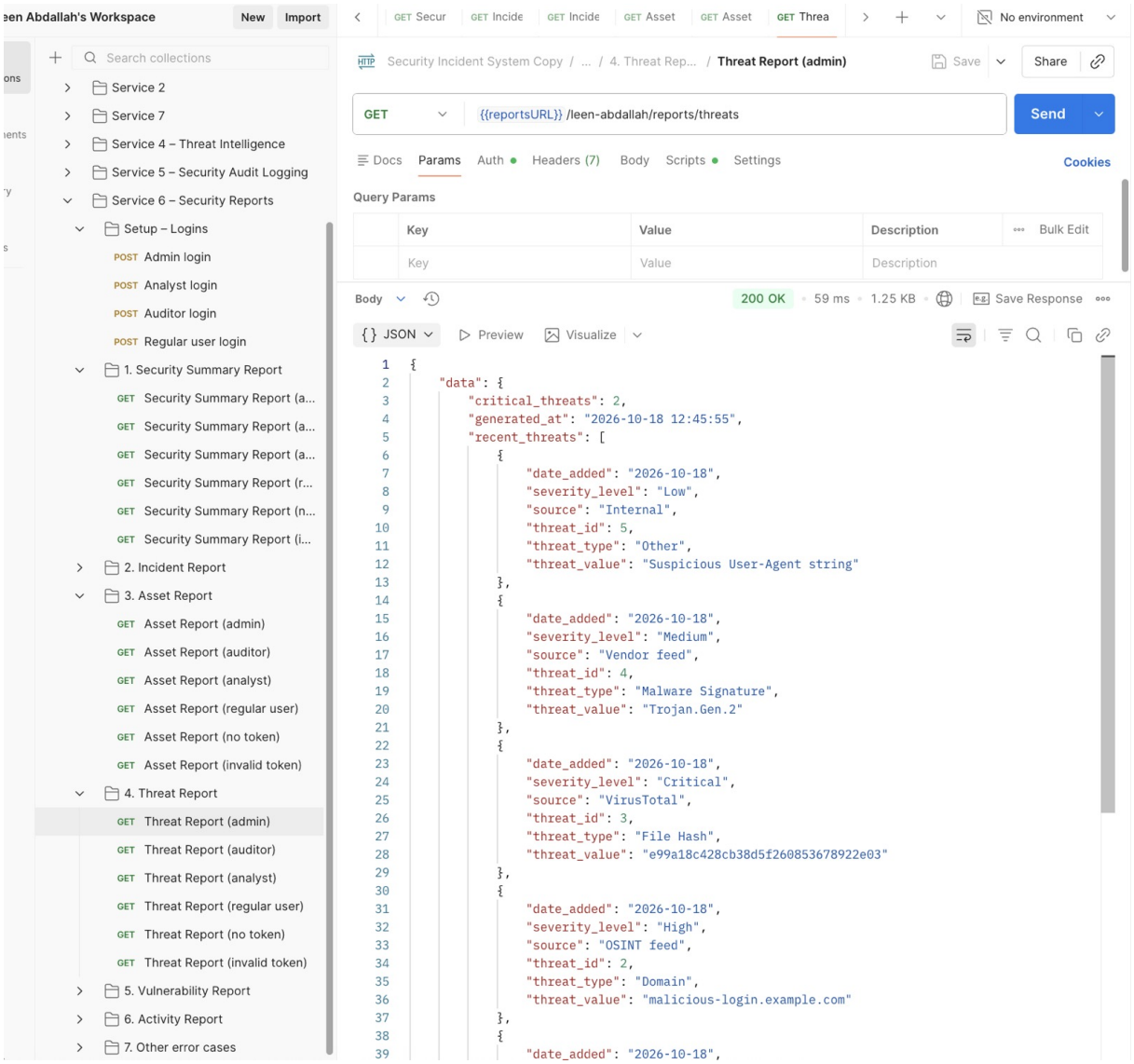


Figure 98: Leen Abdallah: Generate Threat Report (GET /leen-abdallah/reports/threats), 200

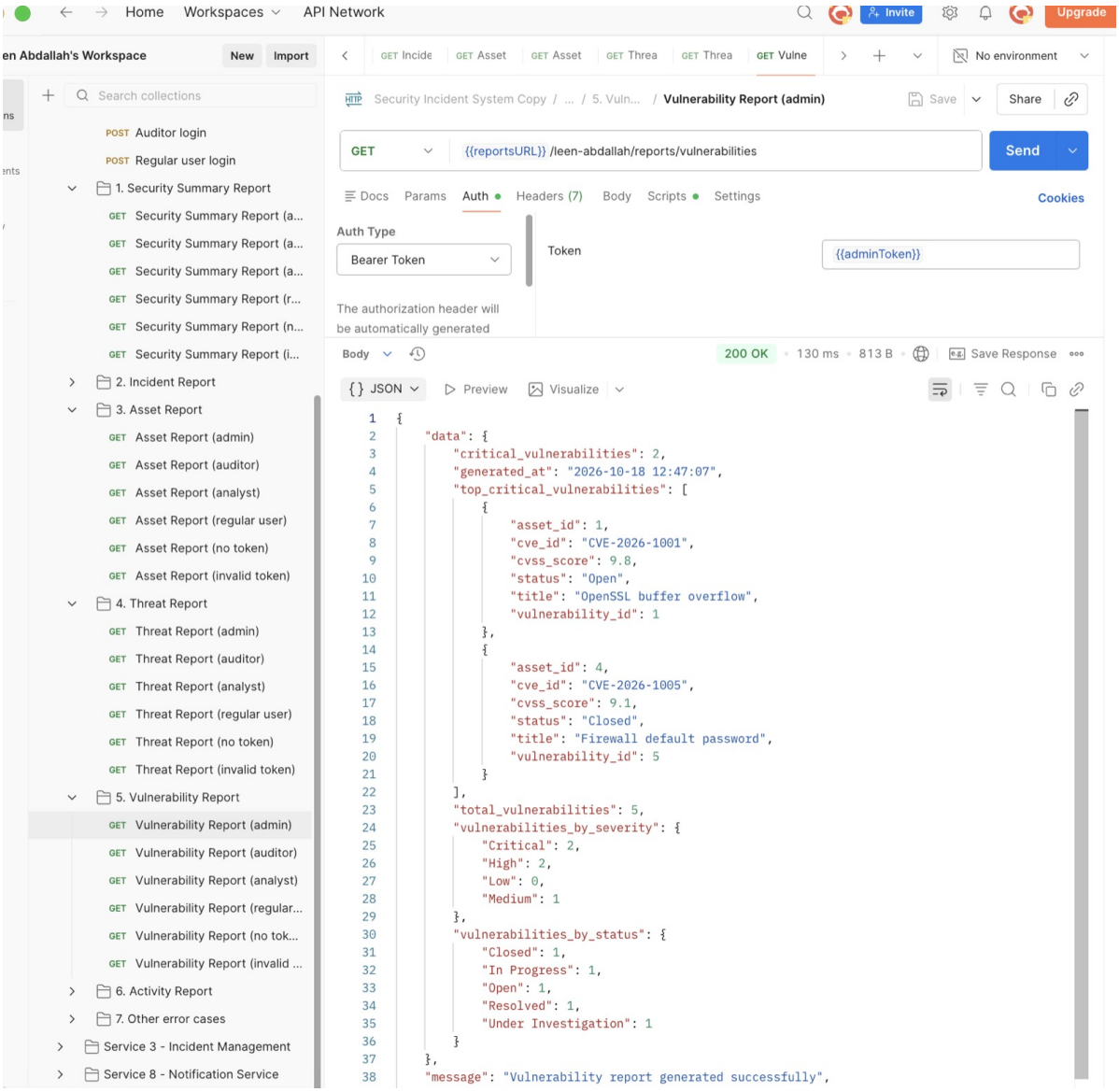


Figure 99: Leen Abdallah: Generate Vulnerability Report (GET /leen-abdallah/reports/vulnerabilities), 200

Figures 100–101: The activity report separates login totals from recent activity. Arbitrary manually entered action names remain user-supplied audit labels.

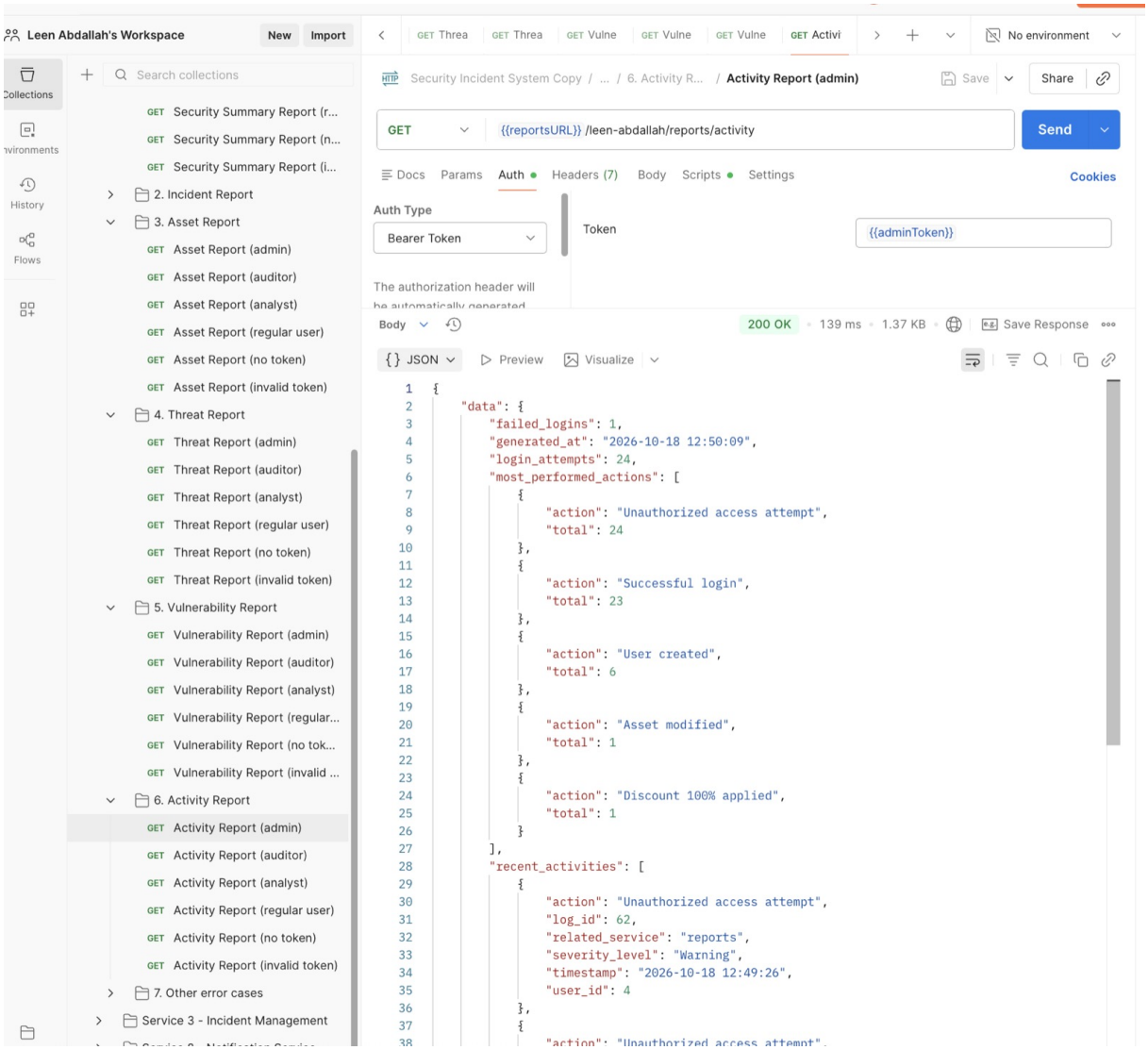


Figure 100: Leen Abdallah: Generate Activity Report (GET /leen-abdallah/reports/activity), 200

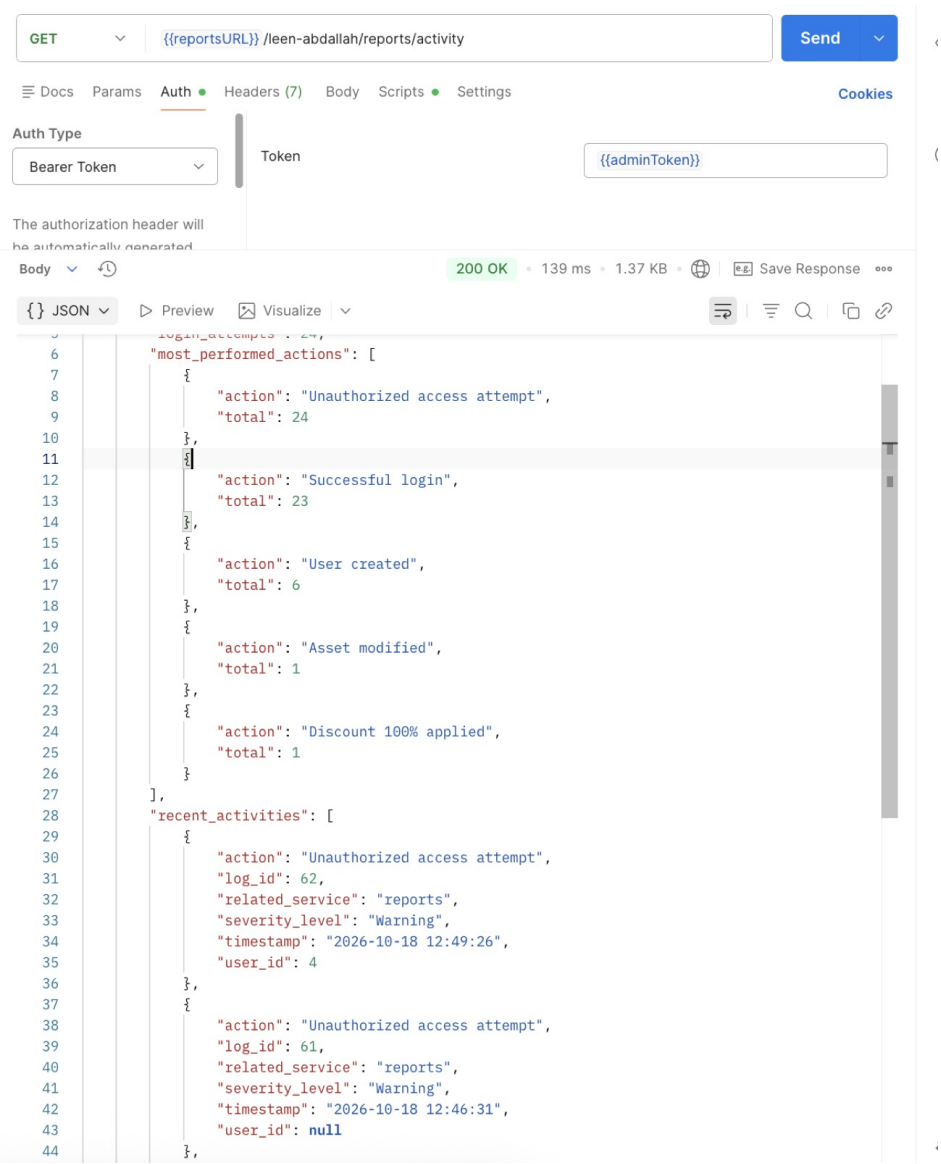


Figure 101: Leen Abdallah: Generate Activity Report: recent activities section, 200

5.7 Service 7: Vulnerability Management (Adam Yahya)

Responsibility. Service 7 associates submitted vulnerability records with assets and tracks remediation. Administrators create and delete records; analysts can update them; auditors can retrieve and search them. The service validates metadata but does not run a vulnerability scanner or verify CVEs against an external database.

API contract. Creation requires `title`, `description`, `cve_id`, `cvss_score`, `severity_level`, `asset_id`, `discovery_source`. Search filters CVE, severity, asset and status. Update accepts description, severity, status, CVSS score and resolution date. The lifecycle is Open → Under Investigation → In Progress → Resolved → Closed; an invalid jump returns `400`.

Persistence and severity. `vulnerabilities.asset_id` references an asset, and each CVE/asset pair is unique. CVSS scores lie between 0 and 10. The model requires the submitted severity to match its score band: Low below 4, Medium from 4 to below 7, High from 7 to below 9, and Critical from 9 to 10. The database stores severity as constrained text. Some sample descriptions do not clearly match the asset type; persistence does not verify the vulnerability.

Implemented APIs.

Method	Endpoint	Access
POST	`/vulnerabilities/AdamYahya`	Administrator
GET	`/vulnerabilities/AdamYahya`	Administrator, Analyst, Auditor
GET	`/vulnerabilities/<id>/AdamYahya`	Administrator, Analyst, Auditor
GET	`/vulnerabilities/search/AdamYahya`	Administrator, Analyst, Auditor
PUT	`/vulnerabilities/<id>/AdamYahya`	Administrator, Analyst
DELETE	`/vulnerabilities/<id>/AdamYahya`	Administrator

Figures 102–106: Creation responses and database views demonstrate Critical, Medium and Low sample records and the constrained severity column. The score/severity relationship is implemented in the model.

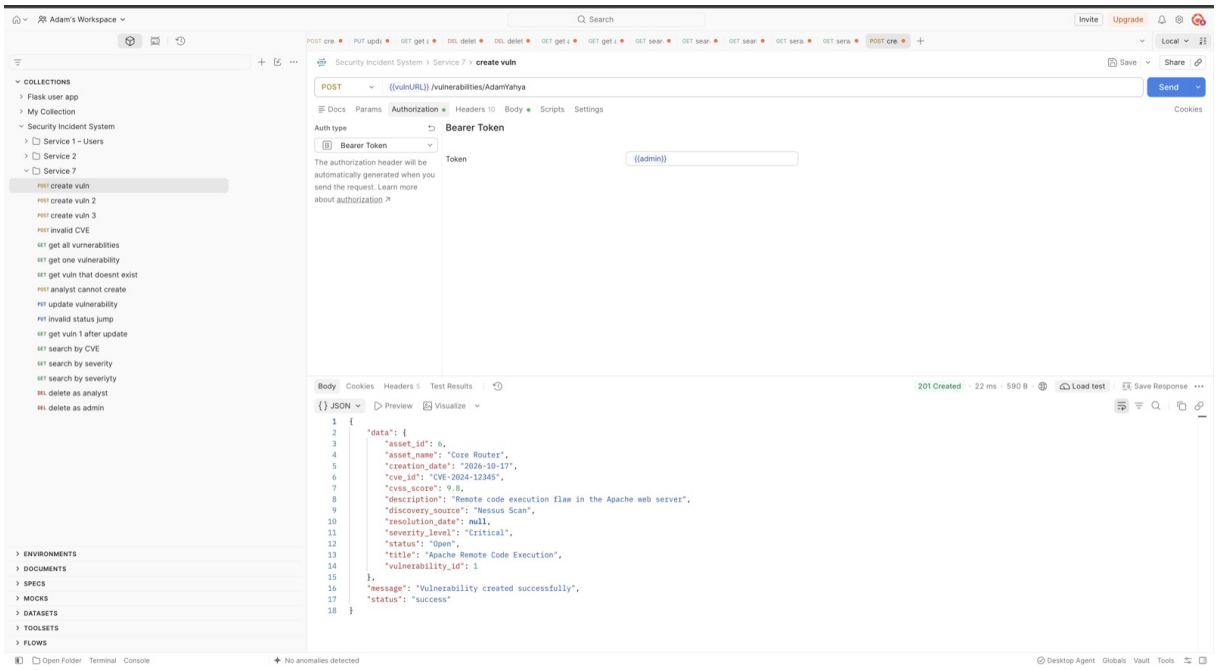


Figure 102: Adam Yahya: Add Vulnerability (POST /vulnerabilities/AdamYahya): Critical vulnerability, 201

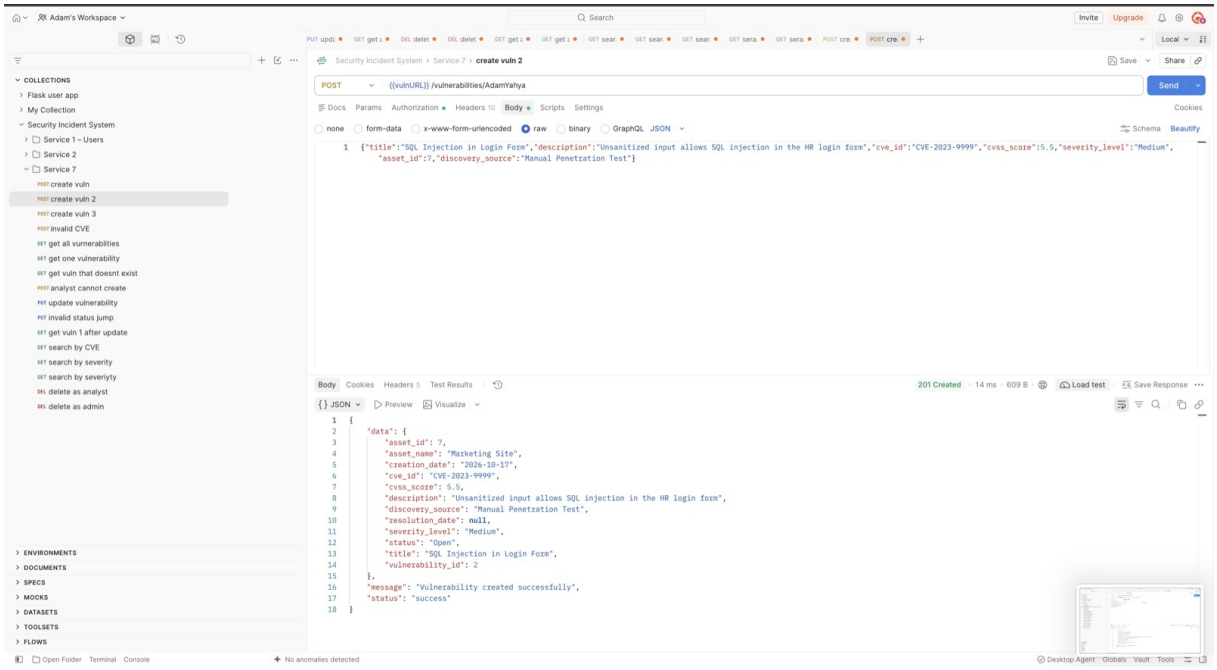


Figure 103: Adam Yahya: Add Vulnerability: Medium vulnerability, 201

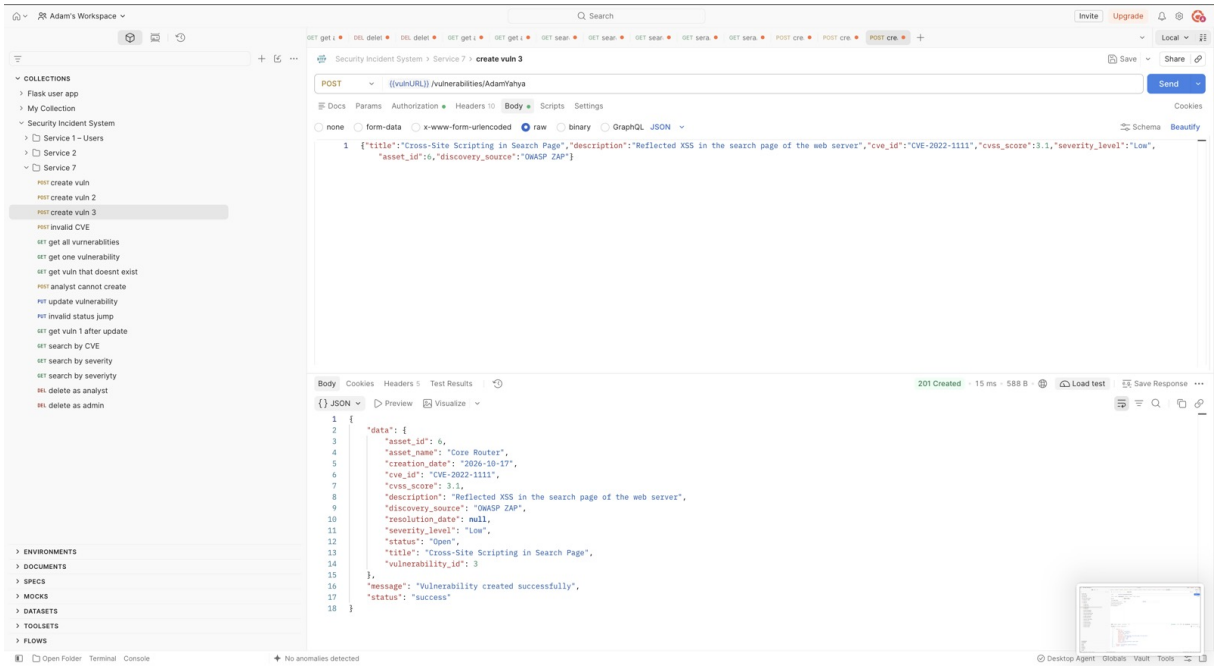


Figure 104: Adam Yahya: Add Vulnerability: Low vulnerability, 201

```
aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT vulnerability_id, title, cve_id, cvss_score, severity_level, asset_id, status FROM vulnerabilities;"
vulnerability_id  title                          cve_id          cvss_score  severity_level  asset_id  status
----------------  -----------------------------  --------------  ----------  --------------  --------  ------
1                 Apache Remote Code Execution   CVE-2024-12345  9.8         Critical        6         Open
2                 SQL Injection in Login Form    CVE-2023-9999   5.5         Medium          7         Open
3                 Cross-Site Scripting in Search Page  CVE-2022-1111  3.1  Low             6         Open
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 105: Adam Yahya: Database: vulnerabilities table with the inserted records

```
aazelmac@Adams-MacBook-Air Lab6 % sqlite3 database/security_incident.db ".schema vulnerabilities"
CREATE TABLE vulnerabilities (
    vulnerability_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title            TEXT NOT NULL,
    description      TEXT NOT NULL,
    cve_id           TEXT NOT NULL,
    cvss_score       REAL NOT NULL CHECK (cvss_score >= 0.0 AND cvss_score <= 10.0),
    severity_level   TEXT NOT NULL
                     CHECK (severity_level IN ('Low', 'Medium', 'High', 'Critical')),
    asset_id         INTEGER NOT NULL REFERENCES assets(asset_id),
    discovery_source TEXT NOT NULL,
    status           TEXT NOT NULL DEFAULT 'Open'
                     CHECK (status IN ('Open', 'Under Investigation', 'In Progress',
                                       'Resolved', 'Closed')),
    creation_date    TEXT NOT NULL DEFAULT (date('now')),
    resolution_date  TEXT,
    UNIQUE (cve_id, asset_id)
);
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 106: Adam Yahya: Database: schema showing how severity is stored (TEXT with a CHECK constraint)

Figures 107–111: Retrieval provides the before state for vulnerability 1; an analyst update and subsequent GET provide the matching after state. An absent ID returns 404.

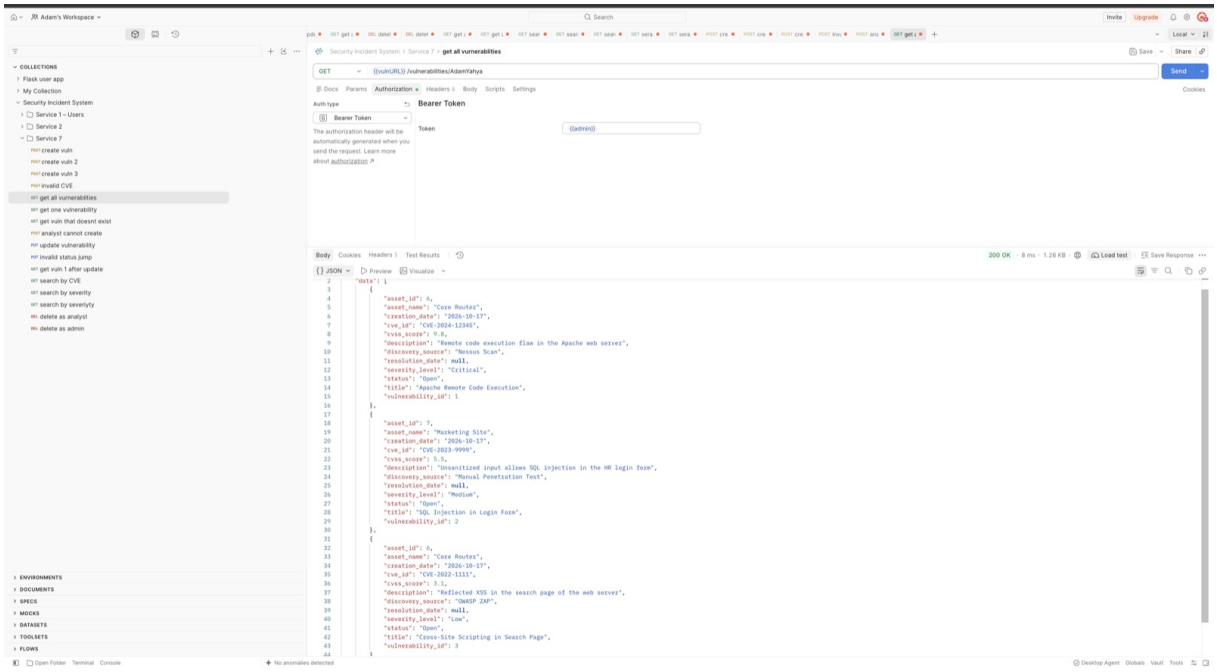


Figure 107: Adam Yahya: Get All Vulnerabilities (GET /vulnerabilities/AdamYahya): 200

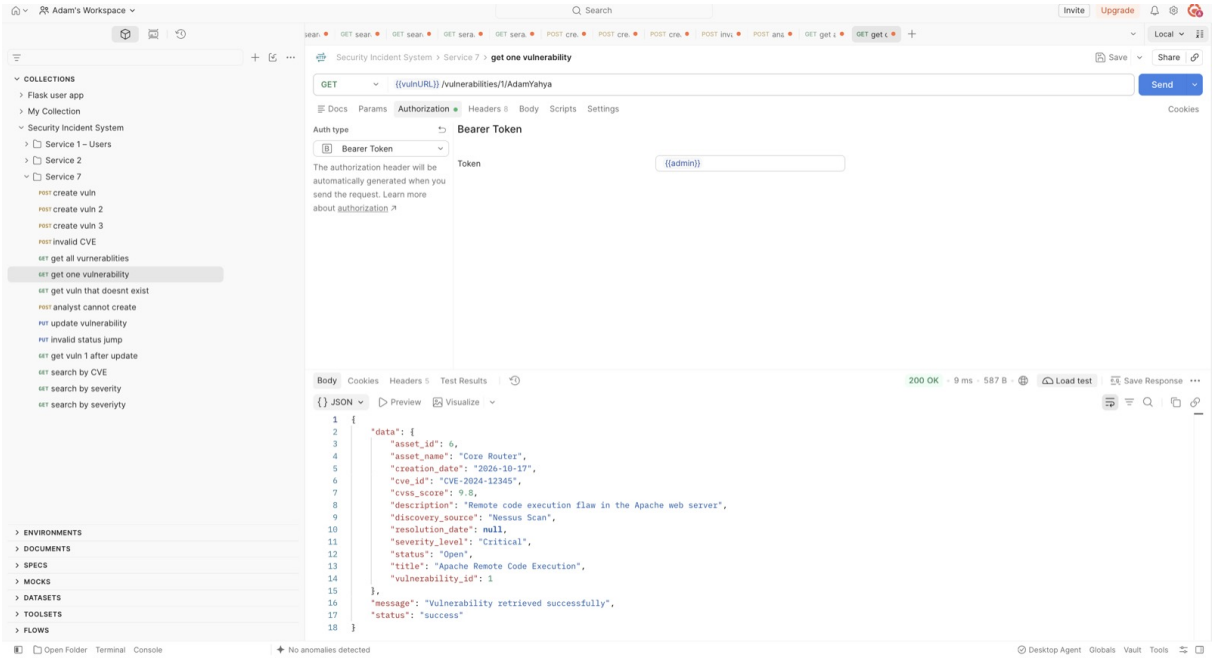


Figure 108: Adam Yahya: Get Vulnerability: vulnerability 1 before the update, 200

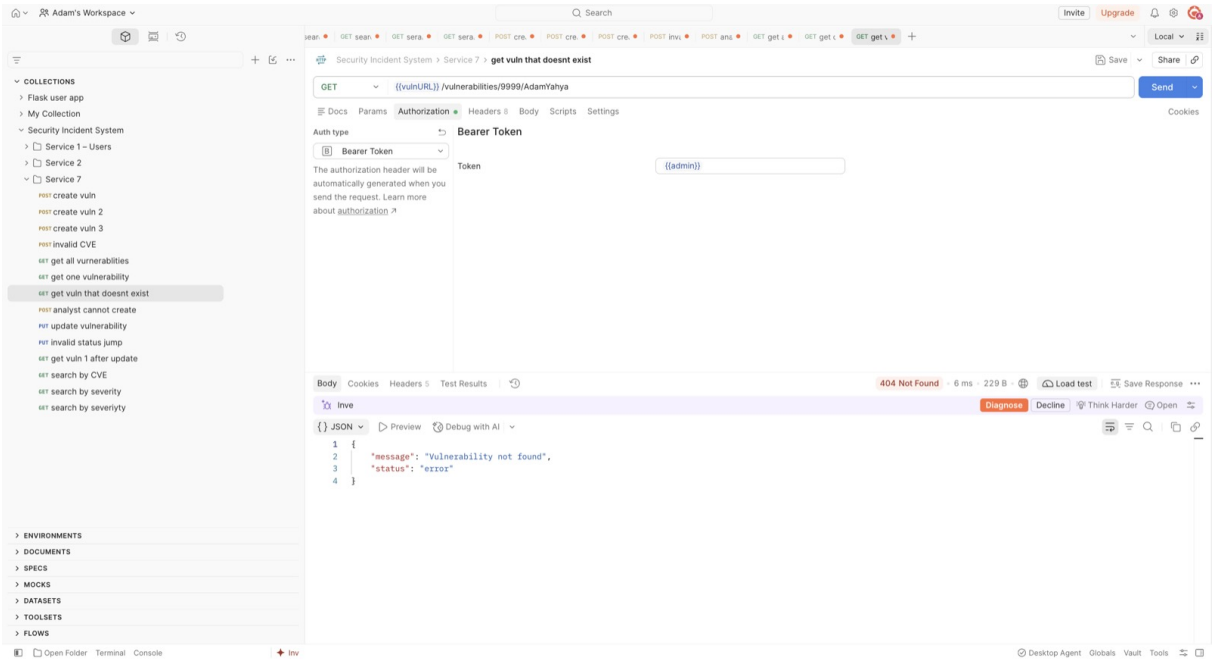


Figure 109: Adam Yahya: Get Vulnerability: invalid ID, 404

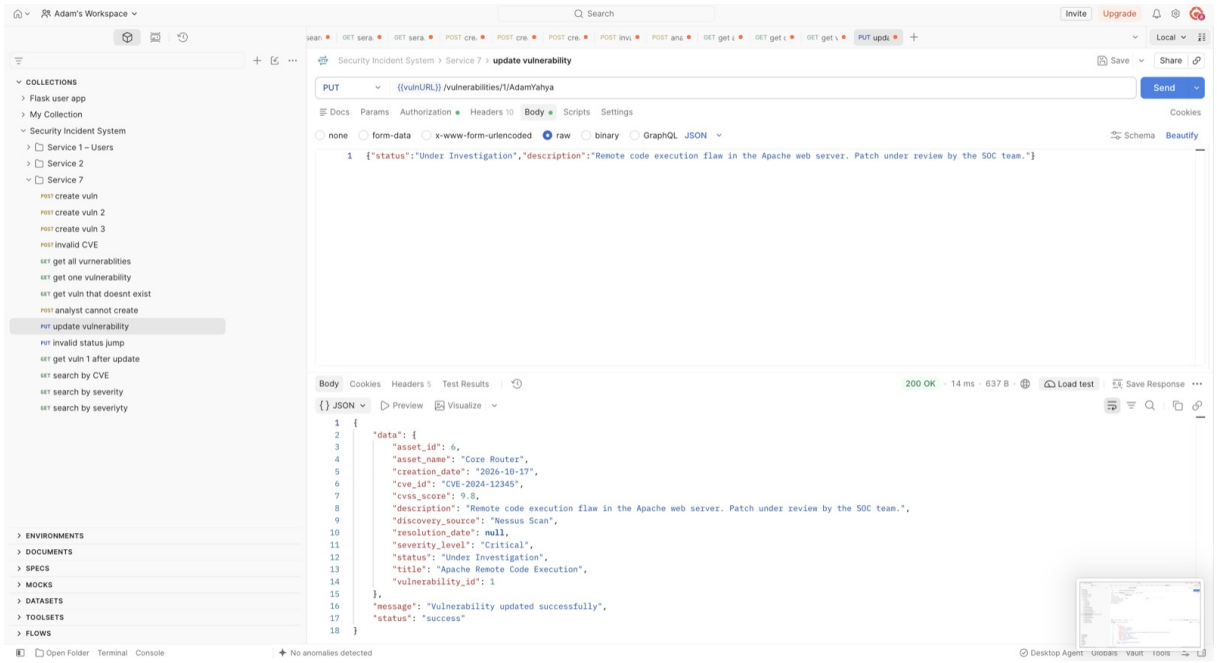


Figure 110: Adam Yahya: Update Vulnerability (PUT /vulnerabilities/:id/AdamYahya): analyst updates status, 200

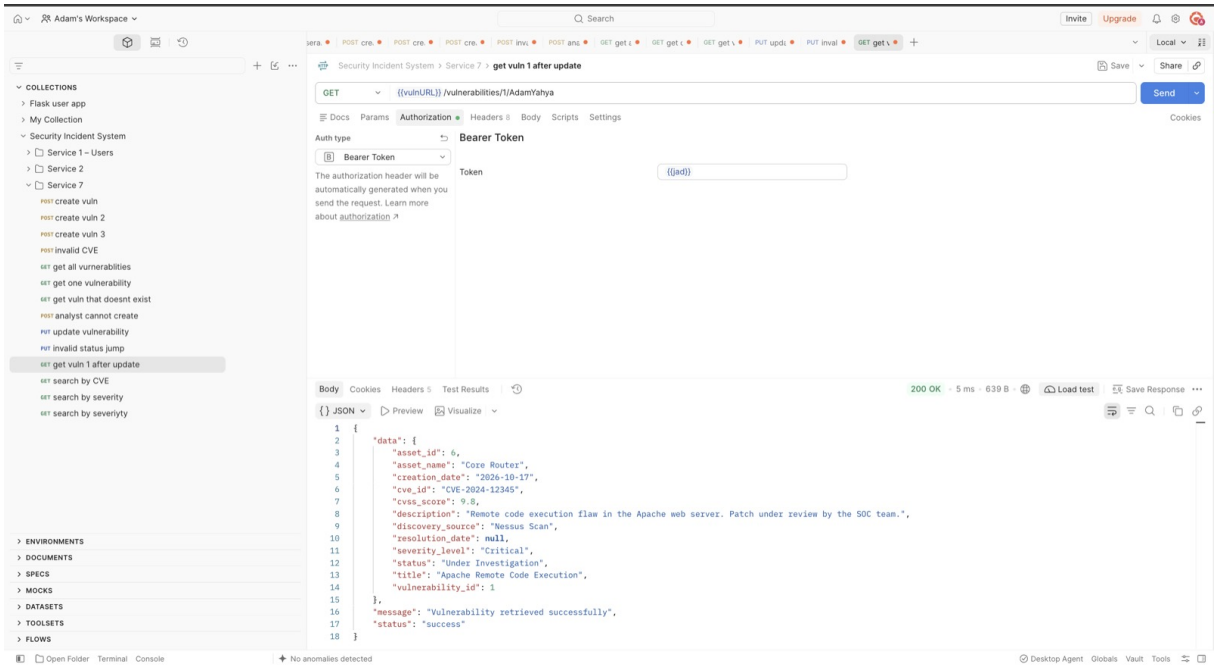


Figure 111: Adam Yahya: Get Vulnerability: vulnerability 1 after the update, 200

Figures 112–115: The four searches demonstrate CVE, severity, asset and status filters against the captured dataset.

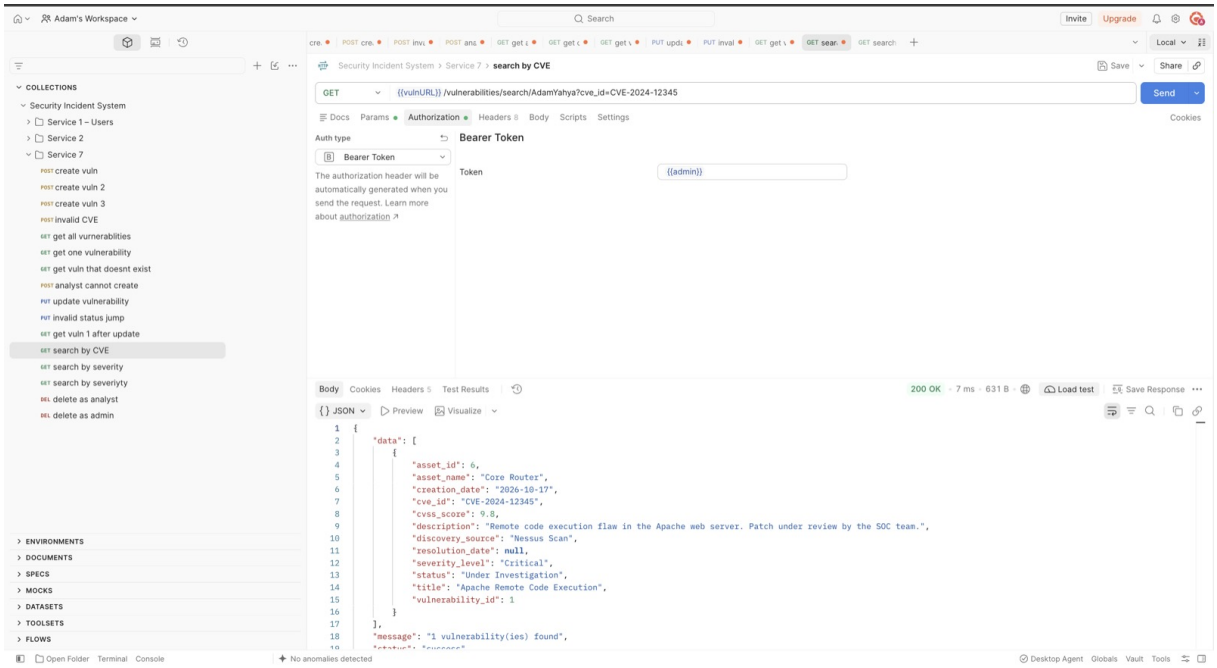


Figure 112: Adam Yahya: Search Vulnerabilities (GET /vulnerabilities/search/AdamYahya): by CVE identifier, 200

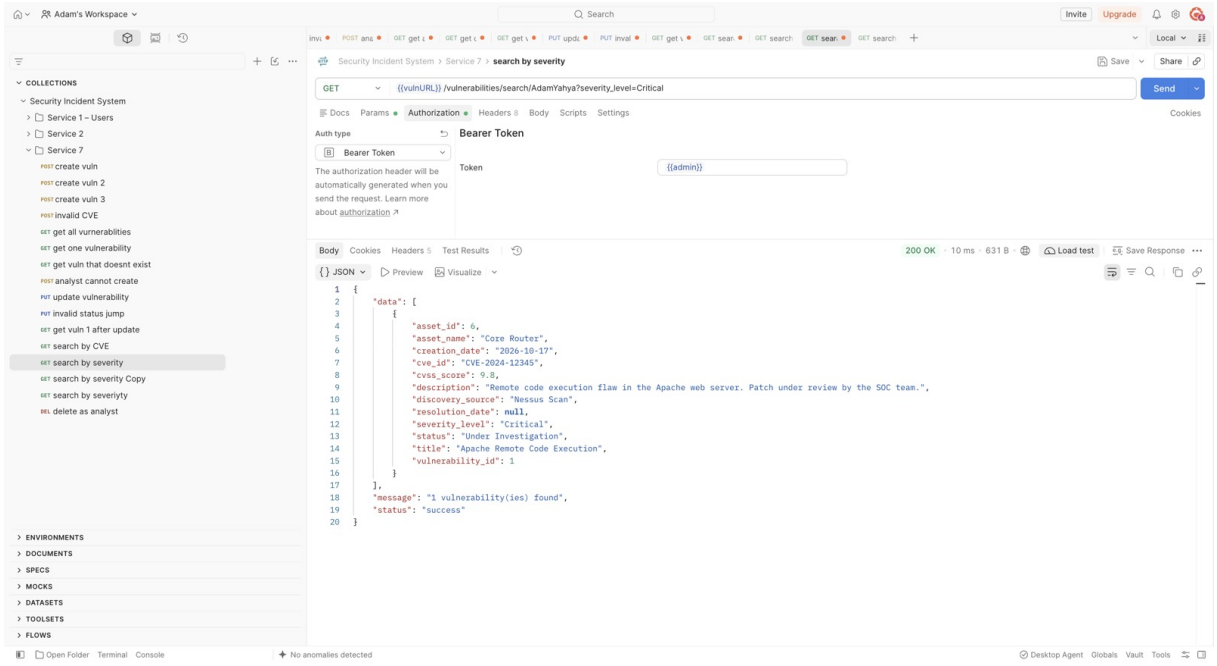


Figure 113: Adam Yahya: Search Vulnerabilities: by severity, 200

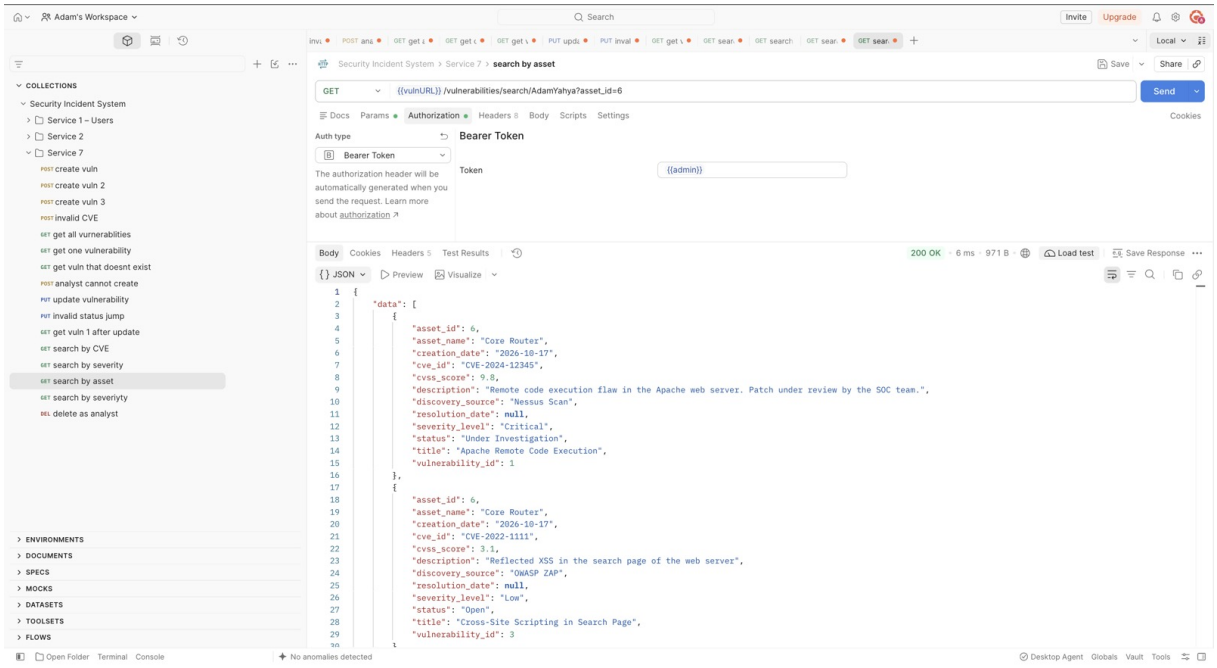


Figure 114: Adam Yahya: Search Vulnerabilities: by asset, 200

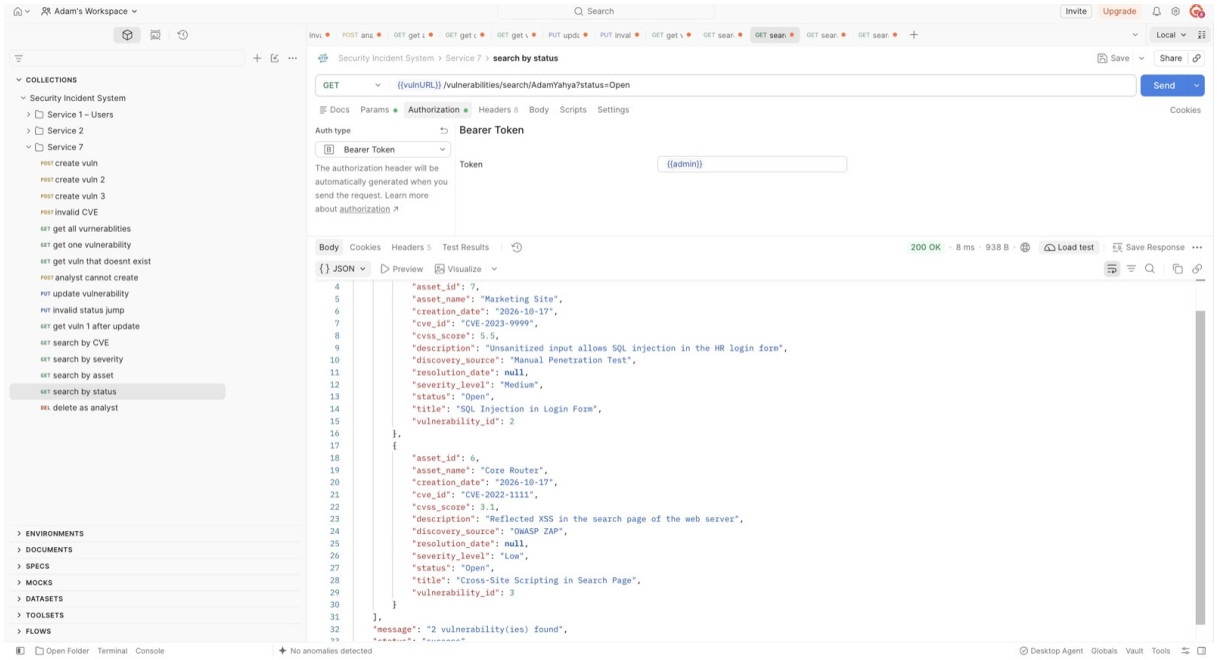


Figure 115: Adam Yahya: Search Vulnerabilities: by status, 200

Figures 116–117: Administrator deletion is followed by stored-row evidence and a database count comparison. These captures establish removal of the selected record, without implying deletion of its asset.

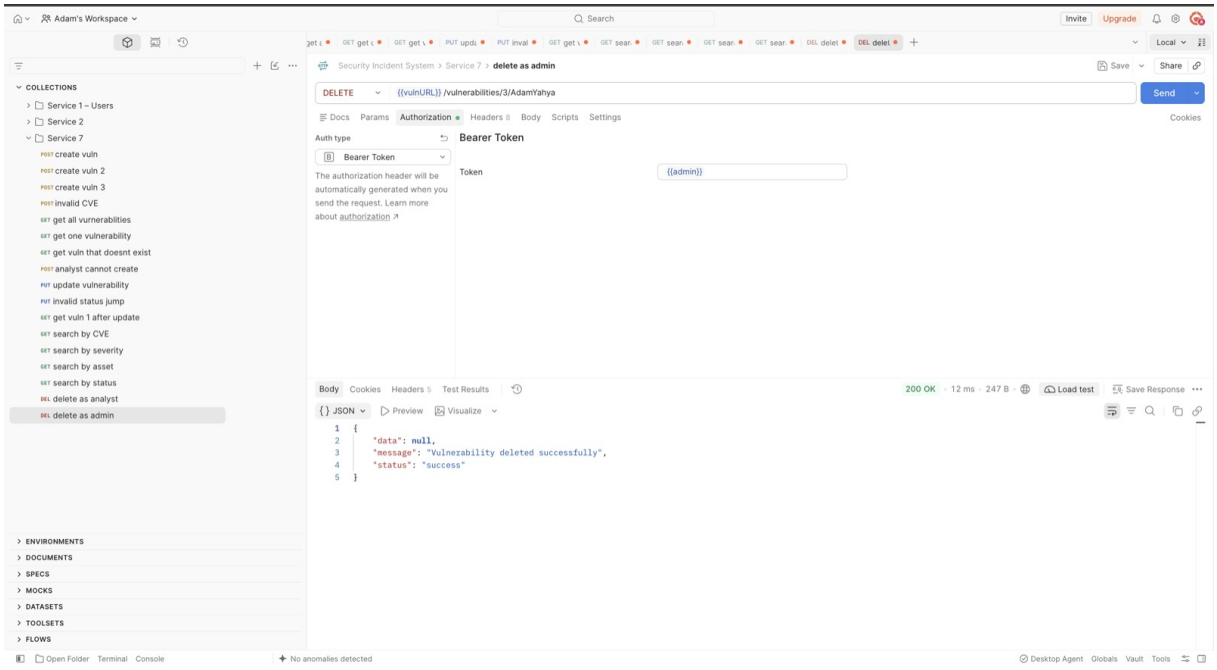


Figure 116: Adam Yahya: Delete Vulnerability (DELETE /vulnerabilities/:id/AdamYahya): administrator, 200

```
[aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT vulnerability_id, title, ]
cve_id, severity_level, asset_id, status FROM vulnerabilities;"
vulnerability_id  title                       cve_id          severity_level  asset_id  status
----------------  --------------------------  --------------  --------------  --------  -------------------
1                 Apache Remote Code Execution  CVE-2024-12345  Critical        6         Under Investigation
2                 SQL Injection in Login Form   CVE-2023-9999   Medium          7         Open
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 117: Adam Yahya: Database: vulnerabilities table after the deletion

```
);
[aazelmac@Adams-MacBook-Air Lab6 % sqlite3 -header -column database/security_incident.db "SELECT (SELECT COUNT(*) FROM vul]
nerabilities) AS vulnerabilities_left, (SELECT COUNT(*) FROM assets) AS assets_left;"
vulnerabilities_left  assets_left
--------------------  -----------
2                     8
aazelmac@Adams-MacBook-Air Lab6 %
```

Figure 118: Adam Yahya: Database: row counts after vulnerability deletion

5.8 Service 8: Notification and Alert Management (Youssef Al Hajj Youness)

Responsibility. Service 8 stores notifications and tracks whether users have read them. Incident assignment creates an Incident Assignment notification automatically. Administrators can create the other demonstrated notification types manually.

API contract. Creation requires `user_id`, `title`, `message`, `notification_type`, `severity_level`. Supported types are Incident Assignment, Security Alert, System Notification, Threat Alert and Vulnerability Alert. Search filters user, type, severity and status. Analysts are restricted to their own search results; Regular Users cannot use the search route but can retrieve and mark their own notifications as read. Auditors can read and search without marking records as read.

Persistence. `notifications.user_id` references its recipient and cascades on user deletion. Its optional incident link becomes NULL if the incident is deleted. The database stores `is_read` as 0 or 1; the API exposes Unread or Read. The first successful read sets `read_date`; subsequent reads preserve it. The added personal-history, single-record and read captures demonstrate this workflow for a Regular User.

Implemented APIs.

Method	Endpoint (prefix `/Youssef-Al-Hajj-Youness`)	Access
POST	`/notifications`	Administrator
GET	`/notifications/user/<user_id>`	Administrator, Auditor; Analyst and Regular User own only
GET	`/notifications/<id>`	Administrator, Auditor; Analyst and Regular User own only
PUT	`/notifications/<id>/read`	Administrator, Analyst, Regular User
GET	`/notifications/search`	Administrator, Analyst (own), Auditor
DELETE	`/notifications/<id>`	Administrator

Figures 119–126: The automatic-assignment record is retrieved through the specific-notification GET API. Manual creation and history captures demonstrate several notification types and an empty history for a user without notifications.

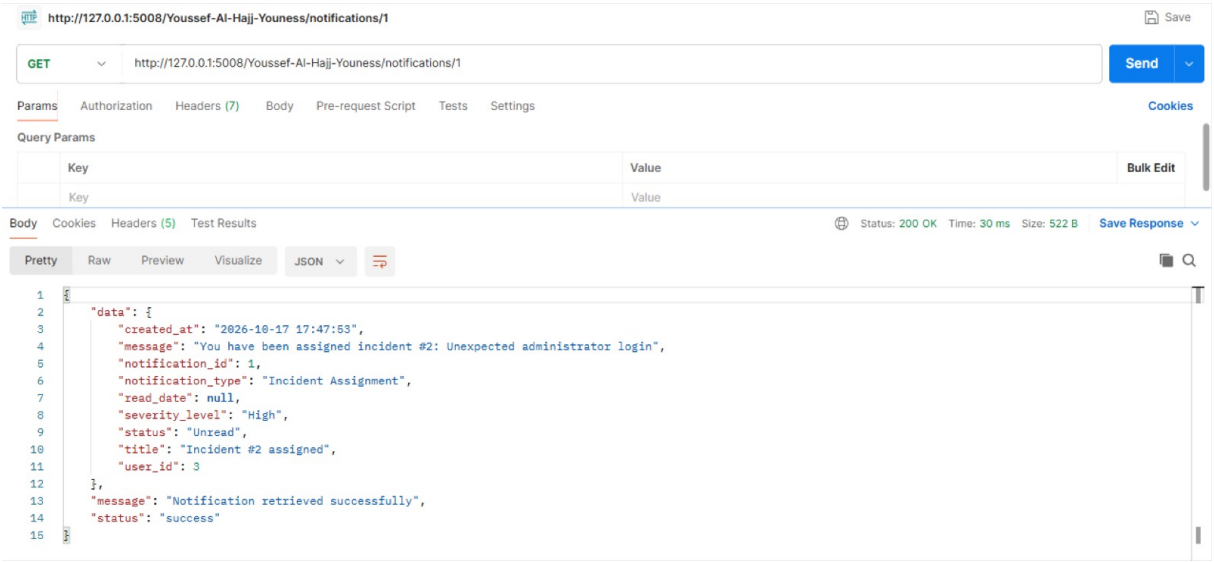


Figure 119: Youssef Al Hajj Youness: GET /Youssef-Al-Hajj-Youness/notifications/1: retrieves the incident-assignment notification, 200

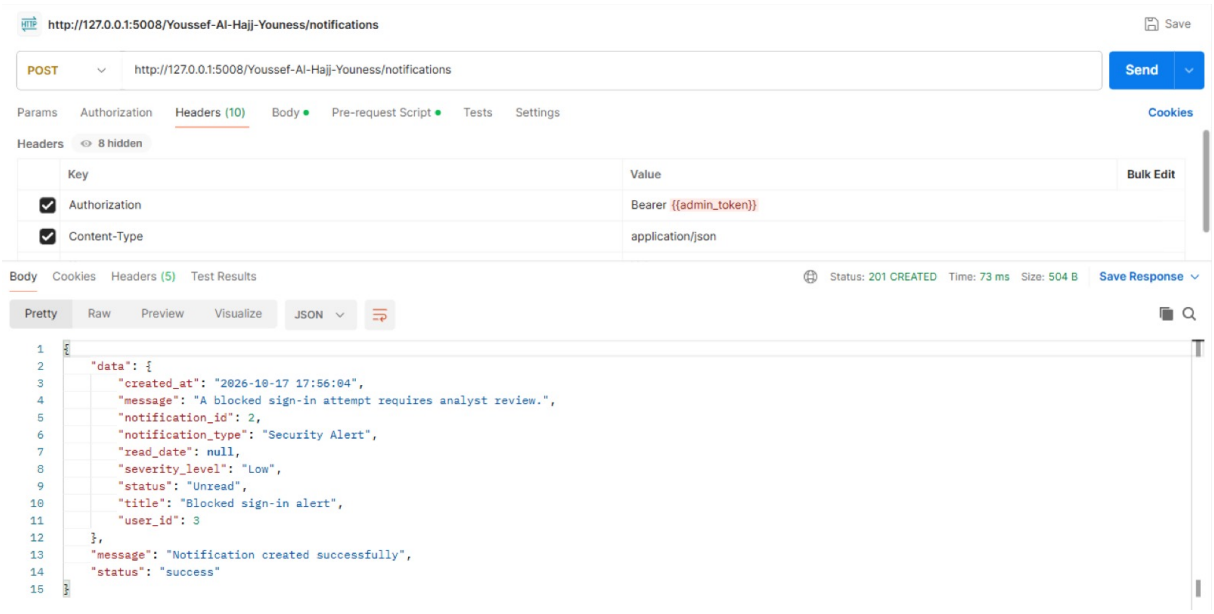


Figure 120: Youssef Al Hajj Youness: Create Notification (POST /Youssef-Al-Hajj-Youness/notifications): 201

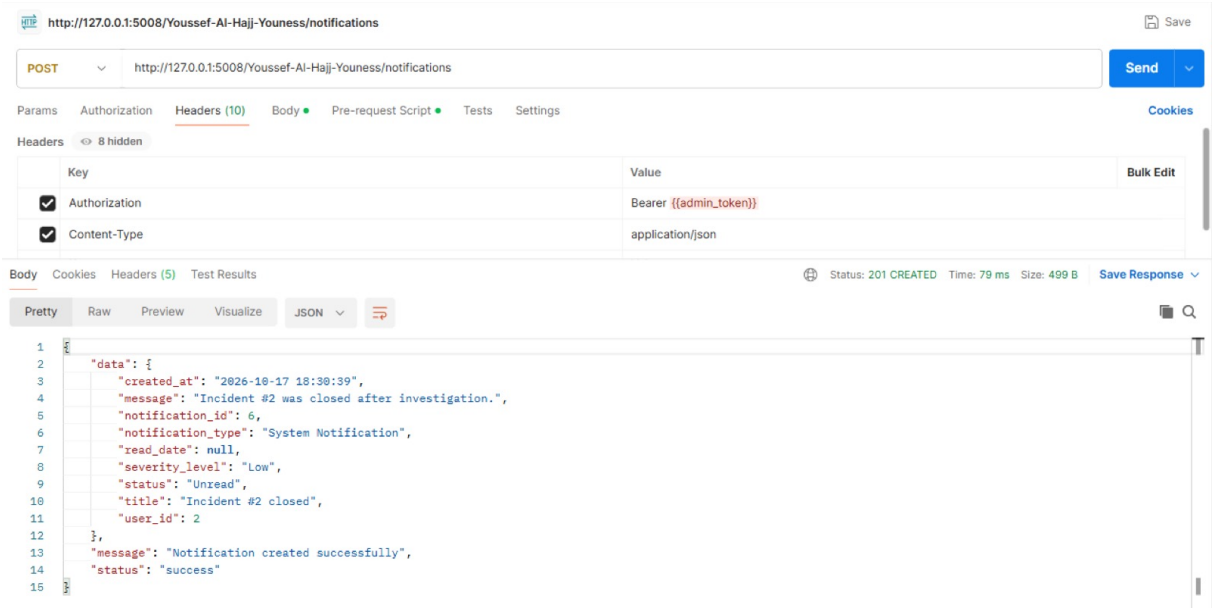


Figure 121: Youssef Al Hajj Youness: Create Notification: personal notification, 201

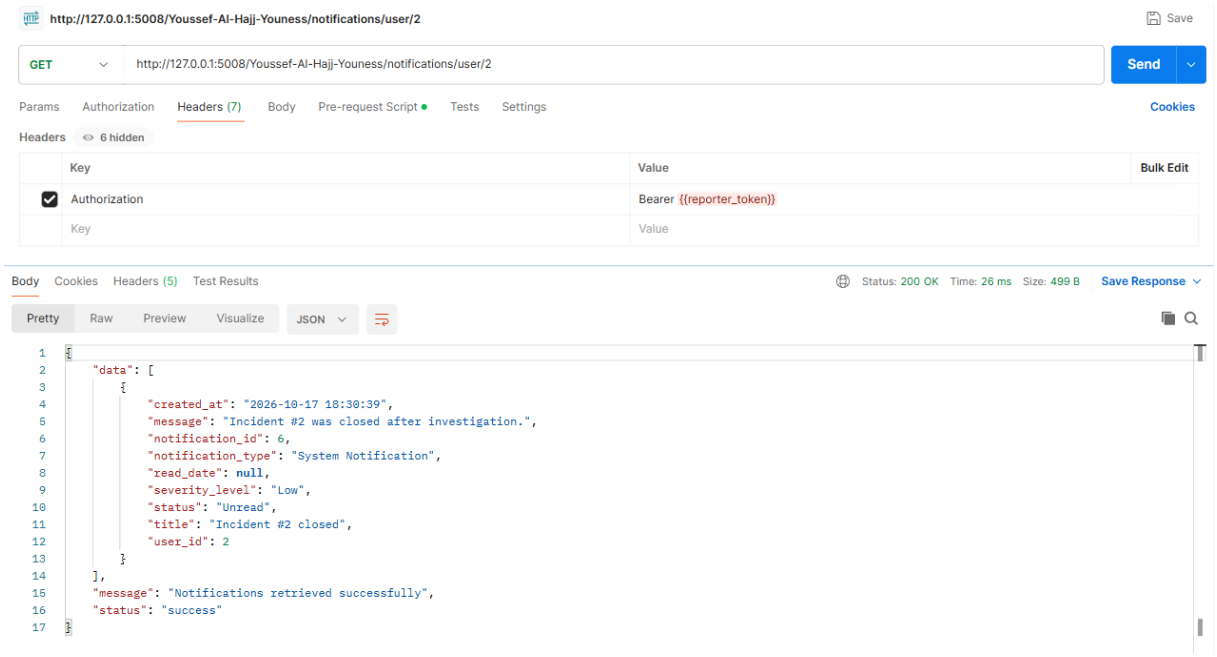


Figure 122: Youssef Al Hajj Youness: Regular User retrieves their personal notification history, 200

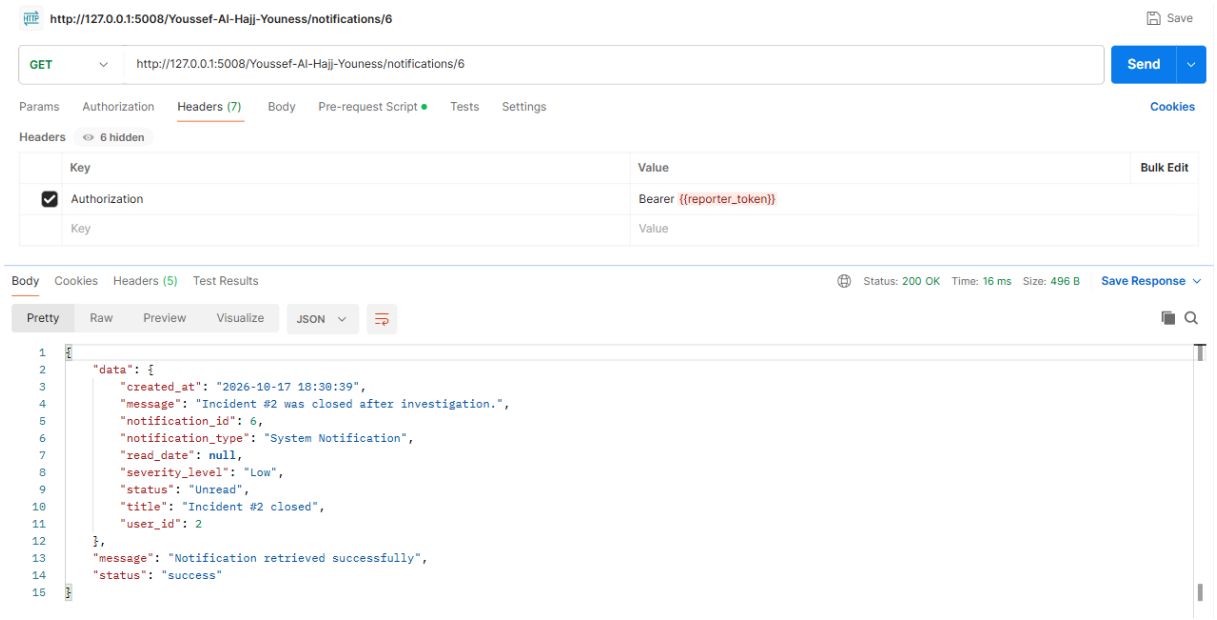


Figure 123: Youssef Al Hajj Youness: Regular User retrieves their own notification 6, 200

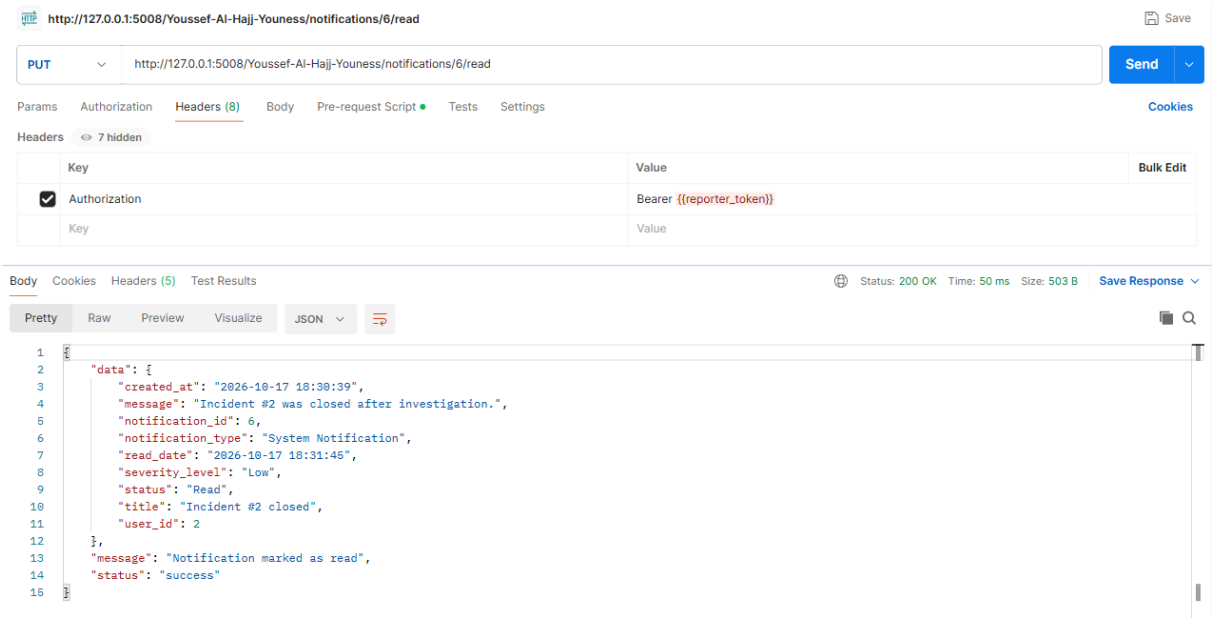


Figure 124: Youssef Al Hajj Youness: Regular User marks their own notification 6 as read, 200

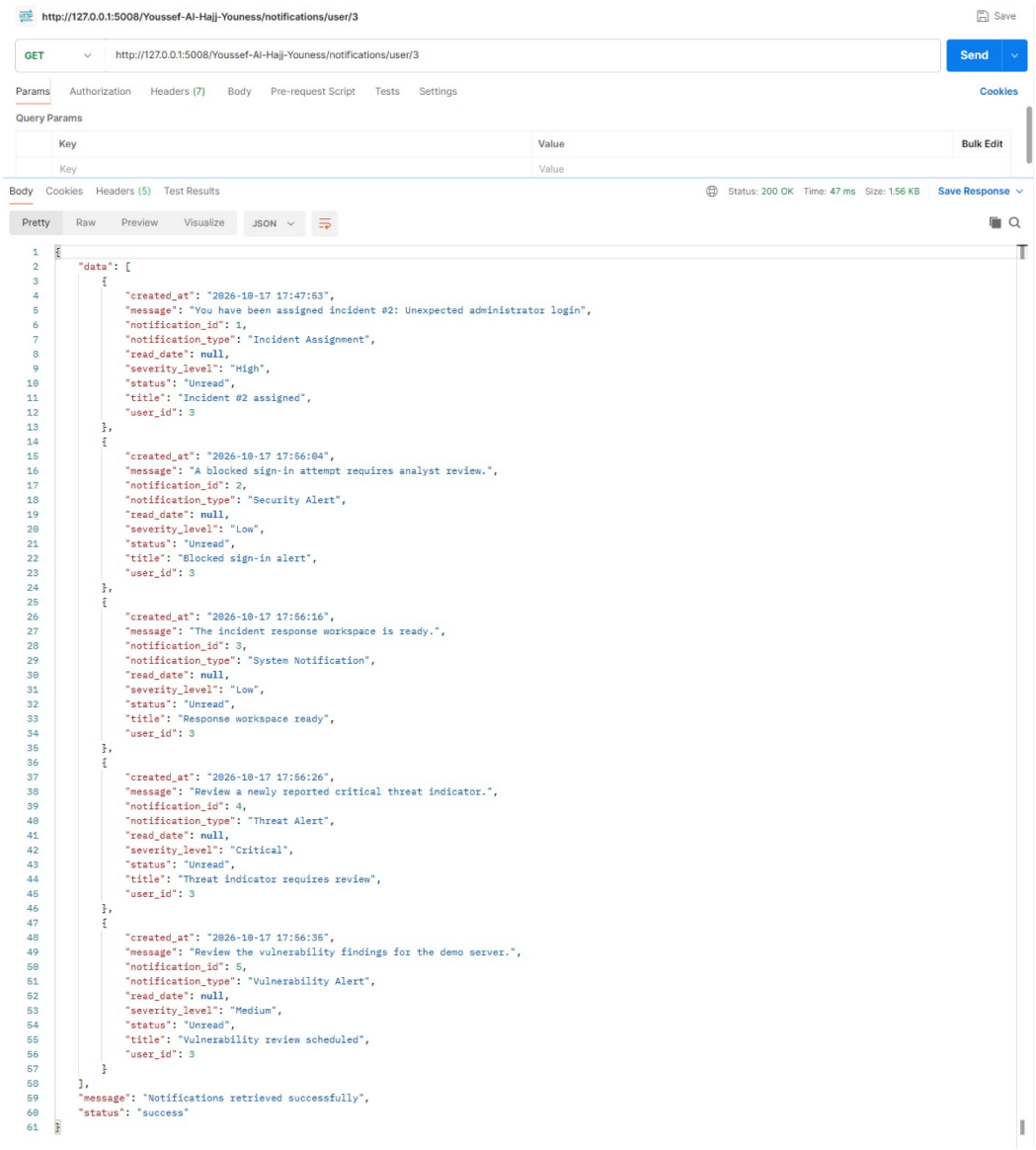


Figure 125: Youssef Al Hajj Youness: Retrieve User Notifications: history with several types, 200

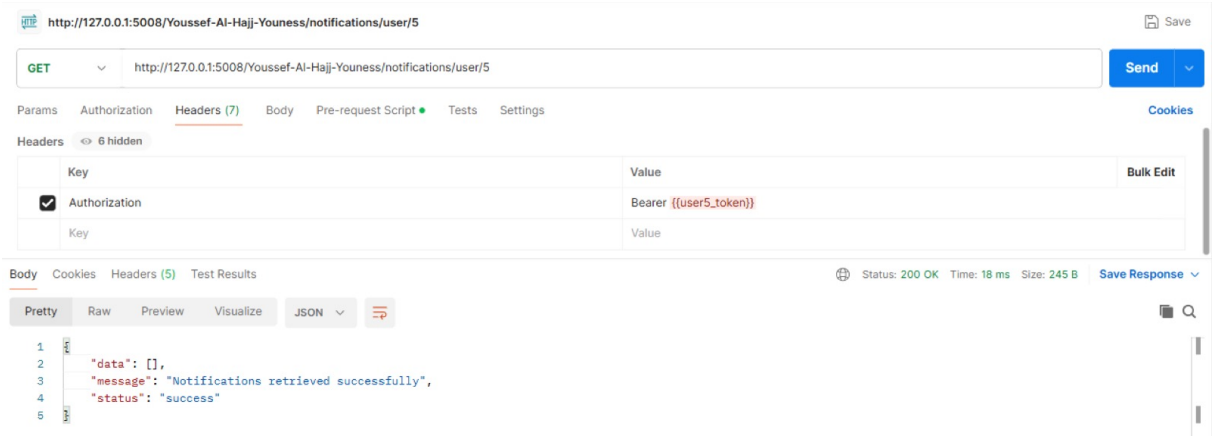


Figure 126: Youssef Al Hajj Youness: Retrieve User Notifications: user without notifications, empty list, 200

Figures 127–128: The read response and paired database query show the Unread-to-Read change.

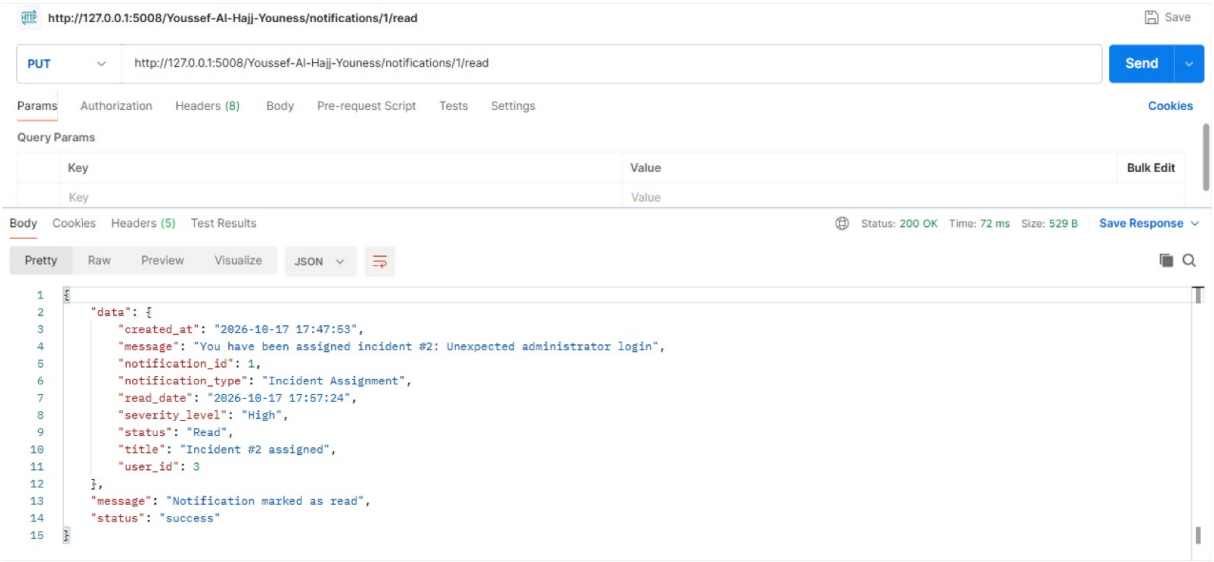


Figure 127: Youssef Al Hajj Youness: Mark Notification as Read (PUT .../notifications/:id/read): 200

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
QUERY FILE: 13_notification_before.sql

SELECT 'BEFORE' AS phase, notification_id AS id, user_id,
       CASE is_read WHEN 0 THEN 'Unread' ELSE 'Read' END AS status,
       created_at, read_date
FROM notifications
WHERE notification_type = 'Incident Assignment'
  AND user_id = (SELECT user_id FROM users WHERE username = 'youssef.younnes')
ORDER BY notification_id;
phase   id  user_id  status  created_at           read_date
------  --  -------  ------  -------------------  ---------
BEFORE  1   3        Unread  2026-10-17 17:47:53  NULL
(1 rows)

DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
QUERY FILE: 13_notification_after.sql

SELECT 'AFTER' AS phase, notification_id AS id, user_id,
       CASE is_read WHEN 0 THEN 'Unread' ELSE 'Read' END AS status,
       created_at, read_date
FROM notifications
WHERE notification_type = 'Incident Assignment'
  AND user_id = (SELECT user_id FROM users WHERE username = 'youssef.younnes')
ORDER BY notification_id;
phase  id  user_id  status  created_at           read_date
-----  --  -------  ------  -------------------  -------------------
AFTER  1   3        Read    2026-10-17 17:47:53  2026-10-17 17:57:24
(1 rows)
```

Figure 128: Youssef Al Hajj Youness: Database: notification record before and after being read

Figures 129–132: Search examples demonstrate combined filters and analyst scoping. The terminal verification repeats the read request and shows the original timestamp is retained.

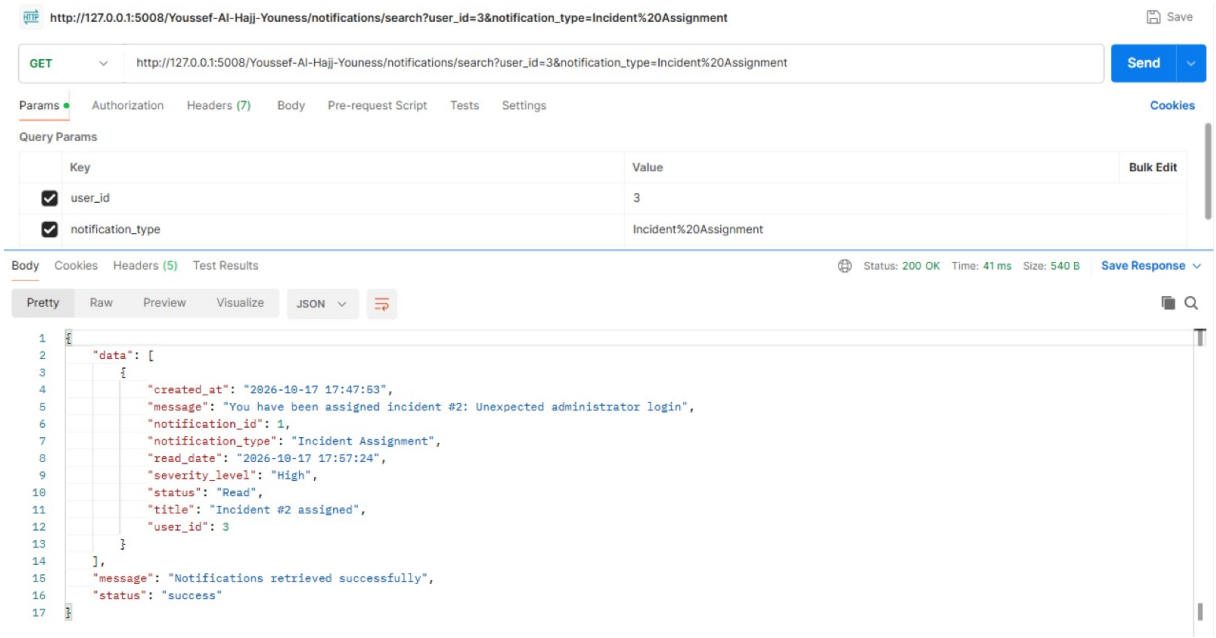


Figure 129: Youssef Al Hajj Youness: Search Notifications: by user and type, 200

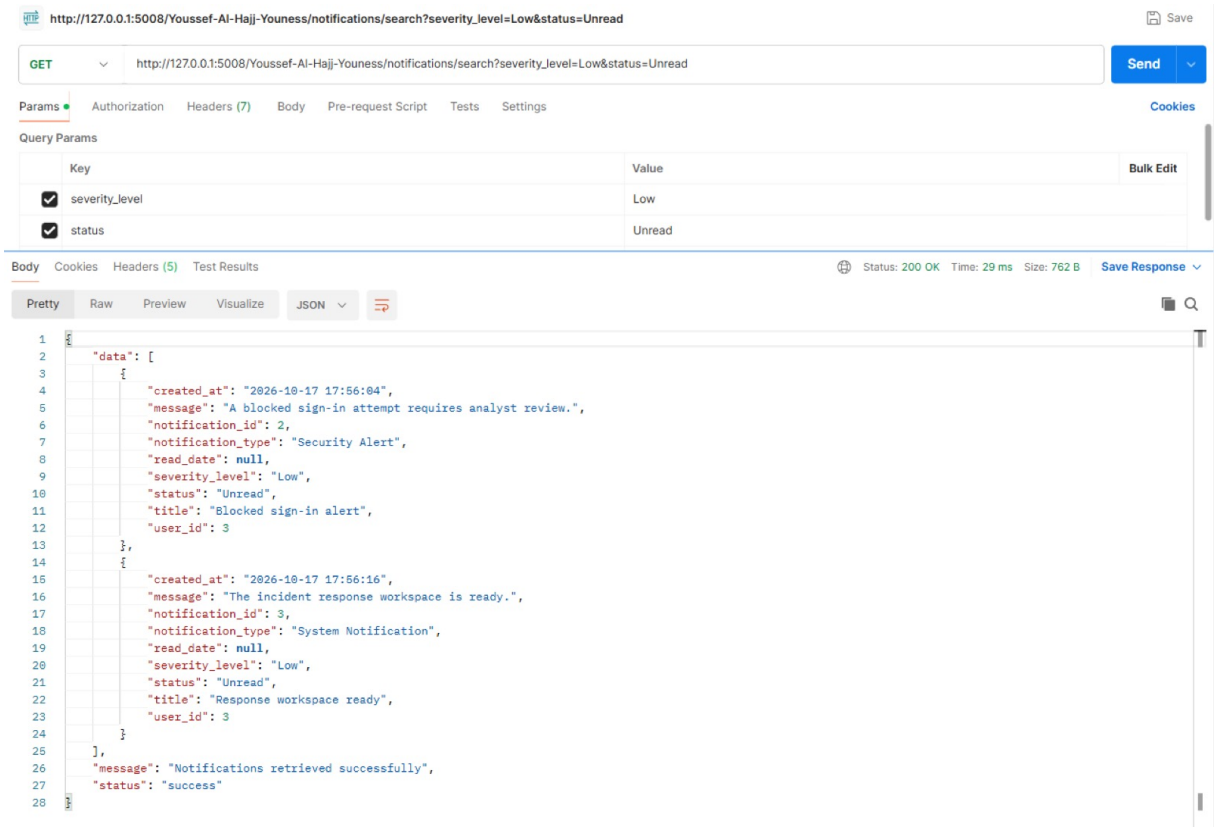


Figure 130: Youssef Al Hajj Youness: Search Notifications: by severity and status, 200

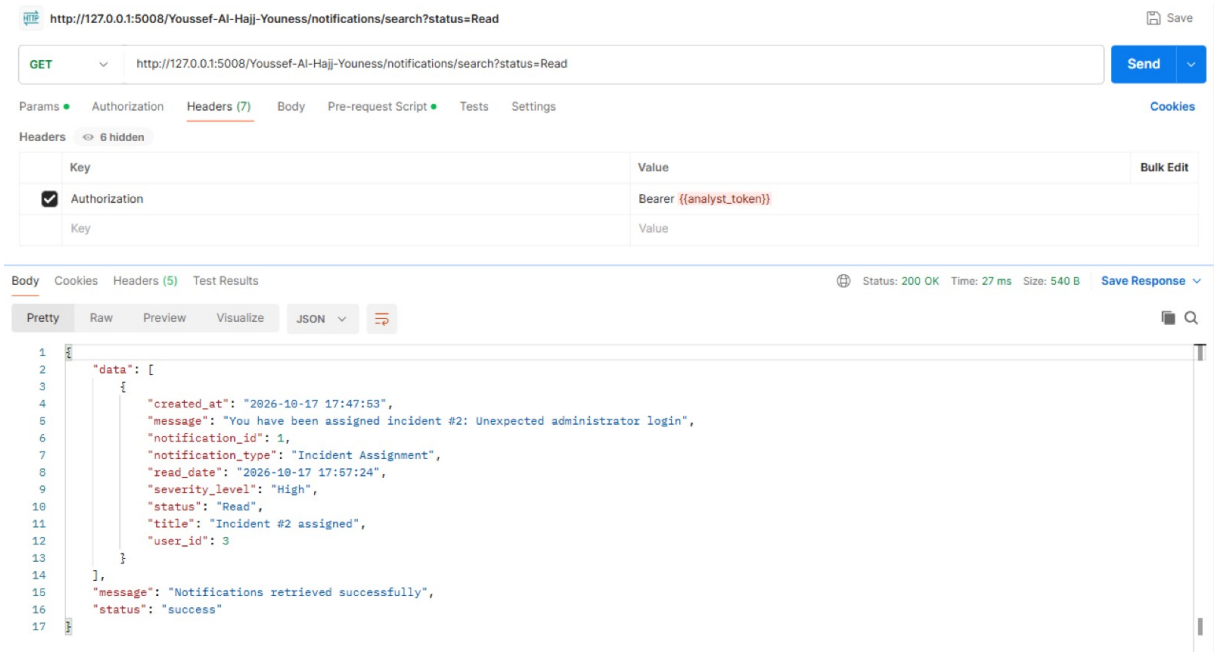


Figure 131: Youssef Al Hajj Youness: Search Notifications: analyst search limited to own notifications, 200

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
CAPTURE FILE: clarity-review-20261017\65_timestamp_evidence.json

BEFORE (2026-10-17 20:31:14.626539 UTC)
SELECT notification_id, user_id, is_read, read_date
FROM notifications WHERE notification_id = 1;
notification_id  user_id  is_read  read_date
---------------  -------  -------  -------------------
1                3        1        2026-10-17 17:57:24
(1 rows)

PUT http://127.0.0.1:5008/Youssef-Al-Hajj-Youness/notifications/1/read
HTTP response: 200 OK

AFTER (2026-10-17 20:31:14.656813 UTC)
SELECT notification_id, user_id, is_read, read_date
FROM notifications WHERE notification_id = 1;
notification_id  user_id  is_read  read_date
---------------  -------  -------  -------------------
1                3        1        2026-10-17 17:57:24
(1 rows)
```

Figure 132: Youssef Al Hajj Youness: Terminal verification: repeated PUT to mark notification 1 as read returns 200 and preserves its first read timestamp

Figures 133–135: Administrator deletion is confirmed by a later 404; the database view also shows structure and audit entries.

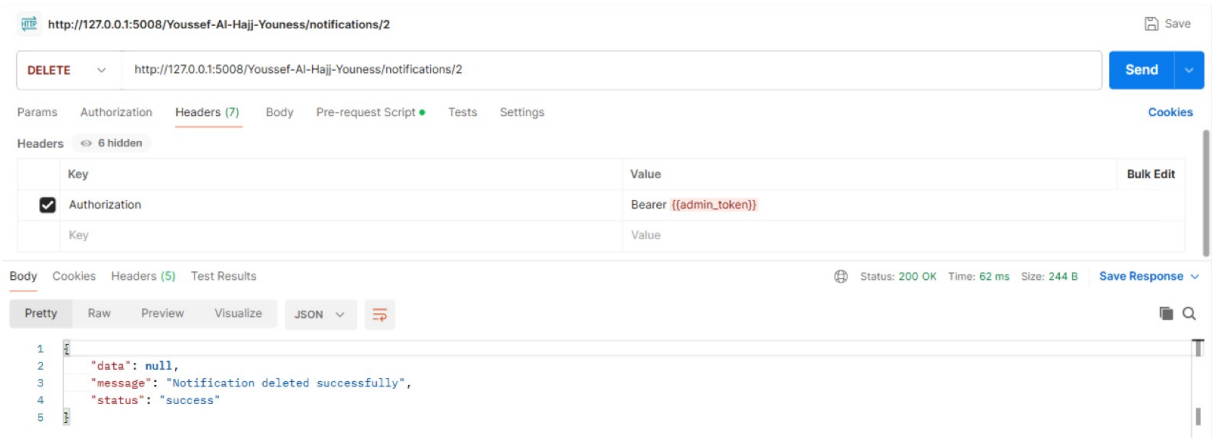


Figure 133: Youssef Al Hajj Youness: Delete Notification (DELETE .../notifications/:id): administrator, 200

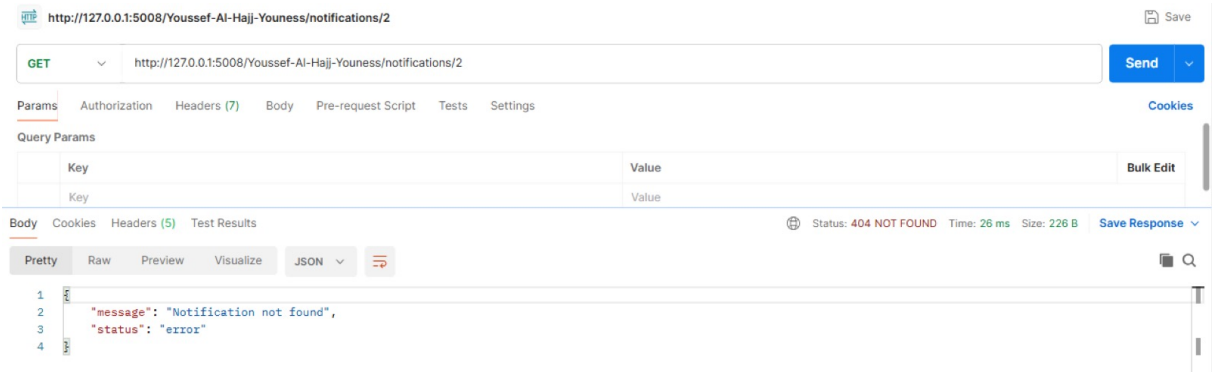


Figure 134: Youssef Al Hajj Youness: Retrieve Specific Notification after deletion, 404

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db

SELECT sql FROM sqlite_master WHERE type='table' AND name='notifications';
CREATE TABLE notifications (
    notification_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id         INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    incident_id     INTEGER REFERENCES incidents(incident_id) ON DELETE SET NULL,
    title           TEXT NOT NULL,
    message         TEXT NOT NULL,
    notification_type TEXT NOT NULL DEFAULT 'System Notification'
                      CHECK (notification_type IN ('Incident Assignment', 'Security Alert',
                                                   'System Notification', 'Threat Alert',
                                                   'Vulnerability Alert')),
    severity_level  TEXT NOT NULL DEFAULT 'Low'
                    CHECK (severity_level IN ('Low', 'Medium', 'High', 'Critical')),
    is_read         INTEGER NOT NULL DEFAULT 0 CHECK (is_read IN (0, 1)),
    created_at      TEXT NOT NULL DEFAULT (datetime('now')),
    read_date       TEXT
);

SELECT n.notification_id AS id, n.user_id, u.username, n.incident_id,
       n.notification_type AS type, n.severity_level AS severity, n.is_read, n.read_date
FROM notifications n JOIN users u USING(user_id) ORDER BY n.notification_id;
id  user_id  username         incident_id  type                 severity  is_read  read_date
--  -------  ---------------  -----------  -------------------  --------  -------  -------------------
1   3        youssef.younnes  2            Incident Assignment  High      1        2026-10-17 17:57:24
3   3        youssef.younnes  NULL         System Notification  Low       0        NULL
4   3        youssef.younnes  NULL         Threat Alert         Critical  0        NULL
5   3        youssef.younnes  NULL         Vulnerability Alert  Medium    0        NULL
6   2        yy123            NULL         System Notification  Low       1        2026-10-17 18:31:45
(5 rows)

SELECT l.log_id, u.role, l.description, l.timestamp
FROM logs l LEFT JOIN users u USING(user_id)
WHERE l.related_service='notifications' AND l.action='Unauthorized access attempt'
ORDER BY l.log_id DESC LIMIT 4;
log_id  role             description                                                                                                  timestamp
------  ---------------  -----------------------------------------------------------------------------------------------------------  -------------------
61      Security Analyst  POST /Youssef-Al-Hajj-Youness/notifications: User does not have permission to perform this operation        2026-10-17 18:33:37
60      Regular User      GET /Youssef-Al-Hajj-Youness/notifications/search: User does not have permission to perform this operation  2026-10-17 18:33:05
59      Auditor           PUT /Youssef-Al-Hajj-Youness/notifications/1/read: User does not have permission to perform this operation  2026-10-17 18:32:49
58      Security Analyst  GET /Youssef-Al-Hajj-Youness/notifications/6: You can only access your own notifications                    2026-10-17 18:32:04
(4 rows)
```

Figure 135: Youssef Al Hajj Youness: Database: notifications table structure and audit entries

The personal-user sequence is shown in Figures 122, 123 and 124. These separate requests demonstrate own-history retrieval, specific-record access and marking the same personal notification as read.

5.9 Service 9: Security Playbook and Response Automation (Youssef Al Hajj Youness)

Responsibility. Service 9 stores response playbooks, ordered action descriptions, assignments and action-status history. It coordinates recorded work; it does not execute shell commands, orchestrate external response tools or automatically attach a playbook to an incident.

API contract. Playbook fields: `playbook_name`, `description`, `incident_category`, `severity_level`. Adding an action requires `action_description`, `step_number`. An update accepts at least one of `status`, `assigned_user_id`, `completion_date`. Administrators and analysts can add or update actions, including actions assigned to another user. Assignees must be existing Active users; the model does not restrict assignment to analysts.

Ordering, completion and persistence. Step numbers are positive and unique within a playbook, and retrieval sorts actions by that number. Gaps are allowed, and completion of earlier steps is not enforced. Pending, In Progress and Completed are valid statuses, without a mandatory transition graph. Completion dates cannot be future dates; an unchanged update preserves existing dates and history, while reopening clears completion. Deleting a playbook cascades to current actions but retains snapshots in `response_action_history`. Those retained rows have no separate history API after the playbook is deleted.

Implemented APIs.

Method	Endpoint (prefix `/Youssef-Al-Hajj-Youness`)	Access
POST	`/playbooks`	Administrator
GET	`/playbooks`	Administrator, Analyst, Auditor
GET	`/playbooks/<id>`	Administrator, Analyst, Auditor
POST	`/playbooks/<id>/actions`	Administrator, Analyst
PUT	`/actions/<id>/status`	Administrator, Analyst
DELETE	`/playbooks/<id>`	Administrator

Figures 136–139: Creation and listing establish multiple playbooks. The analyst adds an action to the first playbook; the administrator's added step belongs to the second, Phishing Triage, rather than extending the same sequence.

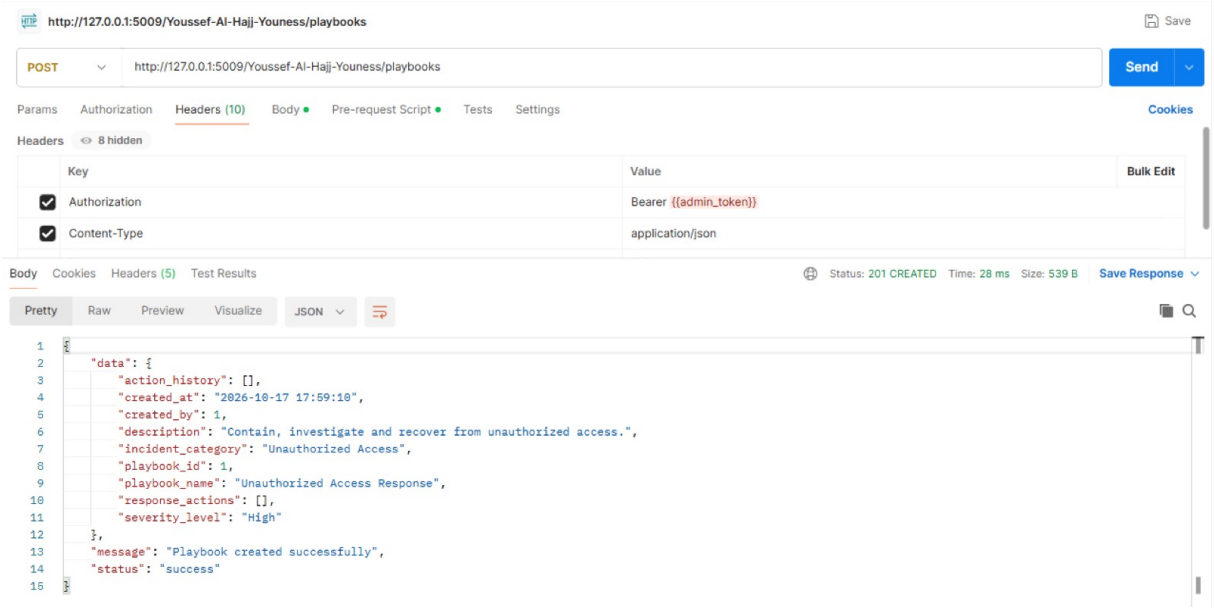


Figure 136: Youssef Al Hajj Youness: Create Response Playbook (POST /Youssef-Al-Hajj-Youness/playbooks): 201

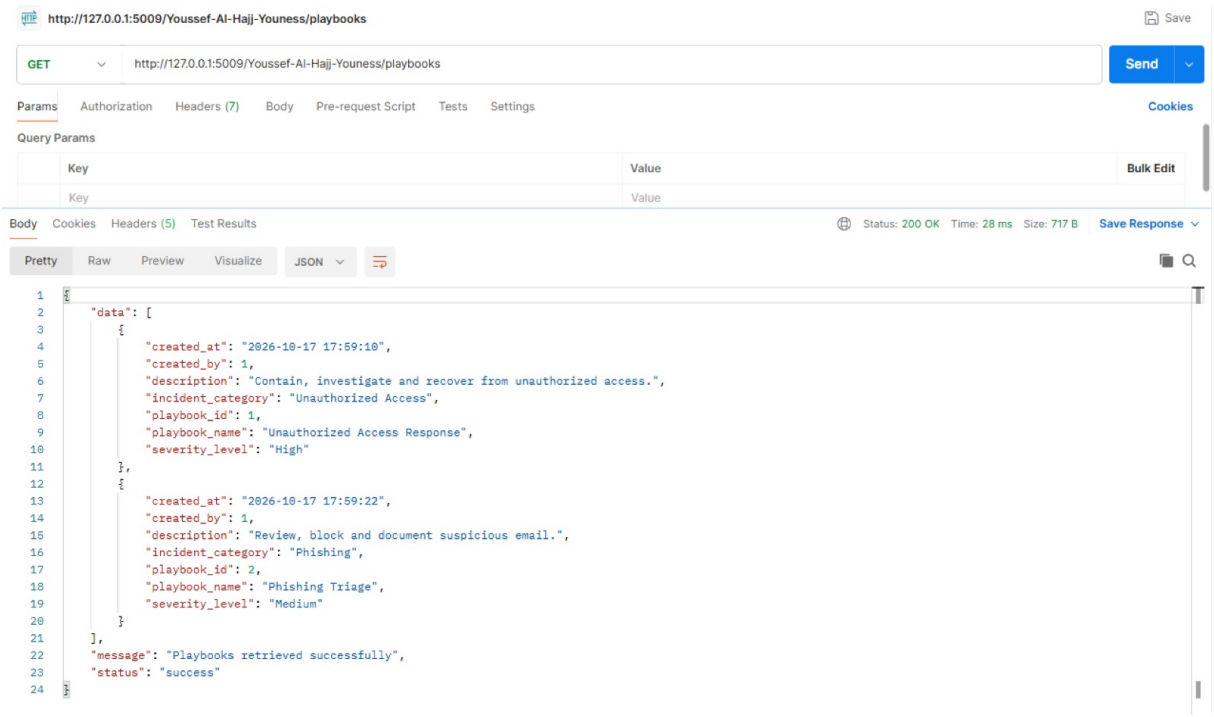


Figure 137: Youssef Al Hajj Youness: Retrieve All Playbooks: several playbooks, 200

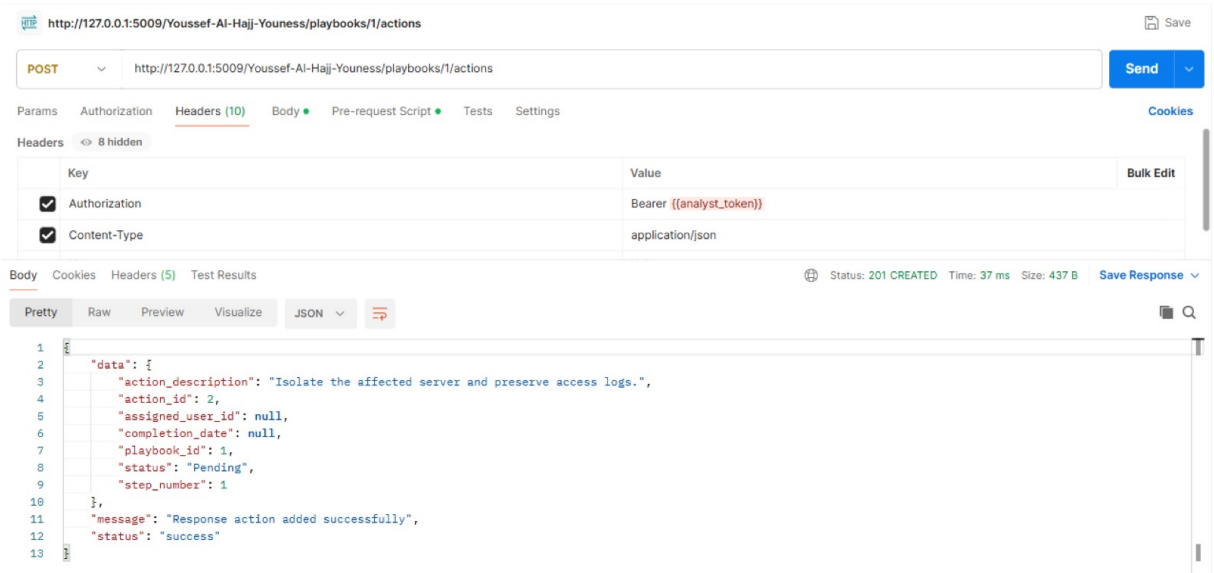


Figure 138: Youssef Al Hajj Youness: Add Response Action (POST .../playbooks/:id/actions): 201

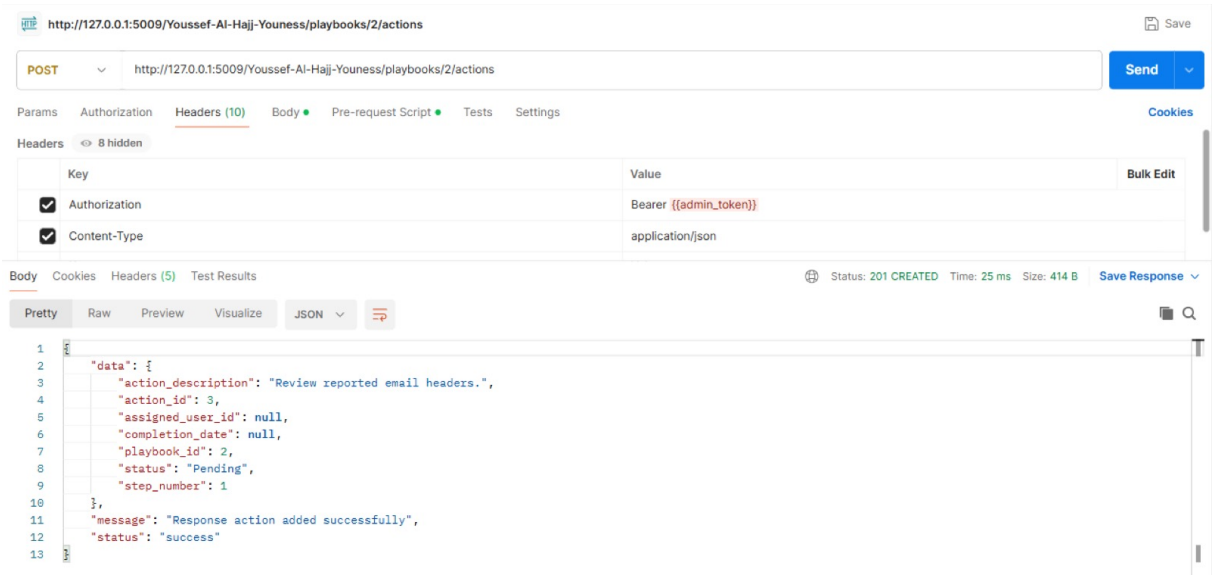


Figure 139: Youssef Al Hajj Youness: Add Response Action: administrator creates step 1 in the second playbook, Phishing Triage, 201

Figures 140–144: Status updates and retrieval show action progress, ordered responses and history for authorized users. This demonstrates recorded execution status, not execution of the described security operation.

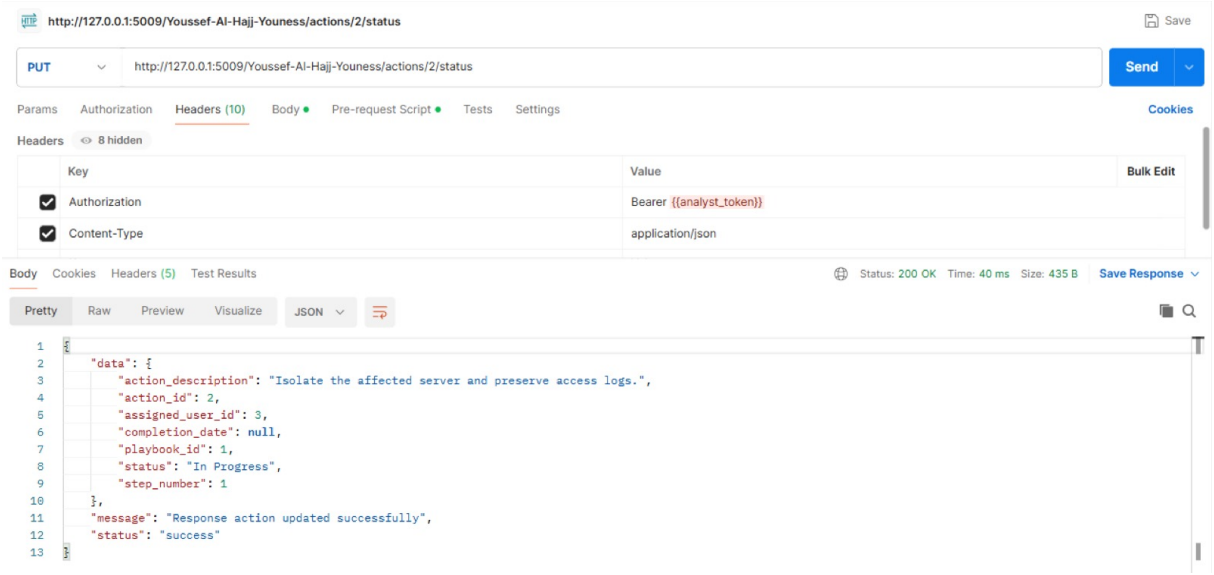


Figure 140: Youssef Al Hajj Youness: Update Response Action Status: Pending to In Progress, 200

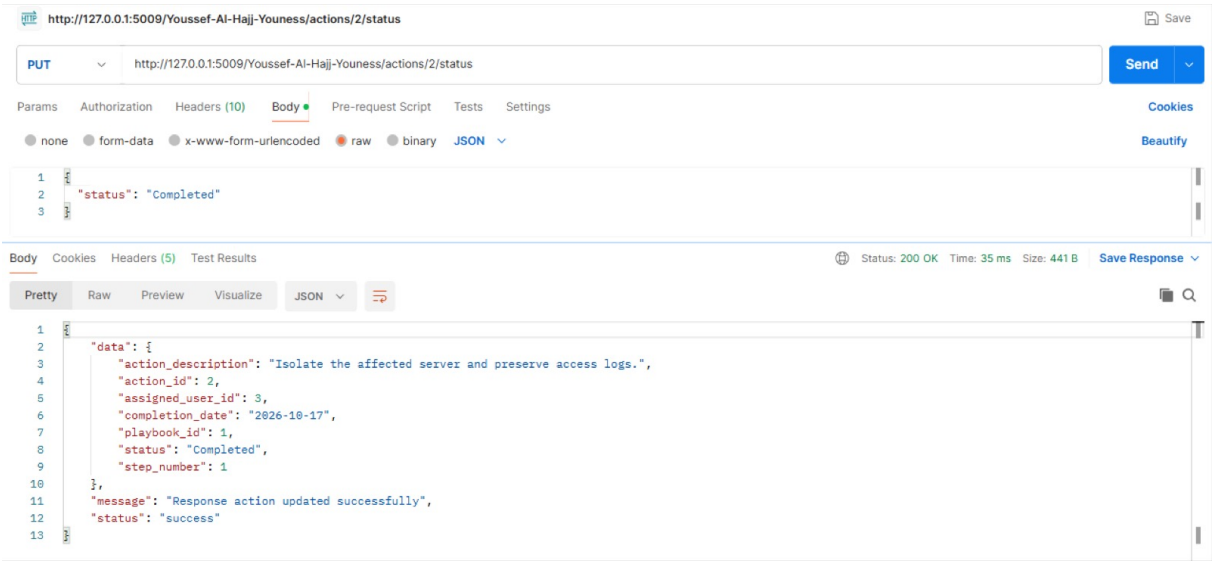


Figure 141: Youssef Al Hajj Youness: Update Response Action Status: In Progress to Completed, 200

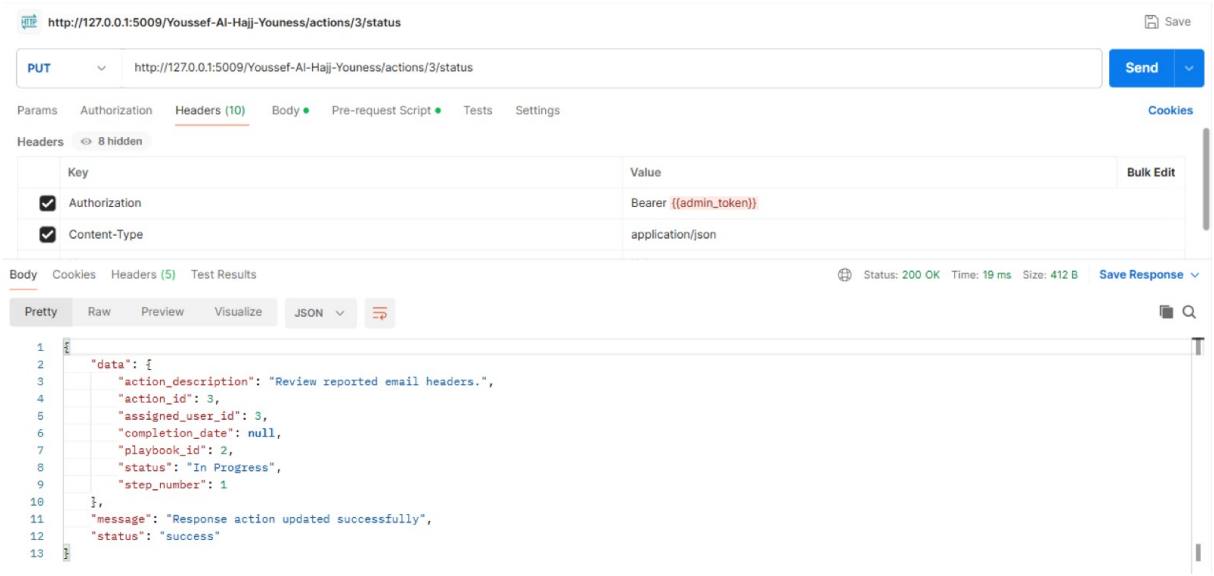


Figure 142: Youssef Al Hajj Youness: Update Response Action Status: administrator update, 200

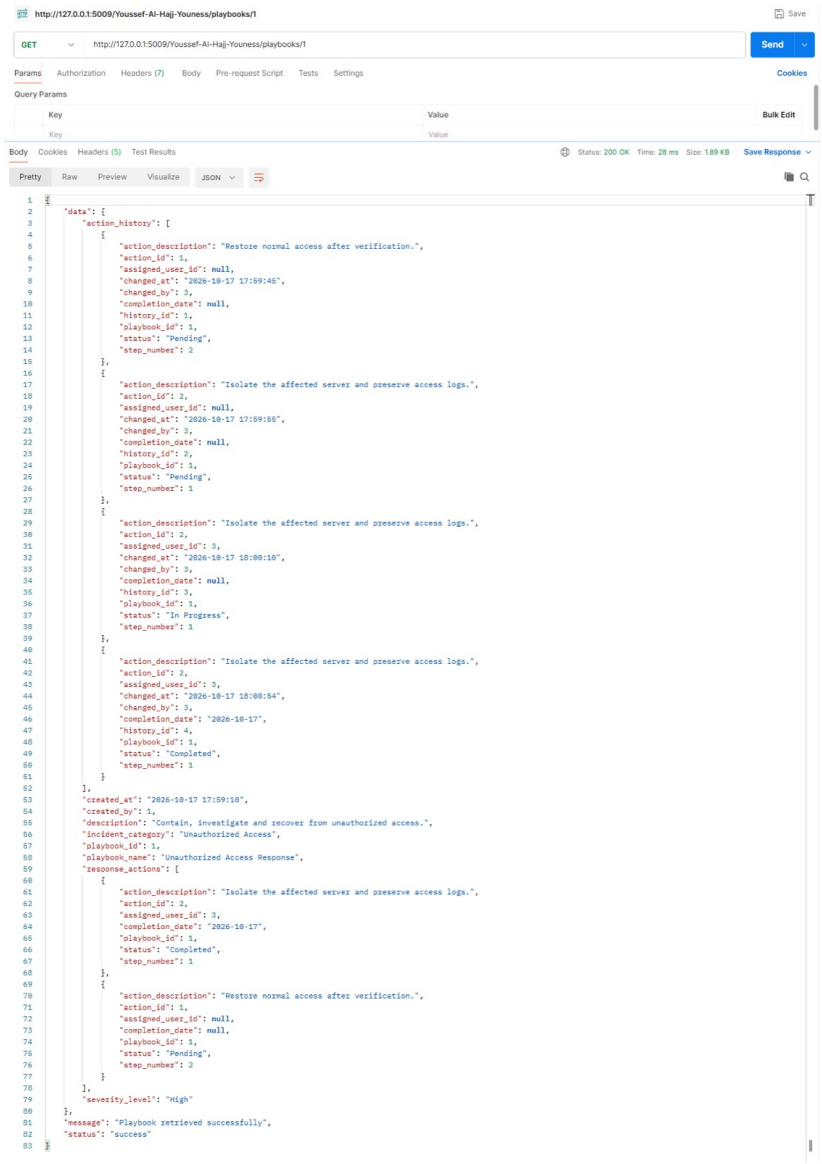


Figure 143: Youssef Al Hajj Youness: Retrieve Specific Playbook: ordered actions and execution history, 200

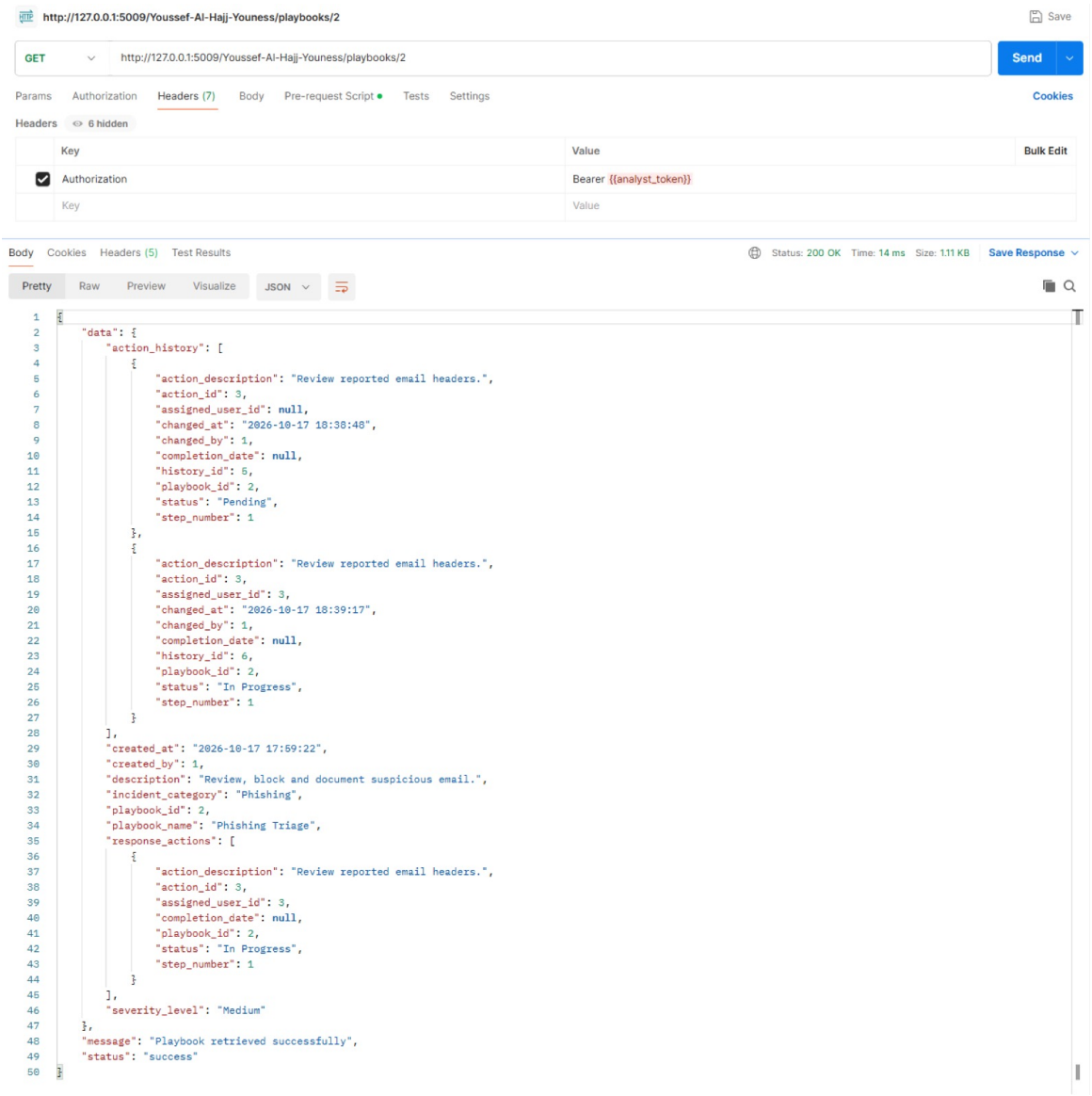


Figure 144: Youssef Al Hajj Youness: Retrieve Specific Playbook: analyst, 200

Figures 145–149: Before-deletion rows, the delete response and a later 404 establish removal of the playbook and current actions. The separate history query shows snapshots remain stored afterward.

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
QUERY FILE: 38_playbook_before.sql

SELECT 'BEFORE' AS phase, playbook_id, playbook_name, incident_category,
       severity_level, created_by
FROM playbooks ORDER BY playbook_id;
phase   playbook_id  playbook_name                incident_category    severity_level  created_by
------  -----------  ---------------------------  -------------------  --------------  ----------
BEFORE  1            Unauthorized Access Response  Unauthorized Access  High            1
BEFORE  2            Phishing Triage               Phishing             Medium          1
(2 rows)

SELECT 'BEFORE' AS phase, action_id, playbook_id, step_number, status,
       assigned_user_id, completion_date
FROM response_actions
WHERE playbook_id = (SELECT playbook_id FROM playbooks
                     WHERE playbook_name = 'Unauthorized Access Response')
ORDER BY step_number;
phase   action_id  playbook_id  step_number  status     assigned_user_id  completion_date
------  ---------  -----------  -----------  ---------  ----------------  ---------------
BEFORE  2          1            1            Completed  3                 2026-10-17
BEFORE  1          1            2            Pending    NULL              NULL
(2 rows)
```

Figure 145: Youssef Al Hajj Youness: Database: playbook and response action rows before deletion

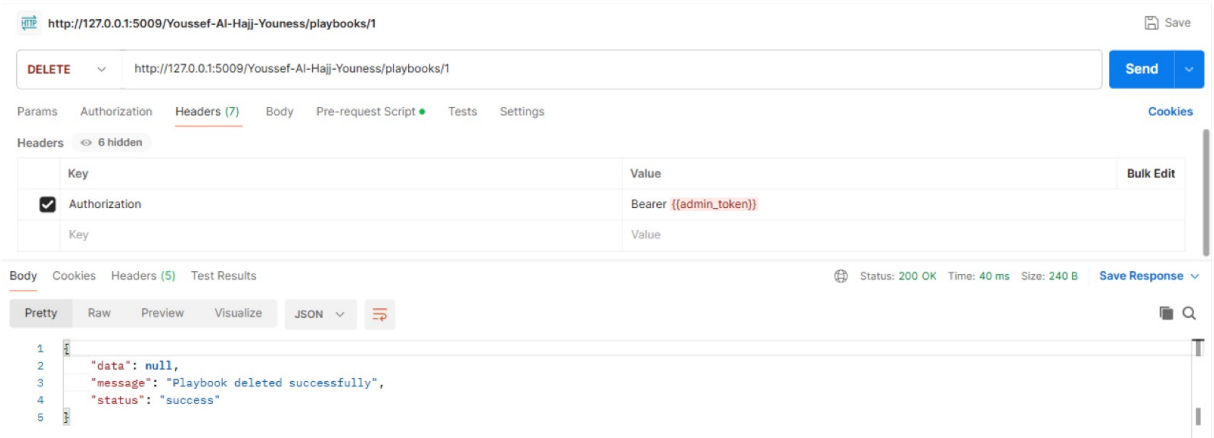


Figure 146: Youssef Al Hajj Youness: Delete Playbook (DELETE .../playbooks/:id): administrator, 200

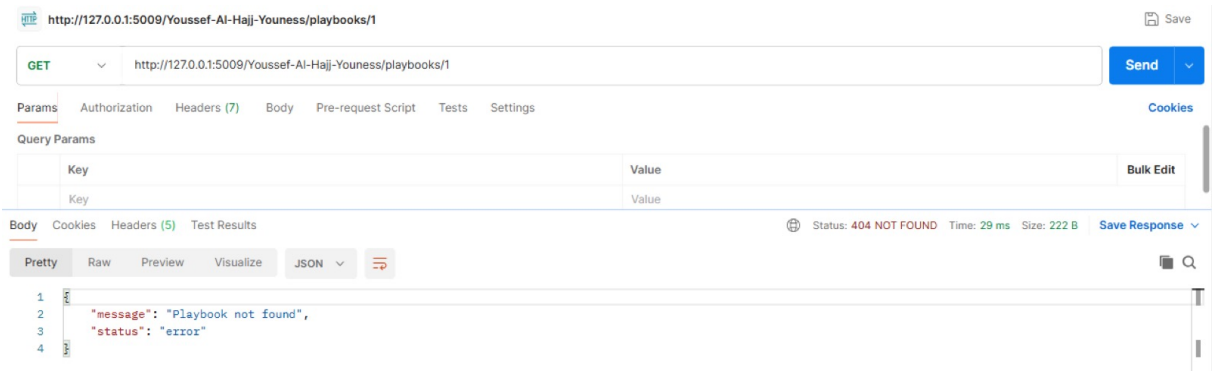


Figure 147: Youssef Al Hajj Youness: Retrieve Specific Playbook after deletion, 404

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db
QUERY FILE: 38_playbook_after.sql

SELECT 'AFTER' AS phase,
       (SELECT COUNT(*) FROM playbooks
        WHERE playbook_name = 'Unauthorized Access Response') AS main_playbooks,
       (SELECT COUNT(*) FROM response_actions WHERE playbook_id IN
          (SELECT playbook_id FROM response_action_history
           WHERE action_description = 'Isolate the affected server and preserve access logs.')) AS main_live_actions;
phase  main_playbooks  main_live_actions
-----  --------------  -----------------
AFTER  0               0
(1 rows)

SELECT action_id, playbook_id, step_number, status, assigned_user_id, completion_date
FROM response_action_history
WHERE action_description IN
  ('Isolate the affected server and preserve access logs.',
   'Restore normal access after verification.')
ORDER BY history_id;
action_id  playbook_id  step_number  status       assigned_user_id  completion_date
---------  -----------  -----------  -----------  ----------------  ---------------
1          1            2            Pending      NULL              NULL
2          1            1            Pending      NULL              NULL
2          1            1            In Progress  3                 NULL
2          1            1            Completed    3                 2026-10-17
(4 rows)

SELECT playbook_id, playbook_name FROM playbooks ORDER BY playbook_id;
playbook_id  playbook_name
-----------  --------------
2            Phishing Triage
(1 rows)
```

Figure 148: Youssef Al Hajj Youness: Database: execution history retained after the playbook is deleted

```
DATABASE: C:\Users\LENOVO\Desktop\435L-evidence\security_incident_demo.db

SELECT sql FROM sqlite_master WHERE type='table' AND name IN ('playbooks','response_actions');
CREATE TABLE playbooks (
    playbook_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    playbook_name     TEXT NOT NULL CHECK (length(trim(playbook_name)) > 0),
    description       TEXT NOT NULL,
    incident_category TEXT NOT NULL CHECK (length(trim(incident_category)) > 0),
    severity_level    TEXT NOT NULL
                      CHECK (severity_level IN ('Low', 'Medium', 'High', 'Critical')),
    created_by        INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
    created_at        TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE TABLE response_actions (
    action_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    playbook_id        INTEGER NOT NULL REFERENCES playbooks(playbook_id) ON DELETE CASCADE,
    action_description TEXT NOT NULL,
    step_number        INTEGER NOT NULL CHECK (typeof(step_number) = 'integer' AND step_number > 0),
    status             TEXT NOT NULL DEFAULT 'Pending'
                       CHECK (status IN ('Pending', 'In Progress', 'Completed')),
    assigned_user_id   INTEGER REFERENCES users(user_id) ON DELETE SET NULL,
    completion_date    TEXT,
    updated_at         TEXT NOT NULL DEFAULT (datetime('now')),
    UNIQUE (playbook_id, step_number),
    CHECK (status = 'Completed' OR completion_date IS NULL)
);

SELECT l.log_id, u.role, l.description, l.timestamp
FROM logs l LEFT JOIN users u USING(user_id)
WHERE l.related_service='playbooks' AND l.action='Unauthorized access attempt'
ORDER BY l.log_id DESC LIMIT 5;
log_id  role             description                                                                                                  timestamp
------  ---------------  -----------------------------------------------------------------------------------------------------------  -------------------
66      Auditor          POST /Youssef-Al-Hajj-Youness/playbooks/2/actions: User does not have permission to perform this operation  2026-10-17 18:36:21
65      Security Analyst POST /Youssef-Al-Hajj-Youness/playbooks: User does not have permission to perform this operation            2026-10-17 18:36:08
64      Regular User     GET /Youssef-Al-Hajj-Youness/playbooks/2: User does not have permission to perform this operation          2026-10-17 18:35:53
63      NULL             GET /Youssef-Al-Hajj-Youness/playbooks/2: Authentication required: send 'Authorization: Bearer <token>'    2026-10-17 18:35:35
40      Security Analyst DELETE /Youssef-Al-Hajj-Youness/playbooks/1: User does not have permission to perform this operation       2026-10-17 18:01:17
(5 rows)
```

Figure 149: Youssef Al Hajj Youness: Database: playbook tables structure and audit entries

6 Database Design

This design was a team decided approach.

Figure 150 shows the eleven tables shared by the nine services. The database is initialized by `database/init_db.py` from `database/schema.sql`. Foreign keys are enforced and enumerations are restricted with CHECK constraints.

6.1 Database Schema Diagram

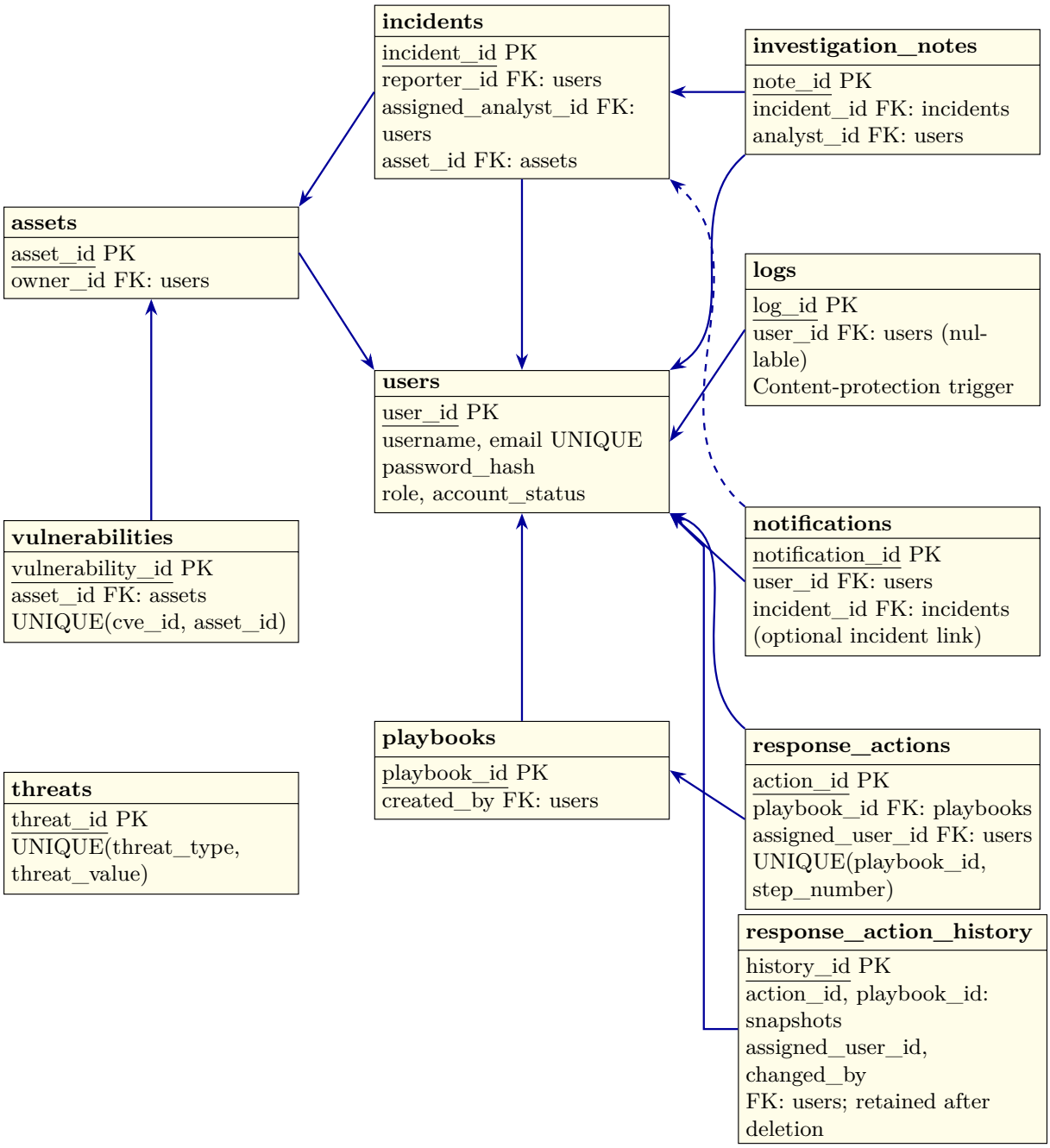


Figure 150: Key schema relationships; arrows point from child to parent. PK denotes a primary key and FK a foreign key. The dashed notification link references incidents. History keeps action/playbook IDs as snapshots, with user references retained as foreign keys. The complete field inventory follows in Section 6.2.

6.2 Tables, Fields, Primary Keys and Foreign Keys

Table	Fields	Keys and constraints
`users`	user_id, full_name, username, email, password_hash, department, role, account_status, created_at, last_login	PK user_id; UNIQUE username, email; role is one of the 4 roles; account_status in {Active, Inactive, Suspended}
`assets`	asset_id, asset_name, asset_type, ip_address, operating_system, department, owner_id, location, criticality_level, status, created_at	PK asset_id; FK owner_id → users; criticality in {Low, Medium, High, Critical}; status in {Active, Inactive, Under Maintenance}
`vulnerabilities`	vulnerability_id, title, description, cve_id, cvss_score, severity_level, asset_id, discovery_source, status, creation_date, resolution_date	PK vulnerability_id; FK asset_id → assets; UNIQUE (cve_id, asset_id); cvss between 0 and 10
`incidents`	incident_id, title, description, category, severity_level, reporter_id, asset_id, assigned_analyst_id, status, creation_date, resolution_date	PK incident_id; FK reporter_id, assigned_analyst_id → users; FK asset_id → assets
`investigation_notes`	note_id, incident_id, analyst_id, note_content, created_at	PK note_id; FK incident_id → incidents (cascade); FK analyst_id → users
`threats`	threat_id, threat_type, threat_value, description, severity_level, source, date_added	PK threat_id; UNIQUE (threat_type, threat_value)
`logs`	log_id, user_id, action, description, timestamp, severity_level, related_service	PK log_id; FK user_id → users (set null on delete); trigger protects content; user link may be nulled
`notifications`	notification_id, user_id, incident_id, title, message, notification_type, severity_level, is_read, created_at, read_date	PK notification_id; FK user_id → users (cascade); FK incident_id → incidents (set null)
`playbooks`	playbook_id, playbook_name, description, incident_category, severity_level, created_by, created_at	PK playbook_id; FK created_by → users (set null)
`response_actions`	action_id, playbook_id, action_description, step_number, status, assigned_user_id, completion_date, updated_at	PK action_id; FK playbook_id → playbooks (cascade); FK assigned_user_id → users; UNIQUE (playbook_id, step_number)
`response_action_history`	history_id, action_id, playbook_id, action_description, step_number, status, assigned_user_id, completion_date, changed_by, changed_at	PK history_id; action and playbook IDs are snapshots kept after deletion; FK assigned_user_id, changed_by → users

6.3 Relationships

- **Users and Assets:** one user owns many assets; each asset has one owner (`assets.owner_id`).

- **Users and Incidents:** a user reports many incidents (`reporter_id`); an analyst can be assigned many incidents (`assigned_analyst_id`).
- **Assets and Incidents:** one asset can be involved in many incidents (`incidents.asset_id`).
- **Incidents and Investigation notes:** one incident has many notes; each note belongs to one incident.
- **Users and Logs:** a user may be linked to many log records; the link can become NULL when the user is deleted.
- **Assets and Vulnerabilities:** one asset has many vulnerabilities; each belongs to one asset.
- **Users and Notifications:** one user receives many notifications.
- **Playbooks and Response actions:** one playbook has many ordered actions.
- **Users and Response actions:** one user can be assigned many actions.

7 Error Handling and Validation

7.1 Error Handling Approach

The services register handlers from `common/errors.py`. Application validation raises `ApiError` or `ValidationError`; the handler returns an HTTP status with a JSON `status` and `message`. Framework errors such as an unknown route or unsupported method use the same envelope. Unexpected exceptions are logged server-side and return a generic `500 Internal server error` response.

```
{ "status": "error",
  "message": "User does not have permission to perform this operation" }
```

Code	Meaning	Observed or implemented example
400	Invalid input or request state	Invalid severity; unknown user supplied to log creation
401	Authentication failure	Missing/invalid token; wrong login credentials
403	Access denied	Disallowed role or another user's notification
404	Missing resource	Permitted caller requests an absent incident
405	Unsupported method	POST to the summary-report GET route
409	Conflict	Duplicate username; skipped incident lifecycle step
500	Unexpected server failure	Generic error; internal details withheld
503	Dependency unavailable	User-service validation or report source unavailable

7.2 Validation Rules

Write operations validate their expected JSON fields, required values, types and allowed enumerations. Shared text validation trims input and checks length and control characters where applied. These rules are operation-specific: GET requests do not require a JSON body, and not every service applies the same integer upper bound. Services using the shared basic ID parser check positive ASCII integers; several newer service models also enforce a signed 64-bit range.

- **Users:** username and email checks, uniqueness, and passwords of 8–128 characters containing upper- and lowercase letters and a digit. Profile updates reject role or status changes.
- **Assets:** allowed type, criticality and status, syntactically valid IPv4/IPv6, and an existing owner. Missing references supplied on creation are validation errors.
- **Incidents:** an existing reporter and asset, allowed severity, and the next lifecycle step. Assignment requires an Active Security Analyst verified through Service 1.
- **Threats:** supported indicator type, valid type-specific format, allowed severity and a nonduplicate type/value pair. Accepted hash syntax does not establish threat reputation.
- **Logs:** an integer ID for an existing user, bounded text and allowed severity; client-generated IDs and timestamps are rejected.

- **Vulnerabilities:** CVE syntax, CVSS in 0–10, matching severity band, an existing asset and ordered status changes. Resolution dates cannot precede creation; unlike action completion dates, the model does not reject all future resolution dates.
- **Notifications:** an existing recipient, supported type and severity, and ownership checks for scoped reads and updates.
- **Playbooks:** positive, unique step numbers; valid statuses; an Active assignee; and completion dates no later than the current date. These checks do not enforce sequential action execution.

7.3 Security Checks and Limits

Repositories bind submitted values as SQL parameters, keeping those values separate from SQL syntax.

Protected routes verify token signature and expiry and reload the current account. Route decorators enforce roles; handlers enforce record ownership or incident assignment. Login returns the same credential-error message for an unknown user and a wrong password.

The shared authorization helpers attempt to record access denials. Failed login has a separate login-failure event, and framework errors are not necessarily authorization events. Standalone audit failures are nonfatal, so a universal guarantee that every 401/403 is persisted would be inaccurate. The log-update trigger and the absent PUT API provide separate content-integrity controls, demonstrated in Section 5.5.

7.4 Screenshots of Validation Tests

Service 1 (Adam Yahya)

Figures 151–152: Duplicate registration returns 409; invalid email returns 400.

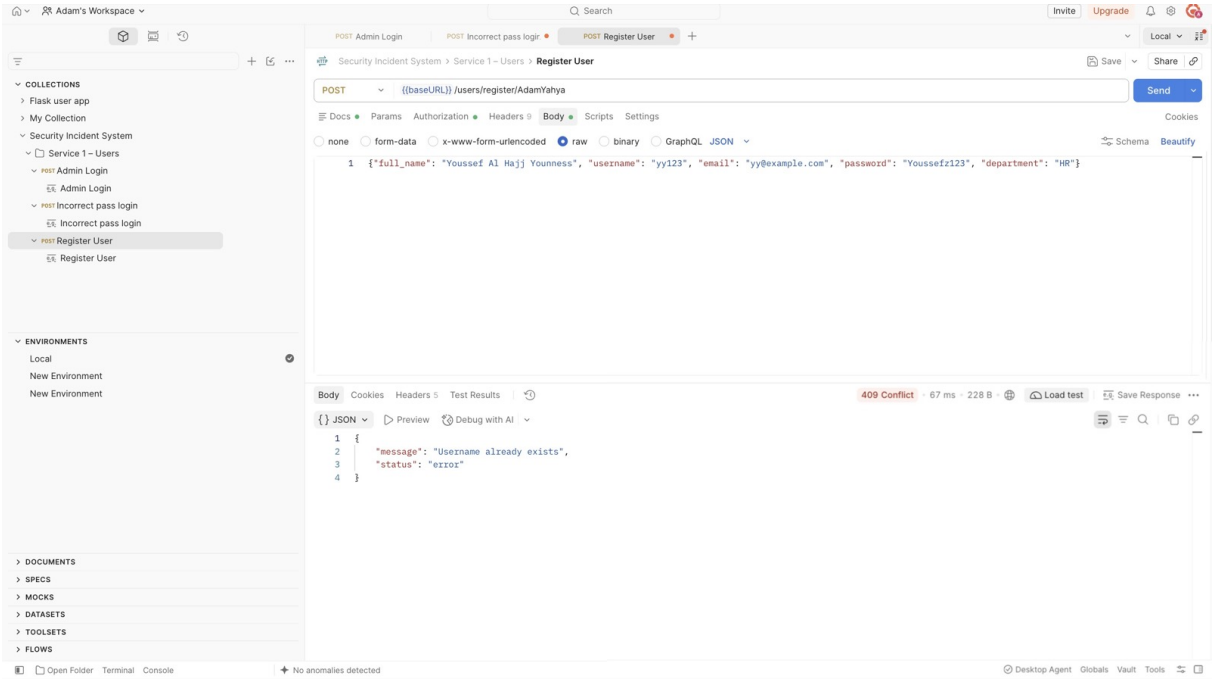


Figure 151: Adam Yahya: Register User: duplicate username rejected, 409

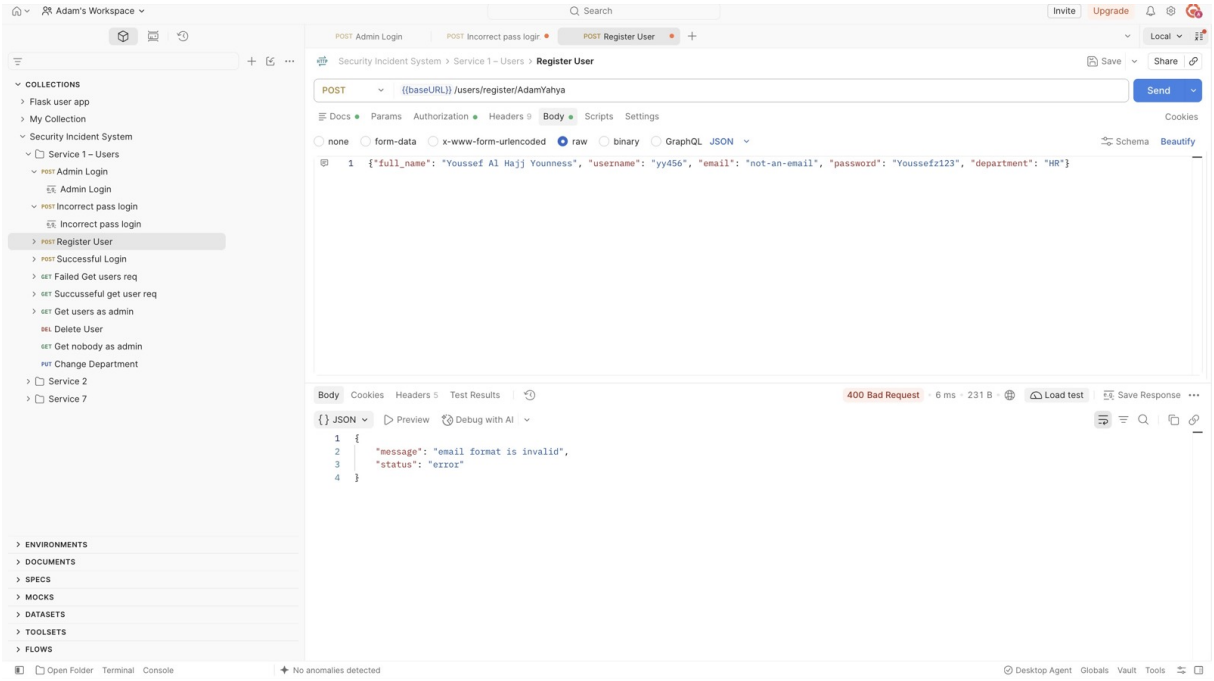


Figure 152: Adam Yahya: Register User: invalid email format rejected, 400

Service 2 (Adam Yahya)

Figures 153–154: Invalid IP syntax and an unsupported asset type each return 400.

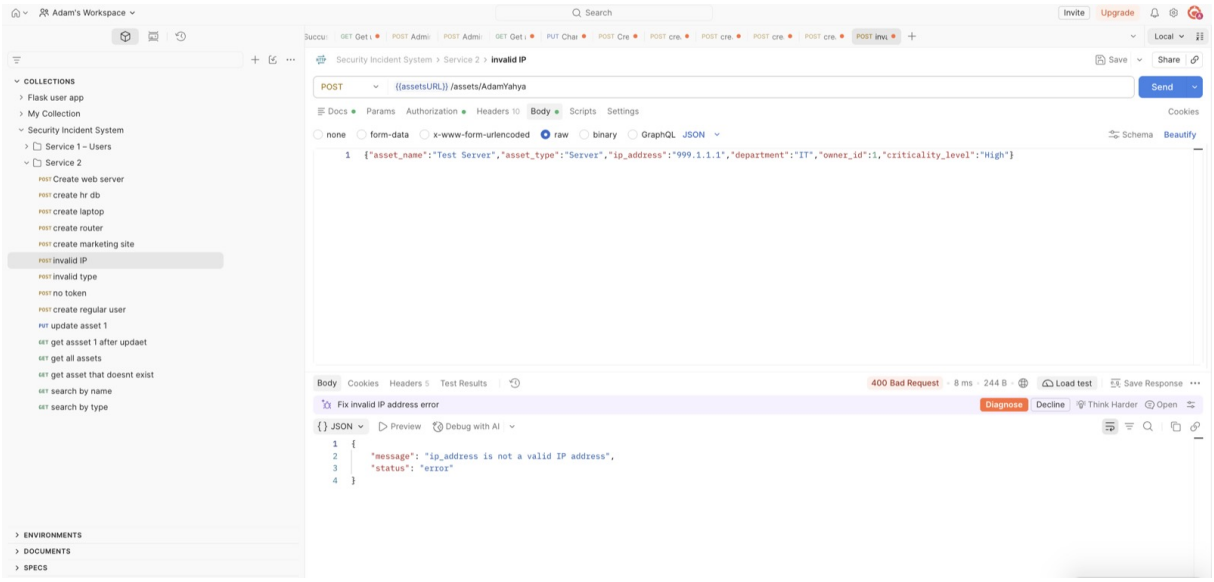


Figure 153: Adam Yahya: Add Asset: invalid IP address rejected, 400

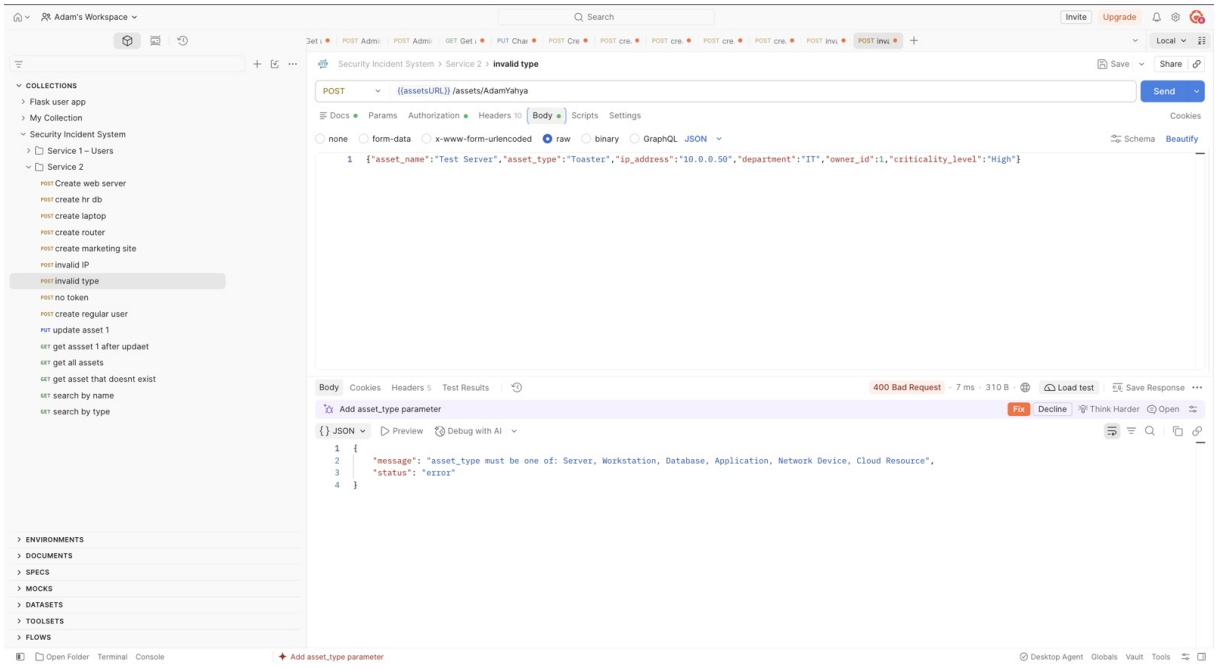


Figure 154: Adam Yahya: Add Asset: invalid asset type rejected, 400

Service 7 (Adam Yahya)

Figures 155–156: Malformed CVE input and a skipped vulnerability status step return 400.

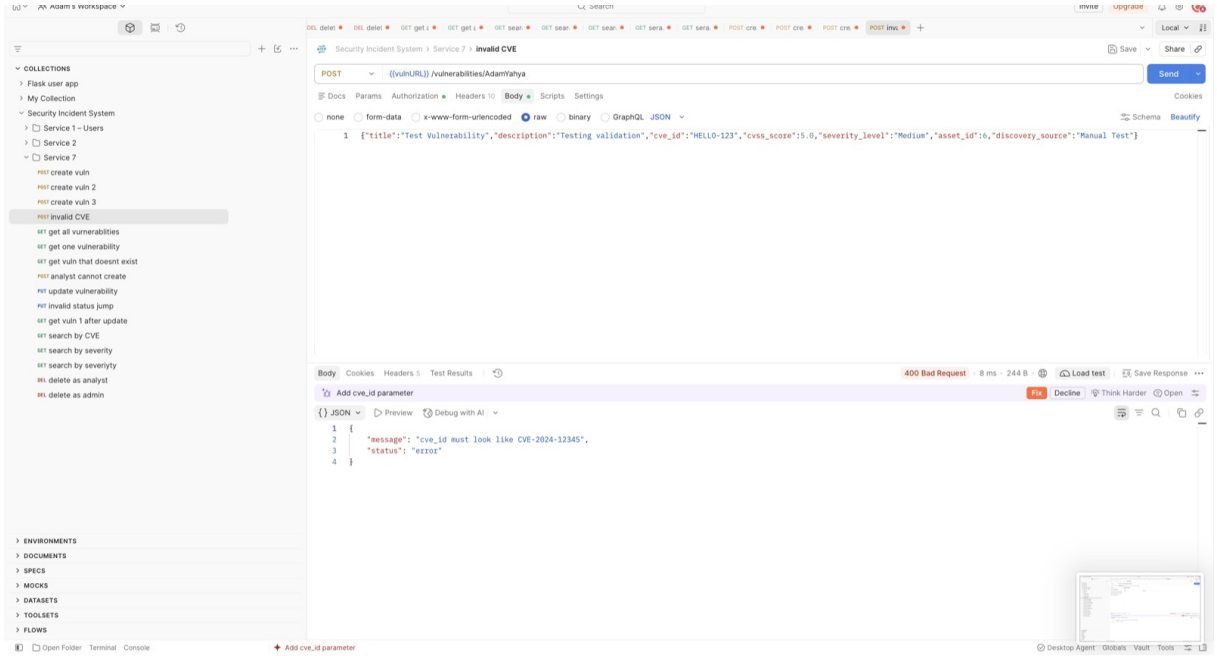


Figure 155: Adam Yahya: Add Vulnerability: invalid CVE identifier rejected, 400

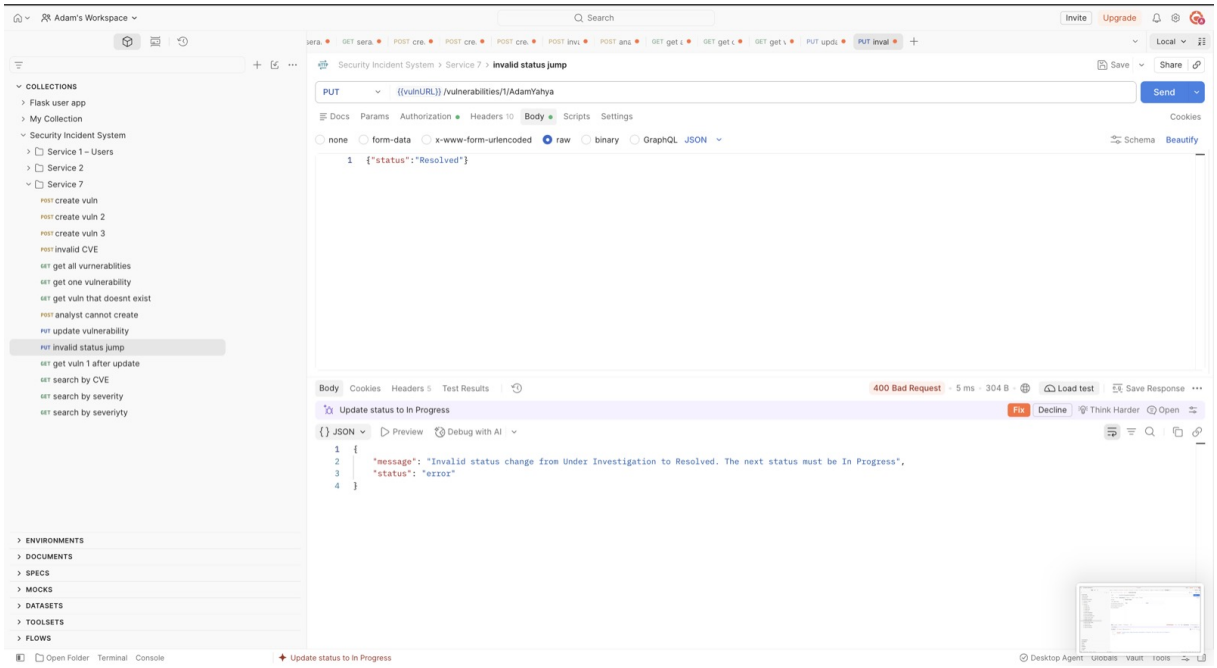


Figure 156: Adam Yahya: Update Vulnerability: invalid status jump rejected, 400

Service 3 (Youssef Al Hajj Youness)

Figures 157–163: The cases distinguish field/severity validation (400), a lifecycle conflict (409), unsuitable assignee (400), and absent incident, reporter or asset records (404).

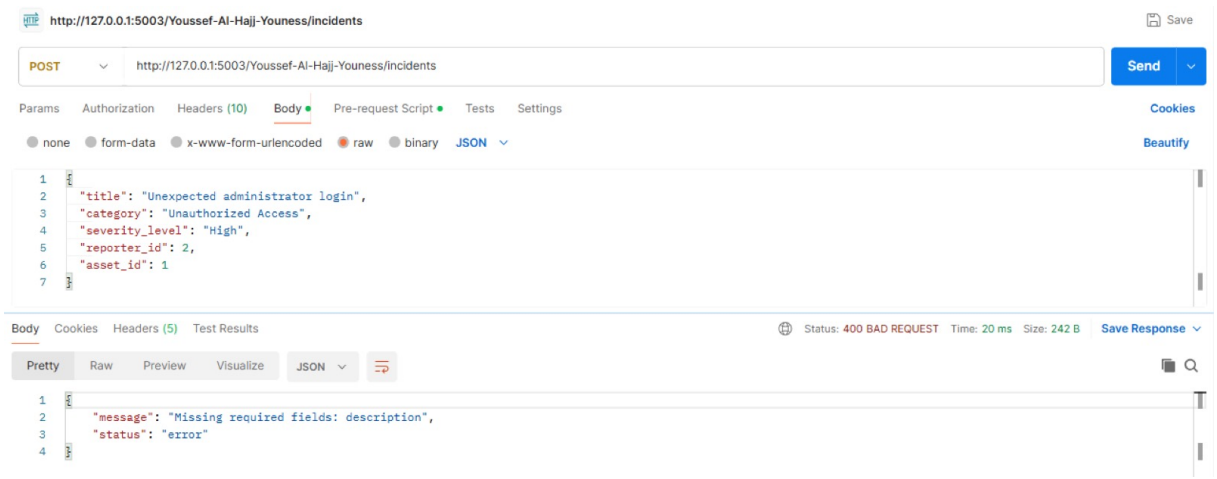


Figure 157: Youssef Al Hajj Youness: Create Incident: missing required field, 400

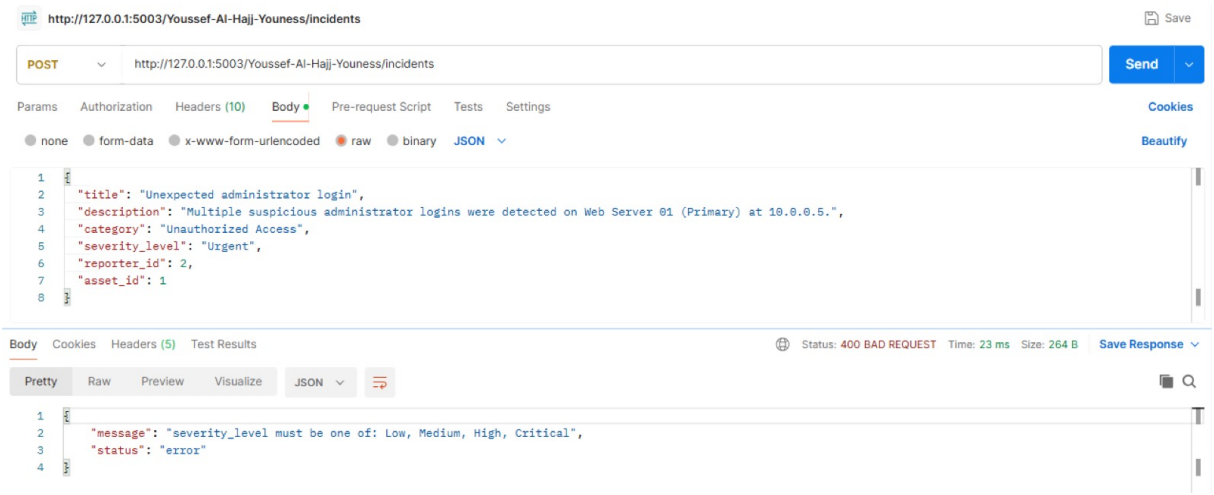


Figure 158: Youssef Al Hajj Youness: Create Incident: invalid severity, 400

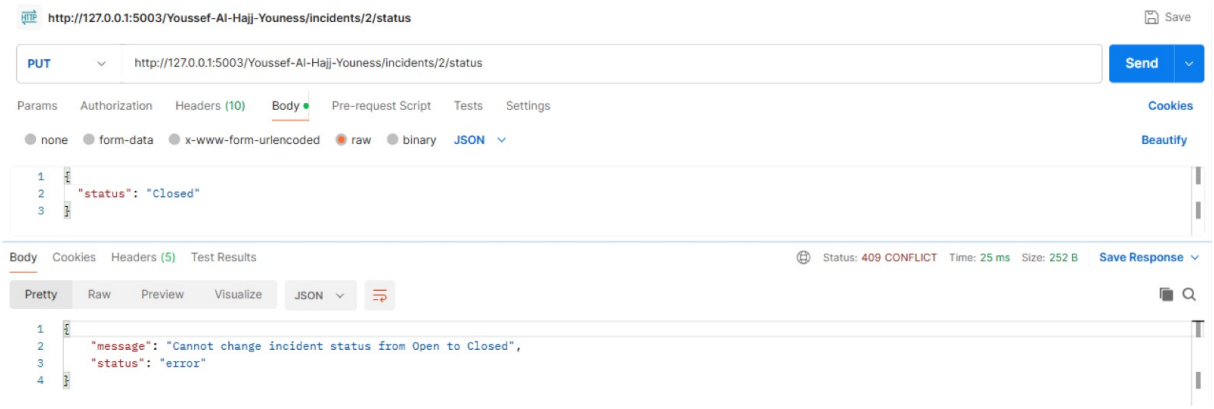


Figure 159: Youssef Al Hajj Youness: Update Incident Status: invalid transition, 409

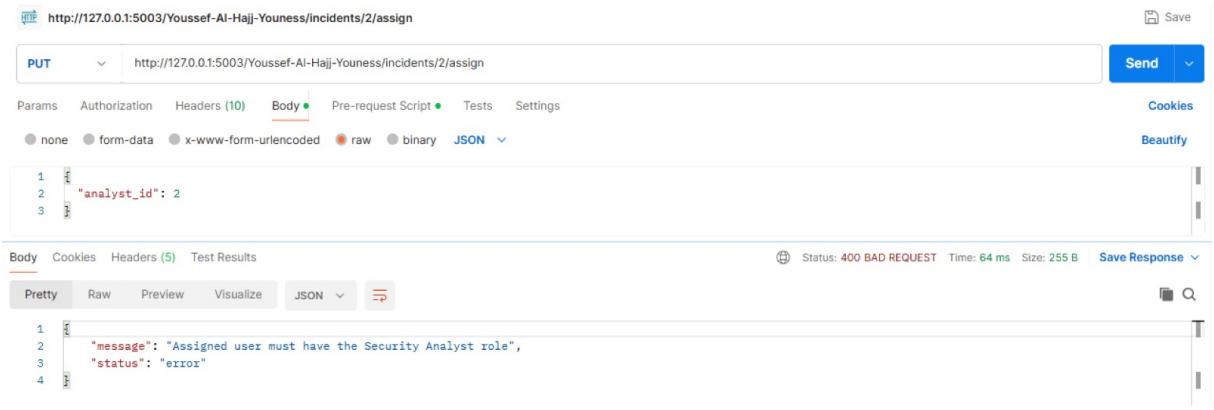


Figure 160: Youssef Al Hajj Youness: Assign Incident: assignee is not a Security Analyst, 400

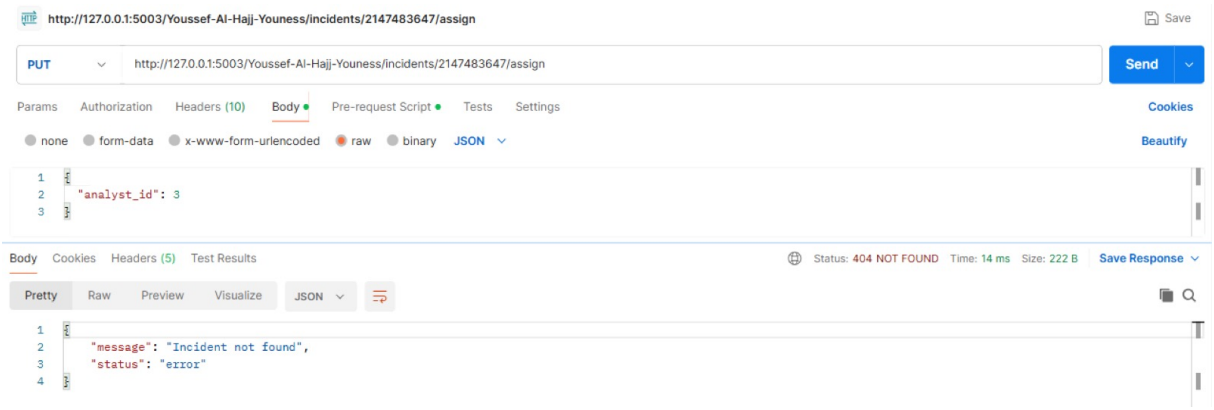


Figure 161: Youssef Al Hajj Youness: Assign Incident: incident does not exist, 404

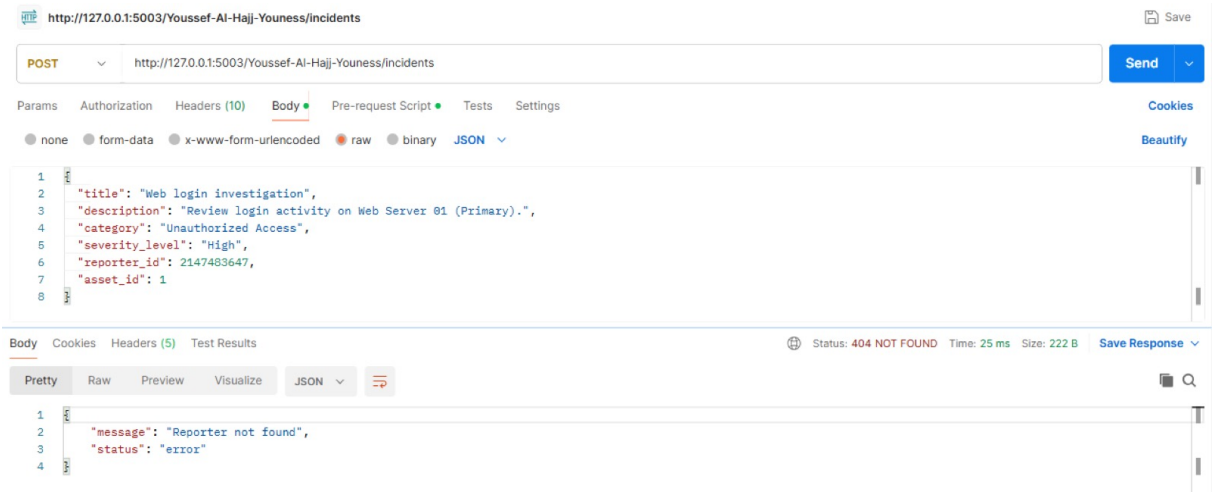


Figure 162: Youssef Al Hajj Youness: Create Incident: reporter does not exist, 404

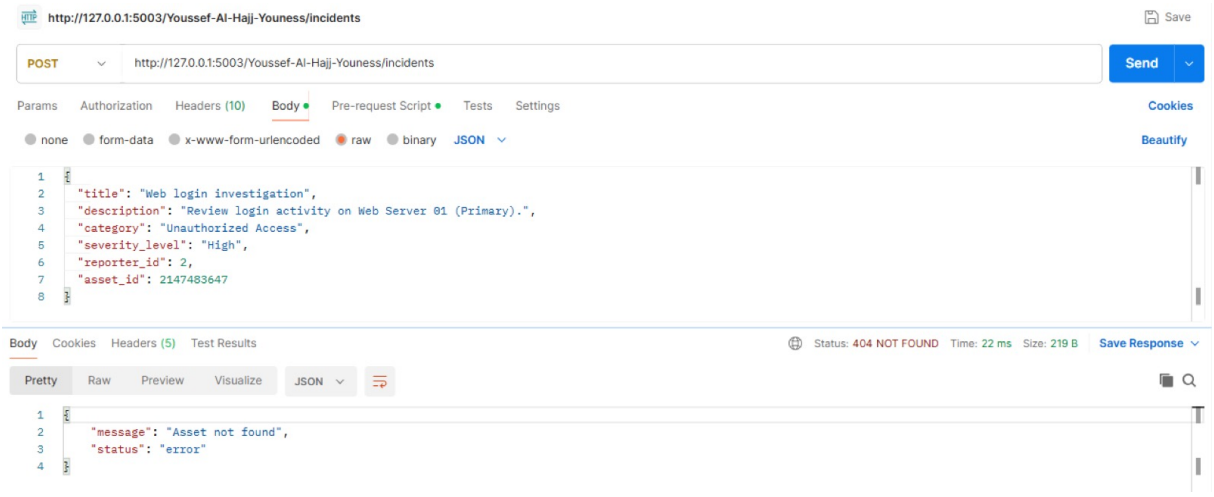


Figure 163: Youssef Al Hajj Youness: Create Incident: asset does not exist, 404

Service 8 (Youssef Al Hajj Youness)

Figure 164: The incomplete notification body is rejected with 400.

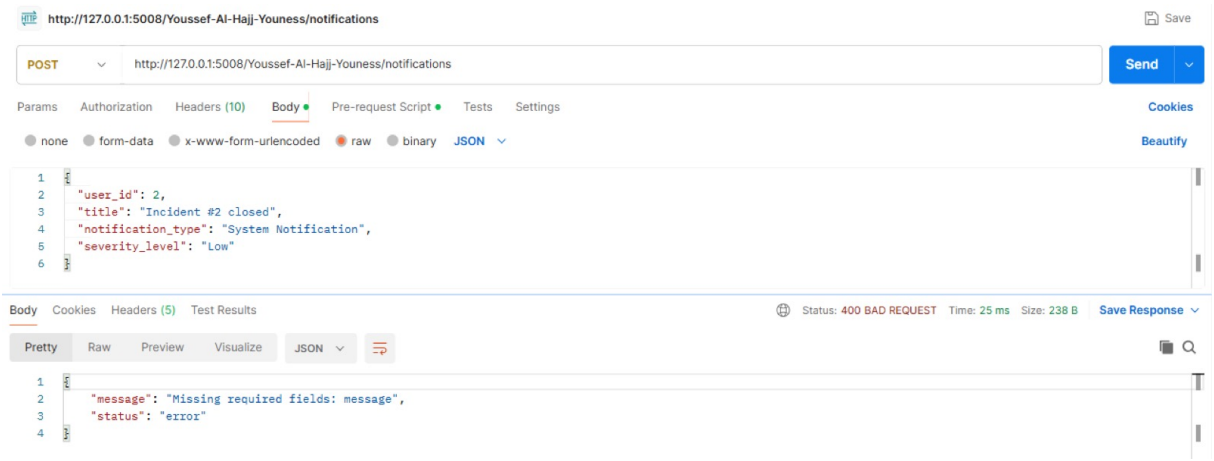


Figure 164: Youssef Al Hajj Youness: Create Notification: missing required field, 400

Service 9 (Youssef Al Hajj Youness)

Figures 165–167: The cases reject a future completion date and a missing playbook field with 400, and a duplicate step number with 409.

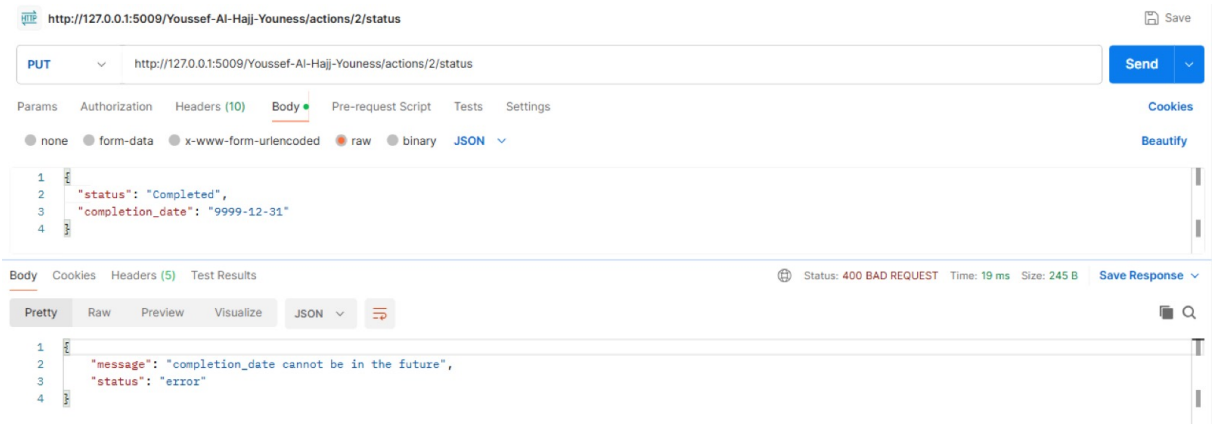


Figure 165: Youssef Al Hajj Youness: Update Response Action Status: future completion date rejected, 400

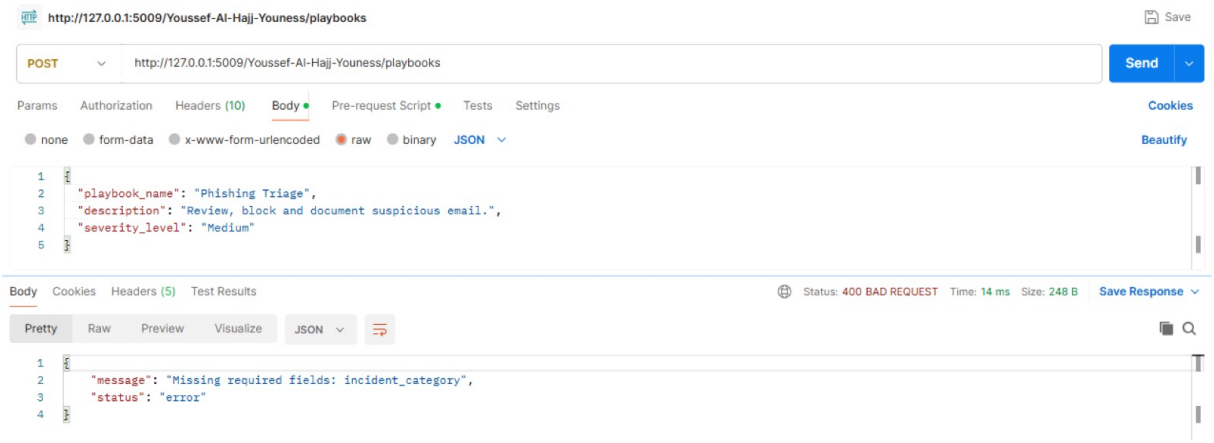


Figure 166: Youssef Al Hajj Youness: Create Playbook: missing required field, 400

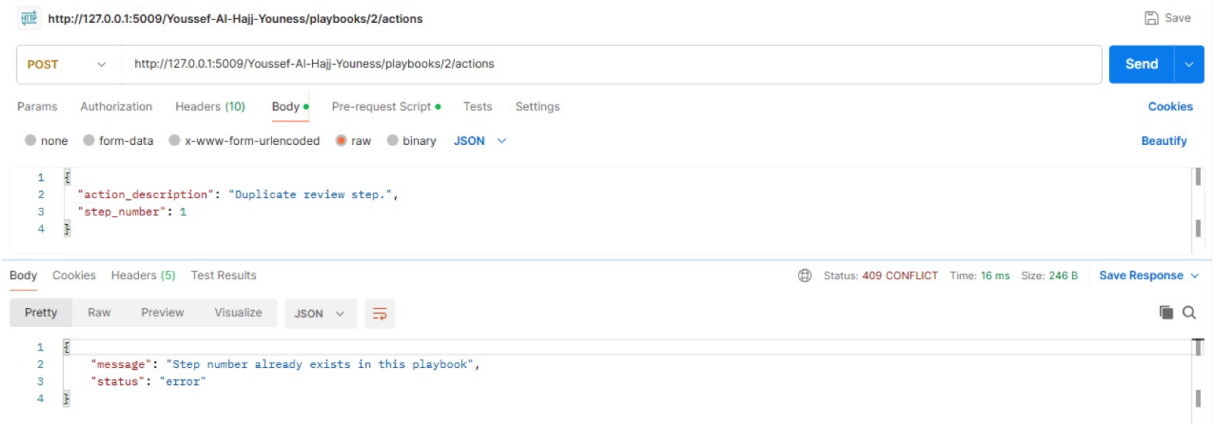


Figure 167: Youssef Al Hajj Youness: Add Response Action: duplicate step number, 409

Service 4 (Leen Abdallah)

Figures 168–178: Completed responses demonstrate duplicate (409), format, enumeration, missing-field, ID, empty-search and update-field validation (400). The retained invalid-severity search capture belongs to the earlier dataset. The analyst update denial appears in Section 9.

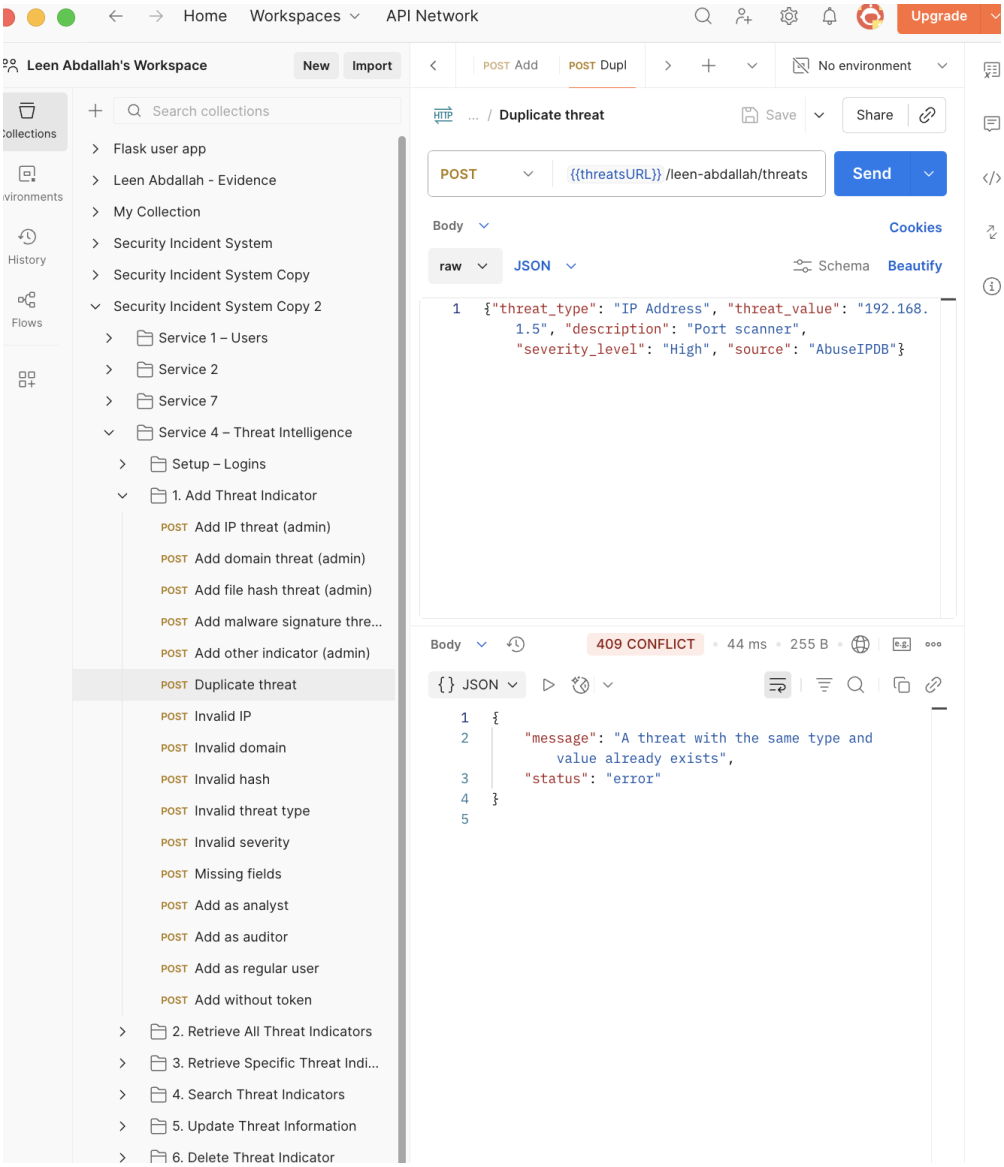


Figure 168: Leen Abdallah: Add Threat Indicator: duplicate type and value, 409

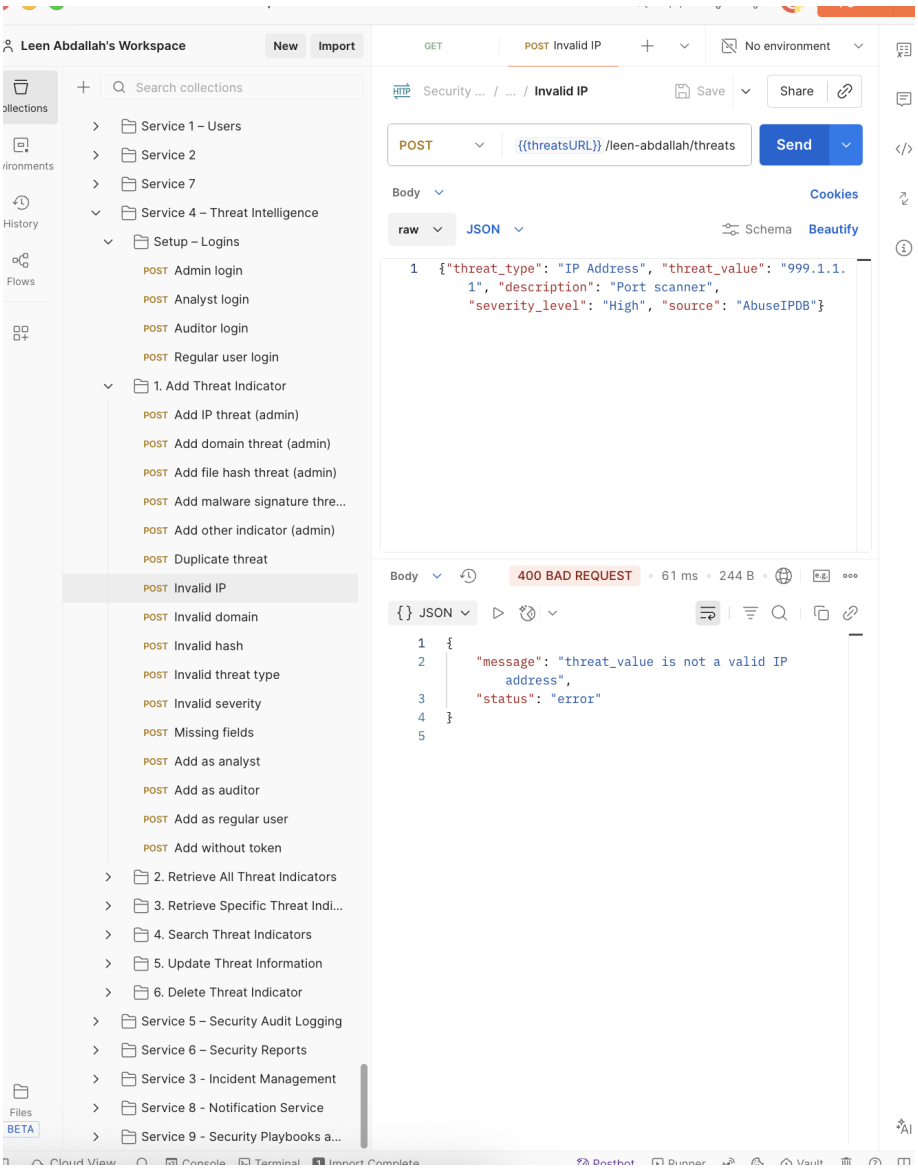


Figure 169: Leen Abdallah: Add Threat Indicator: invalid IP address, 400

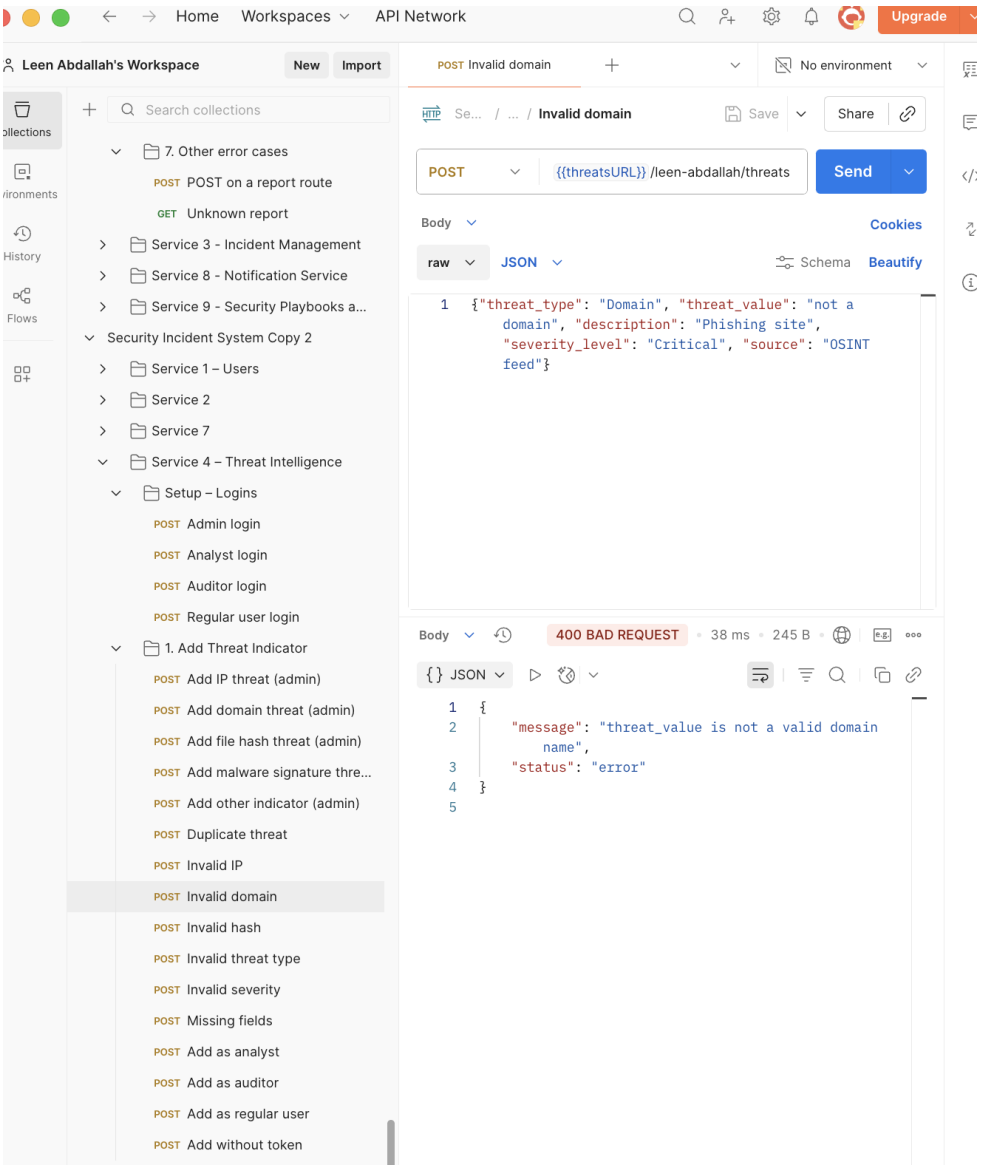


Figure 170: Leen Abdallah: Add Threat Indicator: invalid domain, 400

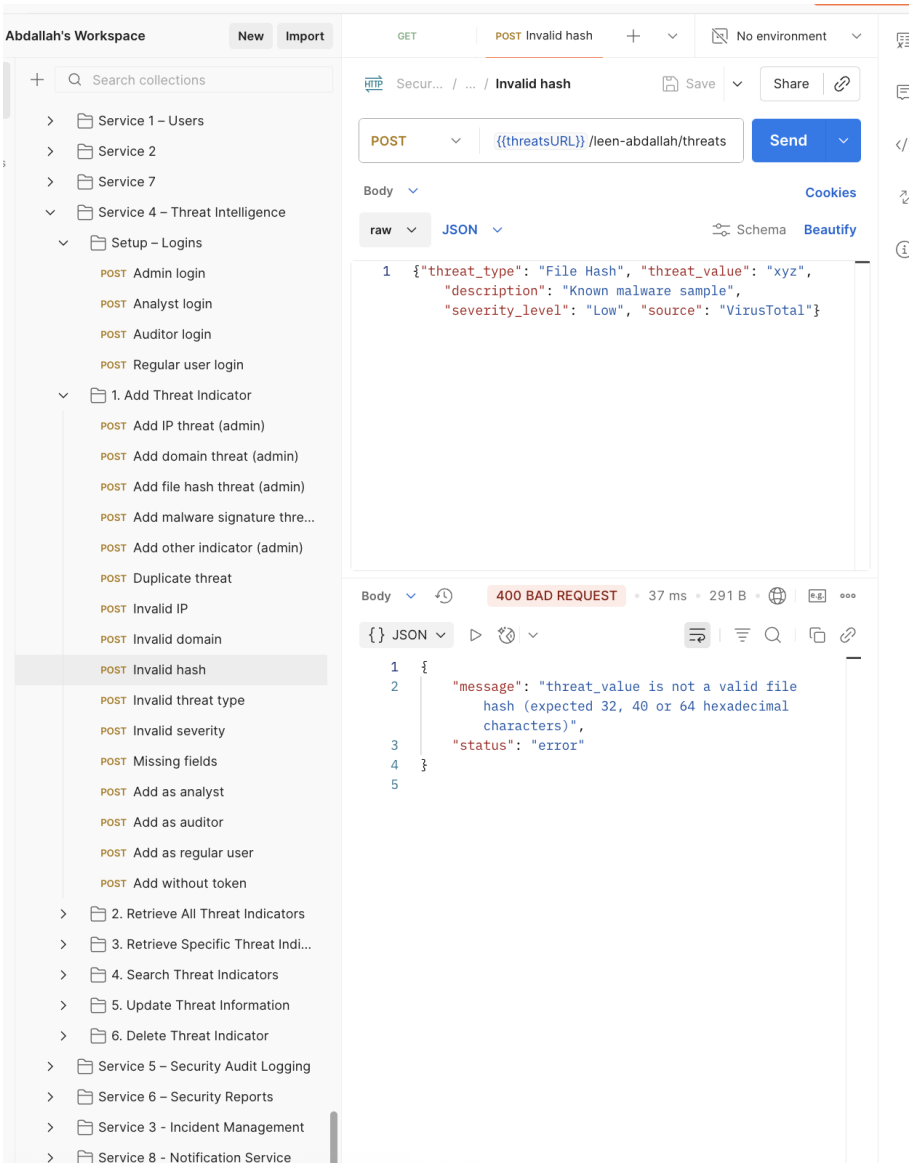


Figure 171: Leen Abdallah: Add Threat Indicator: invalid file hash, 400

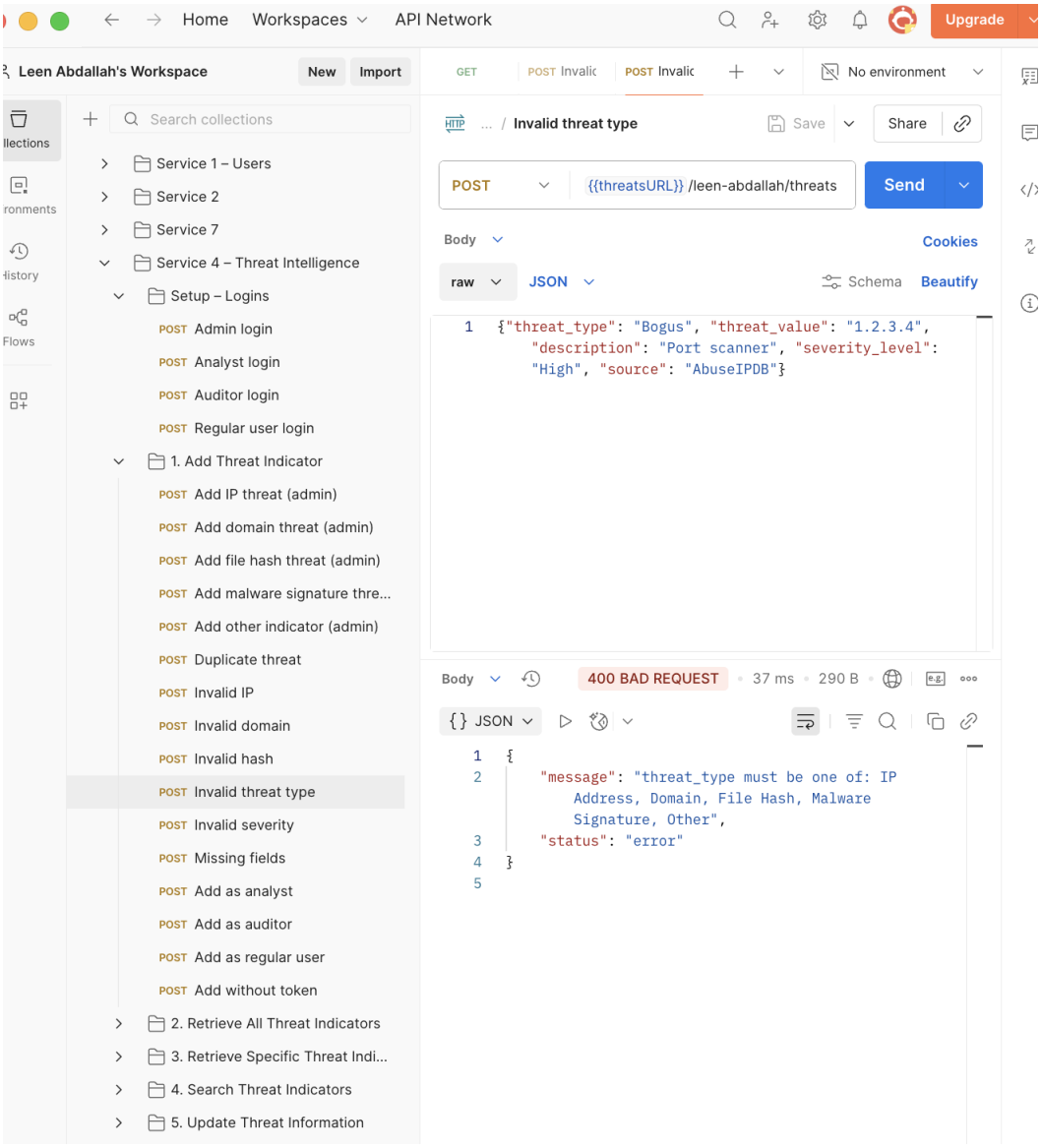


Figure 172: Leen Abdallah: Add Threat Indicator: invalid threat type, 400

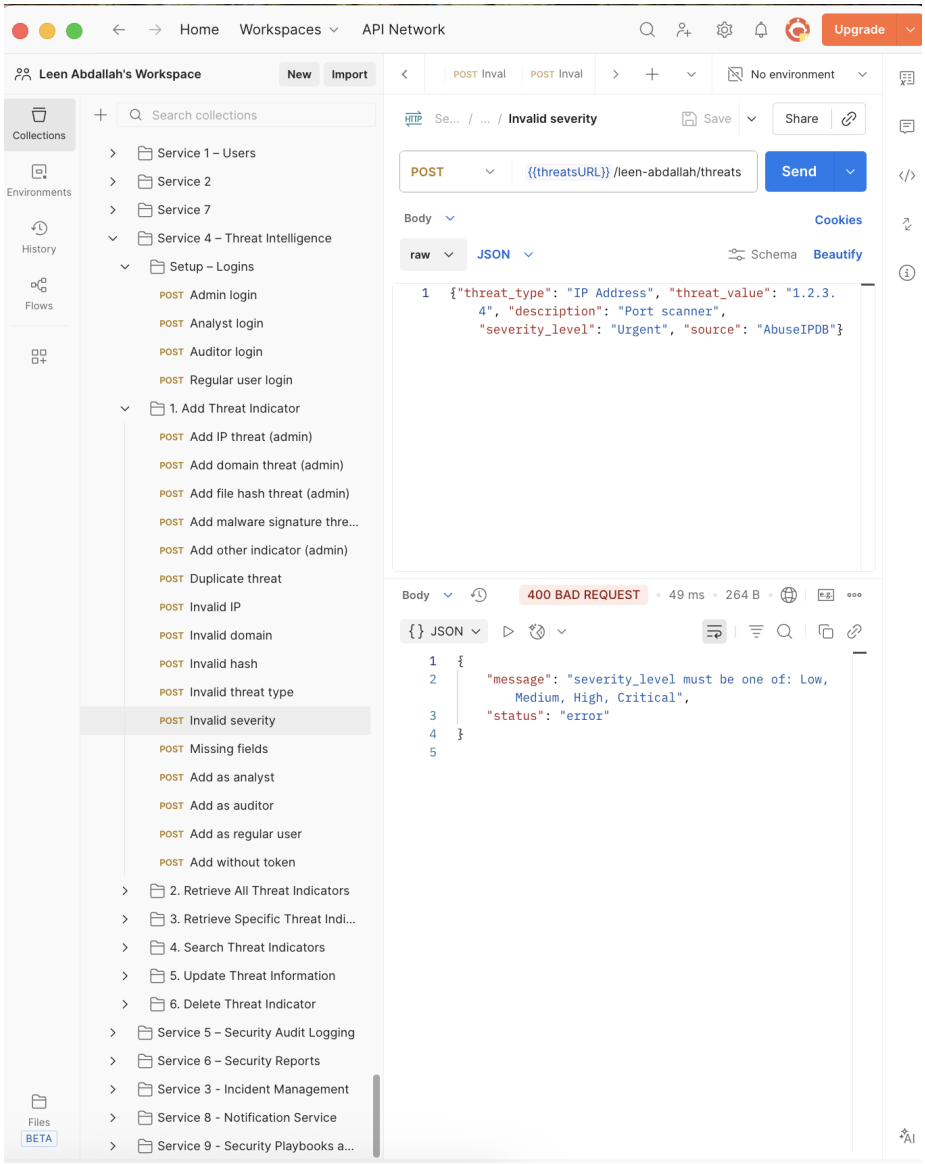


Figure 173: Leen Abdallah: Add Threat Indicator: invalid severity, 400

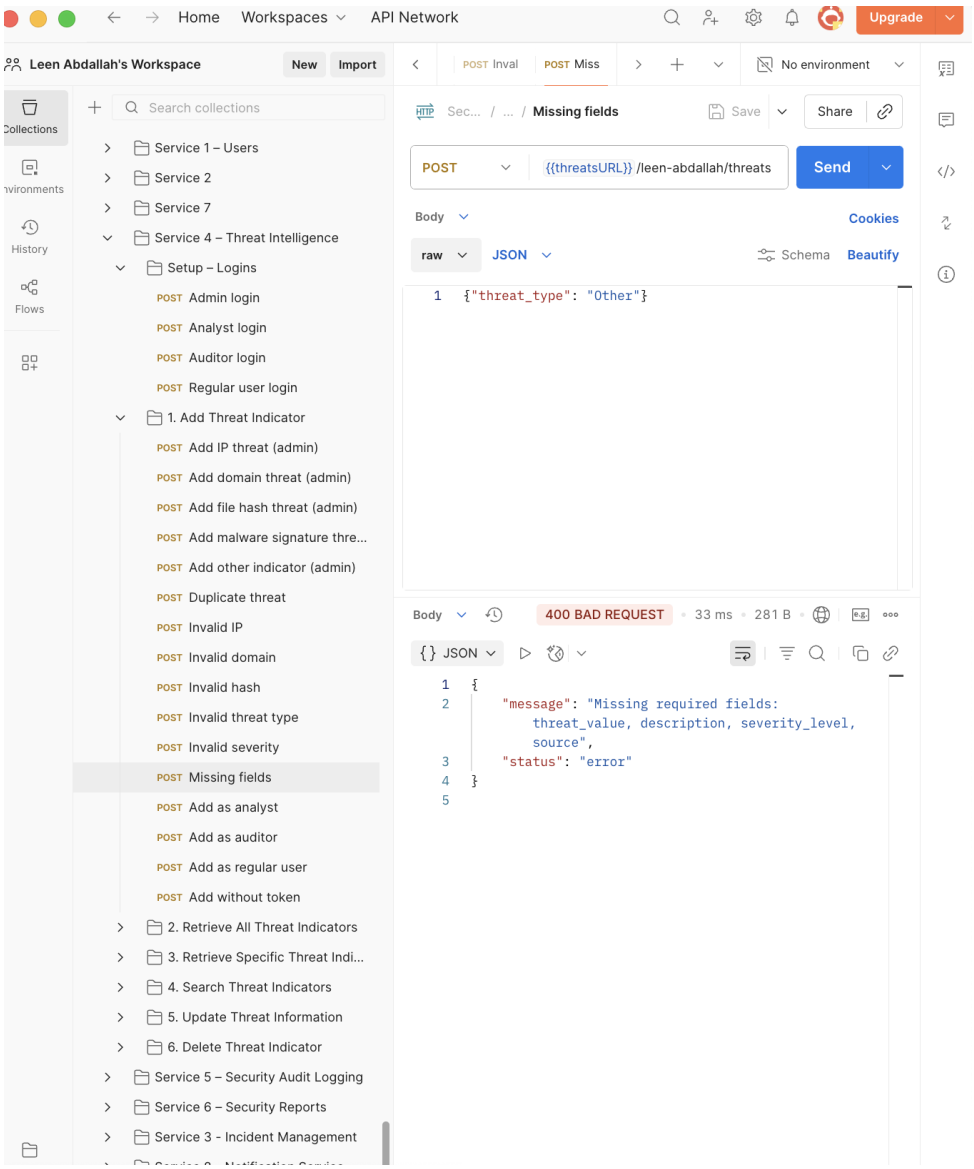


Figure 174: Leen Abdallah: Add Threat Indicator: missing fields, 400

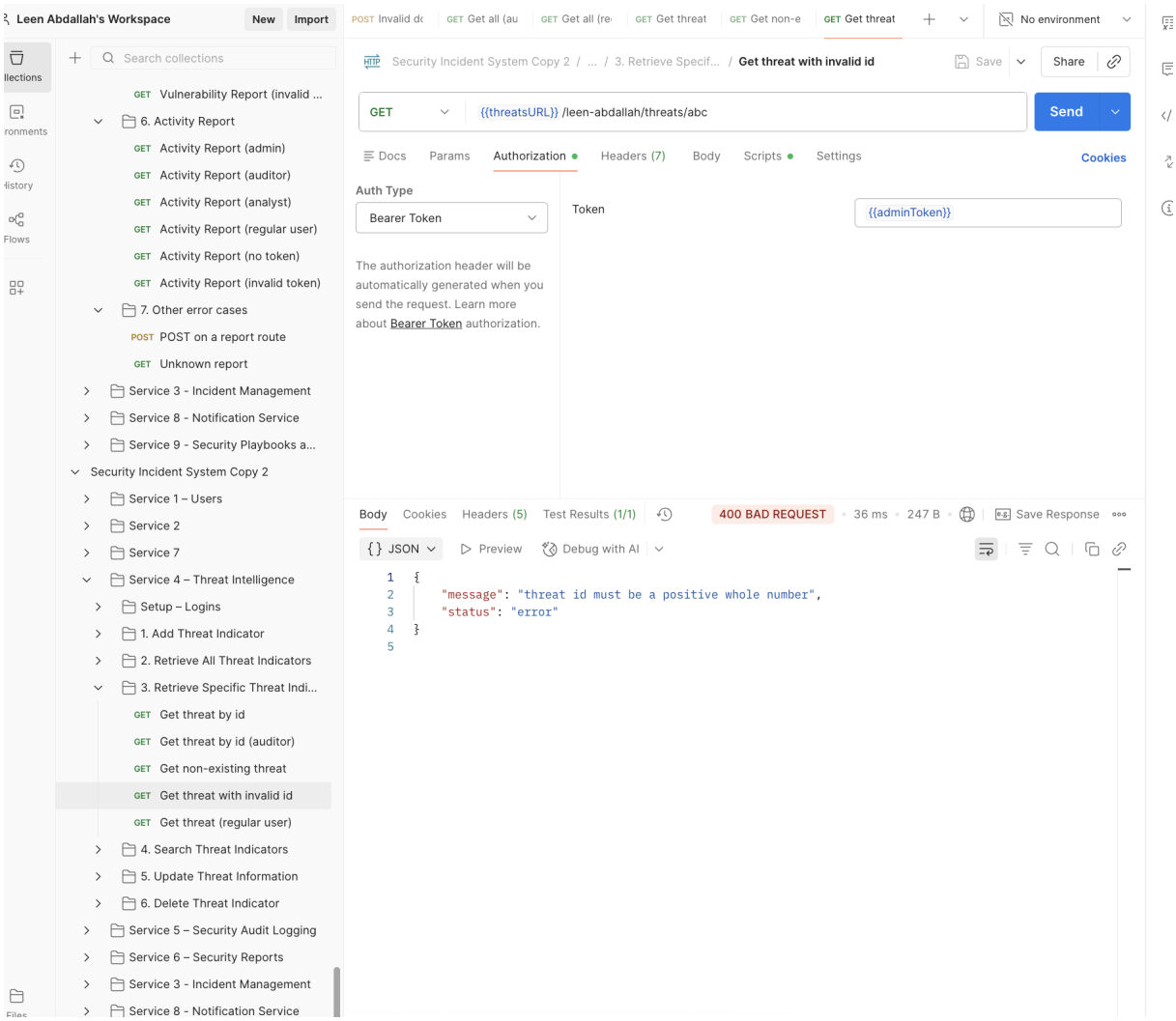


Figure 175: Leen Abdallah: Retrieve Specific Threat Indicator: invalid ID format, 400

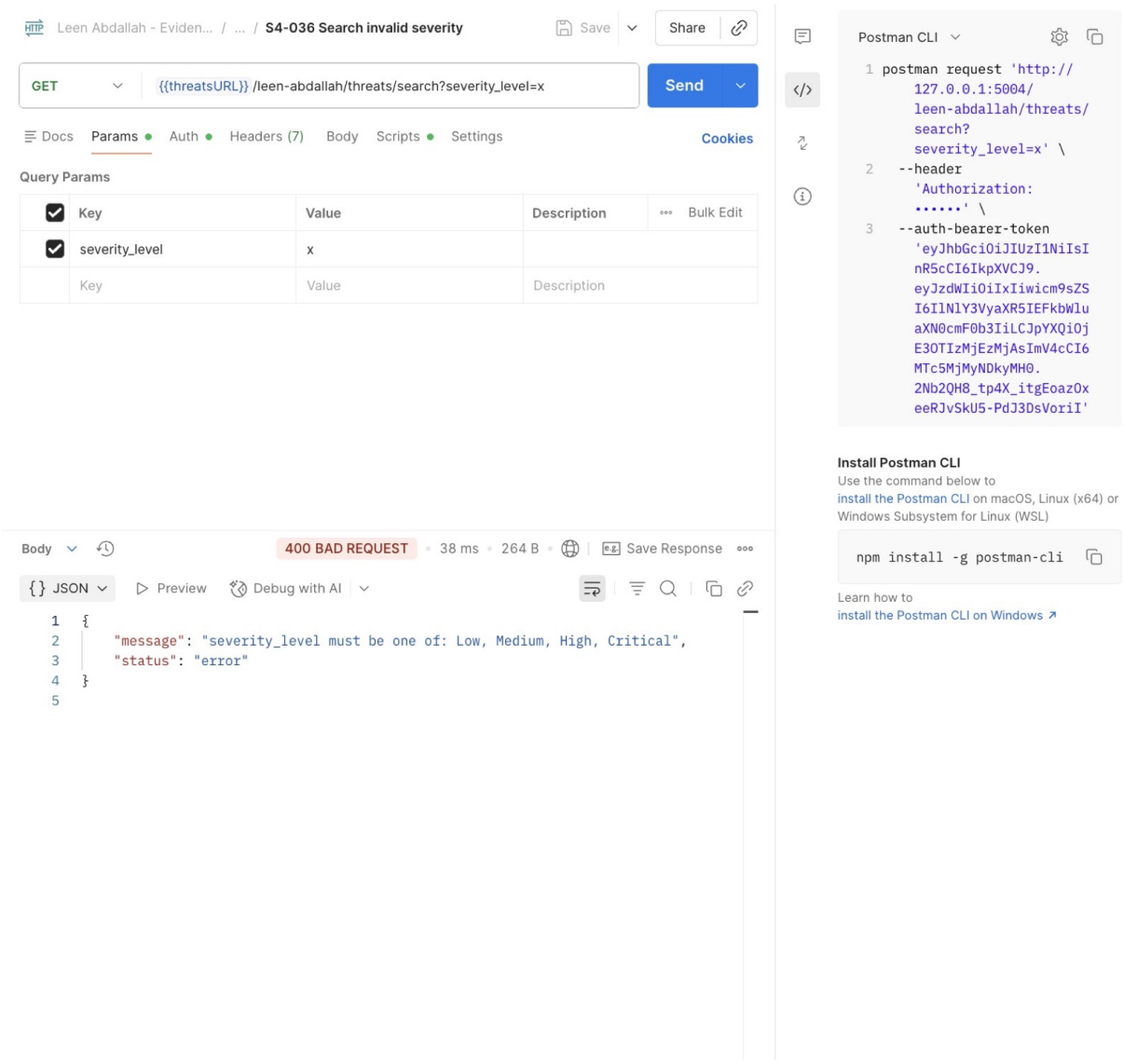


Figure 176: Leen Abdallah: Search Threat Indicators: invalid severity filter, 400

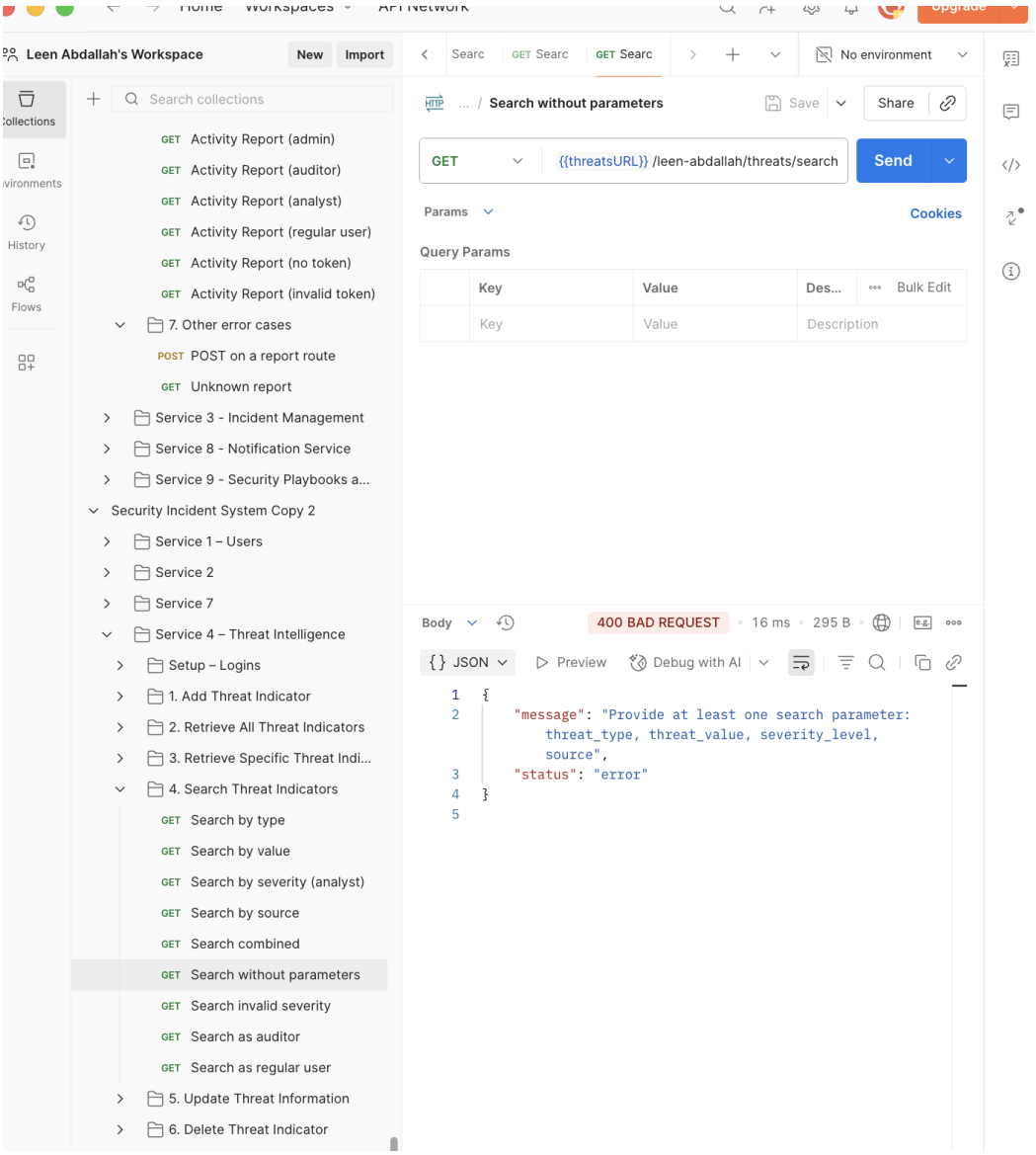


Figure 177: Leen Abdallah: Search Threat Indicators without parameters: at least one filter is required, 400

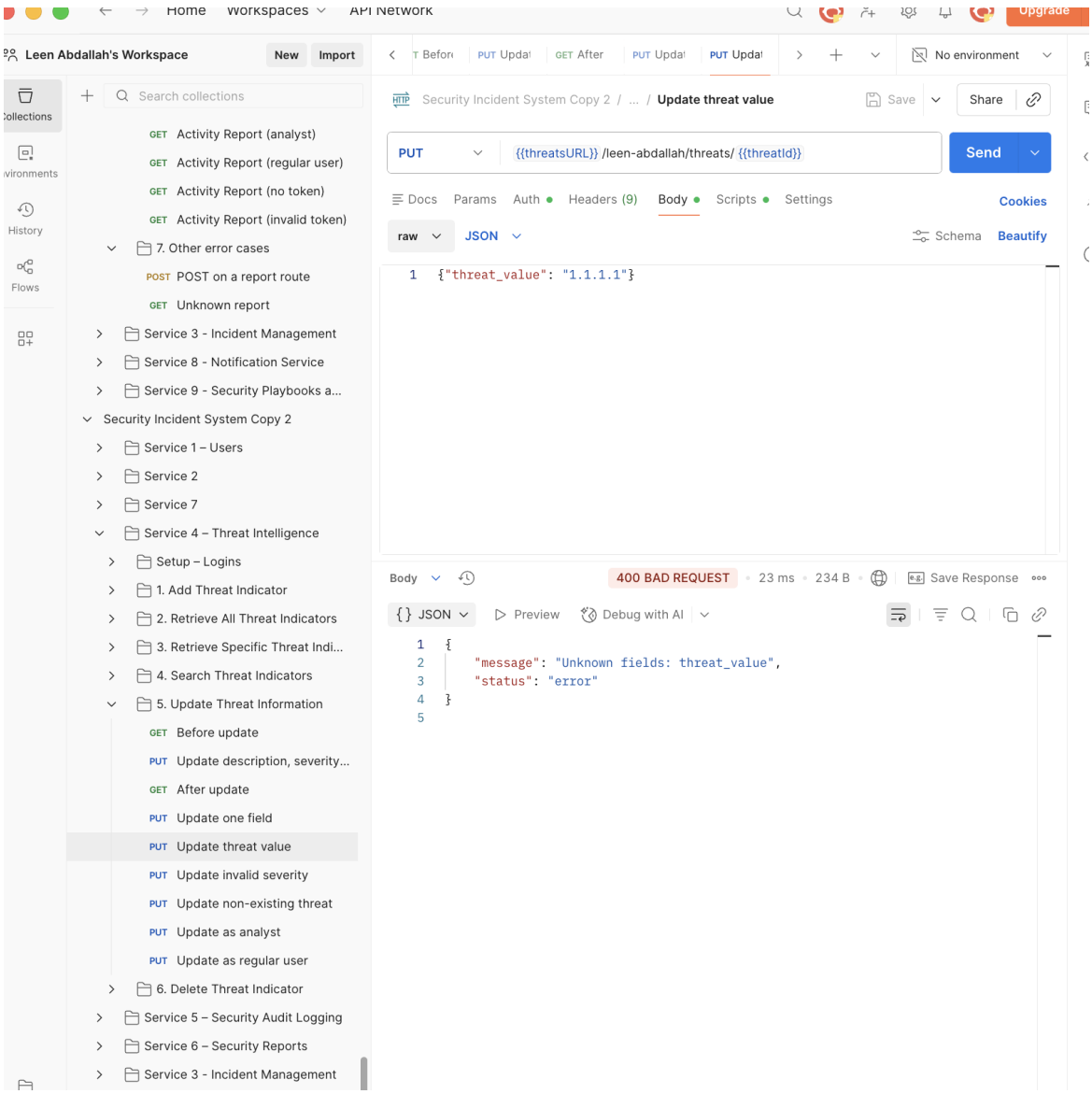


Figure 178: Leen Abdallah: Update Threat Information: changing threat_value is rejected as an unknown update field, 400

The update handler accepts only description, severity and source, validates their values and rejects empty bodies or other fields. Shared authorization checks run before the handler, so an analyst's 403 is an access result rather than evidence of field validation. Invalid or oversized threat IDs return 400; missing records return 404 for permitted callers.

Service 5 (Leen Abdallah)

Figures 179–187: The cases distinguish malformed input (400) from a missing retrieved log (404). In particular, the unknown user supplied to POST is rejected with 400.

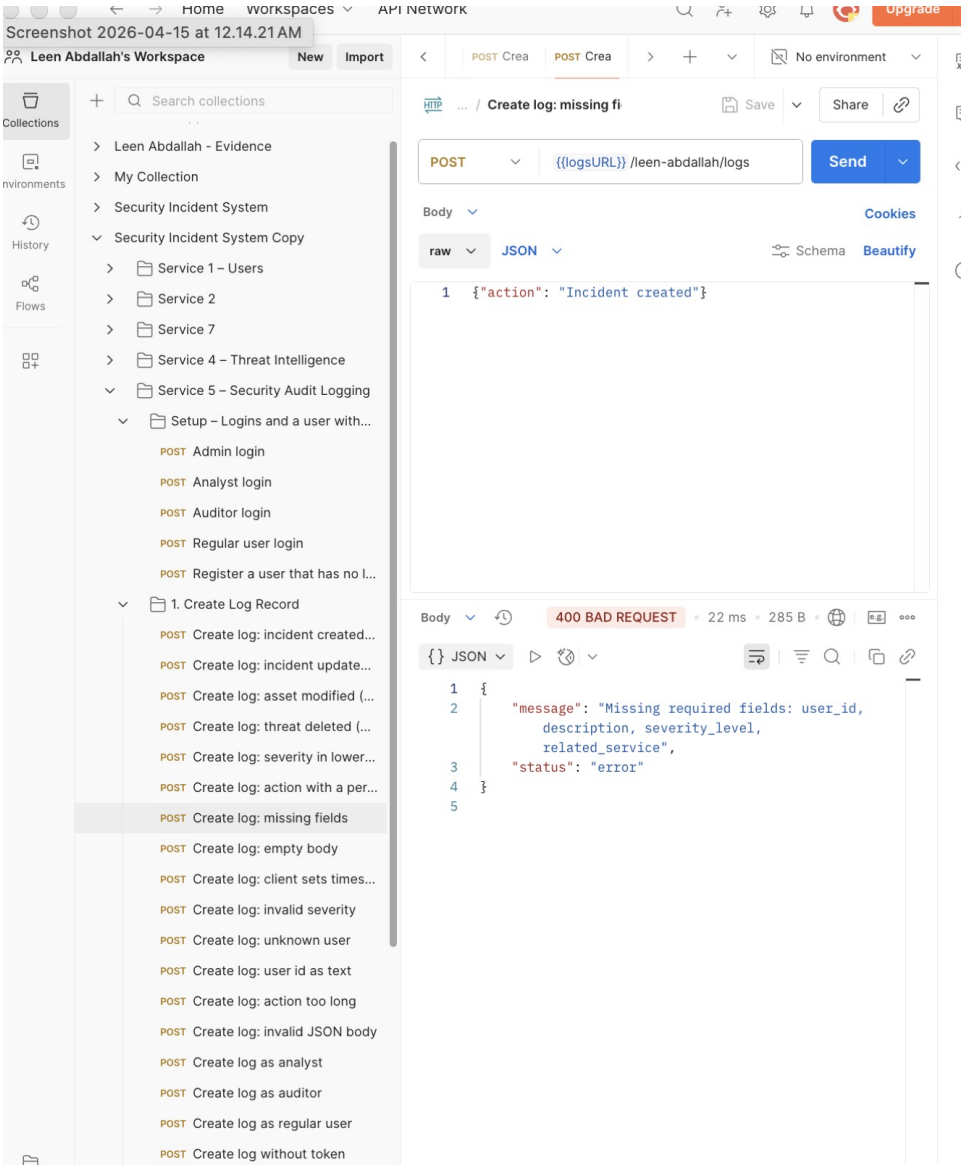


Figure 179: Leen Abdallah: Create Log Record: missing field, 400

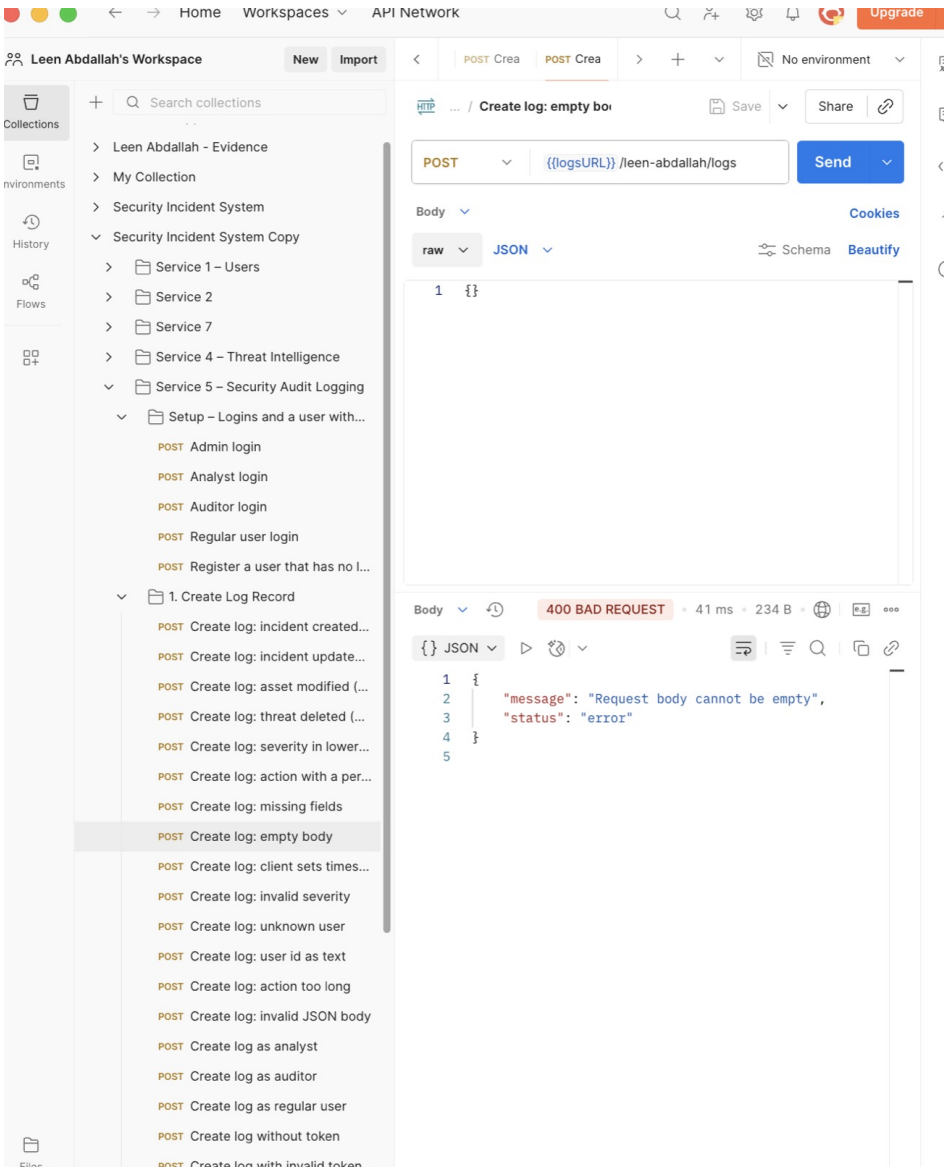


Figure 180: Leen Abdallah: Create Log Record: empty body, 400

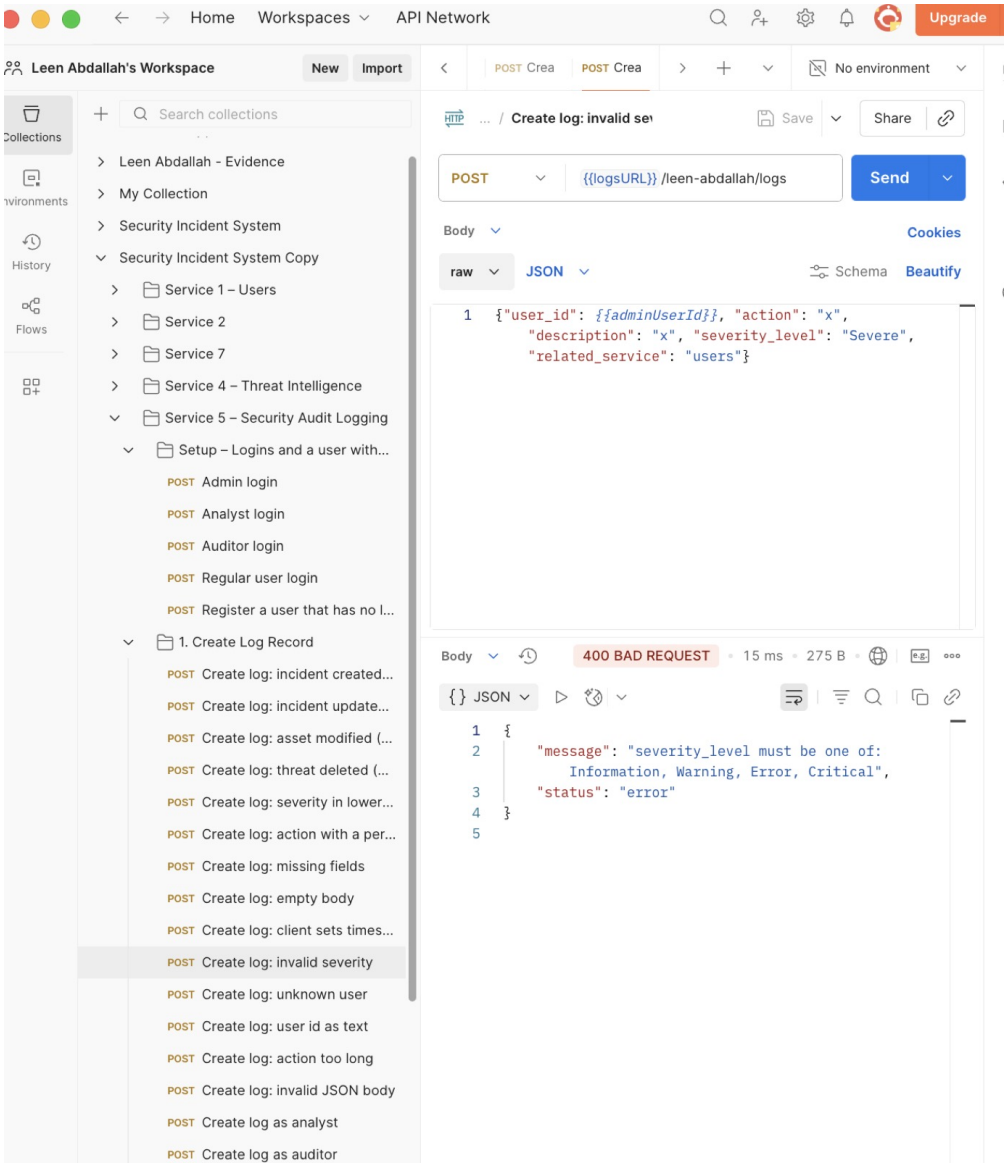


Figure 181: Leen Abdallah: Create Log Record: invalid severity, 400

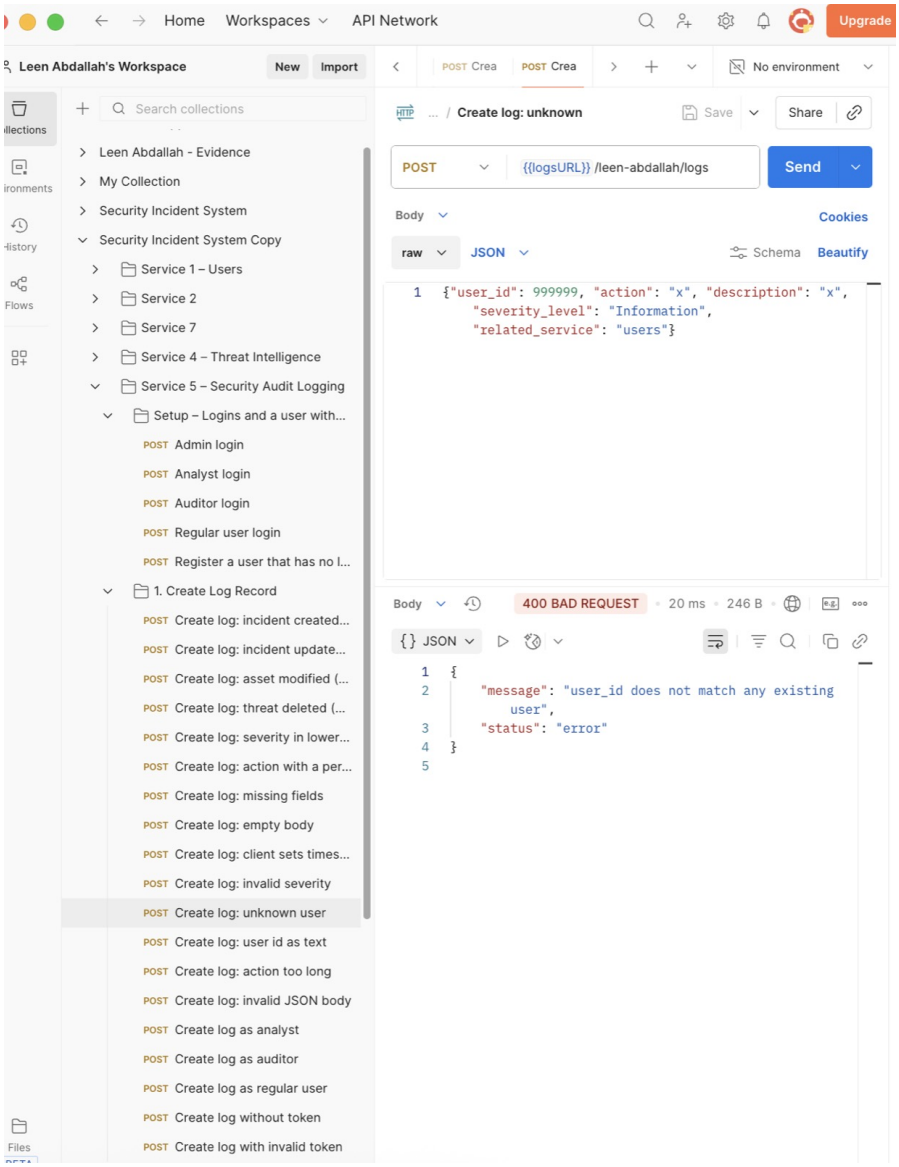


Figure 182: Leen Abdallah: Create Log Record: unknown user ID 999999 rejected, 400 Bad Request

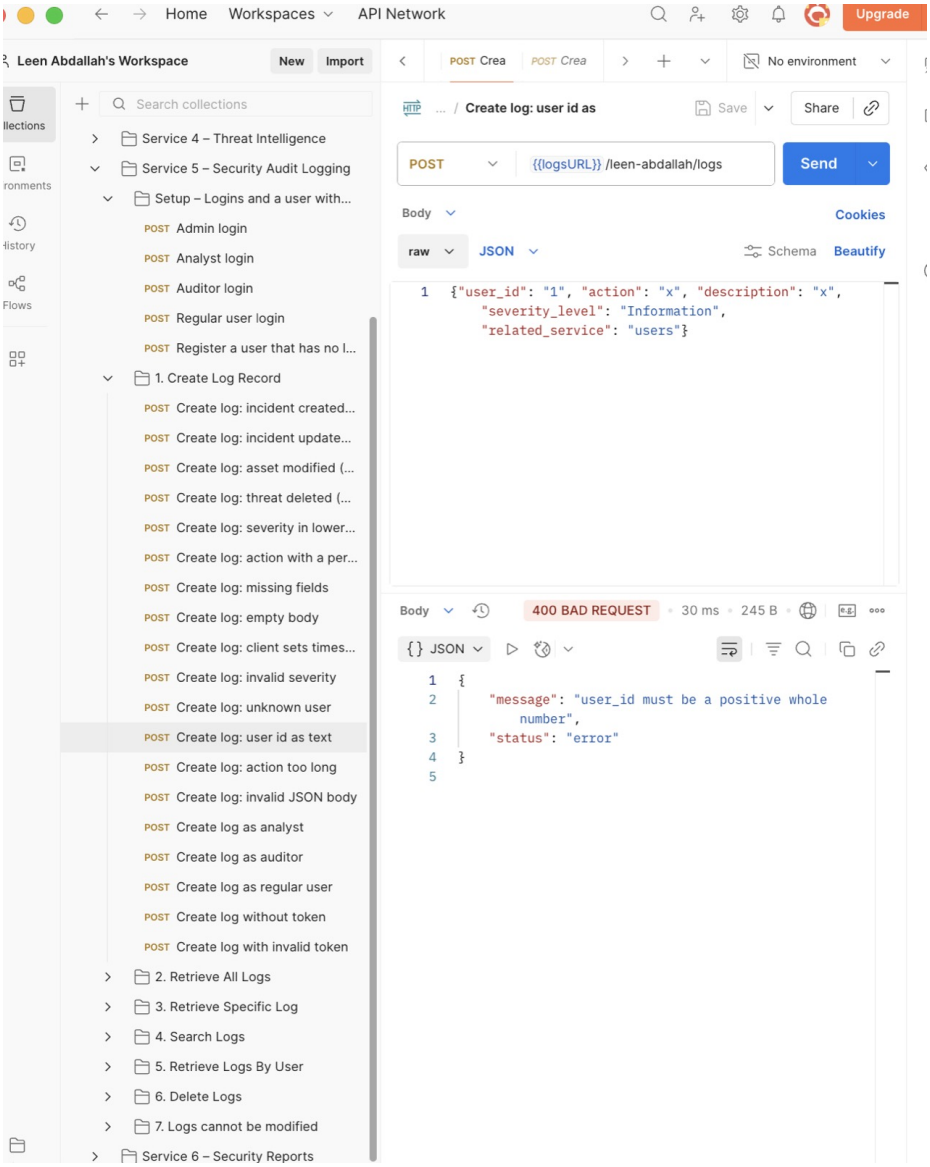
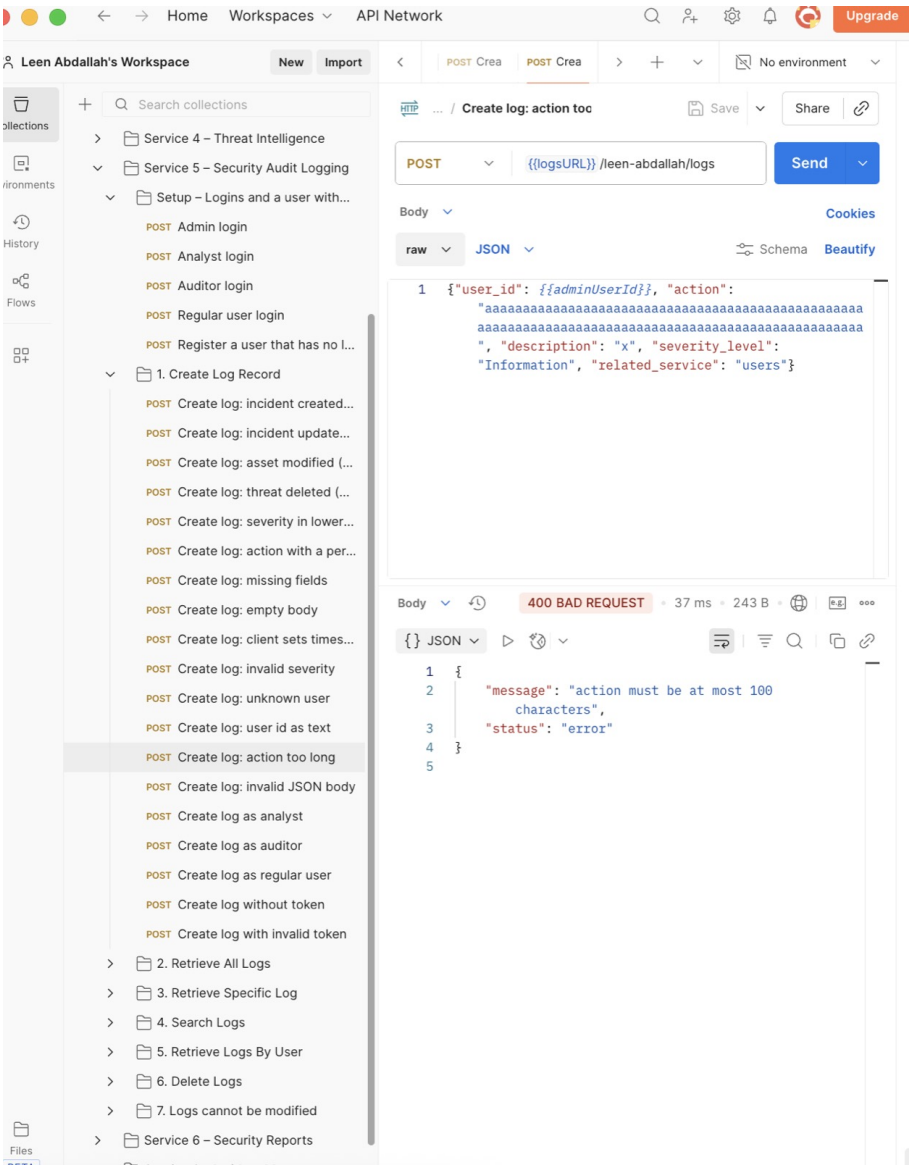


Figure 183: Leen Abdallah: Create Log Record: user_id sent as text, 400


Home
Workspaces
API Network
Upgrade
Leen Abdallah's Workspace
New
Import
Search collections
Service 4 – Threat Intelligence
Service 5 – Security Audit Logging
Setup – Logins and a user with...
POST Admin login
POST Analyst login
POST Auditor login
POST Regular user login
POST Register a user that has no l...
1. Create Log Record
POST Create log: incident created...
POST Create log: incident update...
POST Create log: asset modified (...
POST Create log: threat deleted (...
POST Create log: severity in lower...
POST Create log: action with a per...
POST Create log: missing fields
POST Create log: empty body
POST Create log: client sets times...
POST Create log: invalid severity
POST Create log: unknown user
POST Create log: user id as text
POST Create log: action too long
POST Create log: invalid JSON body
POST Create log as analyst
POST Create log as auditor
POST Create log as regular user
POST Create log without token
POST Create log with invalid token
2. Retrieve All Logs
3. Retrieve Specific Log
4. Search Logs
5. Retrieve Logs By User
6. Delete Logs
7. Logs cannot be modified
Service 6 – Security Reports
Create log: action too
Save
Share
POST
{{logsURL}} /leen-abdallah/logs
Send
Body
Cookies
raw
JSON
Schema
Beautify
{"user_id": {{adminUserId}}, "action":
"aaaa

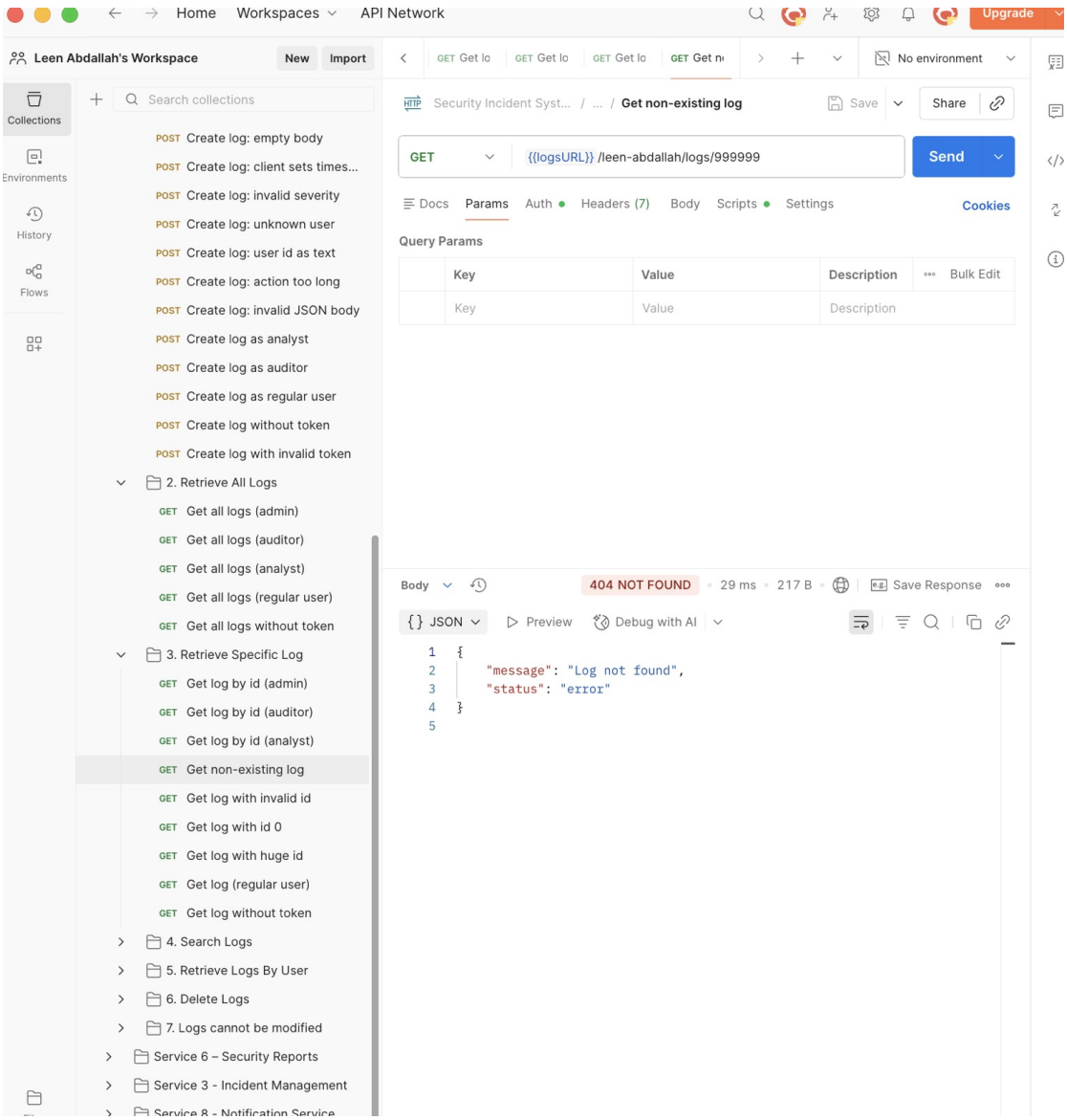


Figure 185: Leen Abdallah: Retrieve Specific Log: non-existing log, 404

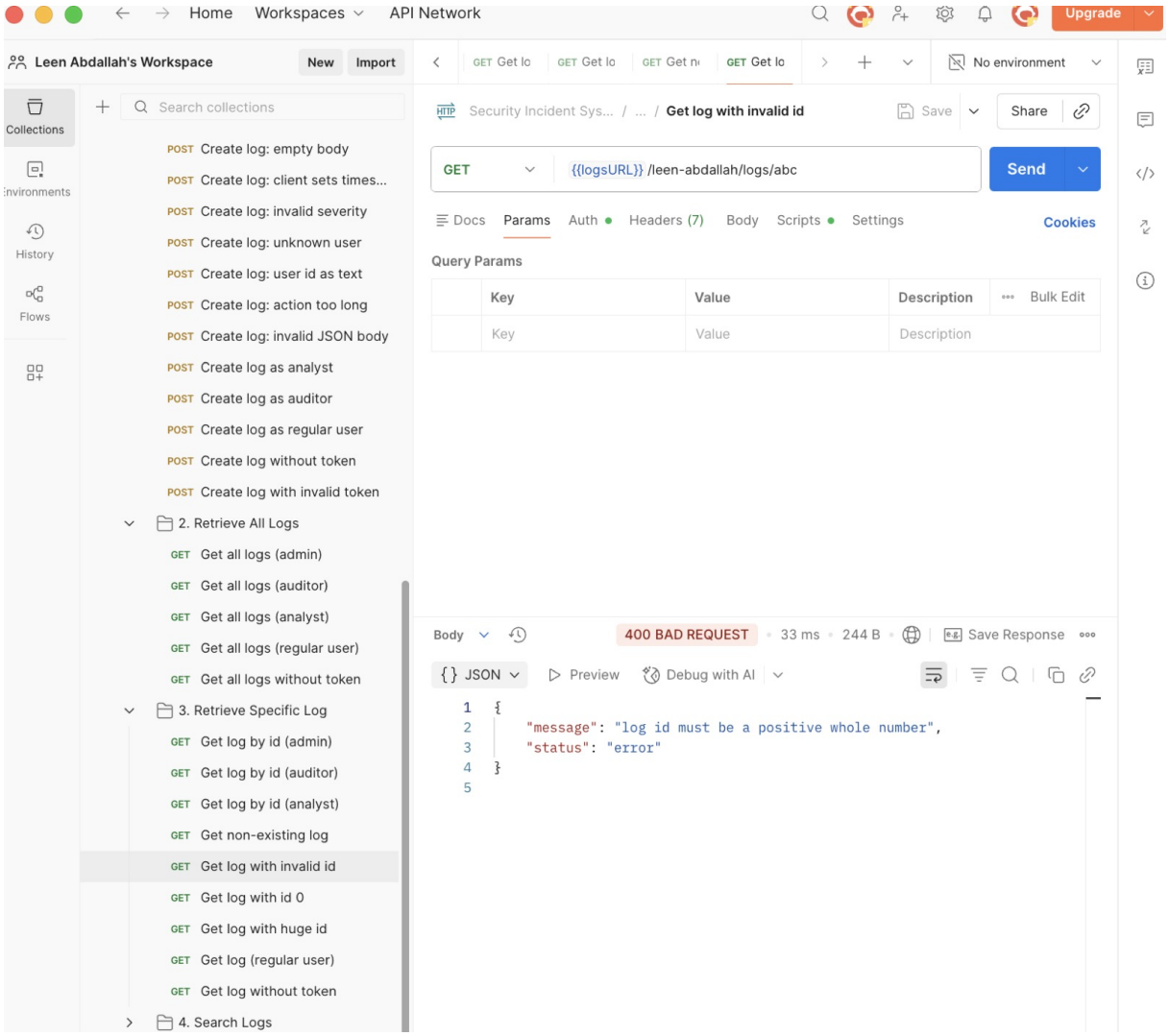


Figure 186: Leen Abdallah: Retrieve Specific Log: invalid ID, 400

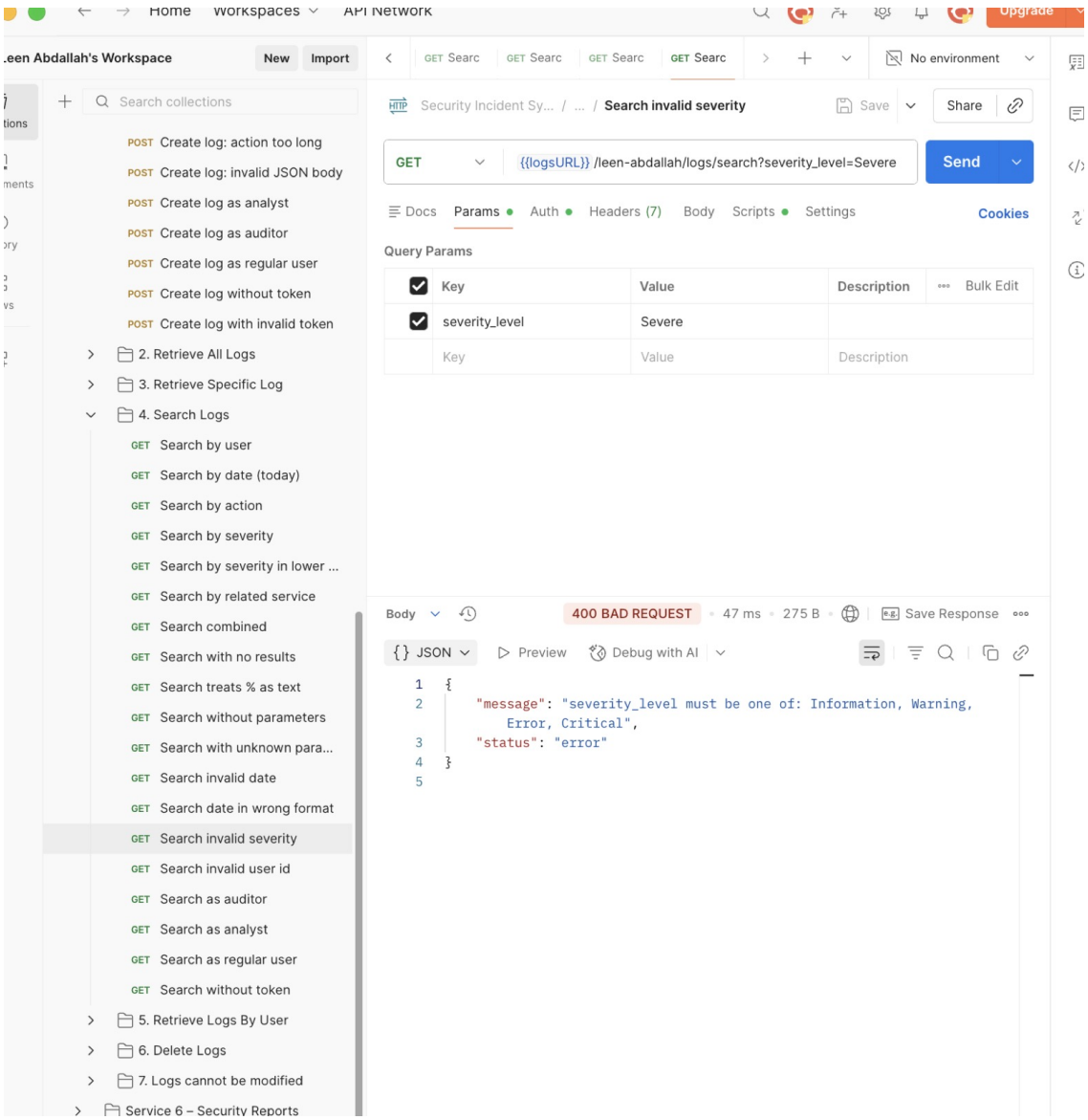


Figure 187: Leen Abdallah: Search Logs: invalid severity filter, 400

Service 6 (Leen Abdallah)

Figures 188–189: The service distinguishes an unsupported POST method (405) from an unknown report route (404).

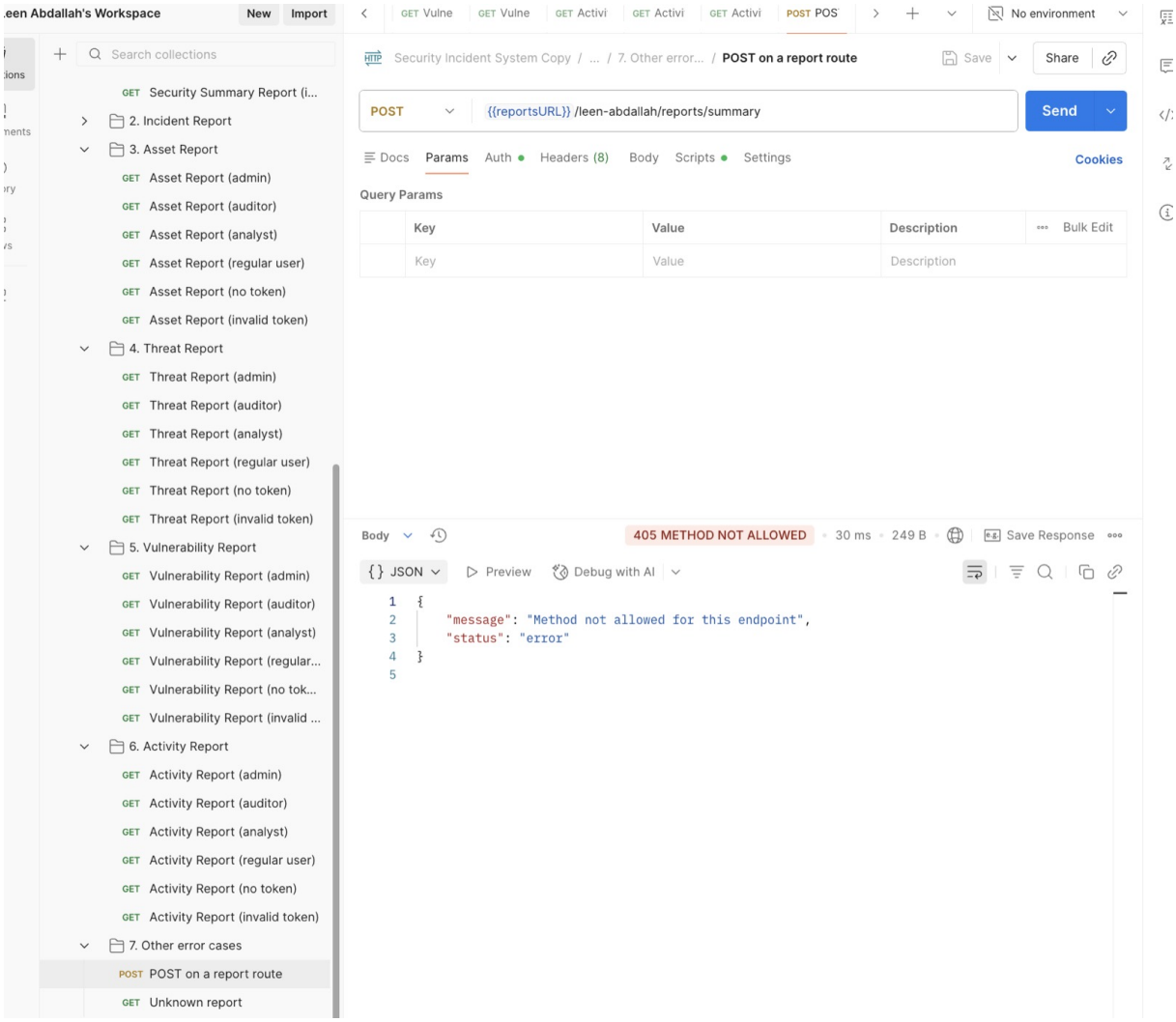


Figure 188: Leen Abdallah: POST on the summary-report GET route returns 405 Method Not Allowed

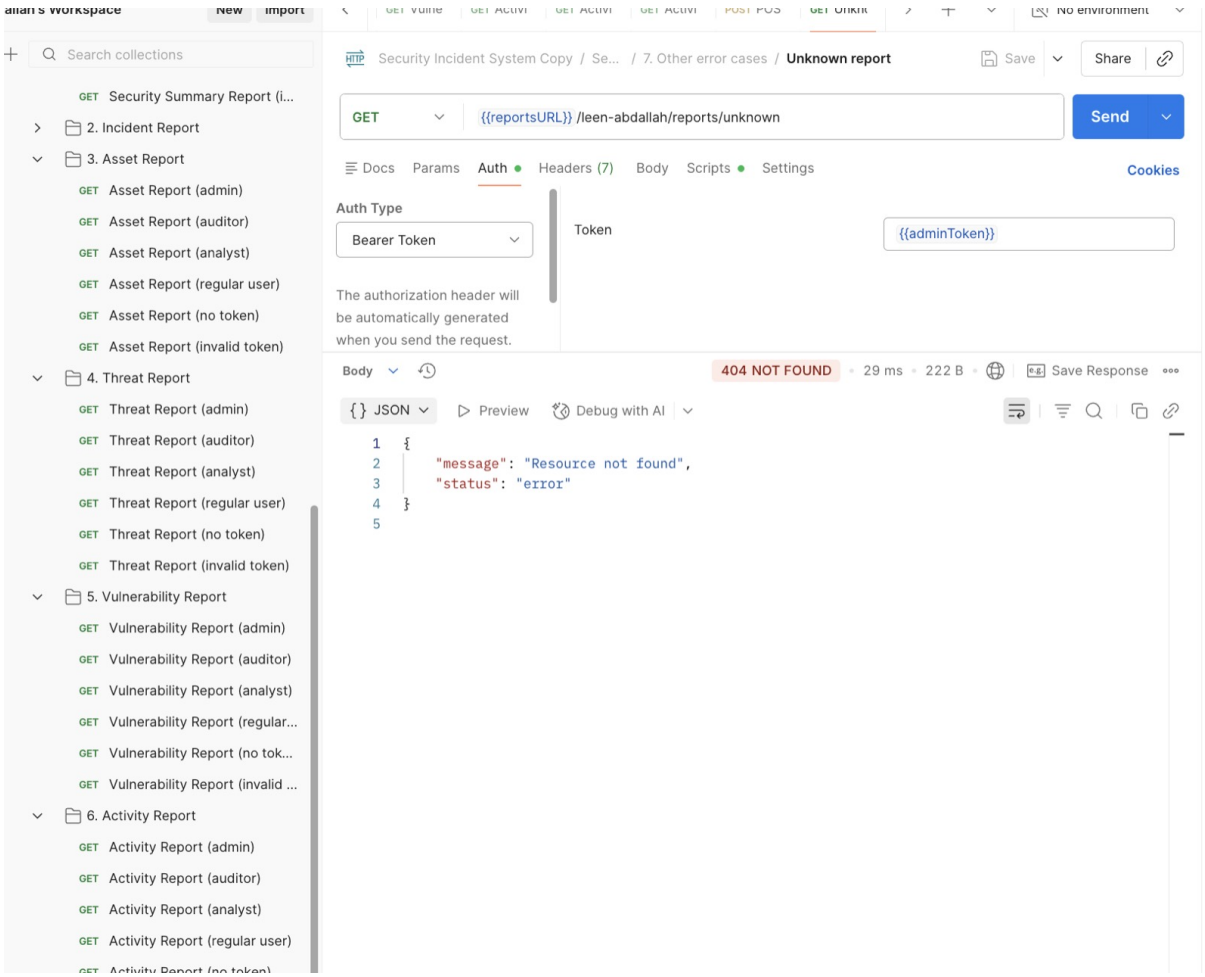


Figure 189: Leen Abdallah: Reports: unknown report name, 404

8 API Documentation

The supplied `postman/Security_Incident_System.postman_collection.json` contains 242 described requests and seven saved responses, not 242 verified passing tests. Service documentation is in `docs/`. Preserve implementer names exactly in request paths.

JSON writes use `Content-Type: application/json`; protected requests add `Authorization: Bearer <token>`. GET routes use path/query values without a body. PUT rows list accepted fields; handlers require at least one permitted change where enforced and reject unknown fields.

Successful responses use `{"status":"success","message":...,"data":...}`; errors use `{"status":"error","message":...}`. Creation normally returns `201`, while retrieval, update and deletion return `200`. Deletion returns a success envelope with null data. Authentication and permission errors are described in Section 9; validation and framework errors are described in Section 7.

8.1 Detailed API Examples

These three complete responses preserve captured values, with normalized key order and indentation. They are transcriptions, not new test results.

8.1.1 Register User (Adam Yahya)

`POST /users/register/AdamYahya` creates an account. Section 5.1 specifies fields and returned data; Figure 3 shows registration/201. Duplicate and invalid-email results appear in Section 7.

8.1.2 Login (Adam Yahya)

`POST /users/login/AdamYahya` returns a token and public user object. Section 9 explains credentials, expiry, account-state checks and captured success/failure responses.

8.1.3 Create Incident (Youssef Al Hajj Youness)

`POST /Youssef-Al-Hajj-Youness/incidents` requires an administrator or a Regular User reporting as themselves. Figure 37 shows this request and the resulting Open incident. Existing reporter and asset references are required.

```
Request:
{
  "title": "Unexpected administrator login",
  "description": "Multiple suspicious administrator logins were detected on Web Server 01 (
    Primary) at 10.0.0.5.",
  "category": "Unauthorized Access",
  "severity_level": "High",
  "reporter_id": 2,
  "asset_id": 1
}
Response 201:
{
  "status": "success",
  "message": "Incident created successfully",
  "data": {
    "asset_id": 1,
    "assigned_analyst_id": null,
```

```
    "category": "Unauthorized Access",
    "creation_date": "2026-10-17",
    "description": "Multiple suspicious administrator logins were detected on Web Server 01 (
    Primary) at 10.0.0.5.",
    "incident_id": 2,
    "investigation_notes": [],
    "reporter_id": 2,
    "resolution_date": null,
    "severity_level": "High",
    "status": "Open",
    "title": "Unexpected administrator login"
  }
}
```


Invalid fields return 400; a Regular User submitting another reporter returns 403; missing reporter or asset records return 404. Authentication failures return 401.

8.1.4 Add Threat Indicator (Leen Abdallah)

POST /leen-abdallah/threats accepts the five fields listed in Section 5.4. Figure 55 returns indicator 1, High, dated 19 October 2026, with 201. The matching stored row appears in Figure 60. Validation and permission errors appear in Sections 7 and 9.

8.1.5 Security Summary Report (Leen Abdallah)

GET /leen-abdallah/reports/summary requires an administrator or auditor and has no request body. Figure 95 contains this response. The totals and the definition of open incidents are reconciled in Section 5.6.


```
Response 200:
{
  "status": "success",
  "message": "Security summary report generated successfully",
  "data": {
    "assets": 4,
    "critical_threats": 2,
    "generated_at": "2026-10-18 12:39:35",
    "open_incidents": 4,
    "total_incidents": 6,
    "total_threats": 5,
    "users": 7
  }
}
```


A missing or invalid token returns 401; disallowed roles return 403; unavailable or unusable source-service responses return 503. A POST to this GET route returns 405.

8.1.6 Mark a Personal Notification as Read (Youssef Al Hajj Youness)

PUT /Youssef-Al-Hajj-Youness/notifications/6/read has no JSON body. Figure 124 shows a Regular User marking their own notification as read. The authenticated user is 2, matching the recipient in the response.


```
Response 200:
{
  "status": "success",
```

```
  "message": "Notification marked as read",
  "data": {
    "created_at": "2026-10-17 18:30:39",
    "message": "Incident #2 was closed after investigation.",
    "notification_id": 6,
    "notification_type": "System Notification",
    "read_date": "2026-10-17 18:31:45",
    "severity_level": "Low",
    "status": "Read",
    "title": "Incident #2 closed",
    "user_id": 2
  }
}
```


Auditors cannot mark notifications as read. Analysts and Regular Users cannot mark another user's notification; ownership denial returns `403`. An administrator requesting an absent record receives `404`. A scoped caller may receive `403` for an absent record as part of the ownership check, so missing-record behavior depends on the caller.

8.2 Response Data by Operation

Service	Successful data payload
Users	Register/update/get one: public account object. List: account list. Login: token, token type and public user object. Password hashes are excluded.
Assets	Create/update/get one: asset object; specific retrieval also includes current related incidents and vulnerabilities. List/search: asset list.
Incidents	Create/get one/assign/status: incident object, with notes on detailed retrieval. List: incident list. Add note: created note object.
Threats	Create/get one/update: indicator object. List/search: indicator list.
Logs	Create/get one: log object. List/search/user history: log list, including an empty list when no entries match.
Reports	Statistics object for the selected report, including generation time and the implemented totals, breakdowns or recent entries.
Vulnerabilities	Create/get one/update: vulnerability object. List/search: vulnerability list.
Notifications	Create/get one/mark read: notification object with public status and read date. User history/search: notification list.
Playbooks	Create: playbook object. List: playbook list. Specific retrieval includes ordered actions and execution-history entries. Add/update action: action object.
All deletion routes	Null data with a success message; dependent-record constraints may prevent deletion.

8.3 Endpoint Summary

The table lists all 54 operation/method combinations. Errors describe expected application responses; unexpected failures can also return `500`. Unsupported methods can return `405` independently of this list. The complete named prefix is required for Services 3–6, 8 and 9.

Method	Endpoint	Required parameters / body	Errors
Service 1: Users (Adam Yahya)			
POST	`/users/register/AdamYahya`	full_name, username, email, password, department	400 401 403 409
POST	`/users/login/AdamYahya`	username or email, password	400 401 403
GET	`/users/AdamYahya`	–	401 403
GET	`/users/<username>/AdamYahya`	path: username	401 403 404
PUT	`/users/<id>/AdamYahya`	path: id; body: full_name, email, department	400 401 403 404 409
DELETE	`/users/<id>/AdamYahya`	path: id	400 401 403 404 409
Service 2: Assets (Adam Yahya)			
POST	`/assets/AdamYahya`	asset_name, asset_type, ip_address, department, owner_id, criticality_level	400 401 403
GET	`/assets/AdamYahya`	–	401 403
GET	`/assets/<id>/AdamYahya`	path: id	400 401 403 404
GET	`/assets/search/AdamYahya`	query: asset_name, asset_type, ip_address, department, criticality_level	400 401 403
PUT	`/assets/<id>/AdamYahya`	asset_name, owner_id, department, criticality_level, status	400 401 403 404
DELETE	`/assets/<id>/AdamYahya`	path: id	400 401 403 404 409
Service 3: Incidents (Youssef Al Hajj Youness), prefix /Youssef-Al-Hajj-Youness			
POST	`/incidents`	title, description, category, severity_level, reporter_id, asset_id	400 401 403 404
GET	`/incidents`	–	401 403
GET	`/incidents/<id>`	path: id	400 401 403 404
PUT	`/incidents/<id>/assign`	analyst_id	400 401 403 404 503
PUT	`/incidents/<id>/status`	status	400 401 403 404 409
POST	`/incidents/<id>/notes`	note_content	400 401 403 404
Service 4: Threats (Leen Abdallah), prefix /leen-abdallah			
POST	`/threats`	threat_type, threat_value, description, severity_level, source	400 401 403 409
GET	`/threats`	–	401 403
GET	`/threats/<id>`	path: id	400 401 403 404
GET	`/threats/search`	query: threat_type, threat_value, severity_level, source	400 401 403
PUT	`/threats/<id>`	description, severity_level, source	400 401 403 404
DELETE	`/threats/<id>`	path: id	400 401 403 404
Service 5: Logs (Leen Abdallah), prefix /leen-abdallah			
POST	`/logs`	user_id, action, description, severity_level, related_service	400 401 403
GET	`/logs`	–	401 403
GET	`/logs/<id>`	path: id	400 401 403 404
GET	`/logs/search`	query: user_id, date, action, severity_level, related_service	400 401 403
GET	`/logs/user/<user_id>`	path: user_id	400 401 403 404

Method	Endpoint	Required parameters / body	Errors
DELETE	`/logs/<id>`	path: id	400 401 403 404
Service 6: Reports (Leen Abdallah), prefix /leen-abdallah			
GET	`/reports/summary`	–	401 403 503
GET	`/reports/incidents`	–	401 403 503
GET	`/reports/assets`	–	401 403 503
GET	`/reports/threats`	–	401 403 503
GET	`/reports/vulnerabilities`	–	401 403 503
GET	`/reports/activity`	–	401 403 503
Service 7: Vulnerabilities (Adam Yahya)			
POST	`/vulnerabilities/AdamYahya`	title, description, cve_id, cvss_score, severity_level, asset_id, discovery_source	400 401 403 409
GET	`/vulnerabilities/AdamYahya`	–	401 403
GET	`/vulnerabilities/<id>/AdamYahya`	path: id	400 401 403 404
GET	`/vulnerabilities/search/AdamYahya`	query: cve_id, severity_level, asset_id, status	400 401 403
PUT	`/vulnerabilities/<id>/AdamYahya`	description, severity_level, status, cvss_score, resolution_date	400 401 403 404
DELETE	`/vulnerabilities/<id>/AdamYahya`	path: id	400 401 403 404
Service 8: Notifications (Youssef Al Hajj Youness), prefix /Youssef-Al-Hajj-Youness			
POST	`/notifications`	user_id, title, message, notification_type, severity_level	400 401 403 404
GET	`/notifications/user/<user_id>`	path: user_id	400 401 403 404
GET	`/notifications/<id>`	path: id	400 401 403 404
PUT	`/notifications/<id>/read`	path: id	400 401 403 404
GET	`/notifications/search`	query: user_id, notification_type, severity_level, status	400 401 403
DELETE	`/notifications/<id>`	path: id	400 401 403 404
Service 9: Playbooks (Youssef Al Hajj Youness), prefix /Youssef-Al-Hajj-Youness			
POST	`/playbooks`	playbook_name, description, incident_category, severity_level	400 401 403
GET	`/playbooks`	–	401 403
GET	`/playbooks/<id>`	path: id	400 401 403 404
POST	`/playbooks/<id>/actions`	action_description, step_number	400 401 403 404 409
PUT	`/actions/<id>/status`	status, assigned_user_id, completion_date (at least one)	400 401 403 404
DELETE	`/playbooks/<id>`	path: id	400 401 403 404

9 Authentication and Authorization

9.1 Authentication Mechanism

The client submits a username or email and password to `POST /users/login/AdamYahya`. Service 1 verifies the stored password hash. Unknown users and incorrect passwords receive the same `401` message; Inactive and Suspended accounts receive `403`. The implementation records login outcomes through the shared audit helper.

Login issues an JWT valid for 60 minutes, with the user ID as subject. Protected requests verify its signature and expiry, then reload the account. Current role and Active status control access; the token's role field is informational. Database-side permission changes affect the next request, although no role/status update API exists.

No refresh, logout or explicit revocation endpoint exists. The default secret and seeded administrator credentials are development settings; production readiness is not established.

9.2 User Roles

Security Administrator Creates and manages the platform's business records, assigns incidents and accesses all reports. Administrator-only deletion routes remain subject to referential constraints and self-deletion rules.

Security Analyst Investigates assigned incidents, adds notes, reads and searches permitted asset/threat/log data, updates vulnerabilities, manages own notifications and adds or updates response actions. Analysts cannot administer users, delete records or generate reports.

Regular Employee/User Registers an account, reads and updates their own permitted profile fields, reports incidents as themselves, follows their own reports and reads or marks their own notifications.

Auditor Reads the permitted records and reports without modifying them through protected operations. Threat search is denied by the implemented Service 4 route; this differs from the assignment's broader general role description.

9.3 Permission Management and Access Control Rules

The `roles_required` decorator authenticates the caller, loads current account data and checks the route's allowed roles. Handlers add ownership and assignment checks when role membership alone is insufficient. For example, an analyst may inspect an incident only when assigned to it, and a Regular User may retrieve only their own notification. Denials attempt to write an audit event; standalone logging is best effort.

Service	Admin	Analyst	User	Auditor
Users	Exposed APIs	None	Own profile	Read
Assets	Exposed APIs	Read one, search	None	Read, search
Incidents	Exposed APIs	Assigned: read, status, notes	Create as self; own read	Read
Threats	Exposed APIs	Read, search	None	Read; no search
Logs	Exposed APIs	Read one, search	None	Read, search
Reports	All six	None	None	All six
Vulnerabilities	Exposed APIs	Read, search, update	None	Read, search

Service	Admin	Analyst	User	Auditor
Notifications	Exposed APIs	Own read, mark read, search	Own read, mark read	Read, search
Playbooks	Exposed APIs	Read; add/update actions	None	Read

9.4 Authentication and Authorization Evidence

The figures pair successful and refused operations, followed by service-specific role and ownership checks. Request names and token-variable labels alone do not prove identity; conclusions follow visible results and reviewed route checks.

Successful authentication

Figures 190–191: The two 200 responses show successful token issuance for distinct accounts.

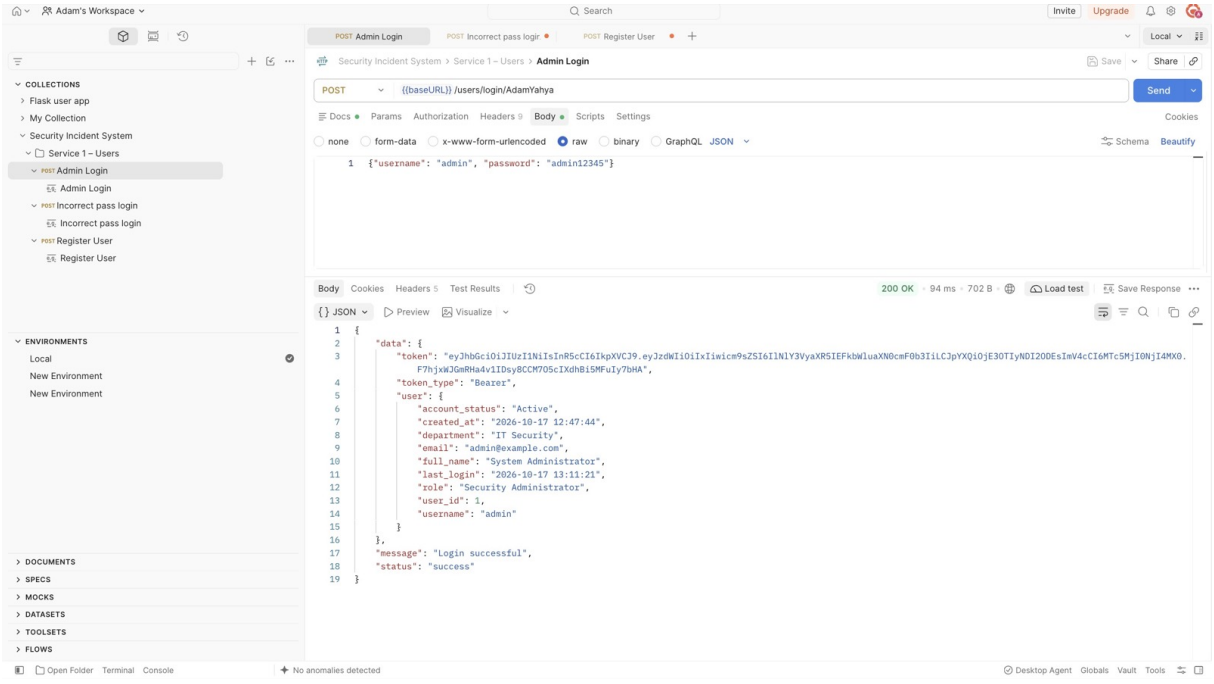


Figure 190: Adam Yahya: Login (POST /users/login/AdamYahya): valid credentials, 200 with token

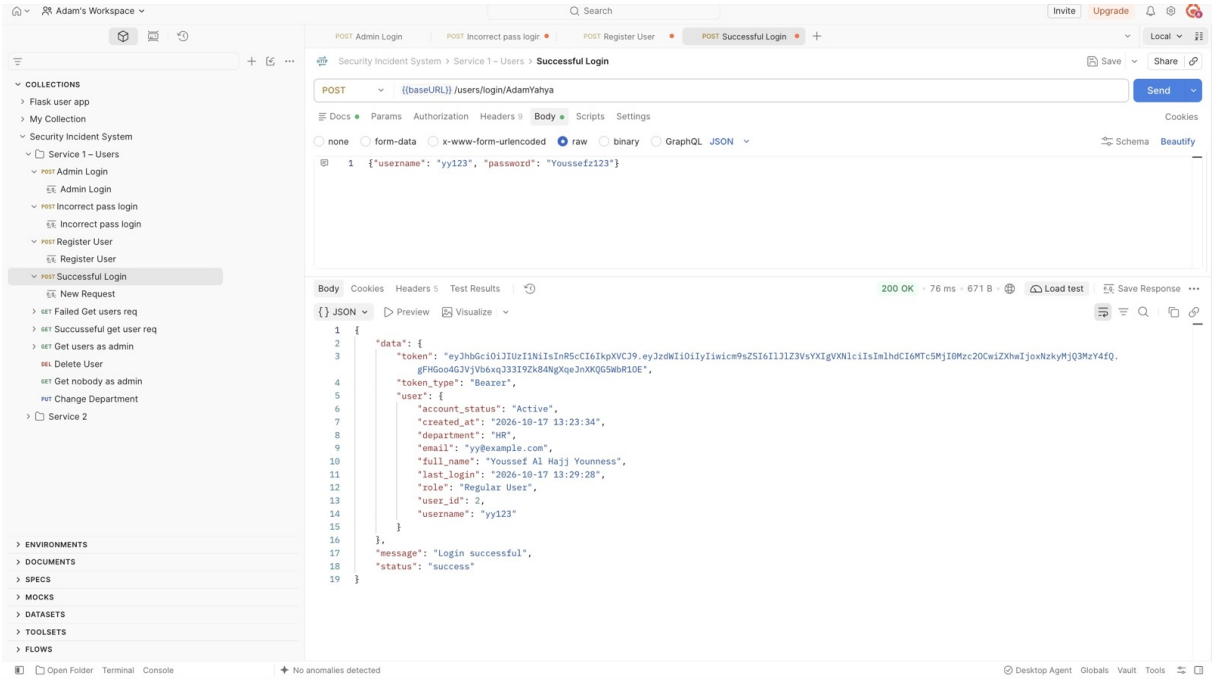


Figure 191: Adam Yahya: Login: a second user logs in successfully, 200

Failed authentication

Figure 192: The wrong password produces 401 with the generic credential-error message.

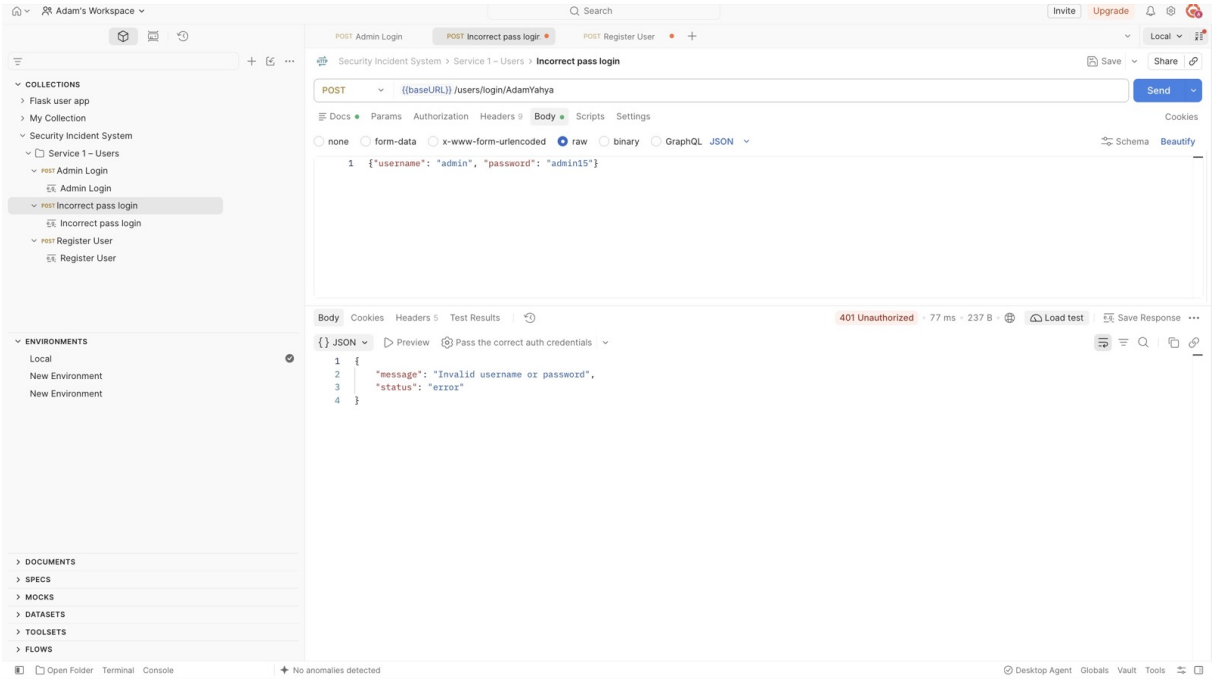


Figure 192: Adam Yahya: Login: wrong password, 401 Invalid username or password

Accessing an API with insufficient permissions, then with correct permissions

Figures 193–197: The user list rejects a Regular User and accepts an administrator. Threat creation likewise rejects nonadministrators and accepts an administrator.

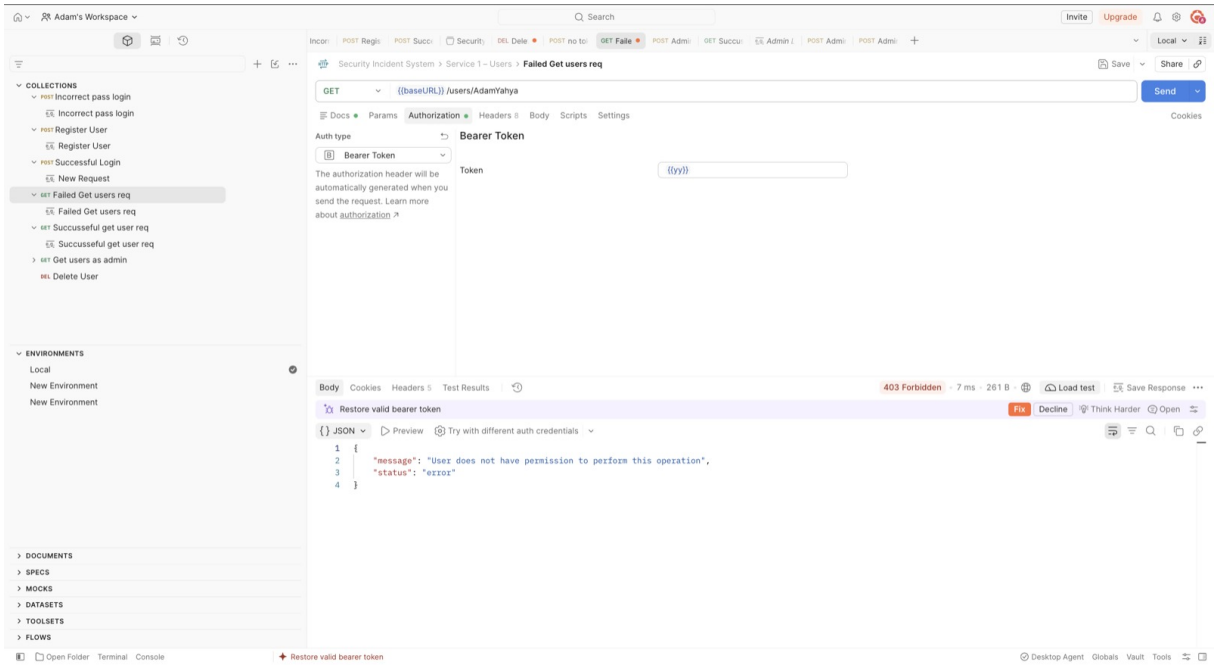


Figure 193: Adam Yahya: Get All Users (GET /users/AdamYahya): Regular User, insufficient permissions, 403

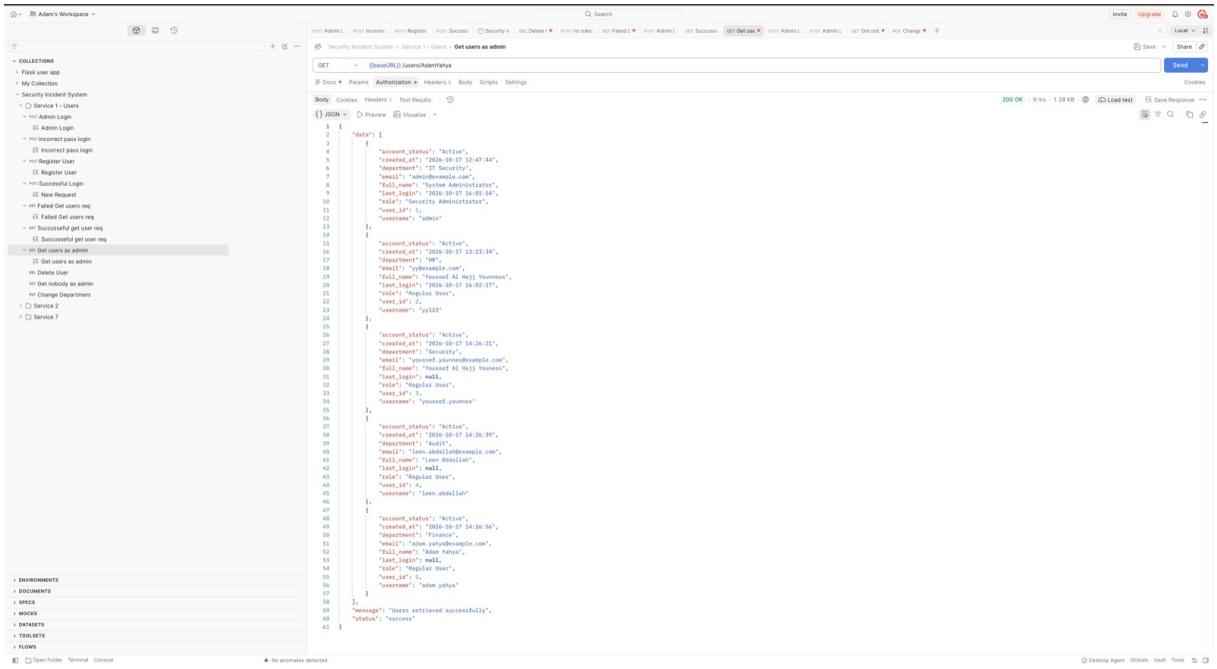


Figure 194: Adam Yahya: Get All Users: the same API with the Security Administrator, 200

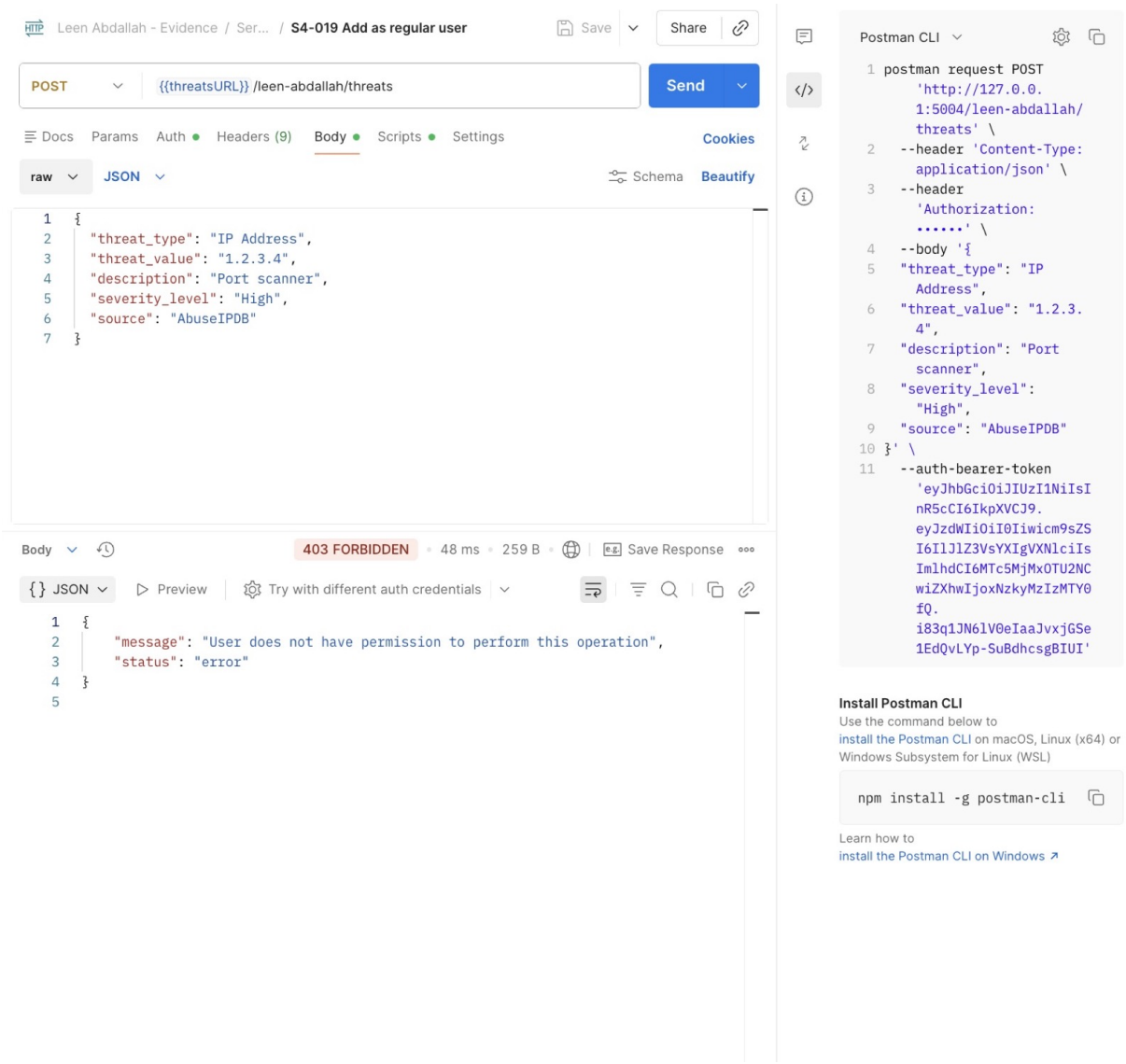


Figure 195: Leen Abdallah: Add Threat Indicator: Regular User, 403

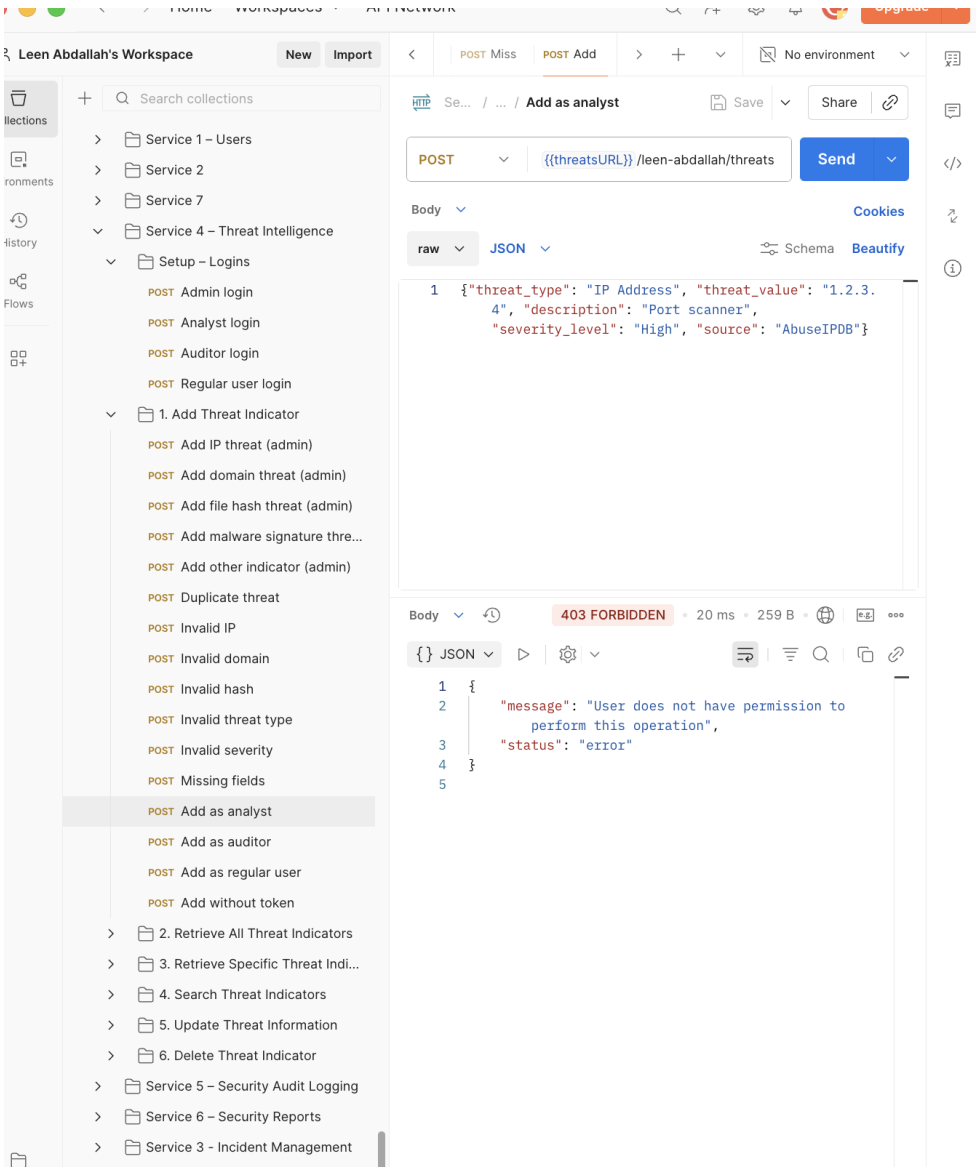


Figure 196: Leen Abdallah: Add Threat Indicator: Security Analyst, 403

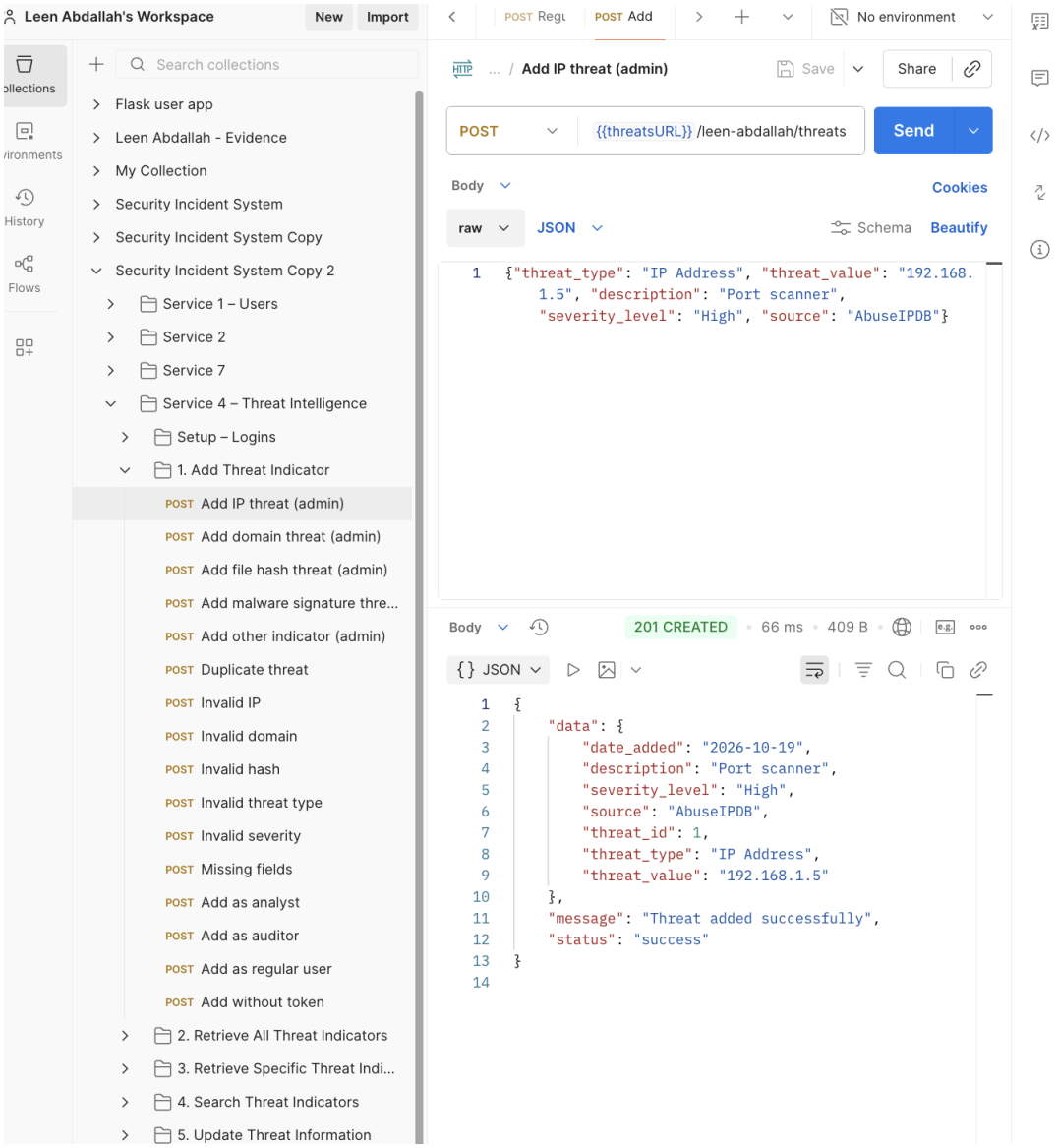


Figure 197: Leen Abdallah: Add Threat Indicator: the same API with the Security Administrator, 201

Further authorization evidence by service

Service 1 (Adam Yahya)

Figures 198–200: The requests distinguish an ownership violation, a missing token and a disallowed deletion role.

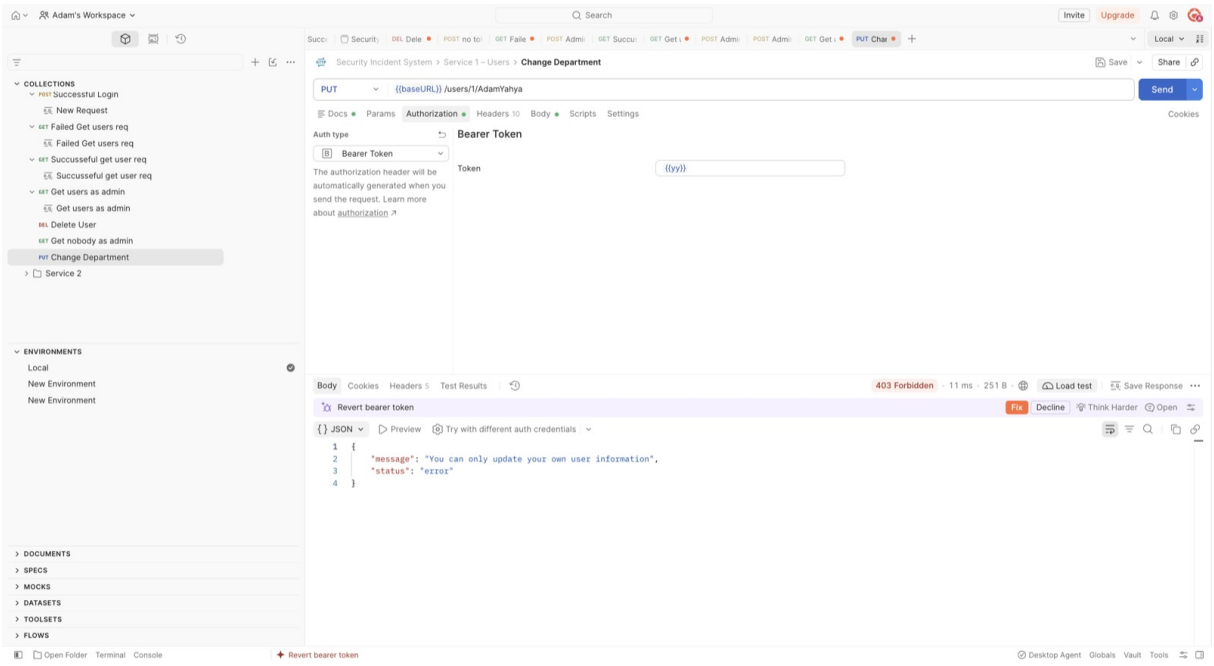


Figure 198: Adam Yahya: Update User: Regular User tries to update another user, 403

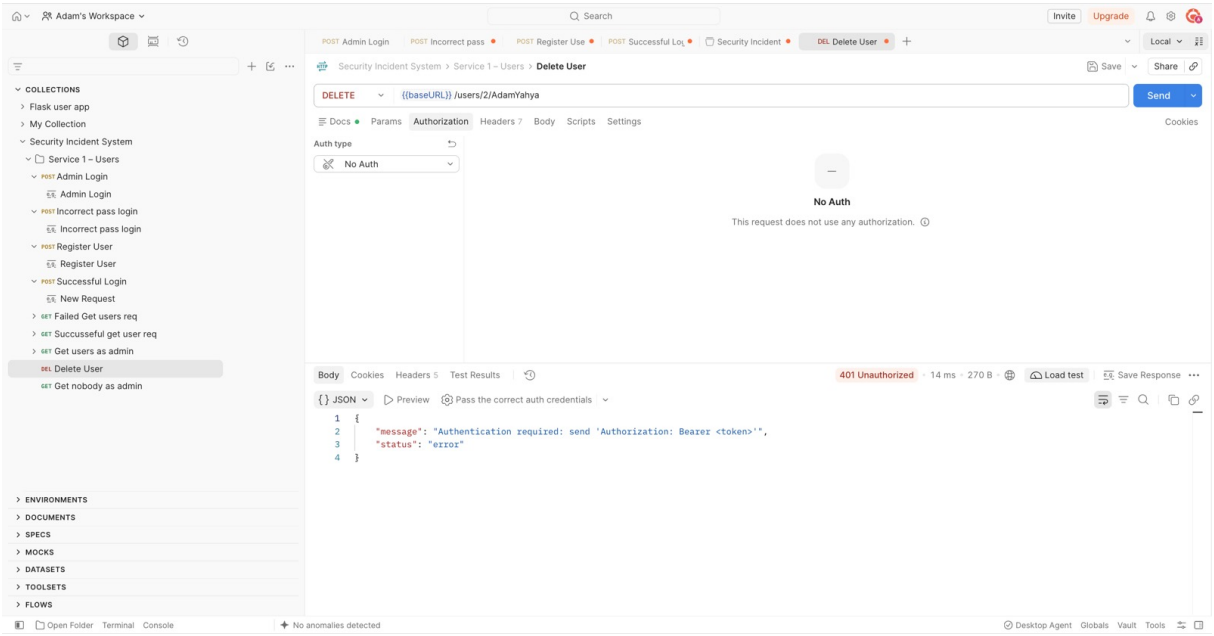


Figure 199: Adam Yahya: Delete User: request without a token, 401

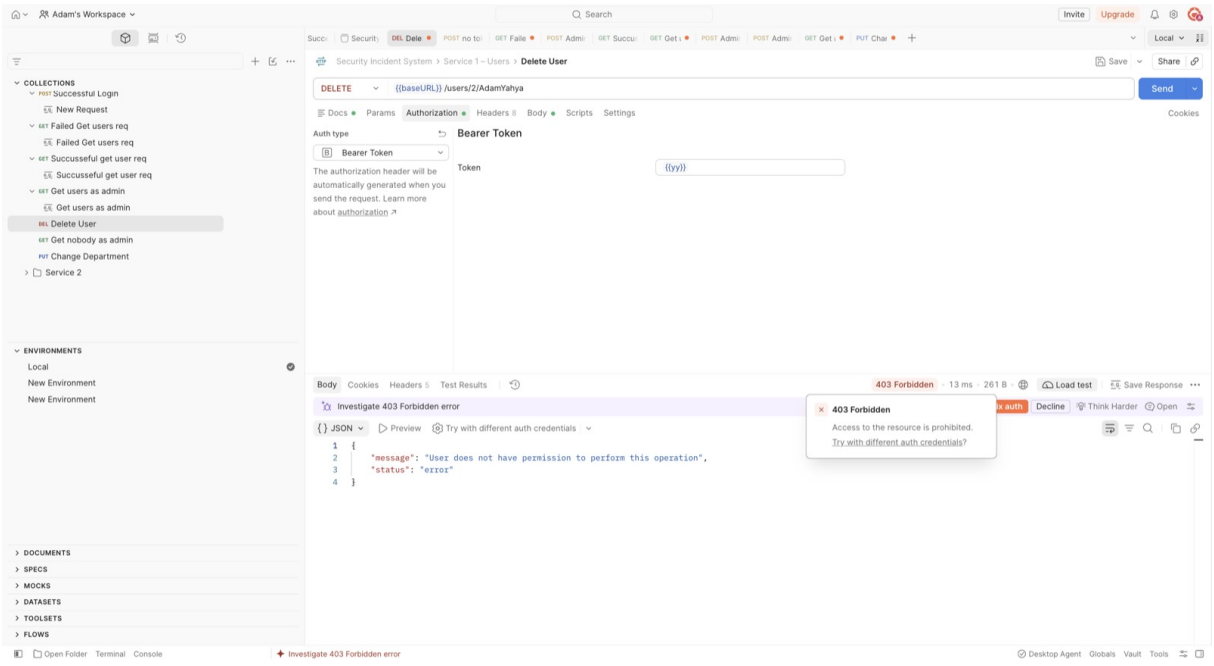


Figure 200: Adam Yahya: Delete User: Regular User is refused, 403

Service 2 (Adam Yahya)

Figures 201–203: The requests reject missing authentication and nonadministrator asset creation or deletion.

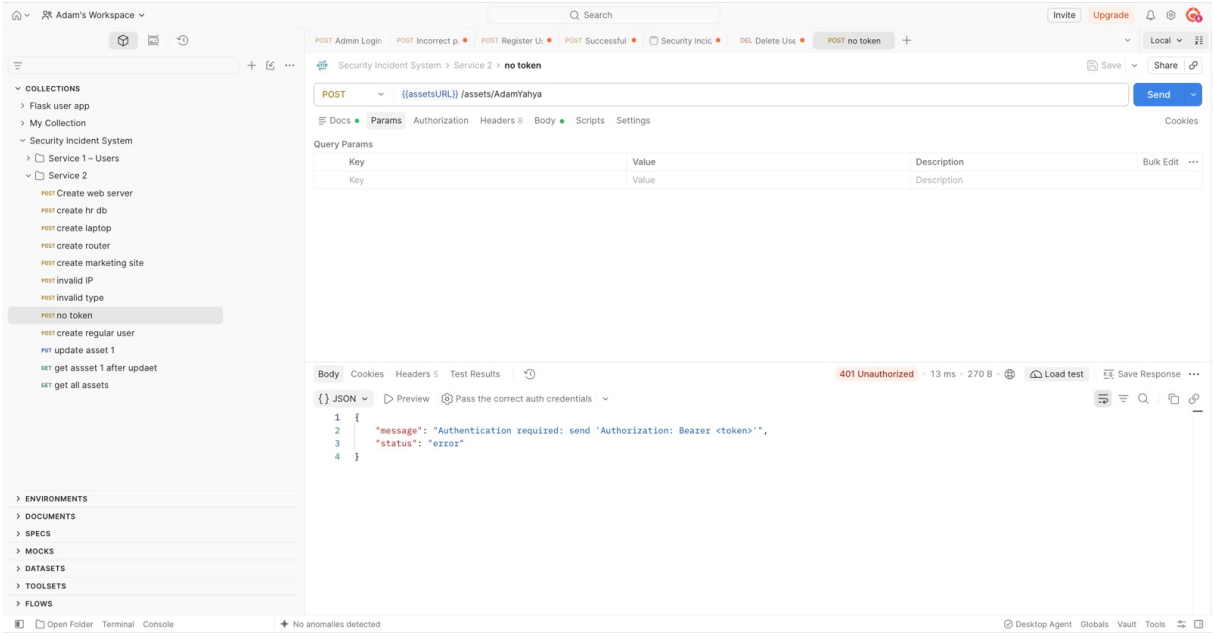


Figure 201: Adam Yahya: Add Asset: request without a token, 401

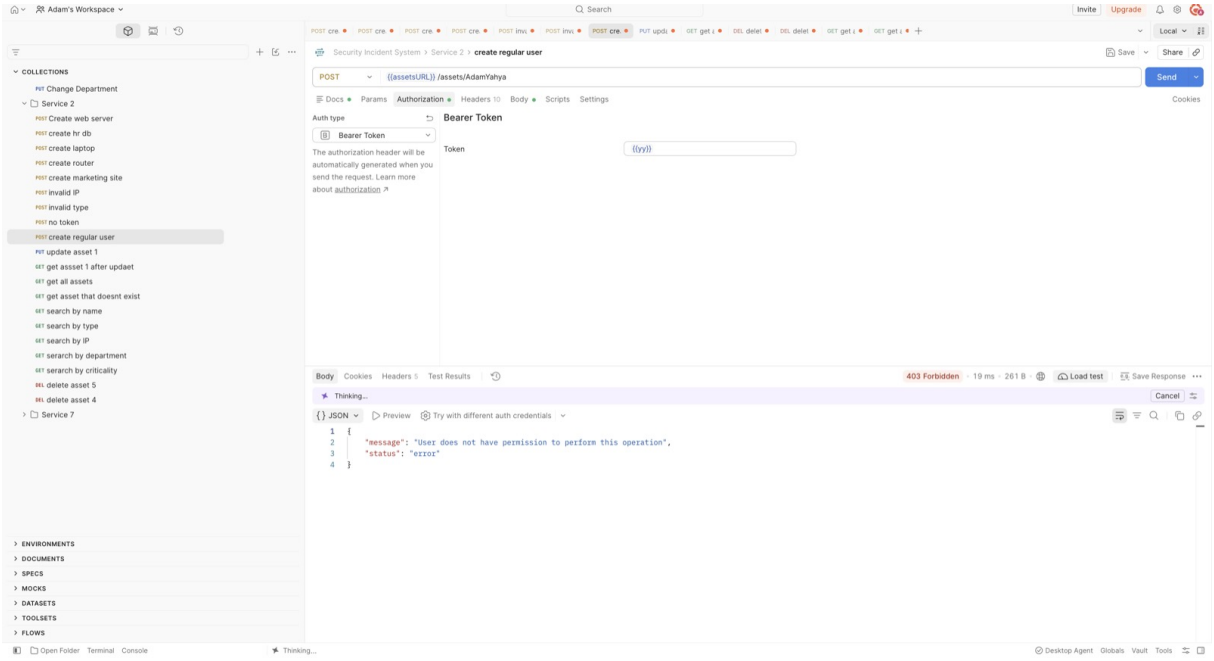


Figure 202: Adam Yahya: Add Asset: Regular User is refused, 403

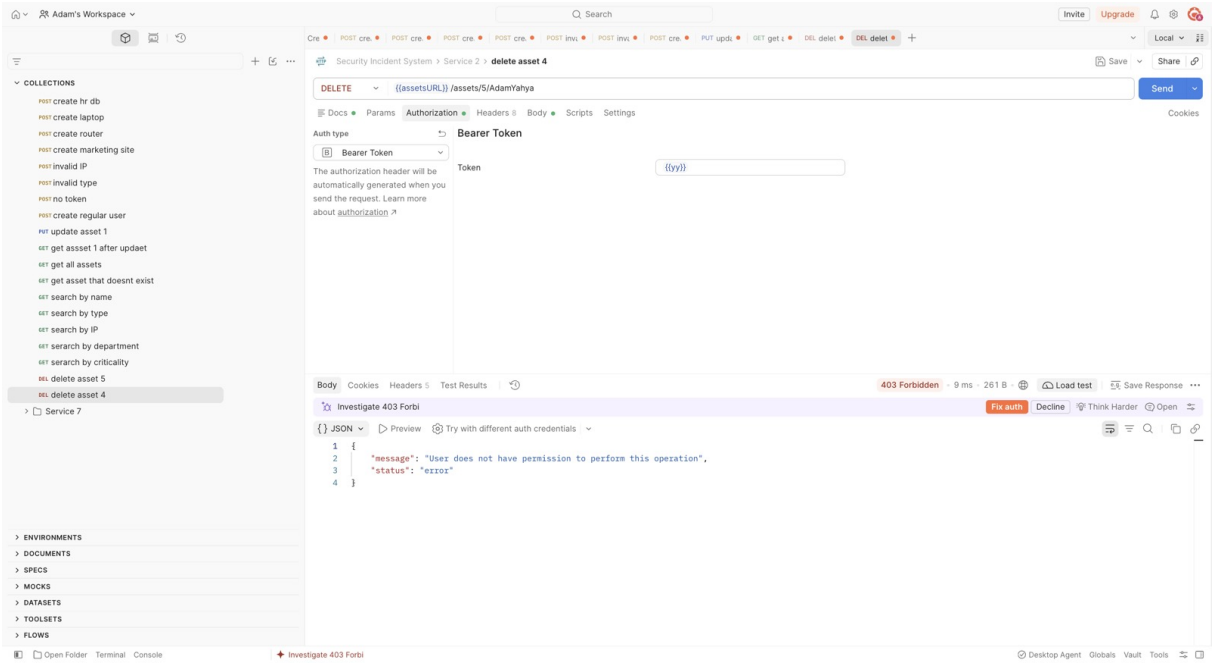


Figure 203: Adam Yahya: Delete Asset: non-administrator is refused, 403

Service 7 (Adam Yahya)

Figures 204–205: Analyst creation and deletion are denied even though analyst updates are permitted.

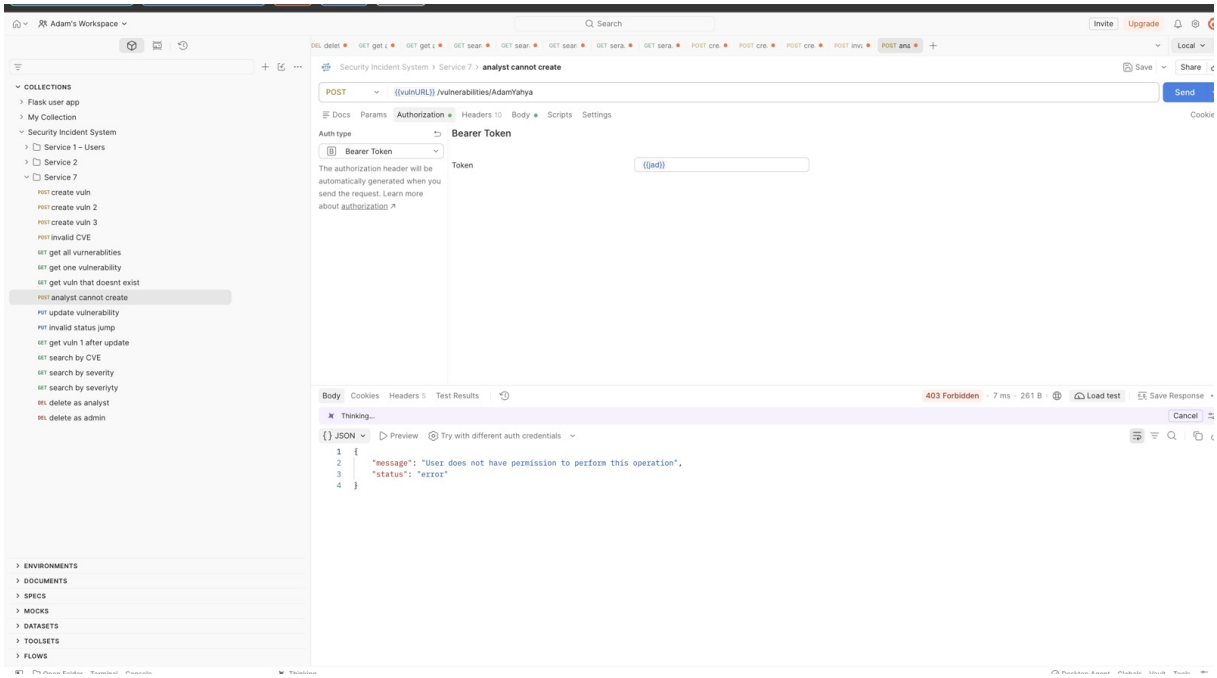


Figure 204: Adam Yahya: Add Vulnerability: Security Analyst is refused, 403

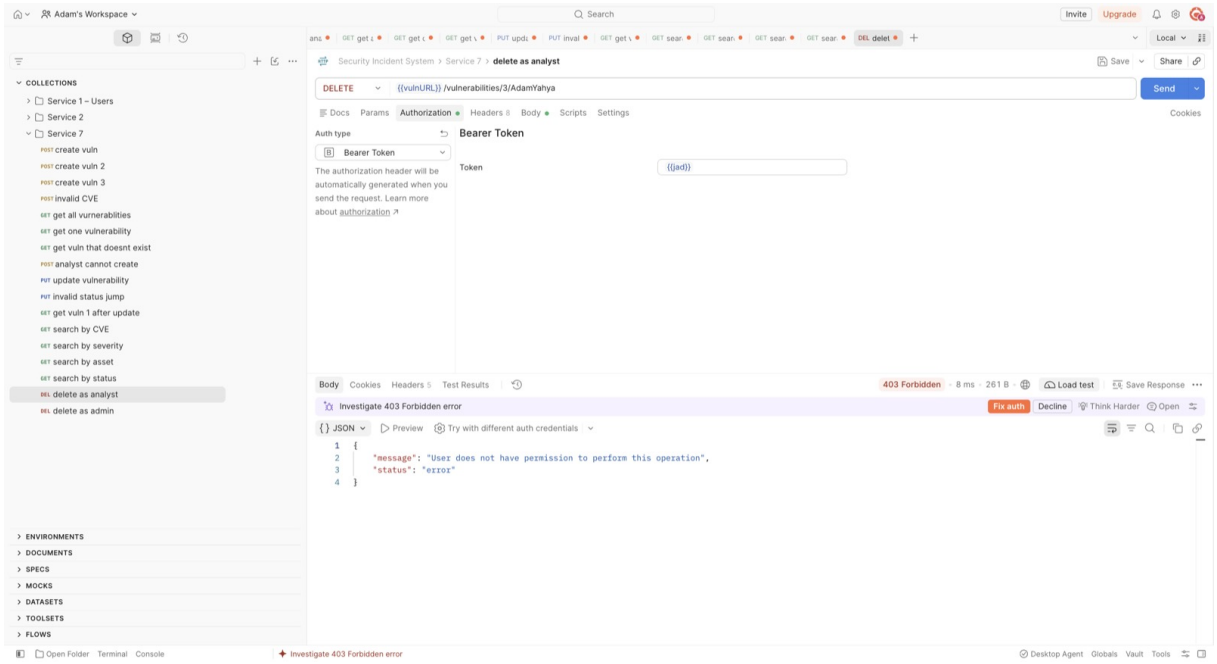


Figure 205: Adam Yahya: Delete Vulnerability: Security Analyst is refused, 403

Service 3 (Youssef Al Hajj Youness)

Figures 206–211: The requests demonstrate authentication, administrator-only assignment, incident ownership/assignment and read-only auditor restrictions.

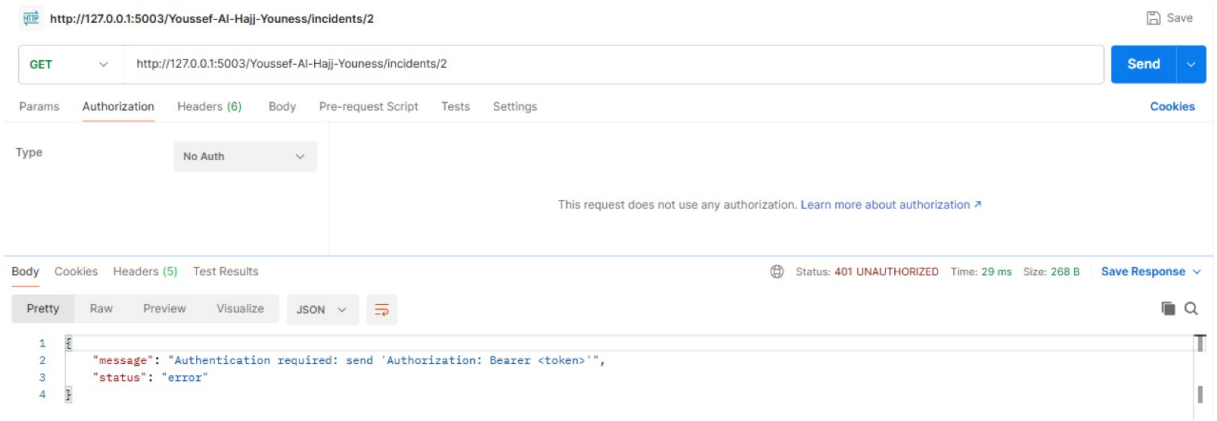


Figure 206: Youssef Al Hajj Youness: Incidents: request without a token, 401

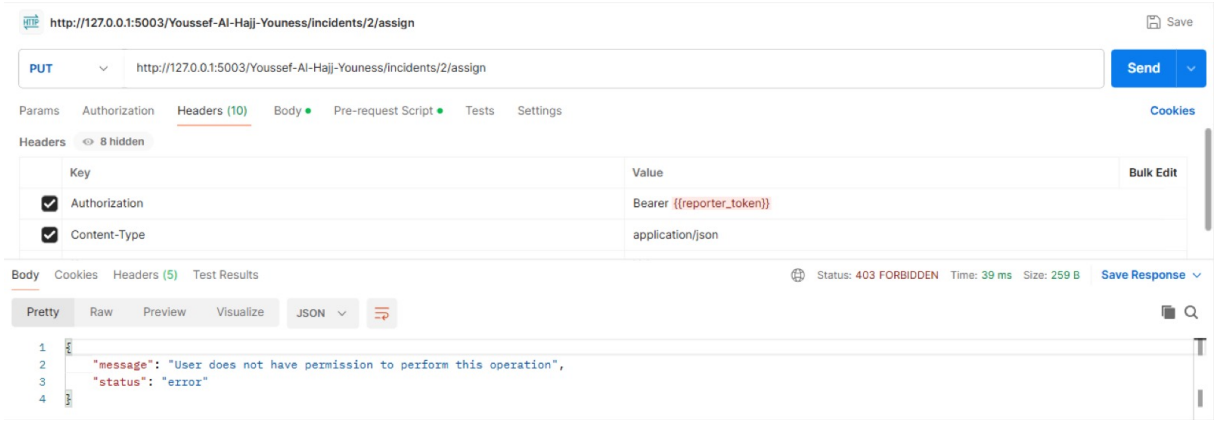


Figure 207: Youssef Al Hajj Youness: Assign Incident: unauthorized assignment attempt, 403

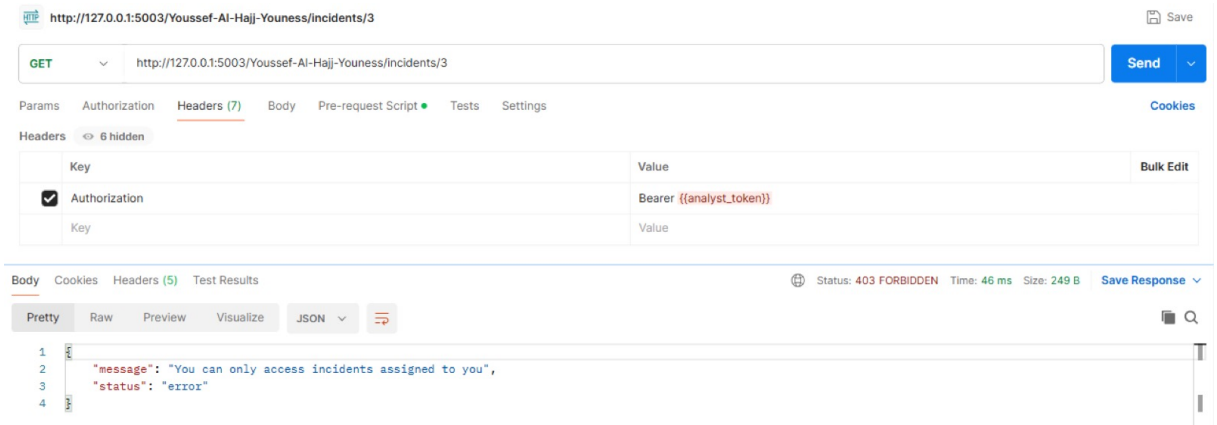


Figure 208: Youssef Al Hajj Youness: Retrieve Specific Incident: analyst not assigned to it, 403

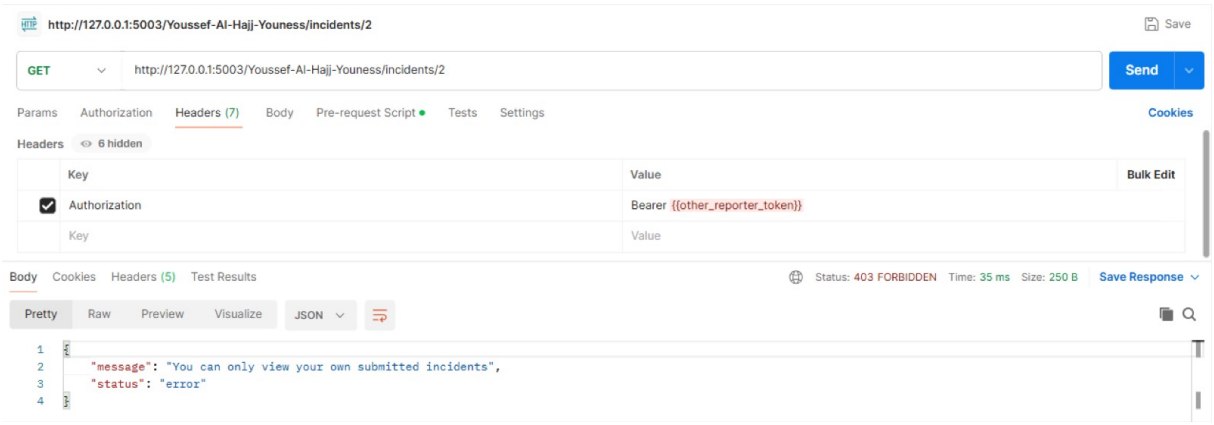


Figure 209: Youssef Al Hajj Youness: Retrieve Specific Incident: another reporter, 403

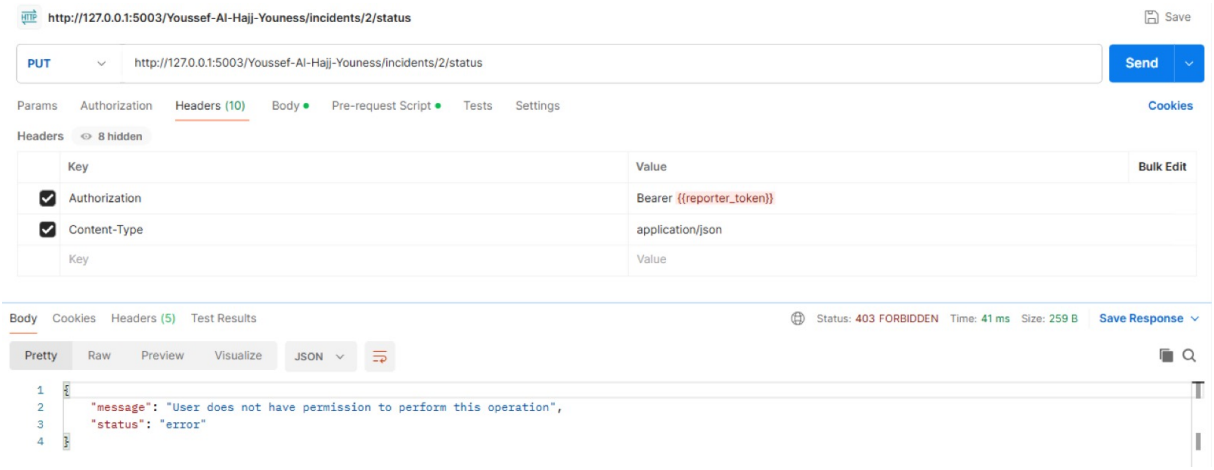


Figure 210: Youssef Al Hajj Youness: Update Incident Status: Regular User is refused, 403

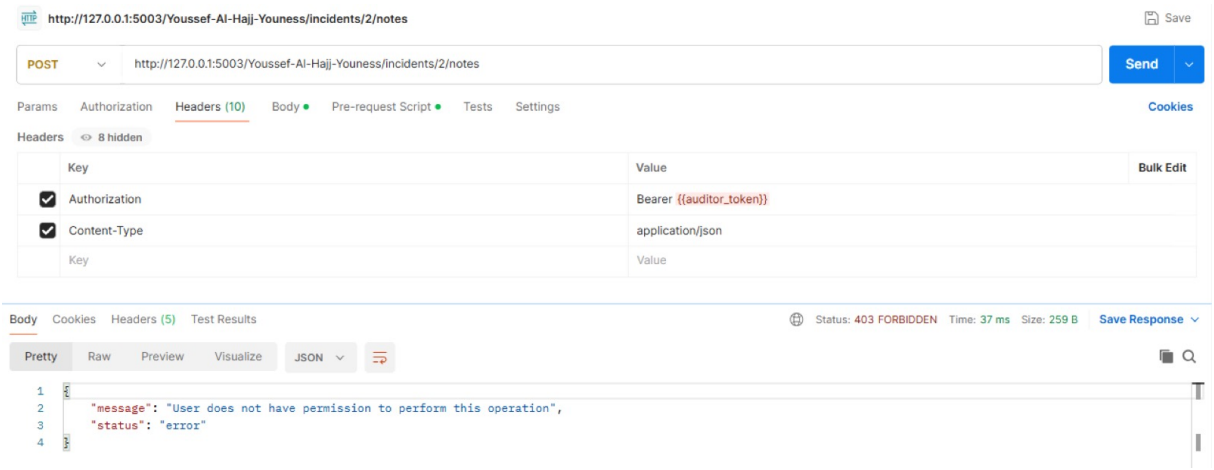


Figure 211: Youssef Al Hajj Youness: Add Investigation Notes: Auditor is refused, 403

Service 8 (Youssef Al Hajj Youness)

Figures 212–218: The requests demonstrate ownership, administrator-only creation/deletion, auditor read-only behavior and the Regular User search restriction. The analyst ownership capture is a GET, not a mark-read request.

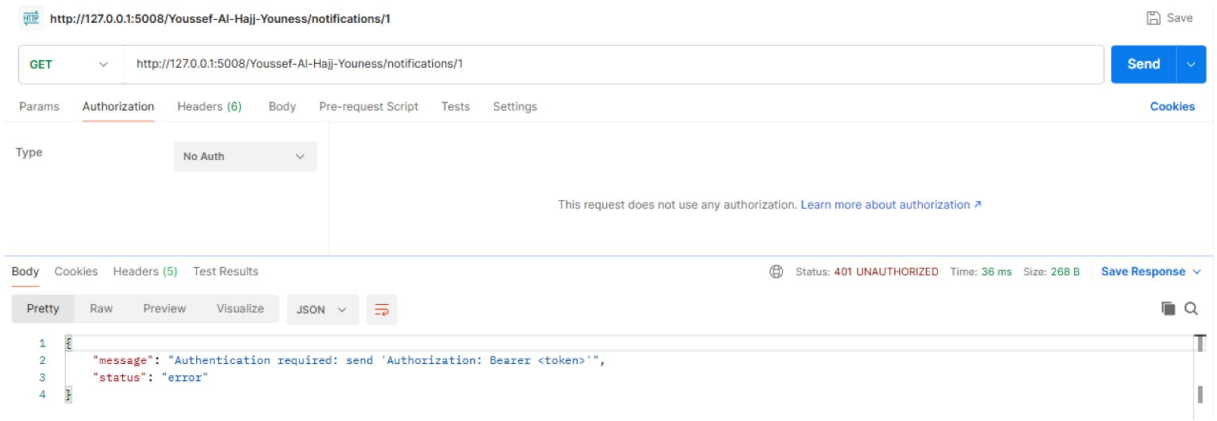


Figure 212: Youssef Al Hajj Youness: Notifications: request without a token, 401

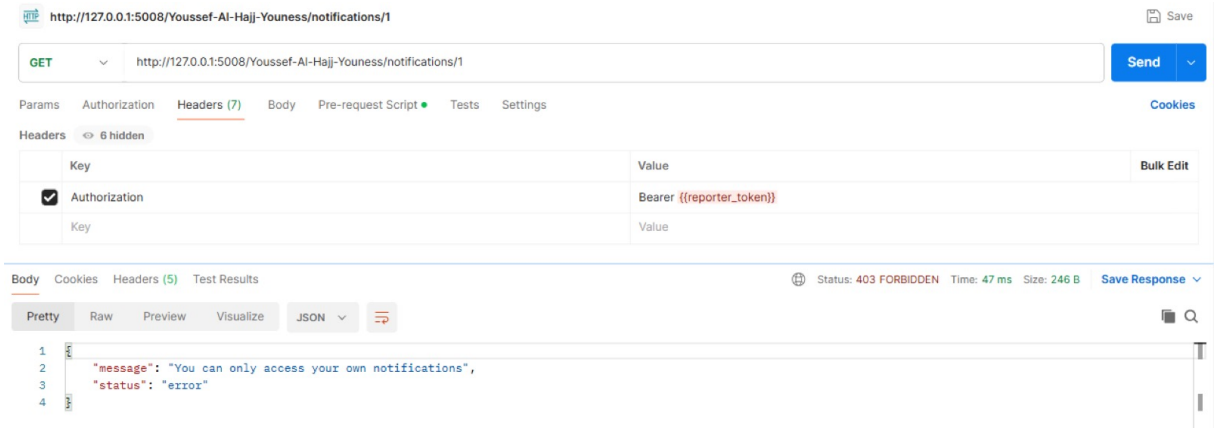


Figure 213: Youssef Al Hajj Youness: Retrieve Notification: another user notification, 403

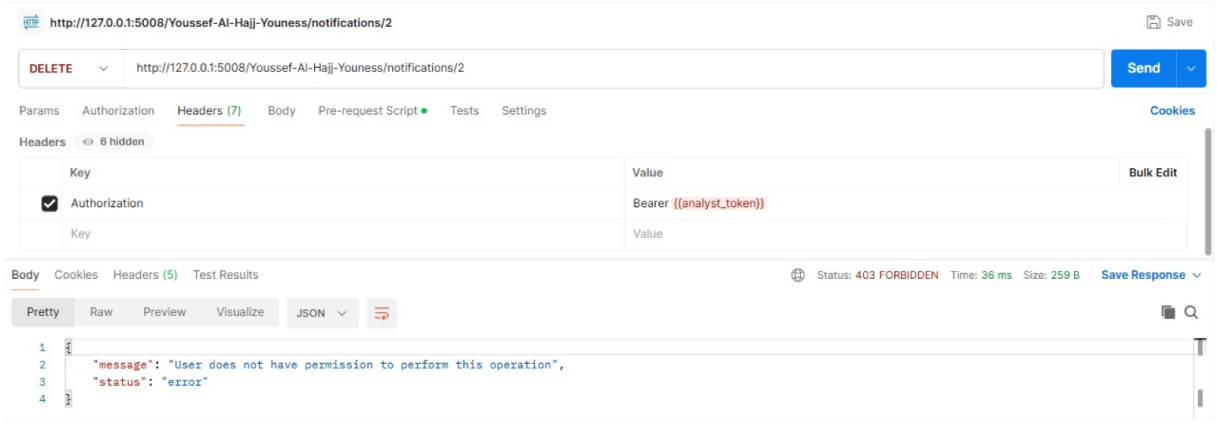


Figure 214: Youssef Al Hajj Youness: Delete Notification: non-administrator, 403

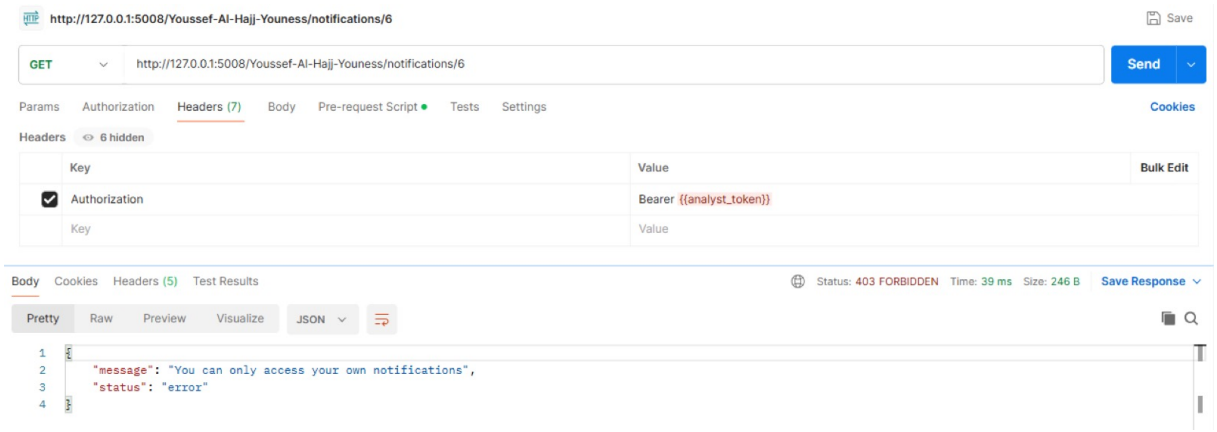


Figure 215: Youssef Al Hajj Youness: GET /Youssef-Al-Hajj-Youness/notifications/6: analyst cannot read another user's notification, 403

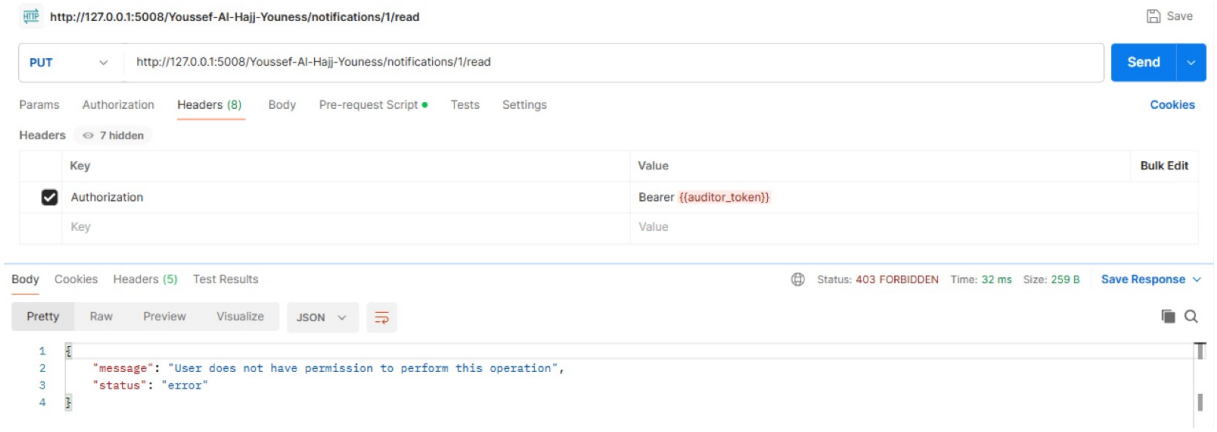


Figure 216: Youssef Al Hajj Youness: Mark as Read: Auditor is refused, 403

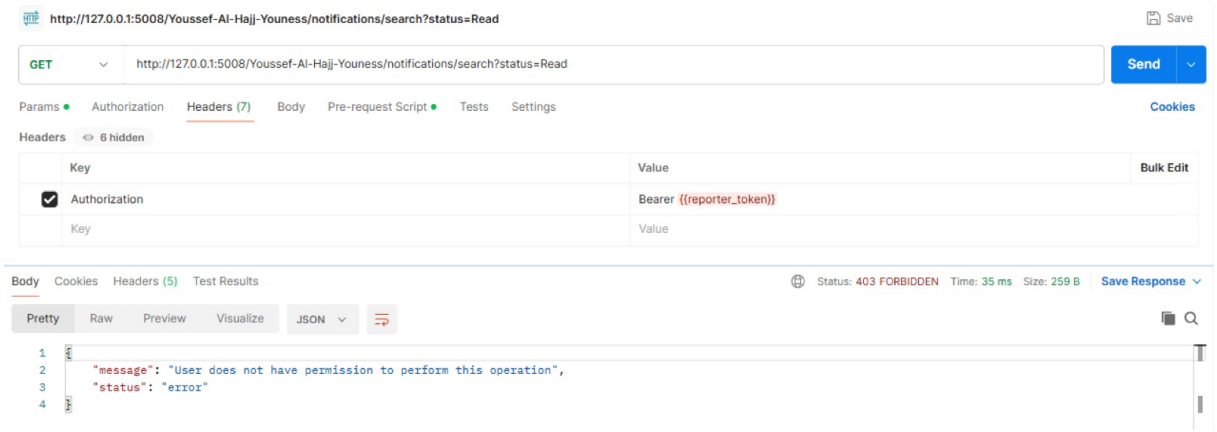


Figure 217: Youssef Al Hajj Youness: Search Notifications: Regular User is refused, 403

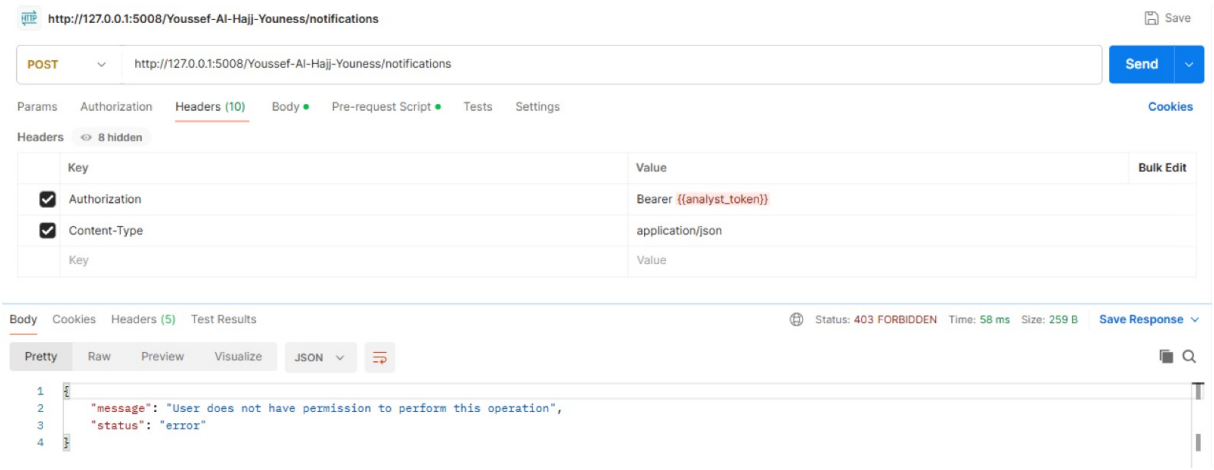


Figure 218: Youssef Al Hajj Youness: Create Notification: Security Analyst is refused, 403

Service 9 (Youssef Al Hajj Youness)

Figures 219–223: The requests demonstrate missing-token denial and the distinct permissions for playbook creation, action changes and deletion.

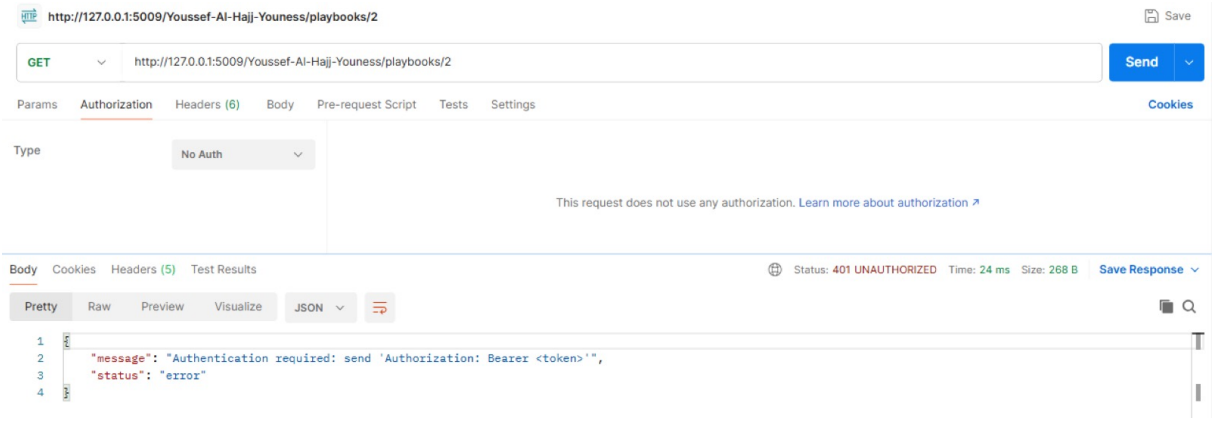


Figure 219: Youssef Al Hajj Youness: Playbooks: request without a token, 401

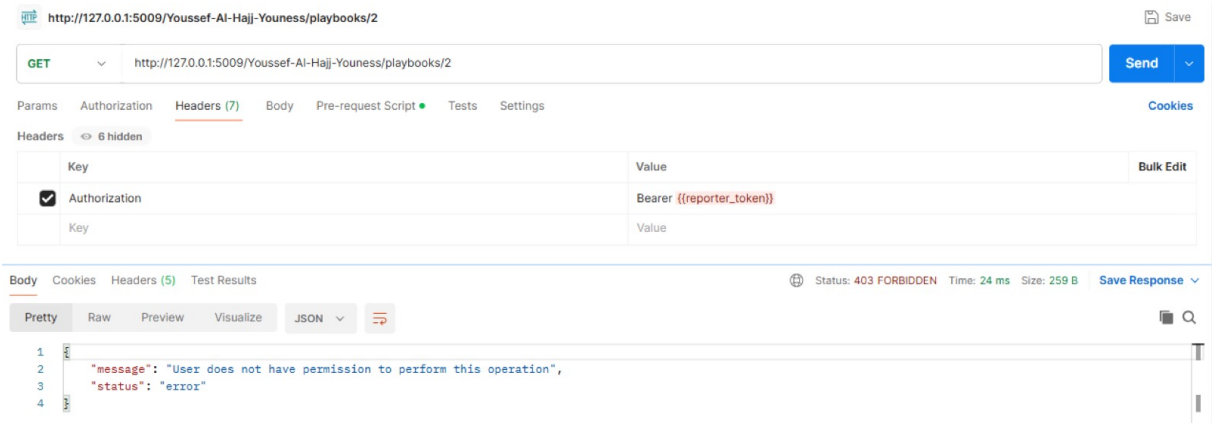


Figure 220: Youssef Al Hajj Youness: Retrieve Playbooks: Regular User is refused, 403

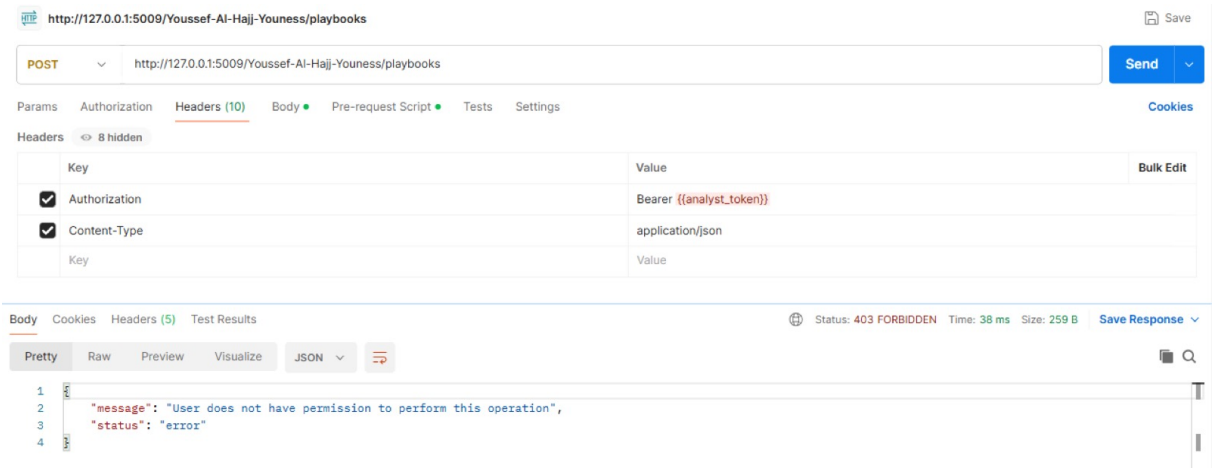


Figure 221: Youssef Al Hajj Youness: Create Playbook: Security Analyst is refused, 403

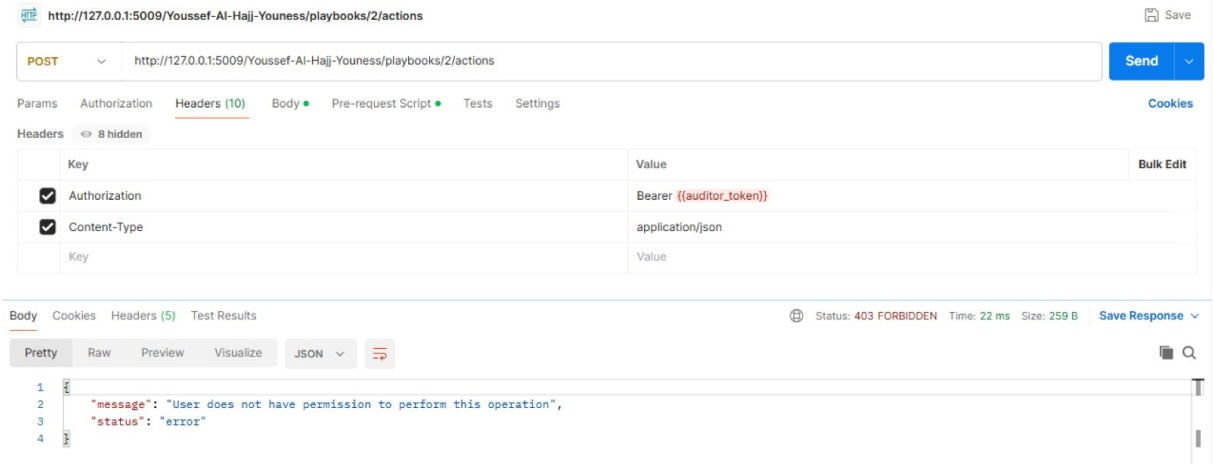


Figure 222: Youssef Al Hajj Youness: Add Response Action: Auditor is refused, 403

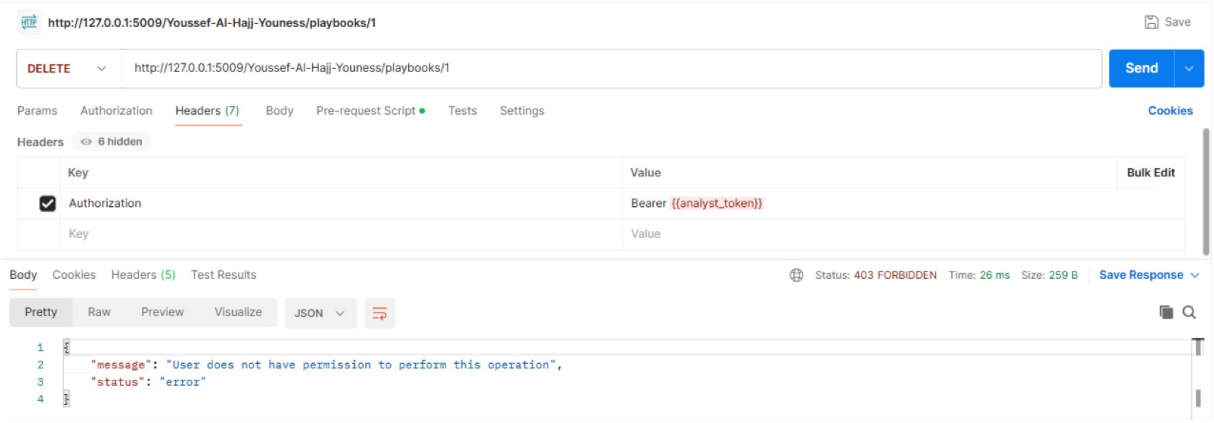


Figure 223: Youssef Al Hajj Youness: Delete Playbook: non-administrator is refused, 403

Service 4 (Leen Abdallah)

Figures 224–233: Completed responses show creation and retrieval restrictions, successful analyst/auditor retrieval, auditor search denial and analyst update denial. The analyst DELETE

capture shows the analyst-token request returning 403 with “User does not have permission to perform this operation”. The Regular User DELETE also returns 403, while a DELETE without a token returns 401. These outcomes match the shared authentication check followed by the administrator-only DELETE role check.

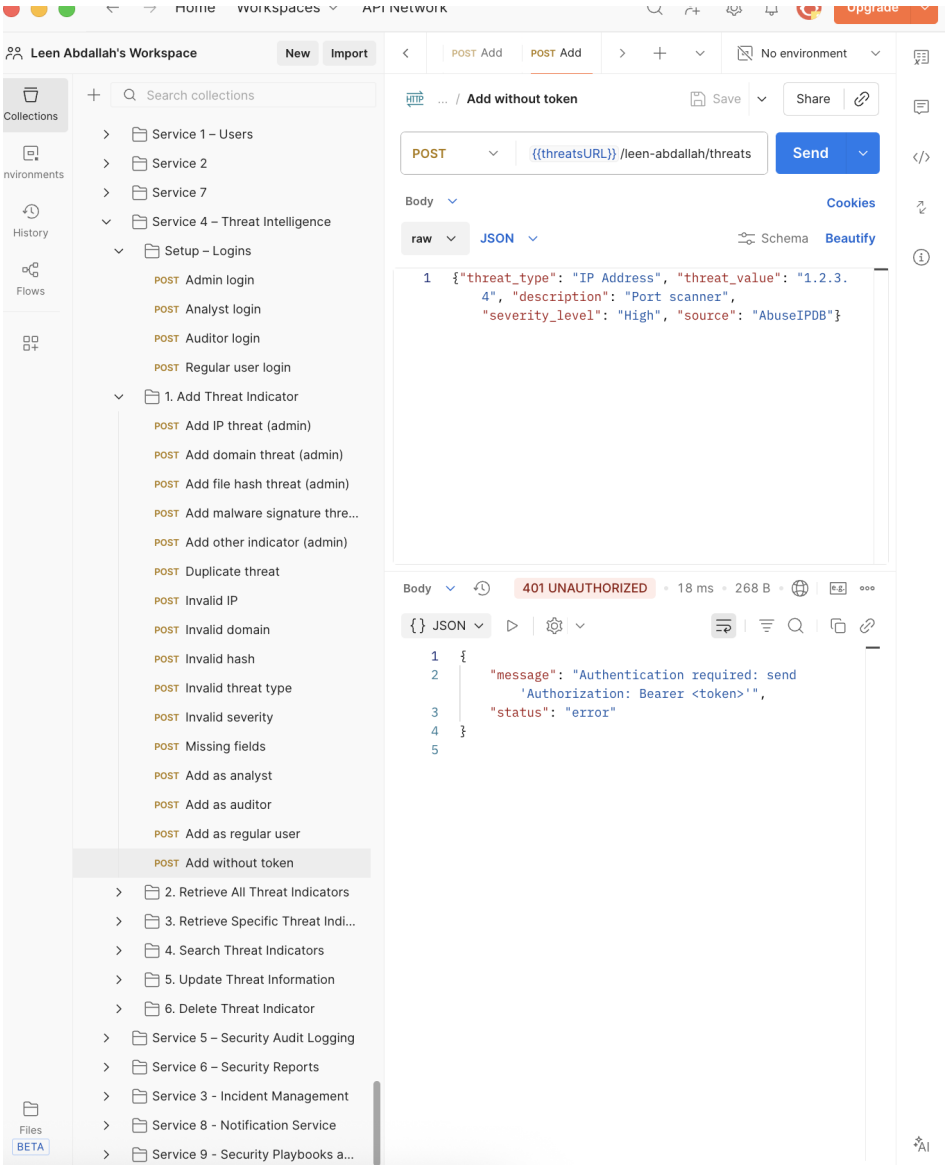


Figure 224: Leen Abdallah: Add Threat Indicator: no token, 401

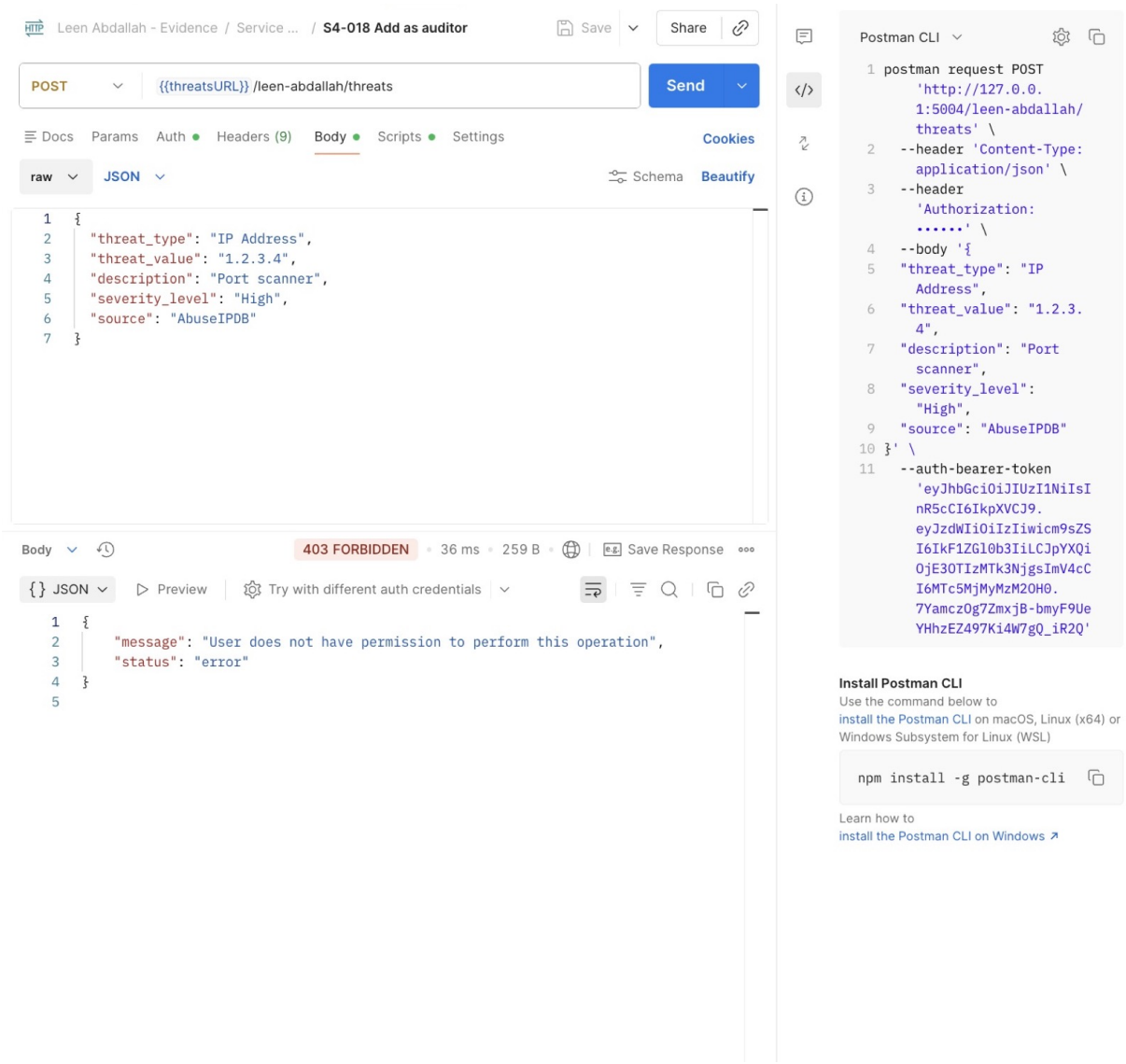


Figure 225: Leen Abdallah: Add Threat Indicator: Auditor is refused, 403

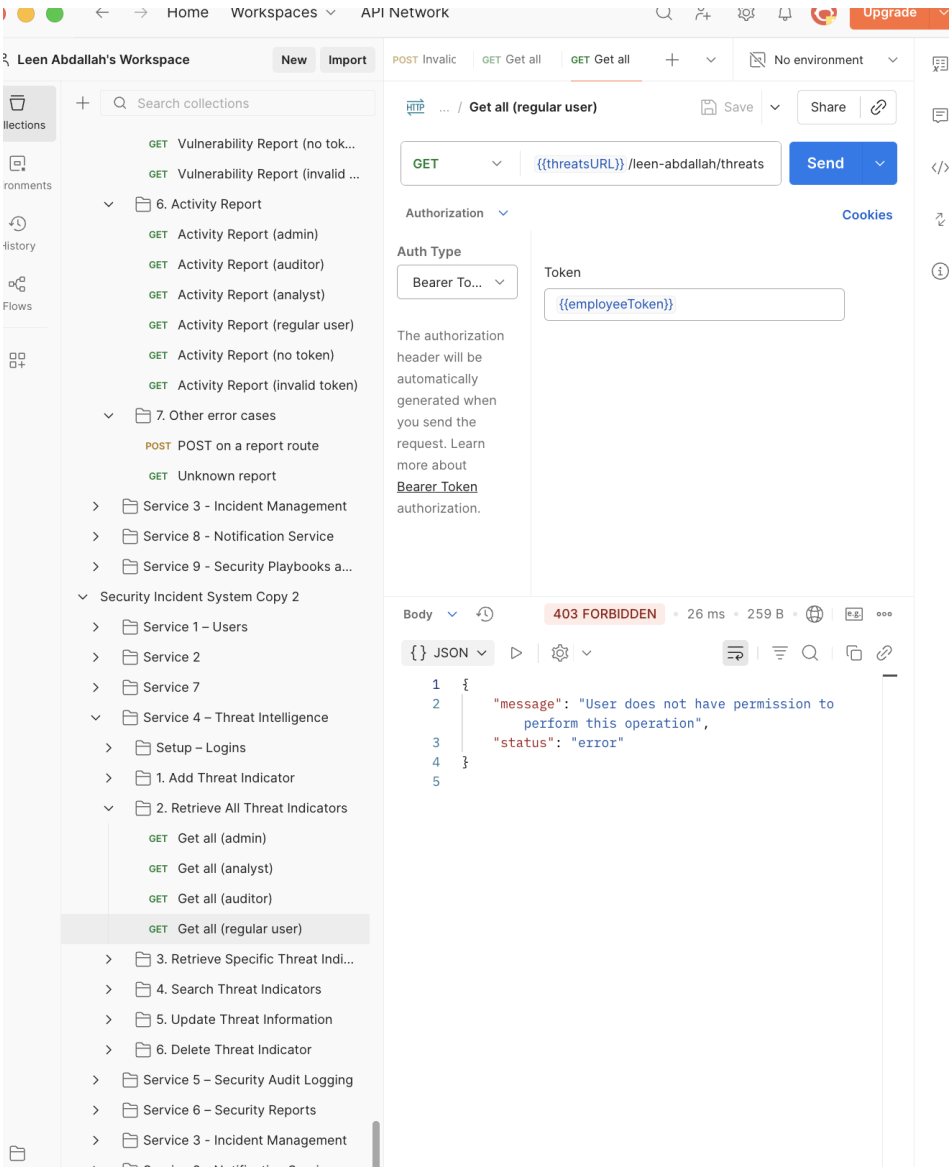


Figure 226: Leen Abdallah: Retrieve All Threat Indicators: Regular User is refused, 403

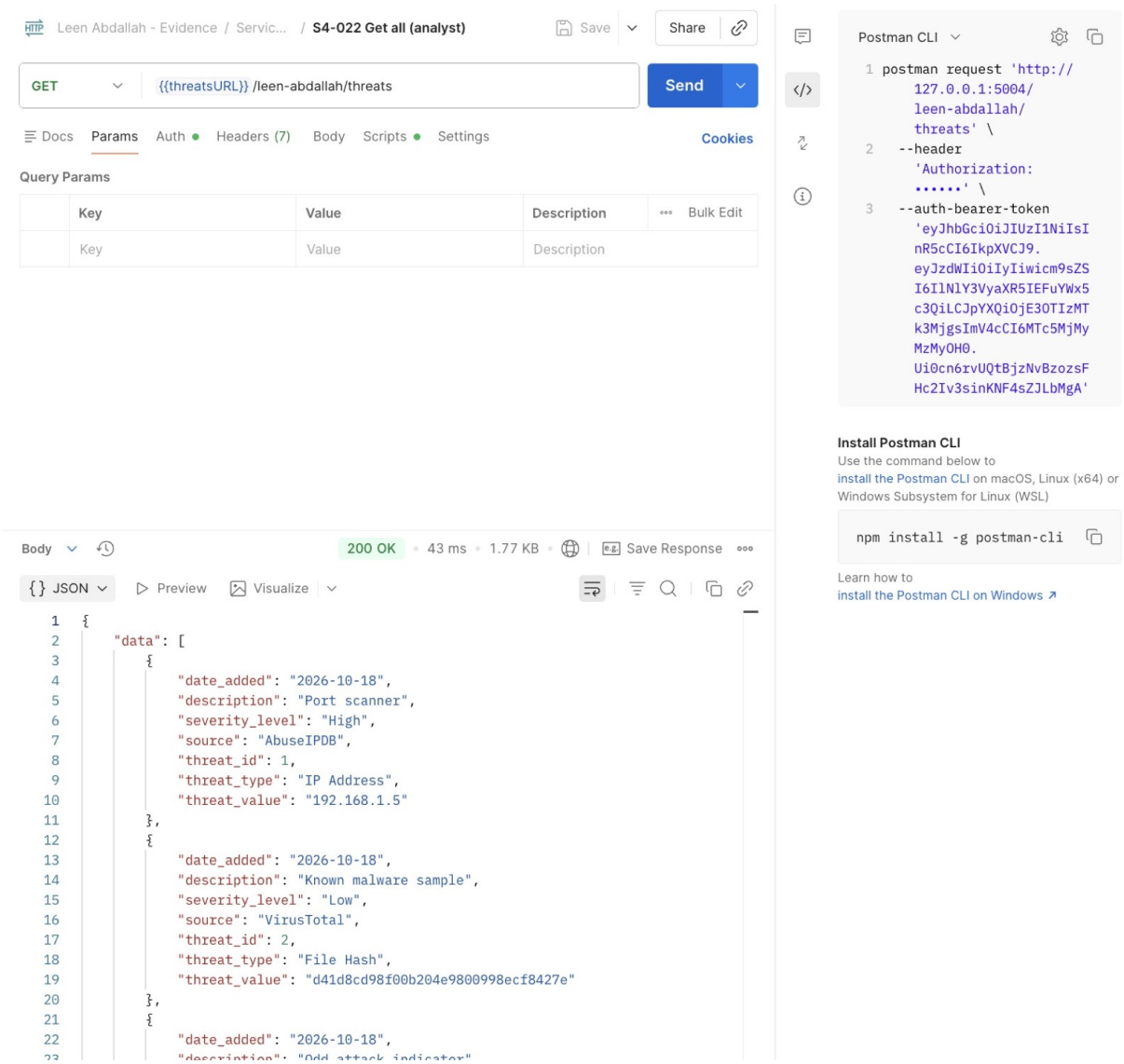


Figure 227: Leen Abdallah: Retrieve All Threat Indicators: analyst allowed, 200

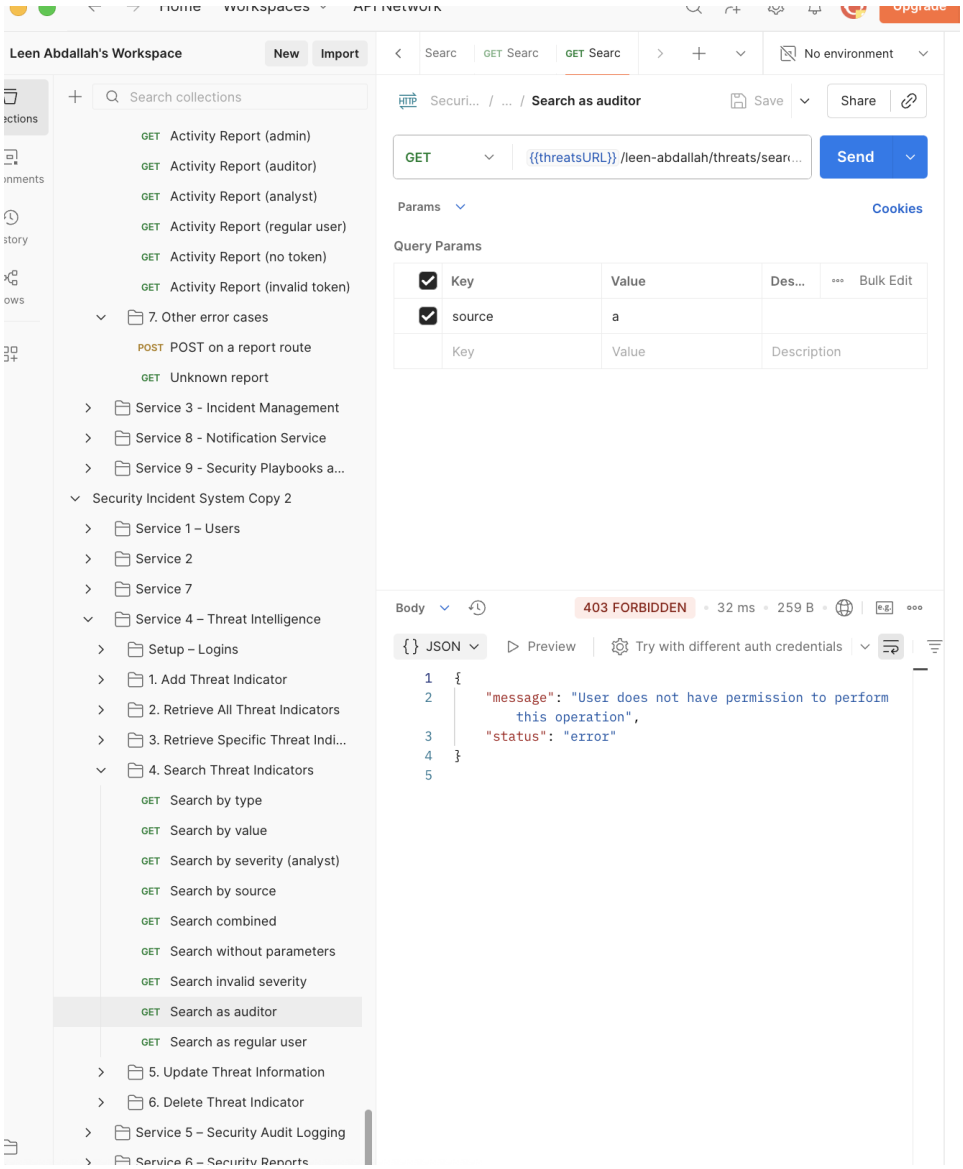


Figure 228: Leen Abdallah: Search Threat Indicators: Auditor is refused, 403

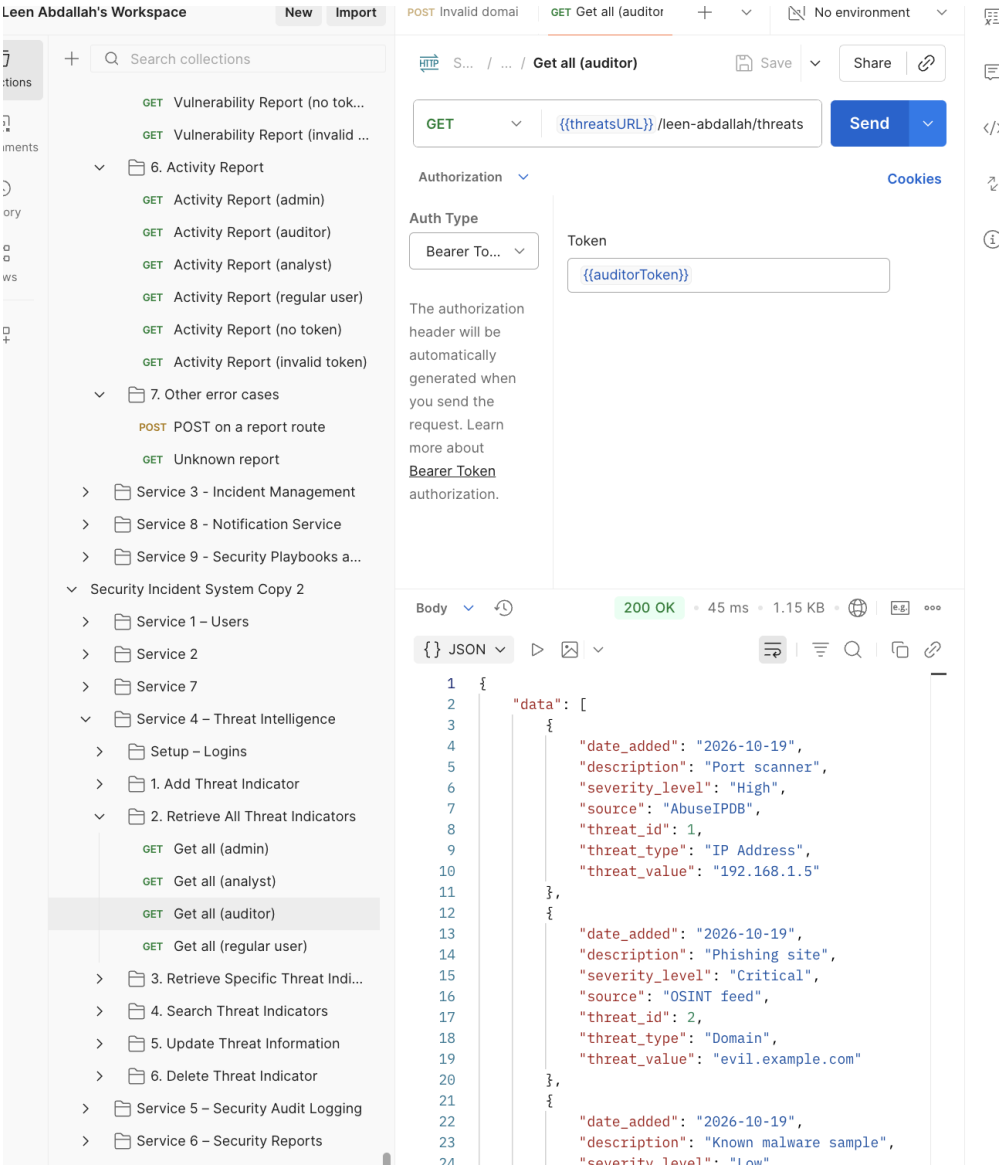


Figure 229: Leen Abdallah: Retrieve All Threat Indicators: Auditor is allowed, 200; visible beginning of the list response

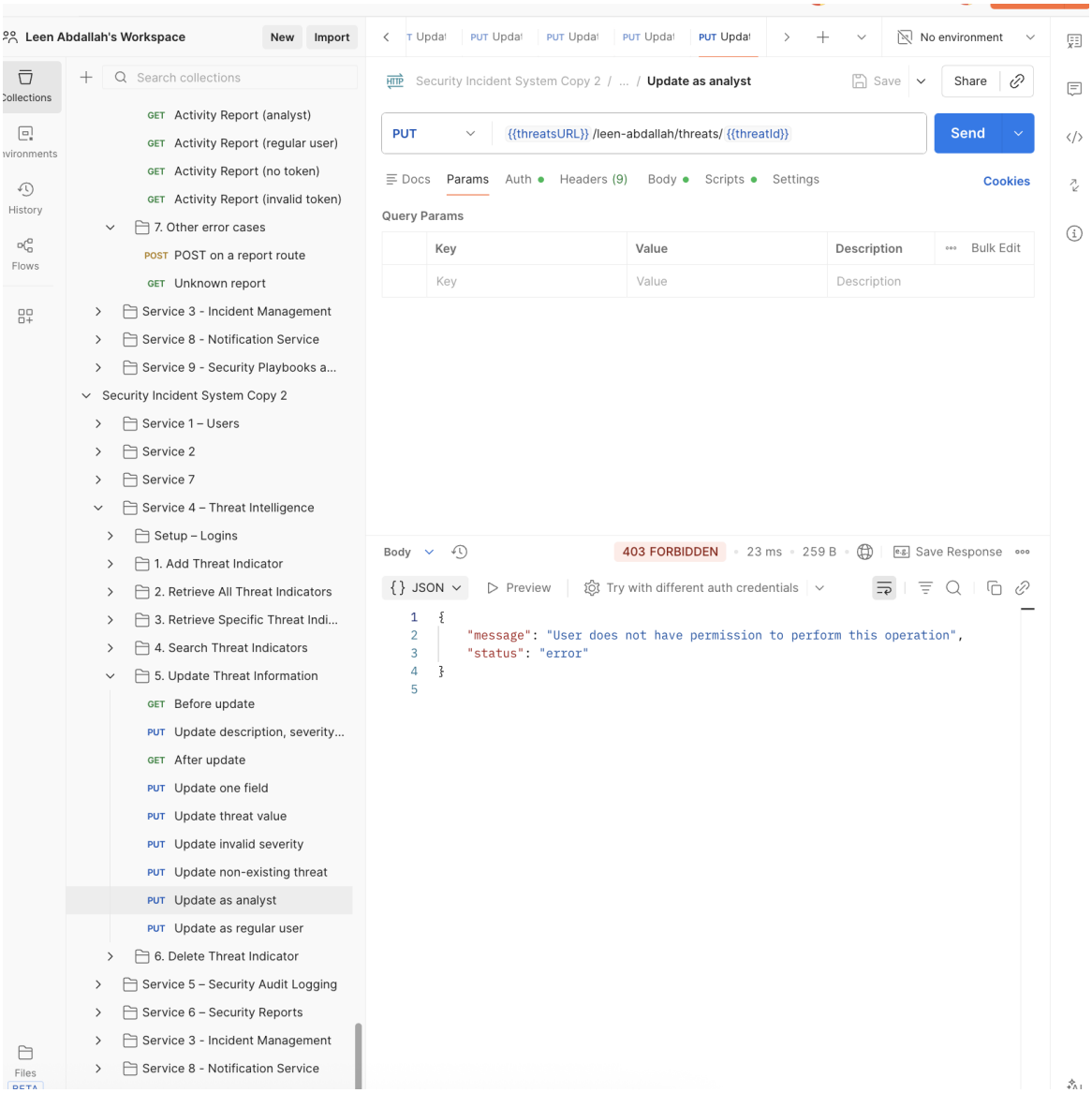


Figure 230: Leen Abdallah: Update Threat Information: Security Analyst is refused, 403

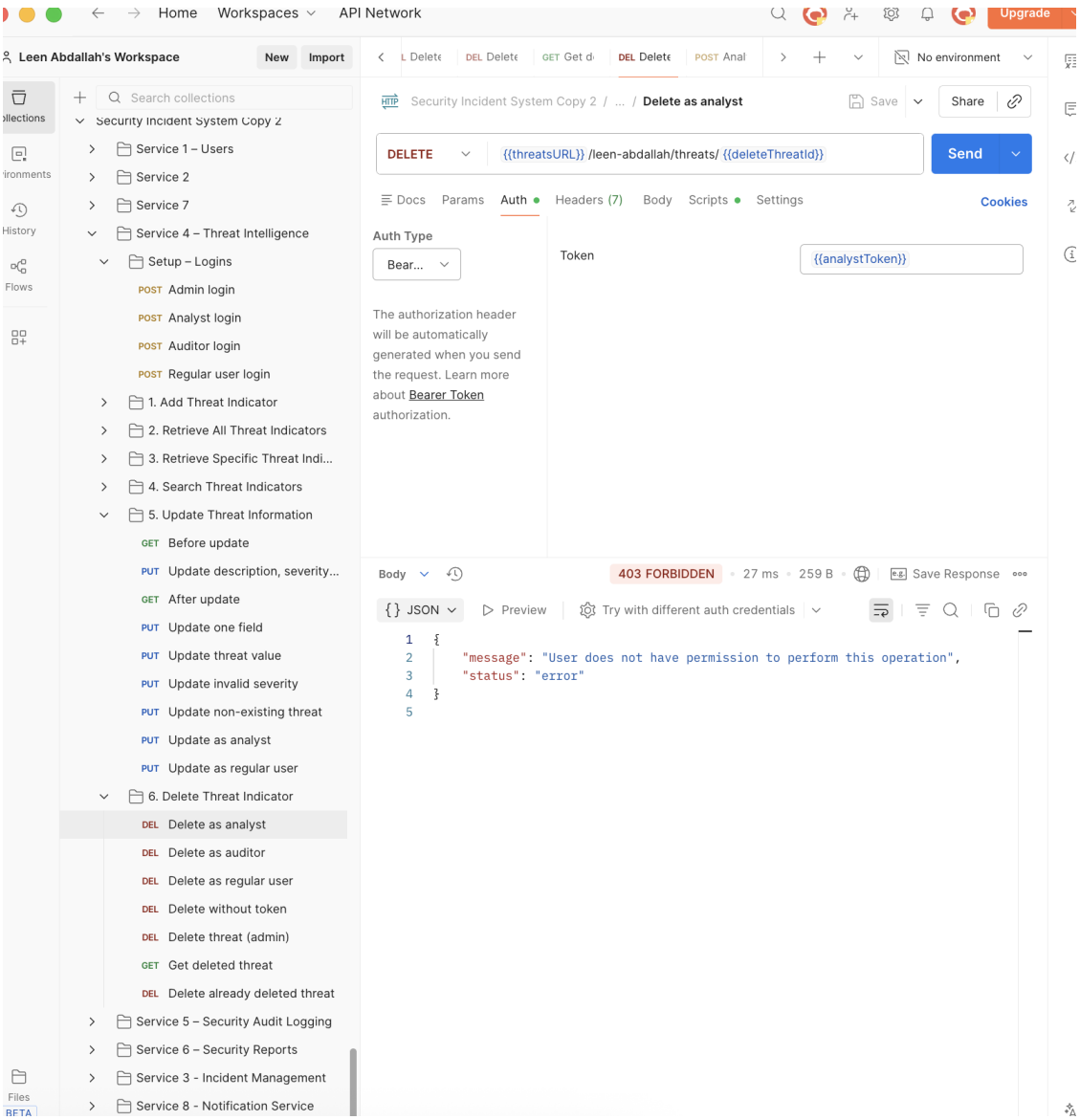


Figure 231: Leen Abdallah: Delete Threat Indicator: Security Analyst is refused, 403

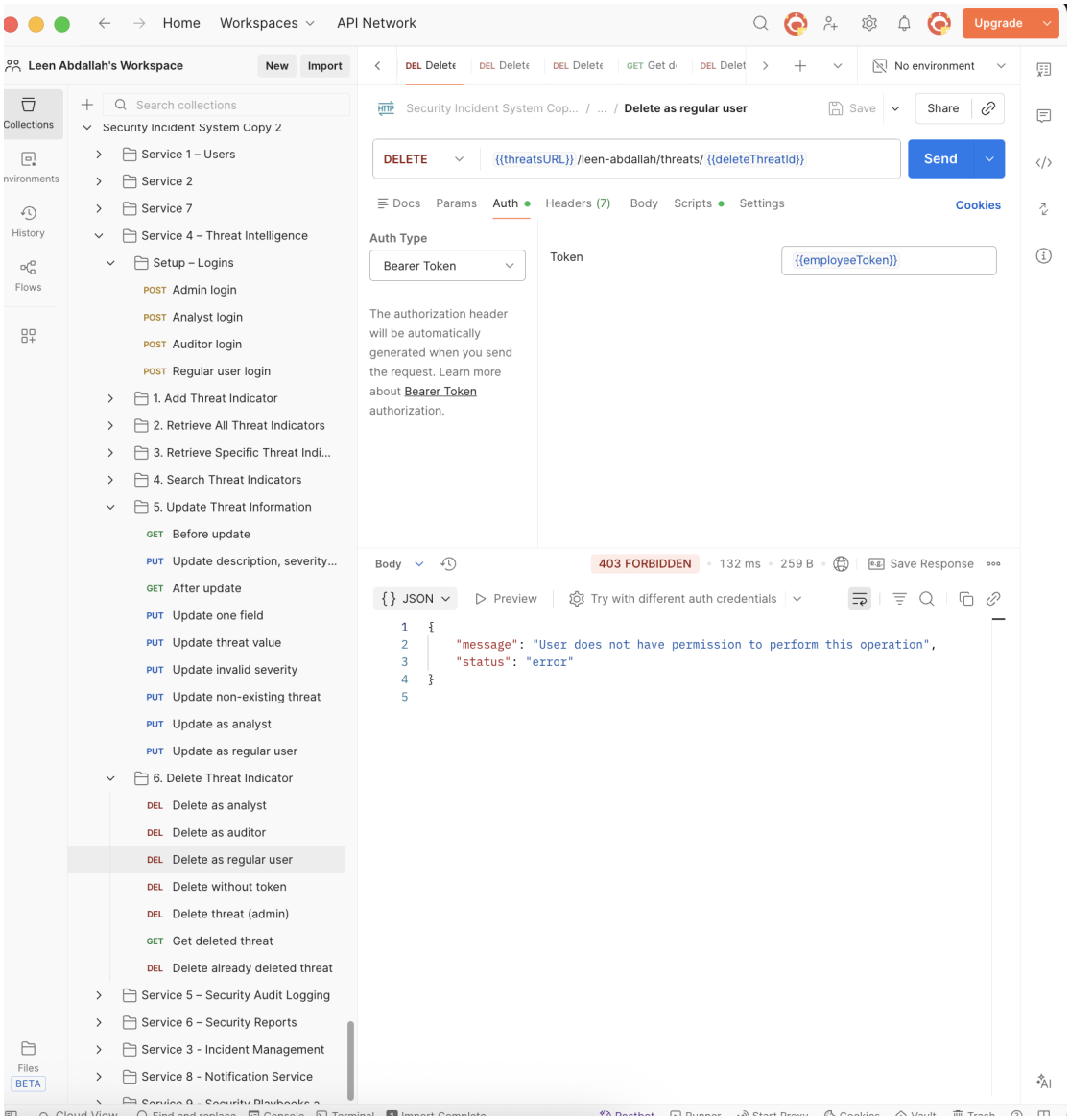


Figure 232: Leen Abdallah: Delete Threat Indicator: Regular User is refused, 403

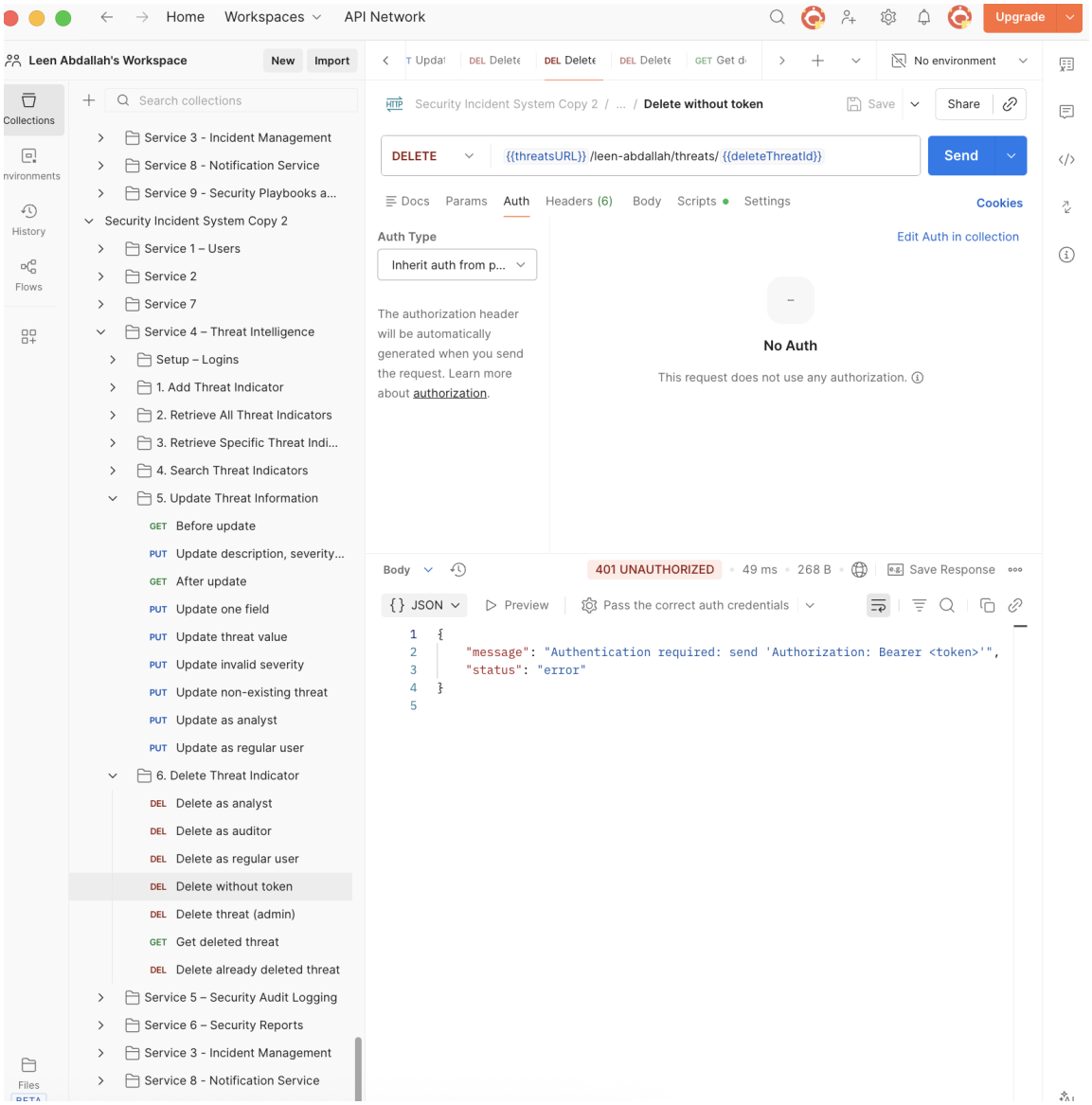


Figure 233: Leen Abdallah: Delete Threat Indicator: no authentication, 401 with the required Bearer-token message

Service 5 (Leen Abdallah)

Figures 234–242: The captures distinguish missing/invalid authentication from role denials, and show auditor list access. Analyst and Regular User deletion attempts return 403.

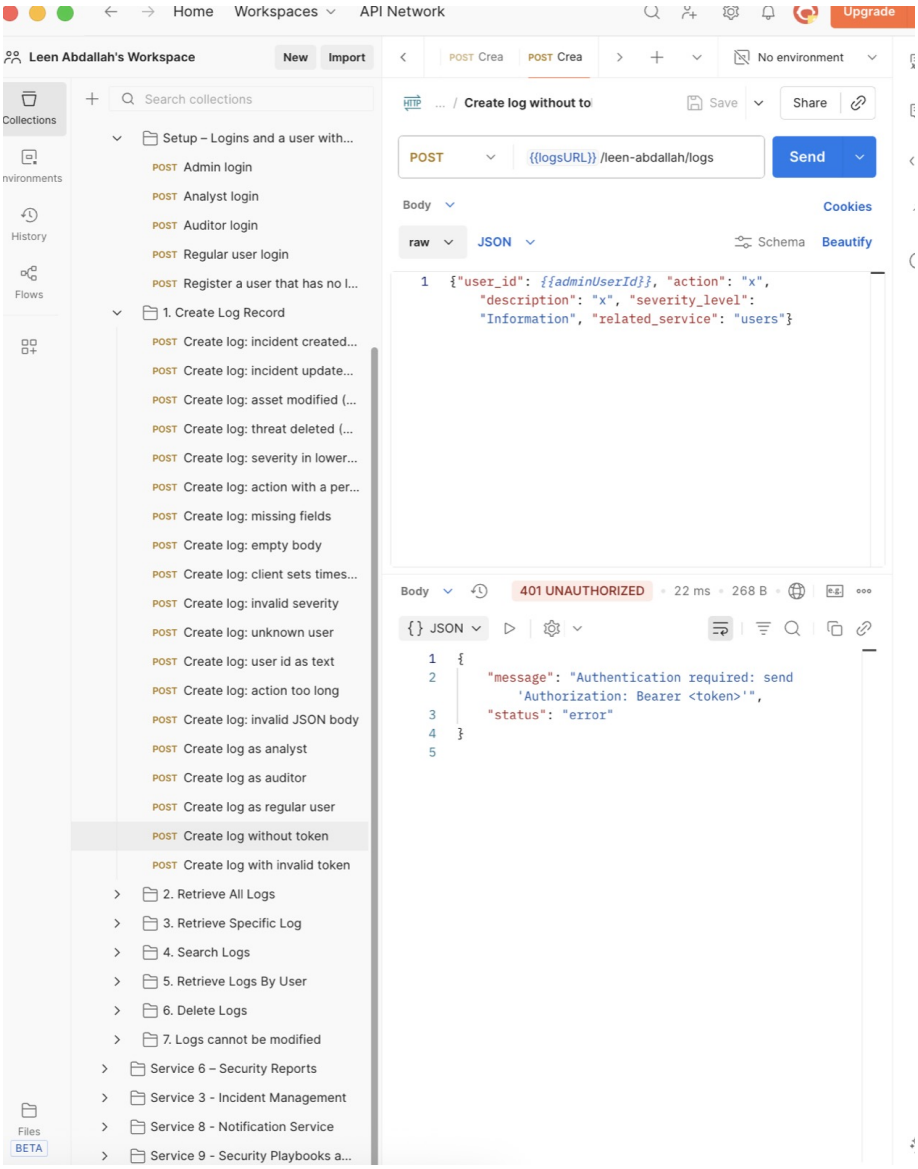


Figure 234: Leen Abdallah: Create Log Record: no token, 401

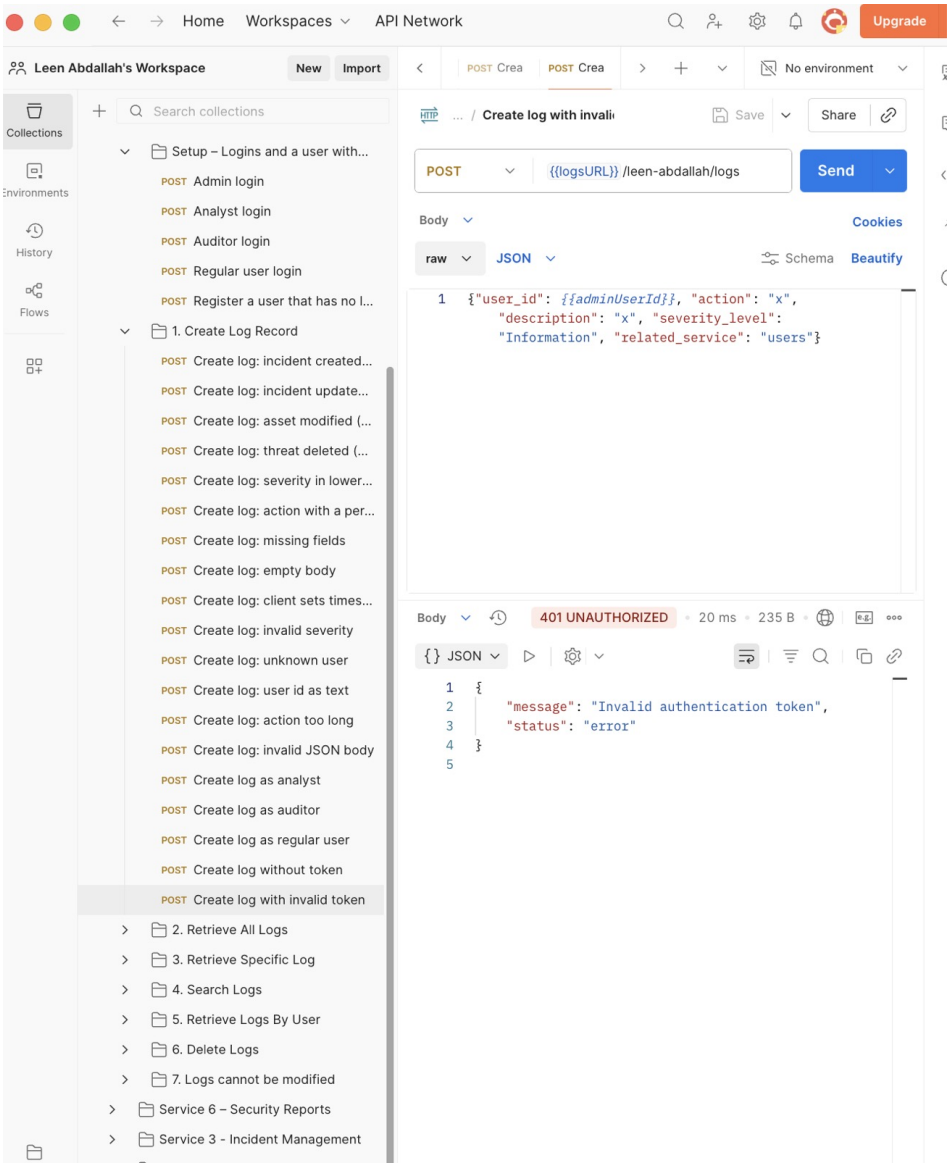


Figure 235: Leen Abdallah: Create Log Record: invalid token, 401

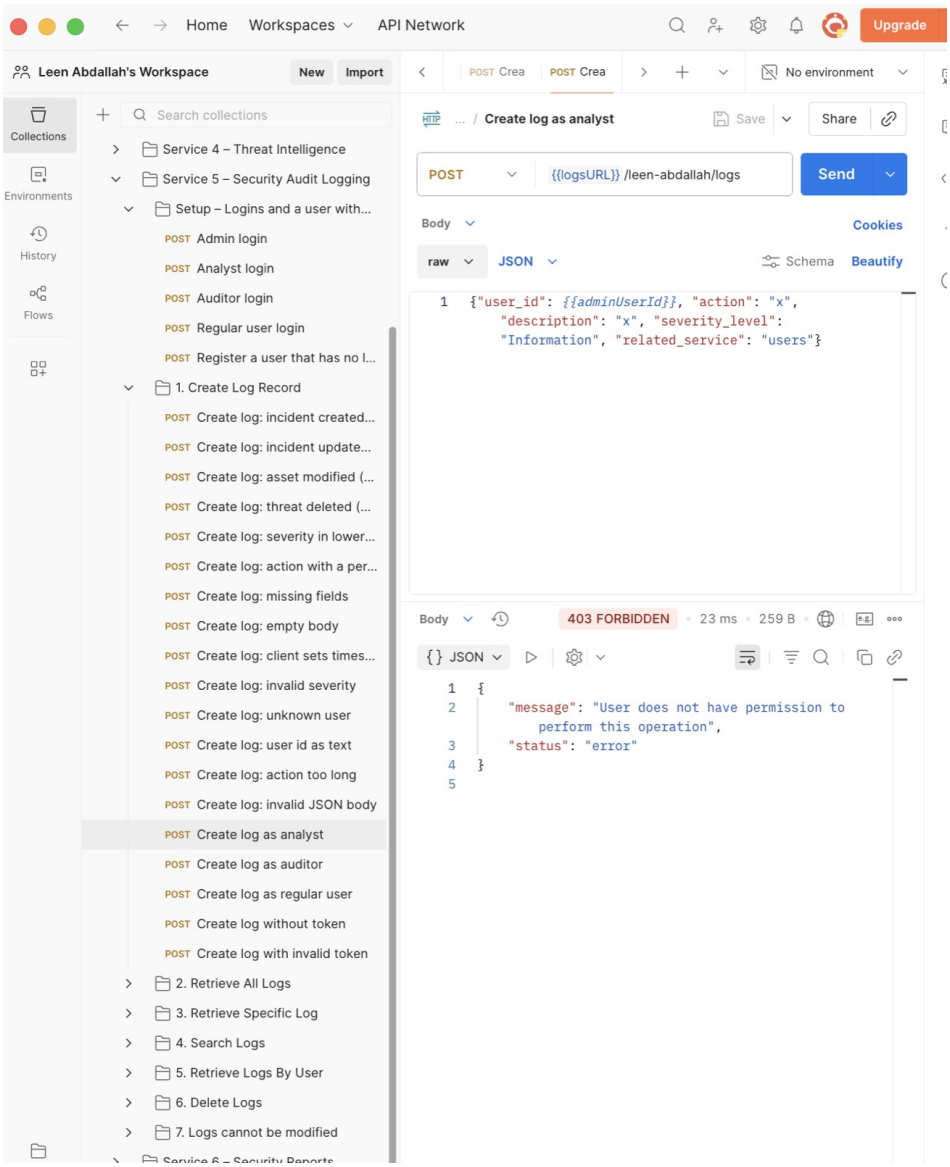


Figure 236: Leen Abdallah: Create Log Record: Security Analyst is refused, 403

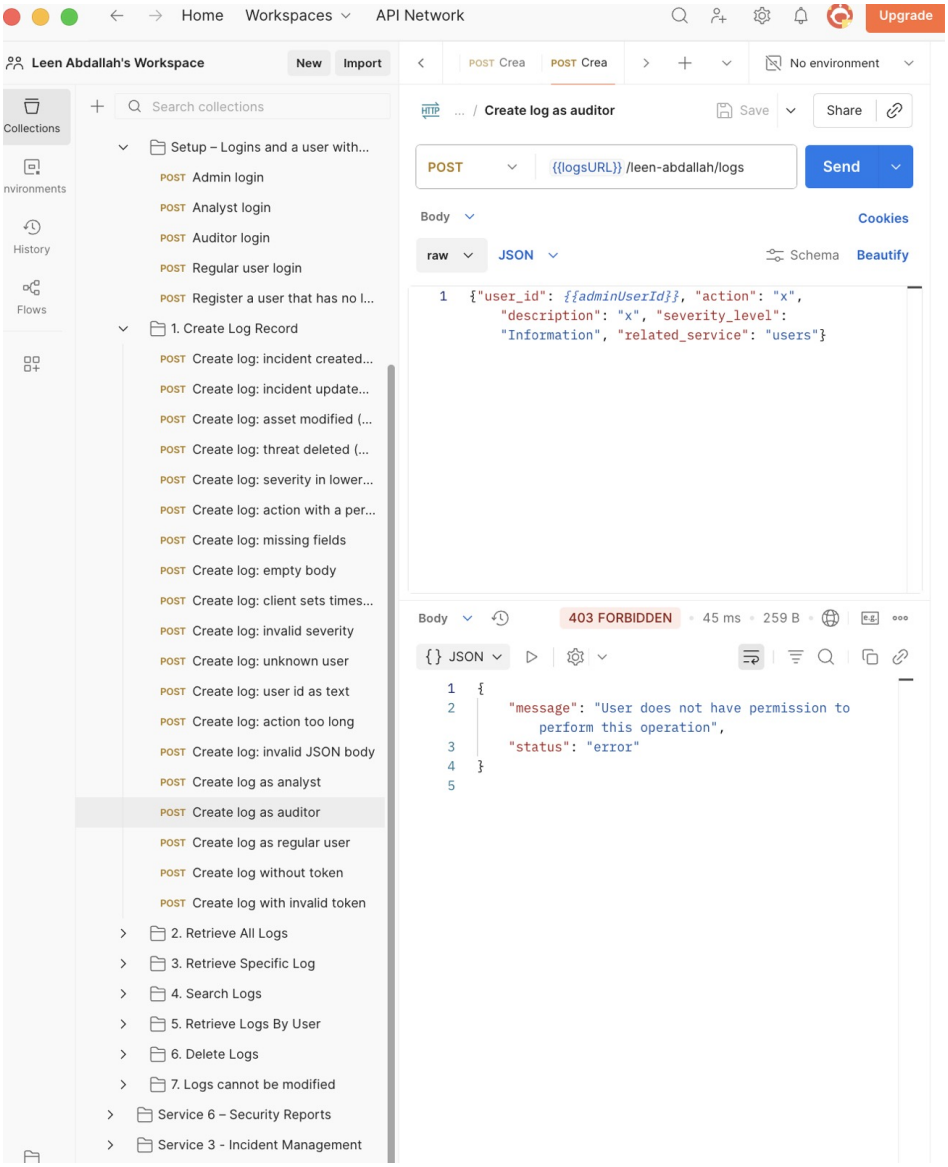


Figure 237: Leen Abdallah: Create Log Record: Auditor is refused, 403

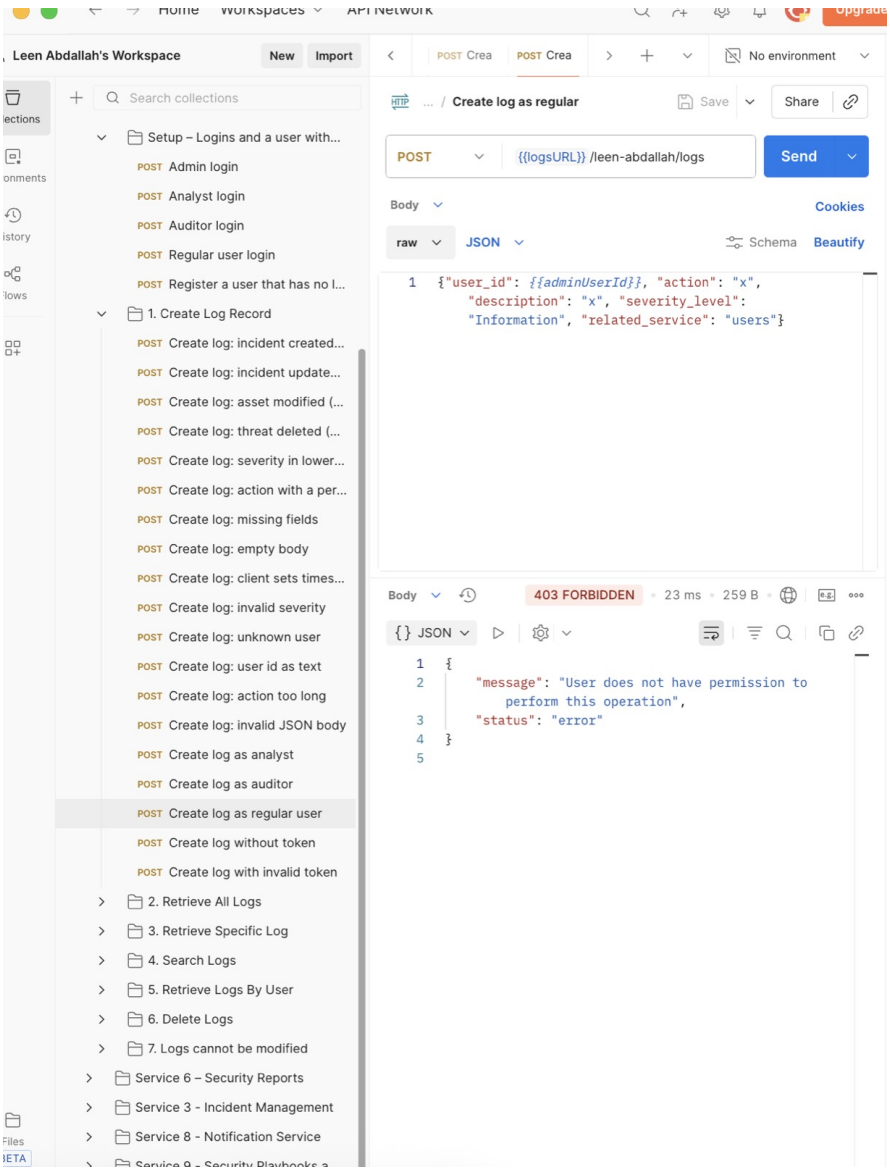


Figure 238: Leen Abdallah: Create Log Record: Regular User is refused, 403

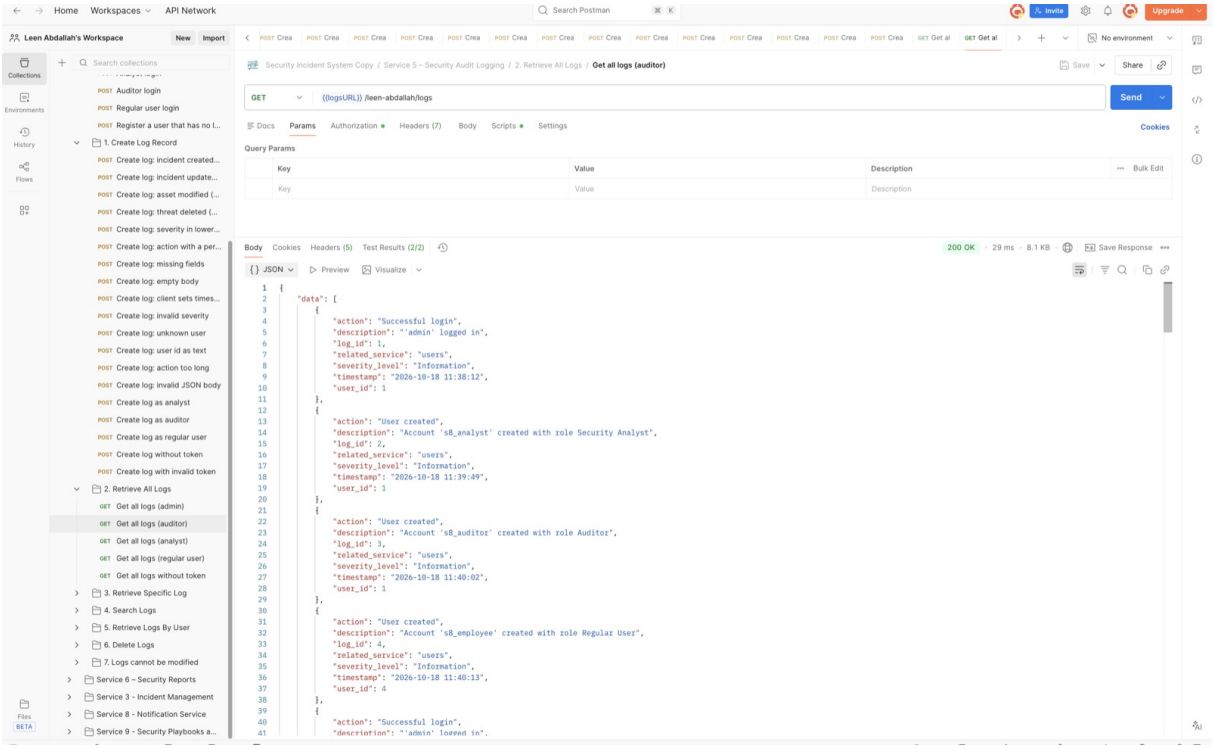


Figure 239: Leen Abdallah: Retrieve All Logs: Auditor allowed, 200

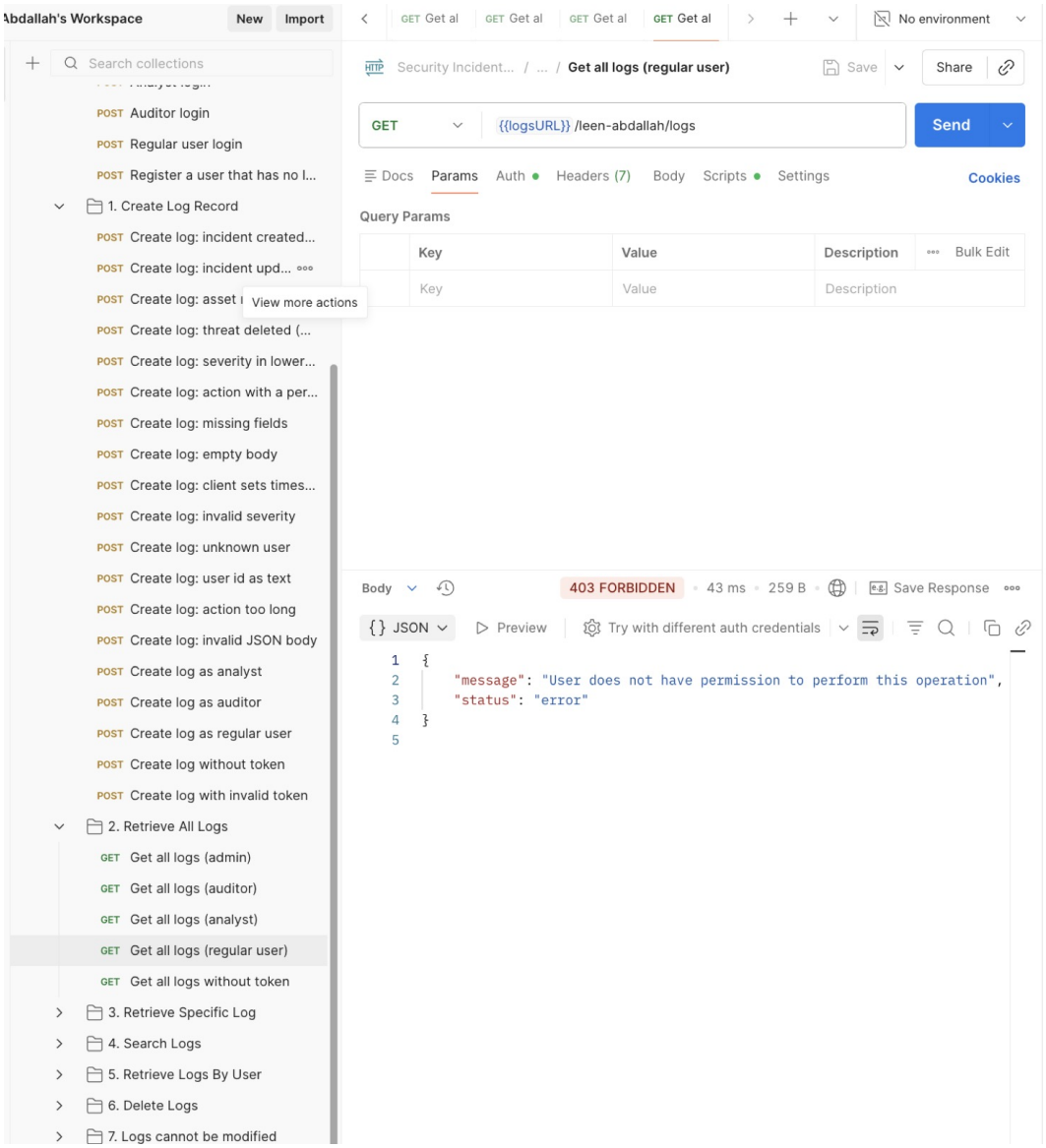


Figure 240: Leen Abdallah: Retrieve All Logs: Regular User is refused, 403

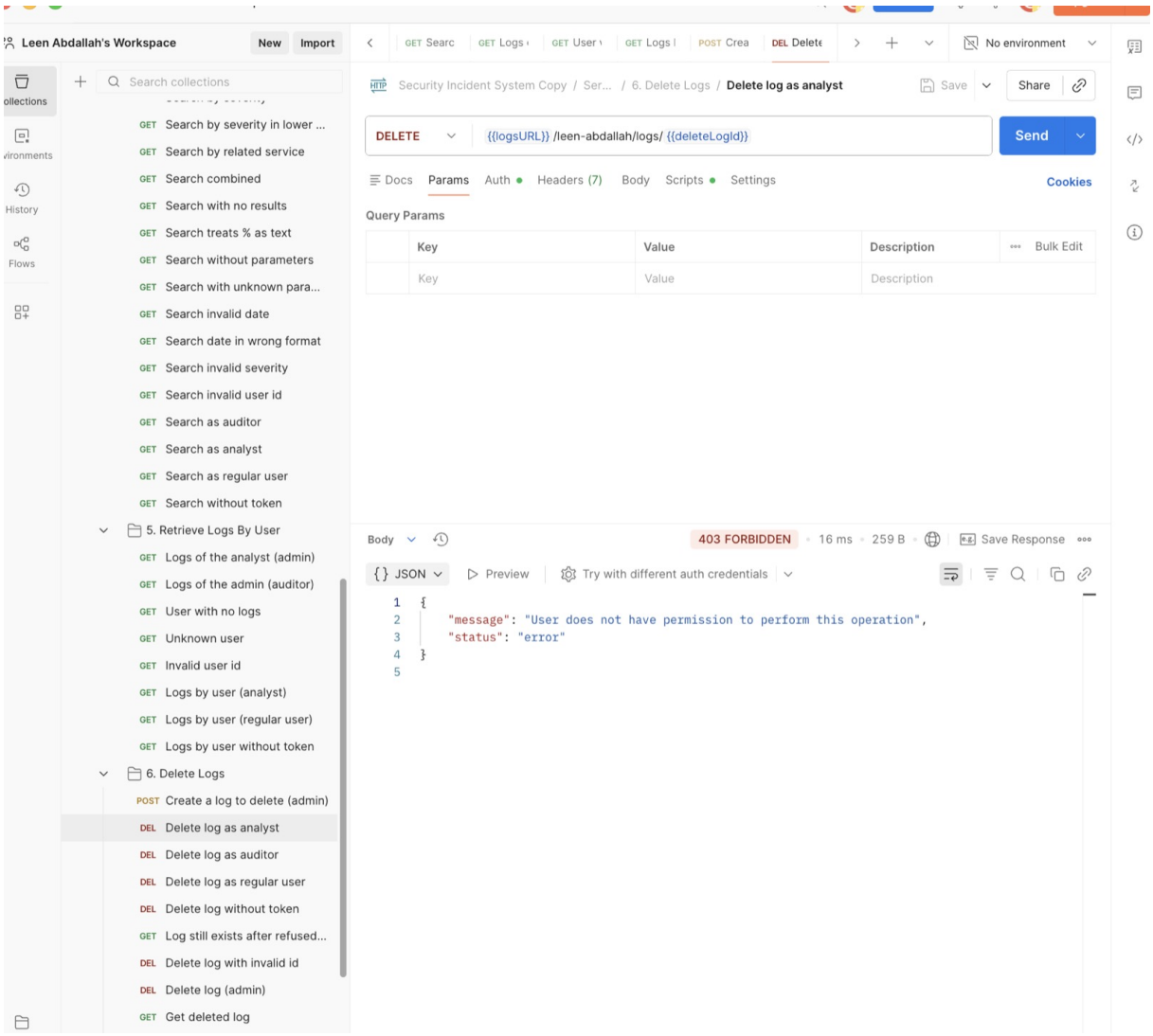


Figure 241: Leen Abdallah: Delete Logs: Security Analyst is refused, 403

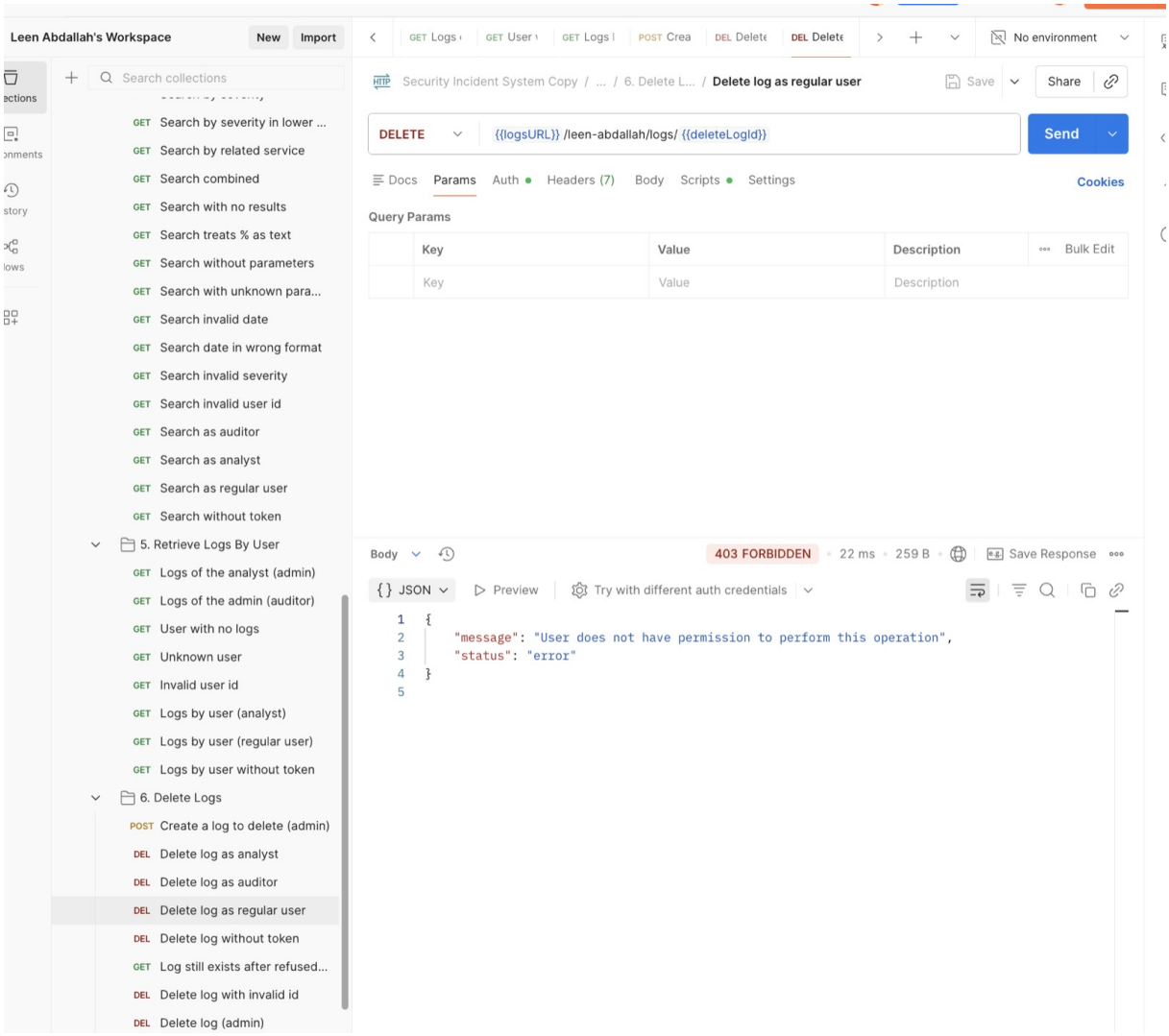


Figure 242: Leen Abdallah: Delete Logs: Regular User is refused, 403

Service 6 (Leen Abdallah)

Figures 243–247: The summary report succeeds for an auditor, rejects analysts and Regular Users, and requires a token. The database query shows the displayed denied report requests were logged.

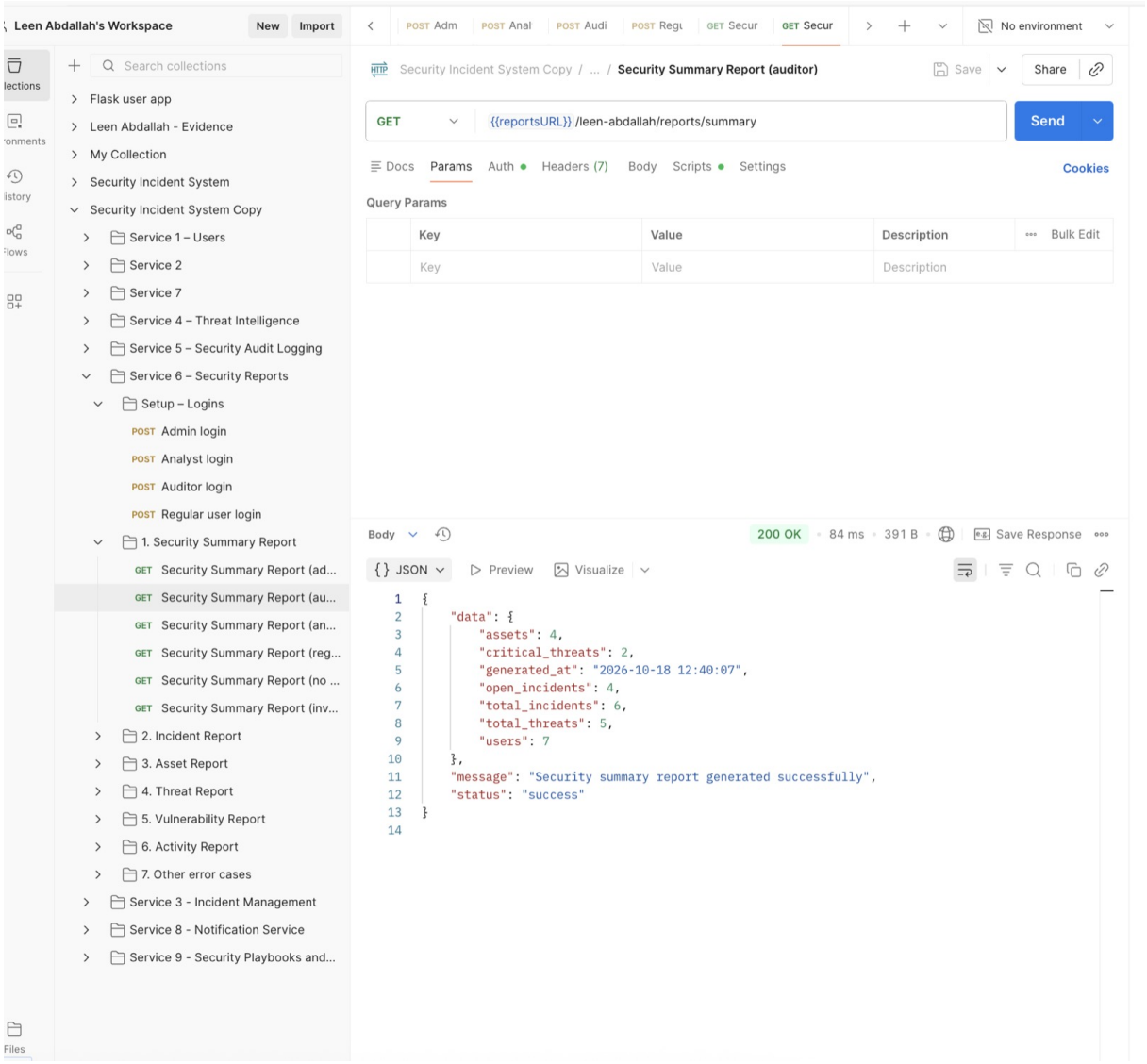


Figure 243: Leen Abdallah: Security Summary Report: Auditor allowed, 200

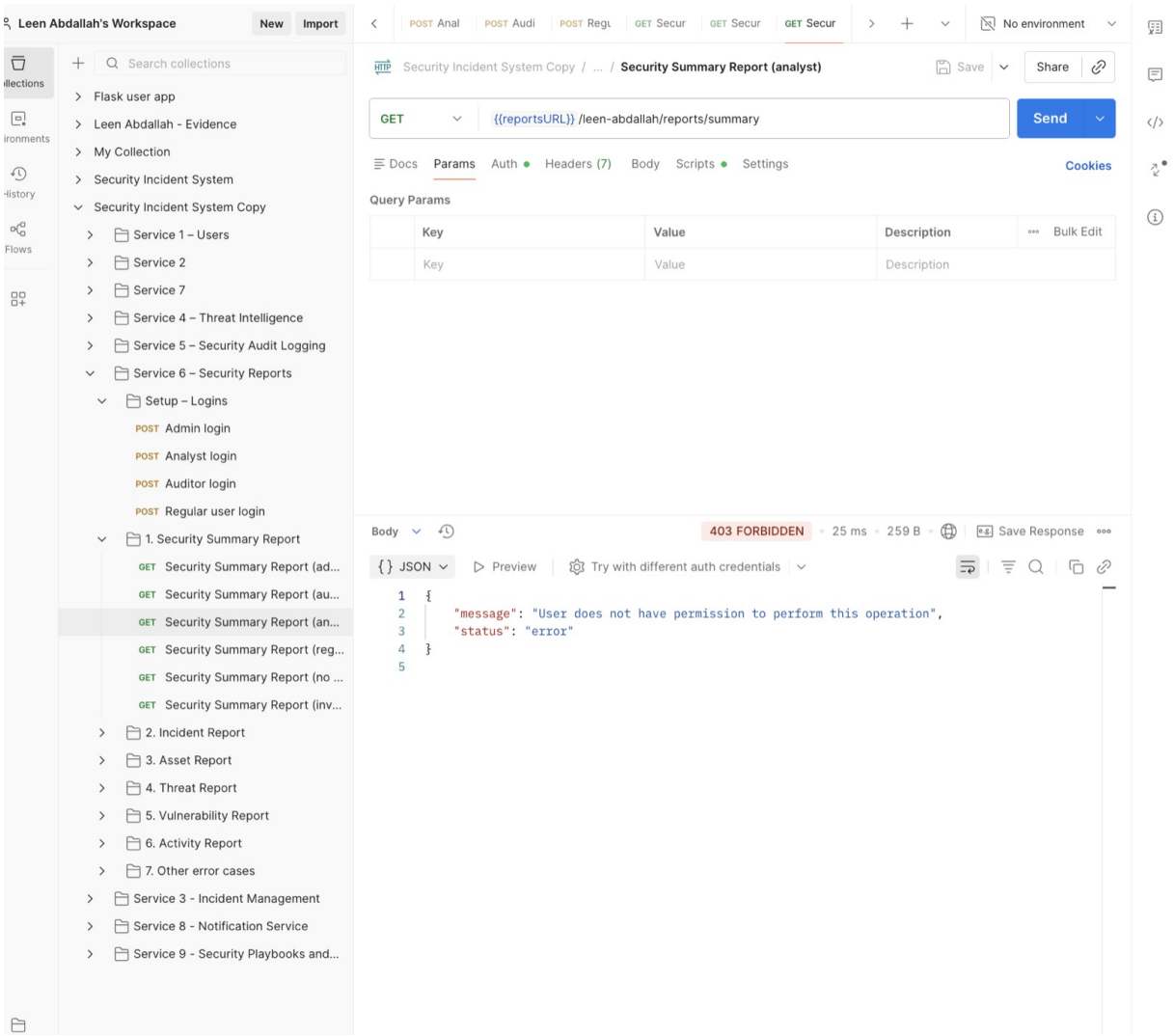


Figure 244: Leen Abdallah: Security Summary Report: Security Analyst is refused, 403

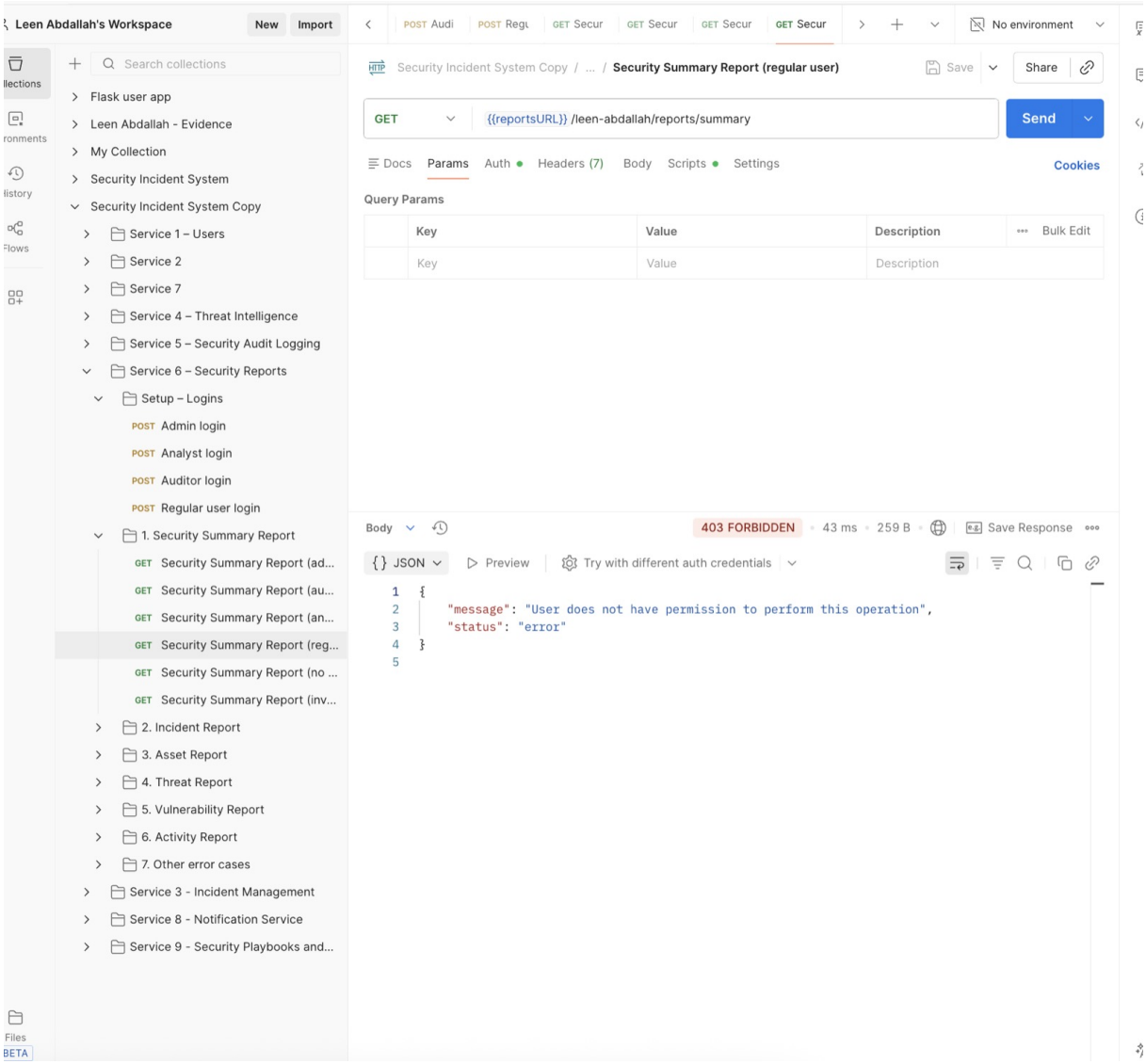


Figure 245: Leen Abdallah: Security Summary Report: Regular User is refused, 403

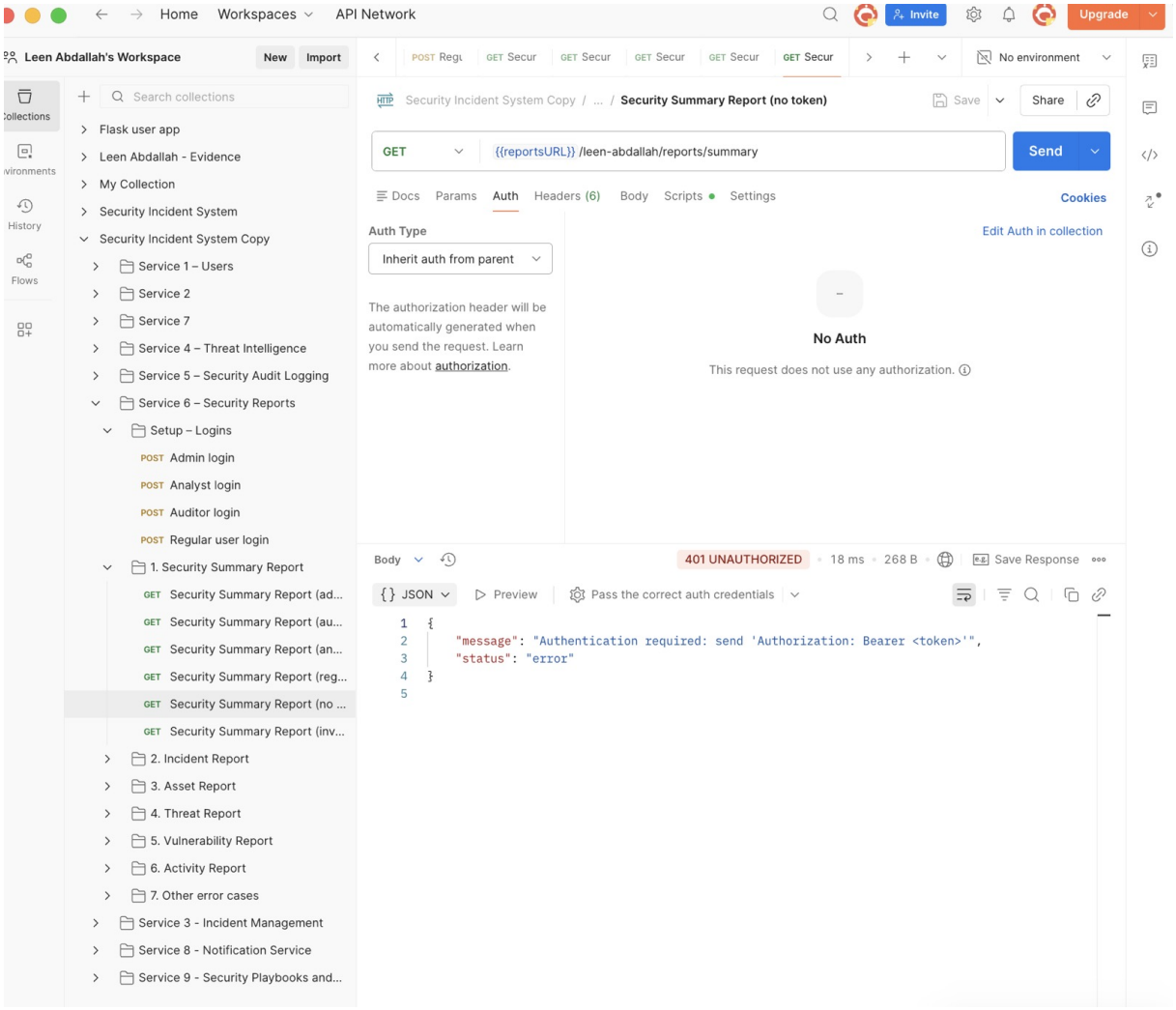


Figure 246: Leen Abdallah: Security Summary Report: no token, 401

```
[leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System % sqlite3 -header -column]
 database/security_incident.db "select log_id,user_id,action,severity_level,related_service from logs
 where related_service='reports' order by log_id desc limit 12;"
log_id  user_id  action                      severity_level  related_service
------  -------  --------------------------  --------------  ---------------
64                Unauthorized access attempt  Warning         reports
63      2         Unauthorized access attempt  Warning         reports
62      4         Unauthorized access attempt  Warning         reports
61                Unauthorized access attempt  Warning         reports
60      4         Unauthorized access attempt  Warning         reports
59      2         Unauthorized access attempt  Warning         reports
58                Unauthorized access attempt  Warning         reports
57      4         Unauthorized access attempt  Warning         reports
56      2         Unauthorized access attempt  Warning         reports
leenabdallah@Leens-MacBook-Air-2 EECE435L-Security-Incident-Response-System %
```

Figure 247: Leen Abdallah: Database: unauthorized report requests recorded in the audit log

10 GitHub and Version Control

10.1 Repository Information

The project repository is Security Incident Response System on GitHub.

The merged pull requests and visible history document feature-branch integration into `main`.

10.2 Commit History

Repository metadata links Adam's aliases to `aazella` and Youssef's to `yy2jisj`. Grouped totals sum to 78, including merges.

Team member	Recorded author names	Commits
Adam Yahya	`aazella`, `Adam Yahya`	17
Leen Abdallah	`leennabdallah`	39
Youssef Al Hajj Youness	`yy2jisj`, `Youssef Al Hajj Youness`	22

Figures 248–255 reproduce the eight commit-history screenshots supplied for this report. Visible entries connect named accounts with implementation, tests, documentation and evidence commits.

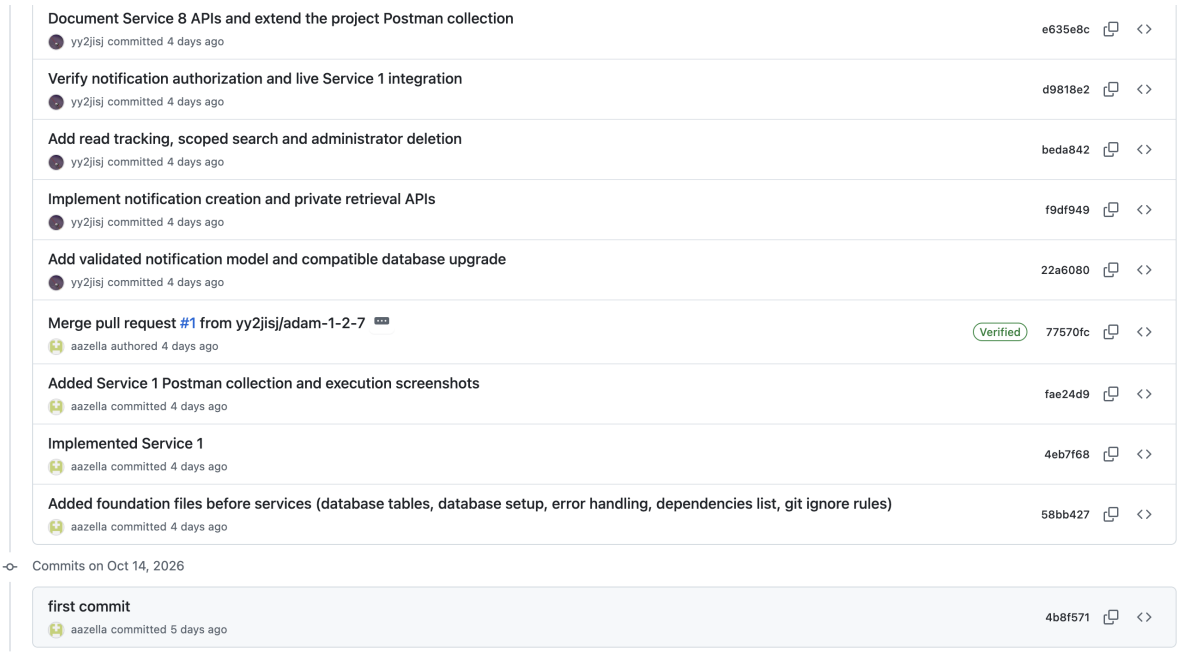


Figure 248: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 14–15 October 2026, initial project foundation, Service 1 implementation and notification development

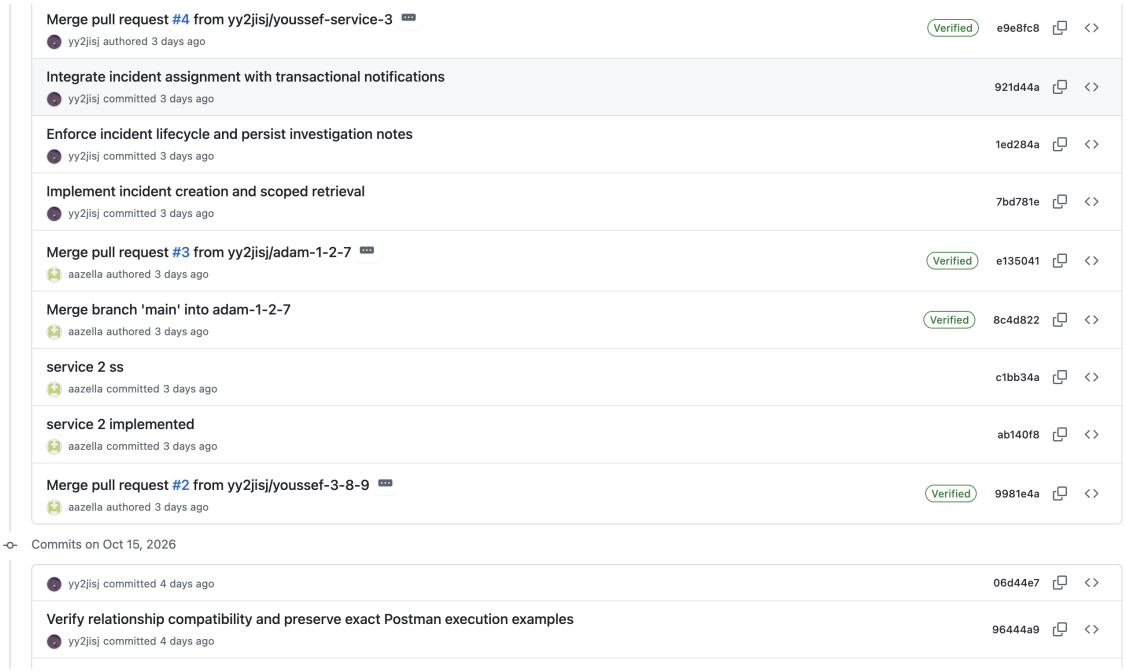


Figure 249: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 15–16 October 2026, Service 2 and asset implementation, incident lifecycle and investigation notes, and merged pull requests #2 to #4

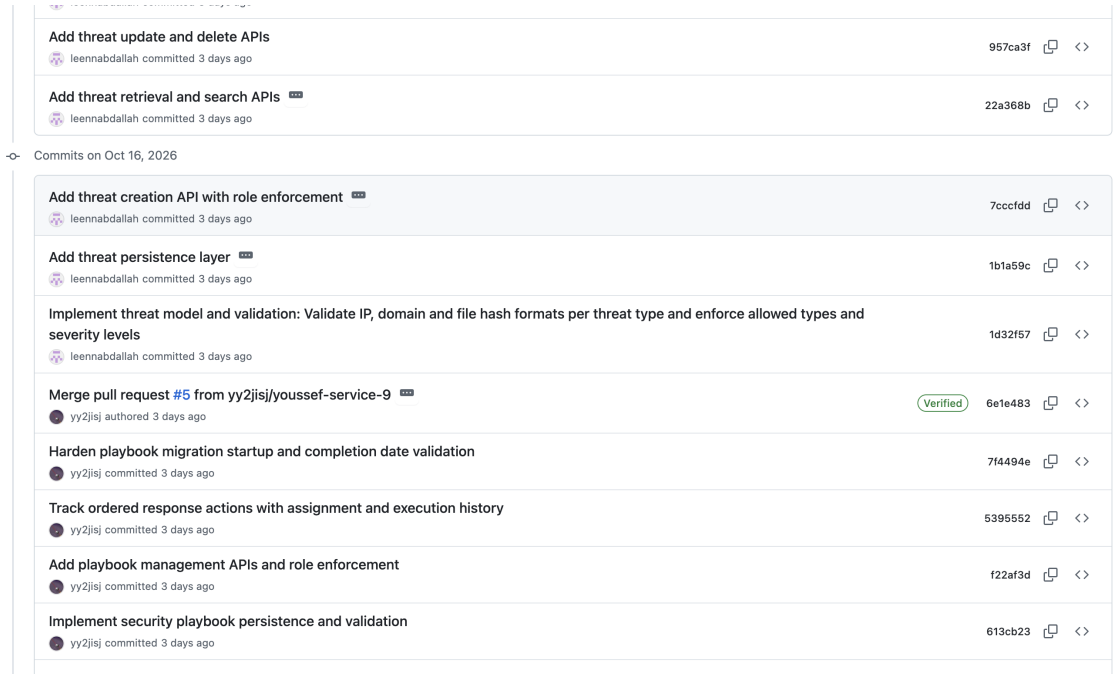


Figure 250: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 16 October 2026, threat model and validation, playbook persistence, ordering and history, and merged pull request #5

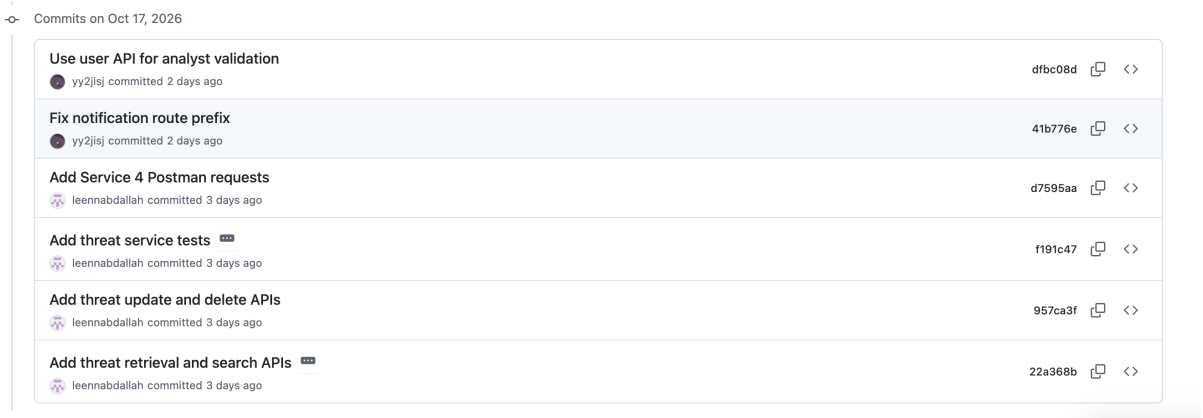


Figure 251: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 16–17 October 2026, threat retrieval, update, delete and test commits, and Service 4 Postman requests

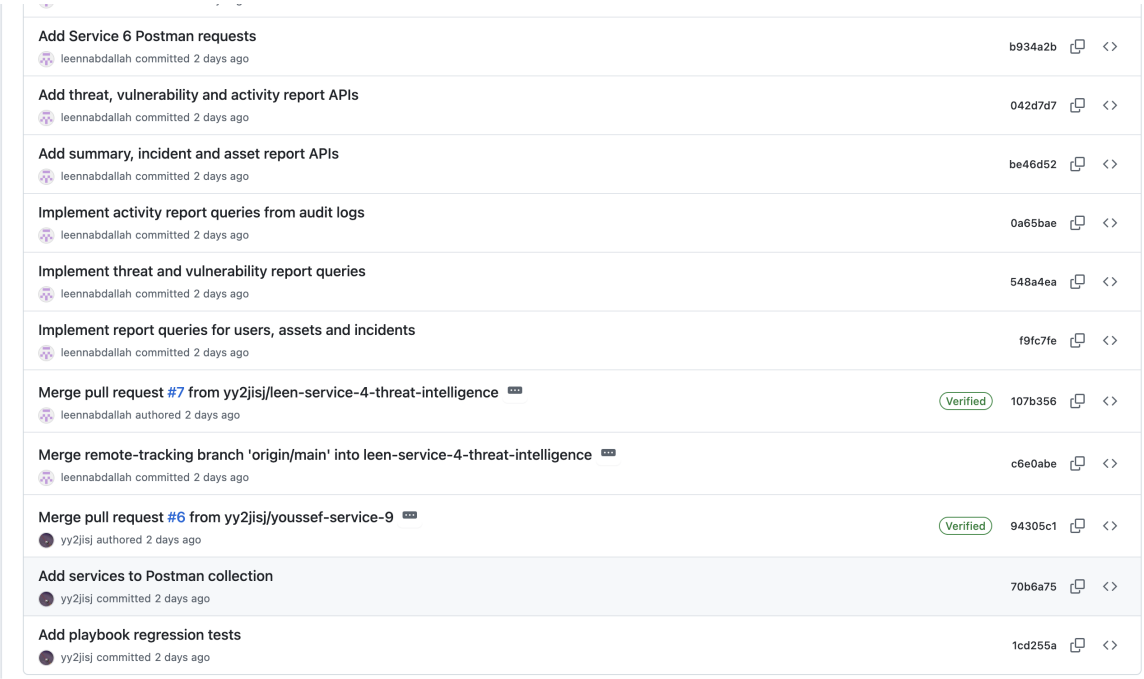


Figure 252: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 17 October 2026, playbook regression tests, report queries and APIs, and merged pull requests #6 and #7

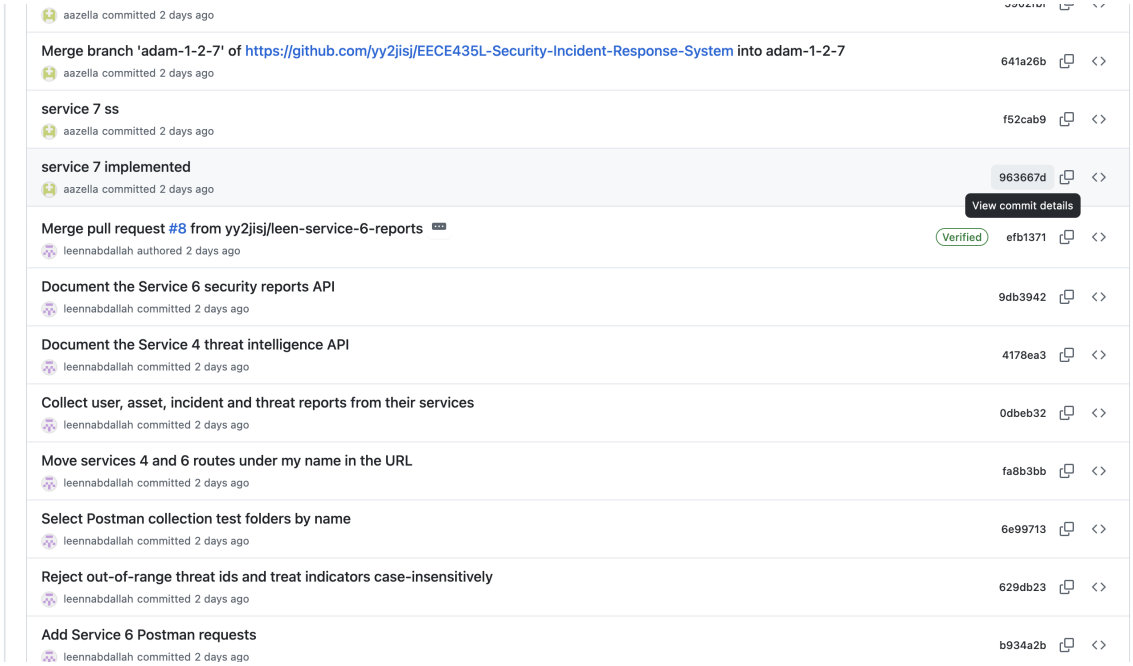


Figure 253: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 17 October 2026, report documentation, Service 7 implementation and merged pull request #8

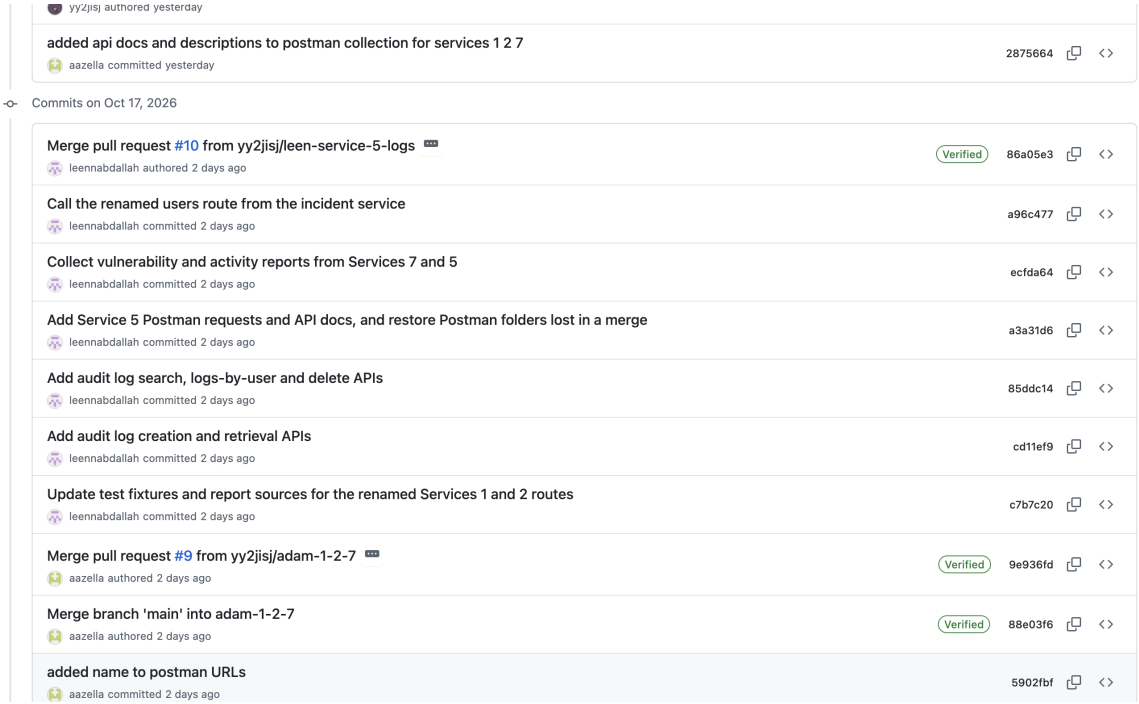


Figure 254: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history: 17 October 2026, audit log APIs, renamed route updates and merged pull requests #9 and #10

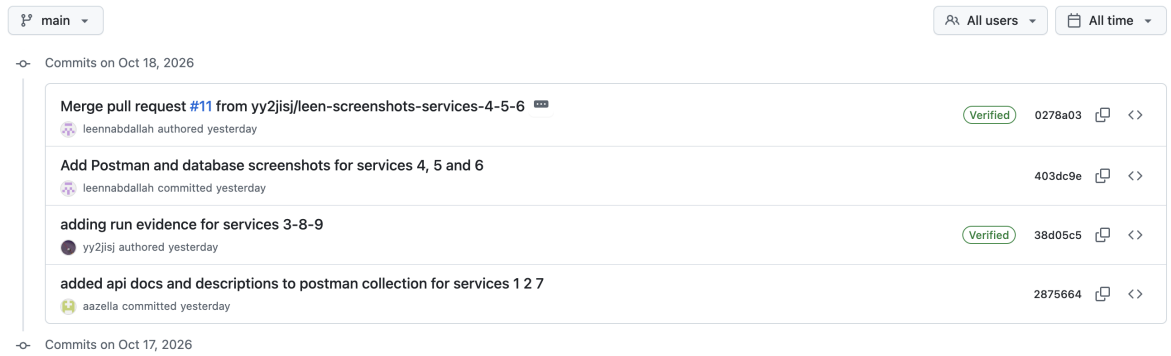


Figure 255: Adam Yahya, Leen Abdallah and Youssef Al Hajj Youness: GitHub history, 18 October 2026: evidence and documentation commits from all three team accounts and the final merged pull request #11

10.3 Team Contribution Evidence

Member	Recorded contribution
Adam Yahya	Services 1, 2 and 7; their API documentation, Postman requests and execution captures.
Leen Abdallah	Services 4, 5 and 6; related tests, API documentation, Postman requests and execution captures.
Youssef Al Hajj Youness	Services 3, 8 and 9, including incident lifecycle, transactional notifications and action history; related tests, API documentation, Postman requests and execution captures.
Team Efforts	Shared foundation and database setup

The commit-history figures show the visible account and commit entries. They are history snapshots.

11 References

These references identify the tools, protocol and course specification. Project source and execution captures establish the implementation-specific behavior.

1. Course material: *Final Project Fall 2026-2027: Sprint 1*, EECE 435L.
2. Labs 1-5 of EECE 435L Course

11.1 Use of AI Tools

The common subdirectory was a mix of previous helper functions that we have used on a similar project in the past, as well as some AI interference that predicted what helper functions might be useful going forward. AI was used to help whenever we got stuck on the syntax while writing the code. Only small chunks of code were taken.
No AI was used for the logic of the system.
All use of AI was monitored with thorough checks from the team members.